
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 15.09.2026, 21:00:17
Файлов исходного кода: 111
Суммарный объём кода: 1.97 MB
Глав/билетов в пособии: 36

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. 6. Графы. Представление графов, DFS, BFS [id: graph-basics]
7. 7. Графы. Планарные. Покраска [id: graph-planarity]
8. 7. Графы. Планарные. Покраска [id: planarity-euler]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articulation-points]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cycle]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: shortest-path-accelerations]
22. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Борувка
24. 19. Графы. Остовное дерево. Прима [id: mst-prim]
25. 20. Графы. Остовное дерево. Борувка [id: mst-kruskal]

- 18. src/data/content_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_segtree.ts
- 27. src/data/tickets/ticket14_20.ts
- 28. src/data/tickets/ticket18_20_mst.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterQuiz.tsx
- 37. src/components/ChapterView.tsx
- 38. src/components/ComplexityViz.tsx
- 39. src/components/DfsBridgesSimulator.tsx
- 40. src/components/DijkstraViz.tsx
- 41. src/components/DPVisualizer.tsx
- 42. src/components/EulerSimulator.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphComponentsViz.tsx
- 45. src/components/GraphTraversalViz.tsx
- 46. src/components/HeapViz.tsx
- 47. src/components/hints/demoKit.tsx
- 48. src/components/hints/demosExtra.tsx
- 49. src/components/hints/demosGraph.tsx
- 50. src/components/hints/demosStruct.tsx
- 51. src/components/hints/MiniDemo.tsx

86. src/layout/header.html

Утилиты

87. src/utils/cn.ts

88. src/utils/exportHtml.ts

89. src/utils/ticketLinks.ts

Скрипты сборки и экспорта

90. scripts/audit-coverage.mjs

91. scripts/audit-terms.mjs

92. scripts/export/build-guide-pdf.mjs

93. scripts/export/build-pdf.mjs

94. scripts/export/capture-viz-shots.mjs

95. scripts/export/collect-code.mjs

96. scripts/export/common.mjs

97. scripts/export/html-to-blocks.mjs

98. scripts/export/render-pages.mjs

99. scripts/verify/browser-checks.tsx

100. scripts/verify/check-links.mjs

101. scripts/verify/check-reference-code.mjs

102. scripts/verify/dom-env.mjs

103. scripts/verify/live-matrix.test.mjs

104. scripts/verify/probe-compiler.tsx

105. scripts/verify/README.md

106. scripts/verify/run-browser-checks.mjs

107. scripts/verify/run-probe.mjs

CI / автоматизация

108. .github/workflows/deploy-pages.yml

109. .github/workflows/rebuild-pdf.yml

110. .github/workflows/verify.yml

Прочее

111. .ai_skills/universal_guide_authoring/SKILL.md

```
contents: read
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
+ group: pages
+ cancel-in-progress: true
+
+ jobs:
+   build:
+     name: Build static site
+     runs-on: ubuntu-latest
+     steps:
+       - name: Checkout
+         uses: actions/checkout@v6
+
+       - name: Setup Node.js
+         uses: actions/setup-node@v7
+         with:
+           node-version: 24
+           cache: npm
+
+       - name: Install dependencies
+         run: npm ci --no-audit --no-fund
+
+       - name: Build for the repository subpath
+         env:
+           VITE_BASE: /algv0/
+         run: npm run build
+
+       - name: Make GitHub Pages fallback
+         run: cp dist/index.html dist/404.html
+
+       - name: Upload Pages artifact
+         uses: actions/upload-pages-artifact@v5
+         with:
+           path: ./dist
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
```

```
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-`${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[
    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommen
          # На Ubuntu политика ImageMagick иногда запре
          # разрешаем то, что нужно пайплайну (text -> p
          for policy in /etc/ImageMagick-6/policy.xml /
            if [ -f "$policy" ]; then
```

```
      name: full-code-export
      path: |
        public/export/full_code_all_pages.pdf
        public/export/full_code_all_pages.txt
      if-no-files-found: error
      retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.*
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-export"
    git push origin "HEAD:${GITHUB_REF_NAME}"
  ....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

 steps:
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, >
    <link rel="icon" type="image/svg+xml" href="%BASE_URL" />
    <title> Дискретка для тупых – Интерактивный гид по
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width, >
+    <link rel="icon" type="image/svg+xml" href="%BASE_URL" />
     <title> Дискретка для тупых – Интерактивный гид по
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen" >
````
```

## `public/favicon.svg`

### Полное содержимое после изменений



```

import { SimulatorHub } from "../components/SimulatorHub"
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeTab, setActiveTab] = useState<"guide" |
 const [activeChapterId, setActiveChapterId] = useState<string>()
 const [modalVizId, setModalVizId] = useState<string>()

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefined;
 const next = index < chapters.length - 1 ? chapters[index + 1] : undefined;

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Стрелки ← → листают темы (как на обучающих сайтах)
 useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target.tagName)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
 }, [goTo, next, prev]);

 const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index, chapters.length]);

 return (
 <div className="min-h-screen bg-slate-950 text-slate-50" >

```

```

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 />
 </main>
</div>
</>
) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);
}
....

```

```
`src/components/Navbar.tsx`
```

```
Полное содержимое после изменений
```

```
```tsx
```

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, C
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  activeTab: 'guide' | 'simulator' | 'compiler';
  setActiveTab: (tab: 'guide' | 'simulator' | 'compiler
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeT
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };
};
```

```
  return (
    <header className="bg-slate-900 border-b border-sla
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
        ap-3 lg:gap-0">
          <div className="flex items-center gap-3 w-full
            <div className="bg-gradient-to-tr from-indigo
              k-0">
                <Sparkles className="w-6 h-6" />
              </div>
            <div className="min-w-0 flex-1">
```

```

        className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
            activeTab === 'compiler'
                ? 'bg-emerald-600 text-white shadow-lg
                : 'text-slate-400 hover:text-white'
        }}
    >
    <Terminal className="w-4 h-4" /> Python
</button>
<a
    id="download-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
    title="Скачать полный PDF сборник всех стра
    >
    <FileDown className="w-4 h-4" /> Скачать PD
</a>
<a
    id="download-txt-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
    title="Скачать полный TXT файл со всем кодом
    >
    <FileText className="w-4 h-4" /> TXT
</a>
<button
    id="download-html-btn"
    onClick={saveBook}
    disabled={busy}
    className="flex items-center justify-center

```



```

-----
export function PythonCompiler() {
  const [code, setCode] = useState(STARTER_CODE);
  const [stdin, setStdin] = useState("");
  const [output, setOutput] = useState("Нажмите «Запустить»");
  const [running, setRunning] = useState(false);
  const [runtimeReady, setRuntimeReady] = useState(false);
  const [runtimeMessage, setRuntimeMessage] = useState("");

  const runCode = useCallback(async () => {
    if (running) return;

    setRunning(true);
    setOutput("");
    setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполнила");

    const stdout: string[] = [];
    const stderr: string[] = [];
    const inputLines = stdin.split(/\r?\n/);

    try {
      const runtime = await getPythonRuntime();
      setRuntimeReady(true);
      setRuntimeMessage("Python готов — код выполняется");

      runtime.setStdout({ batched: (message) => stdout.push(message) });
      runtime.setStderr({ batched: (message) => stderr.push(message) });
      runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
      await runtime.runPythonAsync(code);

      const text = [...stdout, ...stderr].join("");
      setOutput(text || "Готово: программа ничего не выполнила");
    } catch (error) {
      const captured = [...stdout, ...stderr].join("");
      setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
      setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
    } finally {
      setRunning(false);
    }
  }, [code, stdin]);
}

```

```

        <span className="text-[11px] text-slate
</div>
<div className="flex items-center gap-2">
  <button
    type="button"
    onClick={reset}
    className="inline-flex items-center gap-2 text-slate-800 transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" />
  </button>
  <button
    type="button"
    onClick={() => void runCode()}
    disabled={running}
    className="inline-flex items-center gap-2 text-white text-xs font-bold transition-colors"
  >
    {running ? <Loader2 className="w-3.5 h-3.5" /> : "Запустить"}
  </button>
</div>
</div>

<textarea
  value={code}
  onChange={(event) => setCode(event.target.value)}
  onKeyDown={(event) => {
    if ((event.ctrlKey || event.metaKey) && event.defaultPrevented) {
      void runCode();
    }
  }}
  spellCheck={false}
  aria-label="Код Python"
  className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-slate-500"
  placeholder="Напишите Python-код..."

```

```
-----
    <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
      <Info className="w-4 h-4 shrink-0 text-indigo-600" >
        <span>Ctrl или Cmd + Enter запускает код. Введите код в поле ввода.
      </span>
    </div>
    <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
      <Info className="w-4 h-4 shrink-0 text-amber-600" >
        <span>Первый запуск скачивает примерно несколько килобайт кода.
      </span>
    </div>
    <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
      <Info className="w-4 h-4 shrink-0 text-emerald-600" >
        <span>Код выполняется в браузере и не отправляется на сервер.
      </span>
    </div>
  </div>
</div>
</main>
);
}
....
```

Patch относительно `main`

```
````diff
diff --git a/src/components/PythonCompiler.tsx b/src/components/PythonCompiler.tsx
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python и установит библиотеки
+name = "алгоритмы"
+
+def main():
+ print("Привет, мир!")
+ return "Ваш код выполнен успешно!"
+
+main()
+`;
```



```

+
+ const runCode = useCallback(async () => {
+ if (running) return;
+
+ setRunning(true);
+ setOutput("");
+ setRuntimeMessage(runtimeReady ? "Выполняю код..." :
+
+ const stdout: string[] = [];
+ const stderr: string[] = [];
+ const inputLines = stdin.split(/\r?\n/);
+
+ try {
+ const runtime = await getPythonRuntime();
+ setRuntimeReady(true);
+ setRuntimeMessage("Python готов – код выполняется");
+
+ runtime.setStdout({ batched: (message) => stdout
+ runtime.setStderr({ batched: (message) => stderr
+ runtime.setStdin({ stdin: () => (inputLines.length
+ await runtime.runPythonAsync(code);
+
+ const text = [...stdout, ...stderr].join("");
+ setOutput(text || "Готово: программа ничего не выв
+ } catch (error) {
+ const captured = [...stdout, ...stderr].join("")
+ setOutput(`${captured}${captured && !captured.en
+ setRuntimeMessage("Не удалось выполнить код – пр
+ } finally {
+ setRunning(false);
+ }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть резу.
+ };

```

```

late-800 transition-colors"
+
+ >
+ <RotateCcw className="w-3.5 h-3.5" />
+ </button>
+ <button
+ type="button"
+ onClick={() => void runCode()}
+ disabled={running}
+ className="inline-flex items-center
-60 text-white text-xs font-bold transition-colors"
+ >
+ {running ? <Loader2 className="w-3.5
+ {running ? "Запускаю..." : "Запустить"}
+ </button>
+ </div>
+ </div>
+
+ <textarea
+ value={code}
+ onChange={(event) => setCode(event.target
+ onKeyDown={(event) => {
+ if ((event.ctrlKey || event.metaKey) &
+ event.preventDefault();
+ void runCode();
+ }
+ }}
+ spellCheck={false}
+ aria-label="Код Python"
+ className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+ placeholder="Напишите Python-код..."
+ />
+
+ <div className="border-t border-slate-800
+ <label className="block px-4 pt-3 text-[
+ Ввод для input() • по одной строке на
+ </label>
+ <textarea

```

```
+ </div>
+ <div className="flex gap-2 rounded-xl border
+ <Info className="w-4 h-4 shrink-0 text-eme
+ Код выполняется в браузере и не отпра
+ </div>
+ </div>
+ </div>
+ </main>
+);
+}
+...
```

## `vite.config.ts`

### Полное содержимое после изменений

```
```typescript
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algv0/, локальн
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
},
```

Универсальное пособие • полный исходный код

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7

AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

****CRITICAL DIRECTIVE****:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, styles and components required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML elements.

ФАЙЛ 3/111: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2

Gemini App Canvas Instructions

Ты — AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas).
При запросе пользователя создать или сгенерировать новую страницу, ты ****строго**** придерживаешься готовых шаблонов разметки, стилей и компонентов.

Правила для малых страниц (Билетов / Глав)

- **Единый стиль:**** Всегда используй Tailwind CSS и HTML-шаблоны.
- **Структура JSON/TS:**** Каждая новая малая страница должна быть описана в формате `{ "id": "...", "content": ... }` для вставки в ``src/data/content.json``.
- **Строгая структура контента:****
 - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
 - Определение/правило в центре (``border-blue-500`` или ``border-purple-500``).
 - Обязательная сетка аналогий (2 колонки: неформальный язык / строгий язык).

ФАЙЛ 6/111: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

```
{
  "name": "algv0-exam-guide",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "audit:content": "node scripts/audit-coverage.mjs",
    "audit:terms": "node scripts/audit-terms.mjs",
    "export:txt": "node scripts/export/collect-code.mjs",
    "export:png": "node scripts/export/render-pages.mjs",
    "export:pdf": "node scripts/export/build-pdf.mjs",
    "export": "npm run export:txt && npm run export:png",
    "export:guide": "node scripts/export/build-guide-pdf.mjs",
    "export:shots": "node scripts/export/capture-viz-shots.mjs",
    "verify:unit": "node scripts/verify/live-matrix.test.mjs",
    "verify:code": "node scripts/verify/check-reference.mjs",
    "verify:links": "node scripts/verify/check-links.mjs",
    "verify:dom": "node scripts/verify/run-browser-check.mjs",
    "verify:probe": "node scripts/verify/run-probe.mjs",
    "verify": "npm run verify:unit && npm run verify:links",
  },
  "dependencies": {
    "clsx": "2.1.1",
    "lucide-react": "^1.21.0",
    "motion": "^12.40.0",
    "pdfkit": "^0.20.1",
    "react": "19.2.6",
    "react-dom": "19.2.6",
  }
}
```

Универсальное пособие • полный исходный код

```
-----  
npm run dev          # предпросмотр на http://0.0.0.0:5  
npm run build        # сборка в dist/ (single-file)  
````
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **\*\*весь\*\*** исходный код репозитория в о

```
Команда	Шаг	Результат
`npm run export:txt`	код → txt	`public/export/full_code_all_pages.txt`
`npm run export:png`	txt → png	`tmp/export-pages/`
`npm run export:pdf`	png → pdf	`public/export/full_code_all_pages.pdf`
`npm run export`	всё сразу	txt + png + pdf
```

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core  
````
```

Настройки через переменные окружения:

- \* `EXPORT\_DENSITY=200` – DPI растеризации (по умолчанию 300)
- \* `EXPORT\_GRAYSCALE=1` – оттенки серого вместо 1-битного

### Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается **\*\*на каж**  
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док  
и коммитит обновлённые `public/export/full\_code\_all\_pages`  
(с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code\_all\_pages`

## Аудит контента

```
```bash
```

Заголовки

- * Заголовки секций: Используют стандартный тег `<h2>` и
- * Подзаголовки: Используют тег `<h3>`.

Текст и параграфы

- * Обычный текст содержится в тегах `<p>`.
- * Блоки "Шиза" (Аналогии) и другая текстовая инфографика параграфы `<p>`.

Математические формулы

В приложении используется MathJax.

- * В самом исходном коде страницы (и внутри тегов) формулы.
- * Таким образом, при извлечении текстового содержимого математически остаются в валидном Markdown-формате латеха (и

Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- * Они помещены внутрь оберток с классом `<div class="img">`
- * Все отрисовки находятся внутри векторного тега `<svg>`
- * Подпись к картинке, если она есть, идет сразу после `<img>`

Списки

- * Стандартные списки оформлены через `<ul class="list-n">`

Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего

Универсальное пособие • полный исходный код

```
-----
* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**
* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь и BFS**
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой поиск**
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Деревья отрезков**
* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование**
---
```

Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухари»)

Глава	id	Комментарий
15. Планарность: K5, K3,3 и формула Эйлера	`planarity-euler`	т «на пальцах».
16. Флойд (Флойд–Уоршелл)	`floyd`	Есть матричный алгоритм
17. Алгоритм Джонсона	`johnson-algo`	Перевзвешивание
Обзор: Кратчайшие пути и Графы	`intro`	Вводная страница
Эйлер: Путь ≠ Цикл	`euler-path-vs-cycle`	Ключевая концепция

Уровень 4. Есть аналогия, но НЕТ 2D-визуализации («сложно»)

> Замечание: «5. Двумерная матрица префиксных сумм» пока закрыто

Глава	id	Чего не хватает
3. Декартово дерево (Treap)	`treap`	Нет ни визуализации, ни интерактива
8. Графы. Компоненты связности	`graph-components`	Нет визуализации
24. Классы сложности, сведение задач	`complexity-classes`	Нет визуализации

Сводная таблица по всем главам

#	Глава	Аналогий	2D-виз.	SVG	Картинка
---	---	---	---	---	---

ФАЙЛ 9/111: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "types": ["node"],

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",

    /* Path mapping */
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"]
    },

    /* Linting */
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src", "vite.config.ts"]
}
```

```
    },  
  });  
}
```

РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

ФАЙЛ 11/111: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 218 | РАЗМЕР: 11.3 КБ

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";  
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";  
import { chapters } from "../data/content";  
import { Navbar } from "../components/Navbar";  
import { Sidebar } from "../components/Sidebar";  
import { MobileToc } from "../components/MobileToc";  
import { ChapterView } from "../components/ChapterView";  
import { ChapterNav } from "../components/ChapterNav";  
import { SimulatorModal } from "../components/hints/SimulatorModal";  
import { LiteGuide } from "../components/LiteGuide";  
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite`
```

```
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
```

```
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {  
  if (typeof window === "undefined") return { lite: false };  
  const q = new URLSearchParams(window.location.search);  
  const lite = q.has("lite") || q.get("view") === "lite";  
  return { lite, topic: q.get("topic") ?? undefined };  
}
```

```
export default function App() {  
  const liteMode = useMemo(readLiteMode, []);
```

```

    setSidebarMode((m) => {
      const next = m === "wide" ? "compact" : m === "co
      window.localStorage.setItem("sidebarMode", next);
      return next;
    });
  }, []);
const sidebarCollapsed = sidebarMode === "hidden";

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
  const apply = () => {
    const el = contentRef.current;
    if (!el) return;
    el.style.paddingRight = compilerOpen && window.in
  };
  apply();
  window.addEventListener("resize", apply);
  return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
  0,
  chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefi
const next = index < chapters.length - 1 ? chapters[i

const goTo = useCallback((id: string) => {
  setActiveChapterId(id);
  window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
  setCompilerOpen(true);
  requestAnimationFrame(() => {

```

```

<div className="flex flex-col lg:flex-row max-w-
  /* Сайдбар только на десктопе — на мобильном
  <div
    id="sidebar-root"
    className={`hidden lg:block shrink-0 border
      -all duration-200 print:hidden ${sidebarCol
    " : "lg:w-56"}` }
  >
    {!sidebarCollapsed && (
      <Sidebar selectedChapterId={activeChapterId}
    )}
  </div>
  /* Вкладка-переключатель содержания у левого
  <button
    type="button"
    onClick={toggleSidebar}
    className="hidden lg:flex items-center gap-1
    border-slate-700 bg-slate-900/90 px-1 py-3
    aria-label={sidebarMode === "wide" ? "Сжать
    ание"}
    title={sidebarMode === "wide" ? "Сжать содер
    }
  >
    {sidebarCollapsed ? <PanelLeftOpen className="
    <span className="text-[10px] font-bold [wri
  </button>

  <main id="main-content" className="flex-1 min
    /* Шапка главы с быстрыми переходами */
    <div className="flex items-center justify-b
      <div className="min-w-0">
        <div className="text-[10px] uppercase t
          {activeChapter.category || "Универсальн
        </div>
        <h2 className="text-base sm:text-xl font
          {activeChapter.title}
        </h2>
      </div>
    </div>

```

```
        onHeightChange={setCompilerHeight}
        onClose={() => setCompilerOpen(false)}
      />
    )}
  </div>
);
}
```

ФАЙЛ 12/111: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРӨК: 129 | РАЗМЕР: 5.1 K

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

/** Числовые константы TreeWalker: глобального `NodeFile
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.\*\+\?\^\$\{\}\(\)]/

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
```

```

    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
        const surface = m[1].toLowerCase();
        const id = labelToId.get(surface);
        if (!id) continue;

        const usedTerm = perTerm.get(id) ?? 0;
        const usedSurface = perSurface.get(surface) ?? 0;
        if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
        perTerm.set(id, usedTerm + 1);
        perSurface.set(surface, usedSurface + 1);

        if (m.index > last) frag.appendChild(document.createElement("span"));
        const span = document.createElement("span");
        span.className = "term-hint";
        span.dataset.termId = id;
        span.tabIndex = 0;
        span.setAttribute("role", "button");
        span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
        span.textContent = m[1];
        frag.appendChild(span);
        last = m.index + m[1].length;
    }

    if (last === 0) continue;
    if (last < text.length) frag.appendChild(document.createElement("span"));
    textNode.parentNode?.replaceChild(frag, textNode);
}

/**

```

```
.neon-glow-blue {
  box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
}
.neon-glow-emerald {
  box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}
.neon-glow-rose {
  box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
}
.neon-glow-yellow {
  box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
}
.no-scrollbar::-webkit-scrollbar {
  display: none;
}
.no-scrollbar {
  scrollbar-width: none;
}
}

@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
    filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5));
  }
  50% {
    stroke: #fef08a;
    stroke-width: 8;
    filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9));
  }
}

.animate-pulse-gold {
  animation: pulse-gold 1.5s infinite;
}
```

```
    color: #fff;
    outline: none;
}
@media (hover: none) {
    .term-hint {
        border-bottom-style: solid;
        background: rgba(99, 102, 241, 0.12);
    }
}

/* Ссылка на другой билет («...берите Форда–Беллмана (билет
.ticket-link {
    color: #a5b4fc;
    border-bottom: 1px dotted rgba(165, 180, 252, 0.7);
    text-decoration: none;
    border-radius: 3px;
    padding: 0 1px;
    cursor: pointer;
    transition:
        background-color 0.15s ease,
        color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
    background: rgba(99, 102, 241, 0.25);
    color: #fff;
    outline: none;
}

/* Контент главы приходит готовым HTML – чиним его под
.chapter-body {
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}
```



```
    line-height: 1.3 !important;
  }
  .chapter-body :where(h3) {
    font-size: 1.05rem !important;
    line-height: 1.35 !important;
  }
  .chapter-body :where(.p-6, .p-8) {
    padding: 0.9rem !important;
  }
  .chapter-body :where(.text-3xl) {
    font-size: 1.4rem !important;
  }
  .chapter-body :where(.text-2xl) {
    font-size: 1.2rem !important;
  }
  .chapter-body :where(.text-xl) {
    font-size: 1.05rem !important;
  }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}
::-webkit-scrollbar-track {
  background: #0f172a;
}
::-webkit-scrollbar-thumb {
  background: #334155;
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци.
@keyframes demoProgress {
```

```
main {
  padding: 0 !important;
  max-width: none !important;
}

/* текст темы – чёрным по белому, без теней и плашек
article.chapter-body .prose,
article.chapter-body .prose * {
  color: #111827 !important;
  background-color: transparent !important;
  background-image: none !important;
  box-shadow: none !important;
  text-shadow: none !important;
  border-color: #d1d5db !important;
}

article.chapter-body .prose pre,
article.chapter-body .prose code {
  background-color: #f3f4f6 !important;
  color: #111827 !important;
  font-size: 9pt;
}

article.chapter-body .prose table {
  border-collapse: collapse;
}

article.chapter-body .prose th,
article.chapter-body .prose td {
  border: 1px solid #9ca3af !important;
}

article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
  page-break-after: avoid;
}
```

Универсальное пособие • полный исходный код

ФАЙЛ 14/111: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

ФАЙЛ 15/111: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

```
export interface Chapter {
  id: string;
  title: string;
  type: "html" | "markdown";
  content: string;
  description?: string;
  category?: string;
}
```

ФАЙЛ 16/111: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

```
/// <reference types="vite/client" />
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
```

```
<div class="space-y-8">
```

```
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-blue-400">
```

```
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">
      <p class="text-xl font-mono text-white">
      <p class="text-sm font-mono text-slate-300">
    ] [k] </p>
  </div>
```

```
<p class="text-slate-300 text-sm mb-4">Ячейка
лины <span class="font-mono text-emerald-300">
">i</span>. Любой отрезок накрывается <b>двумя
перация <b>идемпотентна</b>: <span class="font-mono text-emerald-300">min</span>, <span class="font-mono text-emerald-300">
, побитовые И и ИЛИ.</p>
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-indigo-300 italic">
  <p class="text-xl font-mono text-white">
  <p class="text-sm font-mono text-slate-300">
вет = op из четырёх блоков kx × ky, накрыва
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Двумя
по столбцам. Отсюда и цена: уровней становится
s="font-mono text-emerald-300">log N</span>
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
  <div class="bg-slate-900 border border-slate-700">
    <p class="text-[10px] uppercase tracking-wider">
```

```

        <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
            Аналогия 2: Торт-пирамидка
        </div>
        <p class="text-slate-300 text-sm mb-4">
            ия, зато красивый, как пирамидка: каждый из
            сех блоков  $2^{kx} \times 2^{ky}$ ), и ответ на запрос
            ой операции нет: крем обратно в торт не запы
             $n(x, x) = x$ ), и цена  $O(1)$  – сто коржей вмес
            <div class="rounded-lg bg-slate-950">
                <div><span class="text-slate-500">
                     $n \ O(1)$ </span></div>
                <div><span class="text-slate-500">
                    = 100 копий матрицы</span></div>
            </div>
            <p class="text-[12px] text-slate-400">
                шает). Менять элементы тоже нельзя. Сумму ч
            </div>
        </div>
        <p class="text-slate-400 text-sm mt-4">Поря
            ерению) → <b>2D: построение</b> (уровни <span
            ос</b> (прямоугольник раскладывается на чет
        </div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-rose-400">
    <p class="text-slate-300 text-sm mb-4">Эти
        за <span class="font-mono text-emerald-300">
        они решают разные задачи.</p>

<div class="bg-slate-800/70 p-5 rounded-lg">
    <p class="text-sm font-bold text-white">
    <p class="text-slate-300 text-sm mb-3">
        цы, у разреженной таблицы – размер, умножен
        <table class="w-full text-xs text-left">
            <thead>
                <tr class="text-slate-500 border">
                    <th class="py-1.5 pr-2 font
```

Почему префиксная тонкая

</div>

<p class="text-slate-300 text-sm mb-1" style="border: 1px solid #000; padding: 5px; display: inline-block; width: 100px; height: 100px; border-radius: 5px; background-color: #f0f0f0; text-align: center; vertical-align: middle;">P[i][j]</p>
ругой прямоугольник выражается через четыре

</div>

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md">

Почему разреженная жирная

</div>

<p class="text-slate-300 text-sm mb-1">ум на его части не вычислишь. Приходится хранить $\log N \times \log M$ на каждой ячейке.</p>

</div>

</div>

<p class="text-slate-300 text-sm mt-4">Вывод: если у нас есть матрица, то для вычисления суммы по строке и столбцу – перекрывающиеся блоки посчитают. Если же у нас есть только строка и столбец – у минимума нет обратной операции. Стало очевидно, что для вычисления суммы по строке и столбцу нужно использовать матрицу.</p>

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500/50">

<h3 class="text-xl font-bold text-amber-400">Проблема с матрицей</h3>

<div class="bg-rose-950/30 p-5 rounded-lg border-rose-500/50">

<ul class="list-disc pl-5 space-y-2 text-slate-300">

Сумма через разреженную таблицу – это неэффективно. Если у нас есть матрица размером $N \times M$, то для вычисления суммы по строке и столбцу нужно использовать матрицу. (билет 5).

Память. $N = M = 1000$ даёт матрицу размером 10^6 элементов. Если у нас есть матрица размером $N \times M$, то для вычисления суммы по строке и столбцу нужно использовать матрицу. считайте ячейки, потом пишите код.

Один log вместо двух. В матрице для вычисления суммы по строке и столбцу нужно использовать матрицу. $\log N \times \log M$ на каждой ячейке.

Выход за границу в запросе. Если у нас есть матрица размером $N \times M$, то для вычисления суммы по строке и столбцу нужно использовать матрицу. 1 и st[0][0].

Нужны обновления. Разреженная матрица – это неэффективно. Если у нас есть матрица размером $N \times M$, то для вычисления суммы по строке и столбцу нужно использовать матрицу. (билет 1), а не она.


```

        else
            st[r][c][kx][ky] = min(
    
```

```

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 &lt;&lt; ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 &lt;&lt; kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);
    
```

Четыре запроса к четырем углам запроса и вместе с ним образуют минимальную сумму.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

Почему это так много? Потому что для каждого запроса нужно вычислить сумму по всем уровням, а запросов очень много.

Вторая половина этого сравнения:

```
<div class="text-center my-6">
  <p class="text-slate-400 text-xs uppercase">
    <p class="my-1 font-bold leading-none">
      <span class="text-2xl text-slate-300">Й</span>
    </p>
  <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Split – таможня
    </div>
    <p class="text-slate-300 text-sm">S
х». Спуск от корня: узел, который вместе с
скрытия</b> – досматривается только одна ве
  </div>
```

```
    <div class="bg-slate-900 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
        Аналогия 2: Merge – стыковка
      </div>
      <p class="text-slate-300 text-sm">M
аг – одно сравнение: чей у у корня больше,
сохраняет порядок x). Итого  $O(h)$  сравнений
    </div>
  </div>
```

```
<div class="bg-slate-900/40 rounded-lg bord
  <p class="text-slate-300 text-sm mb-2">
  <p class="text-slate-400 text-sm mb-1">
быстрее  $n \cdot \log n$ .</p>
  <p class="text-slate-400 text-sm mb-1">
  <p class="text-slate-400 text-sm">• «У6
```



```

<summary class="p-4 font-bold text-slate-700 border-slate-700/50">
    Код: вставка, merge, erase (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300" style="background-color: #f1f3f4; border-radius: 10px; padding: 10px;">
<pre class="bg-slate-950 p-4 rounded" style="background-color: #1e2428; color: #cddce3; border-radius: 10px; padding: 10px;">
def insert(root, key, pri):
    if root is None:
        return Node(key, pri)
    if key <= root.key: # равные x — в левое
        root.left = insert(root.left, key, pri)
    else:
        root.right = insert(root.right, key, pri)
    return root

for key, pri in pairs_sorted_by_pri_desc: # порядок э
    root = insert(root, key, pri)</pre>
<pre class="bg-slate-950 p-4 rounded" style="background-color: #1e2428; color: #cddce3; border-radius: 10px; padding: 10px;">
def merge(a, b): # все x в a < все x в b
    if not a or not b: return a or b
    if a.pri > b.pri:
        a.right = merge(a.right, b); return a
    b.left = merge(a, b.left); return b

def erase(root, key): # удалить произвольный элемент
    if key < root.key: root.left = erase(root.left, key)
    elif key > root.key: root.right = erase(root.right, key)
    else: root = merge(root.left, root.right)
    return root</pre>
</div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "splay-tree",
        title: "4. Splay-дерево",
    },

```

по факту

: ничего не хранит, дереву разрё

```
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Стопка бумаг

```
</div>
<p class="text-slate-300 text-sm">С
тировку не ведёшь, но через неделю самое ну
ерх».</p>
```

```
</div>
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Аналогия 2: Тропинка через га.

```
</div>
<p class="text-slate-300 text-sm">И,
рно вдвое. Первый проход дорогой, зато опла
ы.</p>
```

```
</div>
</div>
```

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Повор
Среднее поддерево  $\beta$  (между  $x$  и  $p$ ) – единств
 $x$ , и под  $p$ .</p>
```

```
<pre class="bg-slate-950 p-4 rounded-lg ove
ading-relaxed">
```

```

      /  \ \
     x    y
    /  \ \
   α    β
      /  \ \
     x    y
    /  \ \
   α    β

      x выше
      -----> α
      /  \ \
     p    r
    /  \ \
   β    γ

      p выше
      <-----
      /  \ \
     p    r
    /  \ \
   β    γ
  </pre>
```

```
<p class="text-slate-300 text-sm mt-3">Обхо
омать дерево поиска поворотами невозможно в
омент «отстёжки», а врезка «Механика одного
```

```
<h4 class="text-lg font-bold text-white mt-3">
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
    амбук из 8 узлов 8→7→6→5→4→3→2→1, ищем 1 (ч
наивно (x, потом x):    3 × «два зига» + Zig → высота 8
Zig-Zig (p, потом x):    3 × Zig-Zig    + Zig → высота 8
бамбук из 16 узлов:      наивно 16 → 16, Zig-Zig 16 → 10<
```

```
<p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mo
изации не будет.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-6">Полн
м 1. Спуск 8 узлов, затем <b>три Zig-Zig по
8 до 6 – линия перестала быть линией. Наивно
о перевёрнутой. Важно не врать себе: на кор
ван не на одном запросе, а в сумме. Следующ
по пути</b>. Отсюда амортизация: дорогой за
дом потенциала: потенциал  $\Phi$  = сумма по узла
ь его; отсюда же теорема о статической опти
ь запросов).</p>
```

```
<div class="bg-slate-900/40 rounded-lg bord
    <p class="text-slate-300 text-sm mb-2">
    <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
    <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
    <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
ое <code class="bg-slate-950 px-2 py-0.5 ro
одиночная операция – до <code class="bg-sla
>O(n)</code>; n вставок подряд – амортизиро
-300 border border-slate-800">O(n log n)</c
```

```
<p class="text-slate-300 text-sm mt-4">Все
```

```

        rotate_up(x)
    else:                                     # ZIG-ZAG: НИЖНЯЯ
        rotate_up(x)
        rotate_up(x)
    return x

def find(root, key):                         # поиск: спуск + s
    cur, last = root, None
    while cur:
        last = cur
        if key == cur.key: break
        cur = cur.left if key < cur.key else cur.right
    return splay(last) if last else None</pre>
    <p class="text-xs text-slate-400">В
        Одна строка – и вся амортизация.</p>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "prefix-sums-2d",
        title: "5. Двумерная матрица префиксных сумм",
        type: "html",
        description: "Сумма подматрицы за O(1): 1D и 2D, дв
        category: "Алгоритмы матрицы",
        content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-v
    </div>

    <div class="space-y-8">

        <div class="bg-slate-700/50 p-6 rounded-xl bord
            <h3 class="text-xl font-bold text-emerald-4

```

```

        <td class="py-2 pr-3 font-mono">
      </td>
    </tr>
  </tbody>
</table>
<p class="text-slate-400 text-xs mt-2">
  ышли за границу» – поэтому в 2D почти всегда
</div>

<div class="grid grid-cols-1 sm:grid-cols-3">
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
    <p class="text-[11px] text-slate-500
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
    <p class="text-[11px] text-slate-500
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-rose-300 t
    <p class="text-[11px] text-slate-500
  </div>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-blue-400 m
  <div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800
      <div class="absolute -top-3 left-4 b
      rder border-emerald-500 shadow-md">
        Аналогия 1: Сетка из вафельных
      </div>
      <p class="text-slate-300 text-sm mb
    о накопилось от угла (0,0) до меня». Хочешь
    у и верни дважды отрезанный угол. Вычитание

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
    Тонкая: одна матрица
  </div>
  <p class="text-slate-300 text-sm mb-2">
    одной матрицы (плюс рамка из нулей). Никаких вычитаний, только сложение.
    прямоугольника от угла. Всё остальное получается.
  </div>
  <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
    <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
      Не умеет: всё, что нельзя вычислить.
    </div>
    <p class="text-slate-300 text-sm mb-2">
      максимума или gcd вычитания не существует:
      е запросы – это <a class="ticket-link" data-ticket="1234567890" href="#">
      платит она за это памятью <span class="font-mono">
    </div>
  </div>

<div class="bg-slate-800/70 p-5 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <p class="text-sm font-bold text-white">
  <table class="w-full text-xs text-left" style="width: 100%; border-collapse: collapse;">
    <thead>
      <tr class="text-slate-500 border-b">
        <th class="py-1.5 pr-2 font-normal">
        <th class="py-1.5 pr-2 font-normal">
        <th class="py-1.5 pr-2 font-normal">
      </tr>
    </thead>
    <tbody>
      <tr class="border-b border-slate-700">
        <td class="py-2 pr-3 font-normal">
        <td class="py-2 pr-3 font-normal">
        <td class="py-2 pr-3 font-normal">
      </tr>
      <tr class="border-b border-slate-700">

```

```

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
    Код: 1D (тот же, что выполняет пане
  </summary>
  <div class="p-5 text-sm text-slate-300
    <pre class="bg-slate-950 p-4 rounde
      = [3, 1, 4, 1, 5, 9, 2, 6]    # исходный ма
n = len(a)                          # префиксов будет n + 1
P = [0]                             # P[0] = 0 – пустой преф
for i in range(1, n + 1):
    P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)</code></pre>
    <p class="text-slate-300 text-sm">П
- часть демонстрации: лента подсвечивает, из
  </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
    Код: 2D – построение и запрос прямо
  </summary>
  <div class="p-5 text-sm text-slate-300
    <pre class="bg-slate-950 p-4 rounde
      = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4

S = []                               # рамка из нуле
for rr in range(n + 1):
    S.append([0] + [0] * m)
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] -

r1, c1, r2, c2 = 1, 1, 2, 2          # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1]
print("итого:", ans)</code></pre>

```

```

    },
    {
      id: "graph-planar-colors",
      title: "7. Графы. Планарные. Покраска",
      type: "html",
      description: "Формула Эйлера и теорема о 4 красках.",
      category: "Графы. Теория",
      content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">7
    <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Графы. Планарные. Покраска
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
        <p class="text-xl font-mono text-white">
          1. Если граф планарный, то он можно вложить в плоскость так, чтобы никакие две
          ребра не пересекались.
          2. Если граф можно вложить в плоскость, то он планарный.
          3. Если граф можно раскрасить 4 цветами, то он планарный.
          4. Если граф планарный, то он можно раскрасить 4 цветами.
        </p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Карта мира
          </div>
          <p class="text-slate-300 text-sm mb-4">
            . Границы не пересекаются в одной точке по-
            ы не сливались цветами, Всегда можно всего 4
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`,
    },
    {
      id: "top-sort",

```



```

    </div>
</section>`,
  },
  {
    id: "scc-kosaraju",
    title: "10. Графы. Компоненты сильной связности (SCC)",
    type: "html",
    description: "Разбиение орграфа на макро-вершины. Алгоритм Косарaju",
    category: "Графы. Продвинутое",
    content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">10.1. Компоненты сильной связности (SCC)
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Сильная связность графа — это свойство, при котором для любых двух вершин  $u$  и  $v$  графа существует путь из  $u$  в  $v$  и обратно.
        <p class="text-xl font-mono text-white">Граф разбивается на компоненты сильной связности (SCC), которые являются максимally сильносвязными подграфами.
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Улицы с односторонним движением
          </div>
          <p class="text-slate-300 text-sm mb-4">Представьте себе карту города, где улицы имеют одностороннее движение. Если вы можете добраться из любой точки города в любую другую, двигаясь только по односторонним улицам, то город является сильно связным.
        </div>
        <div class="bg-slate-800 p-6 rounded-lg">
          <p class="text-slate-300 text-sm mb-4">В графе это означает, что для любых двух вершин  $u$  и  $v$  существует путь из  $u$  в  $v$  и обратно.
        </div>
      </div>
    </div>
  </div>
</section>`,
  },

```

```

        description: "Вершины, удаление которых убивает свя
        category: "Графы. Продвинутые",
        content: `
<section id="graph-articulation" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-rose-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">
        <p class="text-xl font-mono text-white">
          теи > 1).</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            Аналогия 1: Центральный роутер
          </div>
          <p class="text-slate-300 text-sm mb-4">
            правильный роутер - и два сегмента офиса б
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-1"
      <p class="text-lg text-blue-300 italic"
      <p class="text-xl font-mono text-white"
d-длина.</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия: Копаем туннель под .
        </div>
        <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "mst-kruskal",
    title: "18. Графы. Остовное дерево. Краскал",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-emerald-4

```

```

        которое торчит из посещенных в непосещенных
    </div>
    <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                ✦ Аналогия: Заражение вирусом
            </div>
            <p class="text-slate-300 text-sm mb-2">
                ли вирус). Мы стартуем из одного города, и
                Сеть растет только из одного связного куска
            </p>
        </div>
    </div>
</div>
</section>`,
    },
    {
        id: "mst-boruvka",
        title: "20. Графы. Остовное дерево. Борувка",
        type: "html",
        description: "",
        category: "Графы. Остовные",
        content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
                </div>
            <div class="space-y-8">
                <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
                    <h3 class="text-xl font-bold text-emerald-400">
                        <div class="bg-slate-900 p-4 rounded-lg mb-4">
                            <p class="text-lg text-emerald-300 italic">
                                <p class="text-xl font-mono text-white">
                                    ается с соседом.</p>
                                </div>
                            <div class="grid grid-cols-1 gap-6">

```

```

        rder border-emerald-500 shadow-md">
            Аналогия 1: Бумеранг (Префикс
        </div>
        <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
меня перепроверок!</p>
    </div>
</div>
<details class="bg-slate-900/40 rounded-lg
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "string-z-func",
        title: "22. Z-функция",
        type: "html",
        description: "",
        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

    </div>
</section>`,
  },
  {
    id: "complexity-classes",
    title: "24. Классы сложности, сведение задач",
    type: "html",
    description: "",
    category: "Теория сложности",
    content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1 rounded">24</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">Классы сложности</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-purple-400">Сведение задач</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">Пример: сортировка чисел</p>
        <p class="text-base sm:text-lg font-mono">
          <b>P</b>: класс decision-задач, решаемый за полиномиальное время.
          <b>NP</b>: класс decision-задач с сертификатом.
        </p>
        <p>).<br>
          <b>Сведение (Reduction) A ≤<sub>p</sub> B</b>: задача A сводится к задаче B.
        </p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Судоку (P vs NP)
          </div>
          <p class="text-slate-300 text-sm mb-4">Пример сортировки чисел. Класс <b>NP</b> – это класс задач, для которых существует быструю сетку (ответ), он БЫСТРО (за класс P) решается.
        </div>
      </div>
    </div>
  </div>
</section>`
  }
]

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400"

  <div class="bg-slate-900 p-4 rounded-lg mb-4"
    <p class="text-lg text-blue-300 italic"
    <div class="text-left font-mono text-sm"
      <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
      <p><strong class="text-white">Эйлер
      <div class="p-3 bg-slate-950 rounded"
        <p class="text-sm text-blue-300"
        <p><strong class="text-white">П
        <p><strong class="text-white">Ц
      </div>
    </div>
  </div>
</div>

<p class="text-slate-300 text-sm">
  <strong>Билет 13 (Различие пути и цикла
  <br>• <span class="text-green-400 font-
шел-вошёл-вышел).
  <br>• <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap"
  <div class="bg-slate-800 p-6 rounded-lg bor"
    <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
      Аналогия 1: Путь – из дома в бар,
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Ты вышел из дома <strong>(нечётная)
      талый пришёл в бар <strong>(вторая нечётная
      Домой вернуться не смог, потому что
    </p>

```

```

    </div>
</section>`,
  },
  {
    id: "bridges-code",
    title: "11. Графы. Мосты",
    type: "html",
    description: "Мосты через DFS Тарьяна: tin/low, усл
    category: "Графы. Продвинутые",
    content: `<section id="bridges-code" class="mb-20 s
    <div class="flex items-center mb-6">
      <span class="bg-emerald-600 text-white px-4 py-
      <h2 class="text-3xl font-bold text-white">Мосты
    </div>

    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl bord
        <h3 class="text-xl font-bold text-emerald-400">
          Мосты через DFS Тарьяна: tin/low, усл

        <div class="bg-slate-900 p-4 rounded-lg mb-4">
          <p class="text-lg text-emerald-300 ital">
            <div class="text-left font-mono text-sm">
              <p><strong class="text-white">tin[v
              <p><strong class="text-white">low[v
              <div class="p-3 bg-slate-950 rounde
                <span class="text-rose-400 font
                <p class="text-xl font-bold tex
                <p class="text-xs text-slate-400
              </div>
            </div>
          </div>
        </div>
      </div>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-slate-300 text-sm">
          Если из сына u и всех его потомков <strong>
          Единственная ниточка. Порвётся – граф р
        </p>
      </div>
    </div>
  }
}

```



```
-----
<pre class="text-xs font-mono text-
#include <vector>;
#include <algorithm>;

using namespace std;

int n;                // число вершин
vector<vector<int>>> adj; // список смежност
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;           // устанавливаем

    for (int to : adj[v]) {
        if (to == p) continue;          // не идём назад

        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v);                   // рекурсивный в
            low[v] = min(low[v], low[to]);

            if (low[to] > tin[v]) {
                // ЭТО МОСТ!
                // bridge_list.push_back({v, to});
            }
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
}
```

```

        <p><span class="text-emerald-400 font-
        <p><span class="text-rose-400 font-l
    </div>
</div>

<p class="text-slate-300 text-sm">
    <strong>Следствие:</strong> Для планарн
ode>.
    Если ребер больше – граф гарантированно
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
    <div class="bg-slate-800 p-6 rounded-lg bor
        <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
            K5: Полный граф на 5 вершинах
        </div>
        <p class="text-slate-300 text-sm mb-4">
            У K5: <code class="text-white">V=5,
-white">10 &gt; 9</code> – БАХ, непланарен!
            Все 5 вершин соединены со всеми. На
            Теорема Куратовского: K5 и K3,3 – д
            Если внутри графа спрятано <b>подра
            – плоской раскладки не будет.
        </p>
    </div>

<div class="bg-slate-900 p-6 rounded-lg bor
    <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
        K3,3: 3 дома и 3 колодца
    </div>
    <p class="text-slate-300 text-sm mb-4">
        Три дома хотят соединиться с тремя
        Нарисовать без пересечений невозможно
        У него <code class="text-white">V=6
xt-white">9 ≤ 12</code> – проходит, но он в
```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Расписание экзамен

```
</div>
```

```
<p class="text-slate-300 text-sm">В
слот времени. Раскрасить граф = составить р
«берём предметы в каком-то порядке и ставим
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
border border-fuchsia-500 shadow-md">
```

Аналогия 2: Частоты вышек

```
</div>
```

```
<p class="text-slate-300 text-sm">С
цвет, интерференция = ребро. Диапазон частот
регистров в компиляторе: «живые» одновременно
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left"
  <thead>
```

```
<tr class="text-slate-500 border-b"
  <th class="py-2 pr-3 font-b"
  <th class="py-2 pr-3 font-b"
  <th class="py-2 font-bold">
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```

```
<tr class="border-b border-slate-800"
><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta + 1$ <
а свободен</td></tr>
```

```
<tr class="border-b border-slate-800"
d><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$ </td>
/td></tr>
```

```

        веркой цветов соседей; проверка двудольности
border border-slate-800">0(V + E)</code>; т
-300 border border-slate-800">NP-трудно</co

<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код: жадная раскраска и проверка
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounded
я раскраска: не больше  $\Delta + 1$  цвета
color = [-1] * n                # -1 = ещё не покрашен
for v in order:                 # порядок влияет на чи
    used = set()
    for to in graph[v]:
        if color[to] != -1:
            used.add(color[to]) # цвета уже покрашены
    c = 0
    while c in used:             # минимальный свободны
        c += 1
    color[v] = c
chi = max(color) + 1            #  $\chi$ , который выдал жад

# Проверка двудольности = раскраска в 2 цвета (BFS)
from collections import deque
col = [-1] * n
bipartite = True
for start in range(n):
    if col[start] != -1:
        continue
    col[start] = 0
    dq = deque([start])
    while dq:
        v = dq.popleft()
        for to in graph[v]:
            if col[to] == -1:
                col[to] = col[v] ^ 1 # второй цвет

```

```
<p class="text-slate-300 text-sm">
  <strong>Важно:</strong> Это работает то
  епятся к точкам сочленения. То есть, <strong>
  ег - это тупик со степенью 1).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
    <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
      Аналогия: Главный перекресток
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь город, где два района соединены
      её перекроют (удалят вершину) — районы будут
      <strong>хрупкость</strong> системы.
    </p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
    <div class="absolute -top-3 left-4 bg-slate-900 border border-emerald-500 shadow-md">
      Телеком: Центральный свитч
    </div>
    <p class="text-slate-300 text-sm mb-4">
      У тебя несколько компьютеров в одной
      абелем (мостом). Сами свитчи — это точки со
    </p>
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-800">
  <summary class="p-4 font-bold text-slate-400">
    Душный мод: Код и корень DFS (Скрыто)
  </summary>
```

```

        <ul class="list-disc list-inside text-sm text-rose-400">
            <li><strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
            <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
            <li><strong>Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
        </ul>
    </div>
</div>
</section>`,
    },
];

```

ФАЙЛ 19/111: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 145 | РАЗМ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { ticket1Segtree } from "../tickets/ticket1_segtr
import { tickets18to20Mst } from "../tickets/ticket18_20
import { chapters as newChapters } from "../content_temp

/**
 * Содержание пособия = билеты 1–24 из списка тем.
 *
 * Порядок массивов важен: при одинаковом id побеждает
 * версия главы. `newChapters` (content temp) перетирае

```

```
merged.forEach((c) => {
  if (!toRemove.includes(c.id)) {
    dedup.set(c.id, c);
  }
});

// Самые свежие версии страниц перетирают старые.
if (newChapters) {
  newChapters.forEach((c) => {
    if (!toRemove.includes(c.id)) dedup.set(c.id, c);
  });
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
  // Главы без номера билета (введение) стоят ДО билетов
  const numOf = (t: string) => {
    const m = t.match(/(\d+)/);
    return m ? parseInt(m[0], 10) : -1;
  };
  return numOf(a.title) - numOf(b.title);
});

export const chapters: Chapter[] = finalChapters;

/**
 * Билеты, закрытые каждой страницей (для шапки, экспор
 * Страница `mst`, например, закрывает сразу 18, 19 и 2
 */
export const chapterTopics: Record<string, number[]> =
  "segment-trees": [1],
  "sparse-table": [2],
  treap: [3],
  "splay-tree": [4],
  "prefix-sums-2d": [5],
  "graph-dfs-bfs": [6],
  "planarity-euler-formula": [7],
```

```

-----
const tickets = chapterTopics[chapterId] ?? [];
// Глав без билетов (введение) относим к первому разд
if (!tickets.length) return SECTIONS[0];
const first = Math.min(...tickets);
return SECTIONS.find((sec) => first >= sec.from && fi
}

export const activeVizIds = [
  "euler-path-vs-cycle", "bridges-code", "planarity-euler
  "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
  "dijkstra", "bellman-ford", "floyd", "aho-corasick",
  "mst", "string-kmp", "string-z-func", "segment-trees", "s
  "dynamic-programming", "johnson-algo", "graph-articulat
  "complexity-classes", "treap", "splay-tree", "prefix-sum
];

```

ФАЙЛ 20/111: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 207 | РАЗМ

```

-----
/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

```

```

export type DebugValue = null | boolean | number | string

```

```

export interface DebugStep {
  /** Номер строки в коде пользователя (1-based). */
  line: number;
  /** Имя функции-фрейма ("<module>" – верхний уровень)
  func: string;
  /** Локальные переменные: имя → герг (усечённый), для
  locals: Record<string, string>;
  /**

```



```

__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
    pass

def __safe(v):
    try:
        r = repr(v)
    except Exception:
        r = "<...>"
    return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
    """Небольшой JSON-снимок: UI использует его без парсера"""
    if v is None or isinstance(v, (bool, int, str)):
        return v
    if isinstance(v, float):
        return v if v == v and v not in (float("inf"), float("-inf")) else None
    if depth >= 4:
        return __safe(v)
    limit = 80 if depth < 2 else 32
    if isinstance(v, (list, tuple)):
        return [__snapshot(x, depth + 1) for x in list(v)]
    if isinstance(v, (set, frozenset)):
        return [__snapshot(x, depth + 1) for x in list(v)]
    if isinstance(v, dict):
        # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
        # предел 80 незаметно обрезал уровни 4x4 и 8x8
        dict_limit = 192 if depth == 0 else limit
        return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())}
    # deque, heap-подобные и другие итерируемые учебные
    if type(v).__name__ == "deque":
        return [__snapshot(x, depth + 1) for x in list(v)]
    return __safe(v)

```

```

-----
    # PEP 709: с 3.12 компрекеншны встроены во фрейм
    # на своей строке каждую итерацию. Склеиваем по
    if __steps and __steps[-1]["line"] == frame.f_lineno:
        __steps[-1]["locals"] = loc
        __steps[-1]["values"] = values
    else:
        __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
    if len(__steps) >= __CAP:
        # Нельзя просто отключить trace: бесконечный цикл
        # тогда продолжит работать вечно. Мягко прерываем
        __was_truncated = True
        raise __TraceLimit()
    # Финальные локалы программы (после последней строки)
    if event == "return" and frame.f_code.co_filename == __FILE__:
        loc, values = __capture(frame)
        __final.update(loc)
        __final_values.update(values)
    return __tracer

__err = ""
try:
    __sys.settrace(__tracer)
    exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
    pass
except Exception:
    import traceback as __tb
    __err = __tb.format_exc(limit=3)
finally:
    __sys.settrace(None)

__OUT = __json.dumps({
    "steps": __steps,
    "finalLocals": __final,
    "finalValues": __final_values,
    "error": __err,
    "truncated": __was_truncated,
})

```

ФАЙЛ 21/111: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 639 | РАЗМ:

```
import type { DemoKind } from "../components/hints/MiniHint";
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мобильном)
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
  id: string;
```

```
  /** Варианты написания в тексте (регистр не важен). */
```

```
  labels: string[];
```

```
  title: string;
```

```
  /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
  short: string;
```

```
  /** Строгая формулировка / что важно не забыть. */
```

```
  formal?: string;
```

```
  complexity?: string;
```

```
  /** Какая мини-анимация показывается в карточке. */
```

```
  demo?: DemoKind;
```

```
  /** id полного симулятора (см. vizRegistry) – кнопка
```

```
  simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
  {
```

```
    id: "bfs",
```

```
    labels: ["BFS", "обход в ширину", "поиск в ширину"],
```

```
    title: "BFS – обход в ширину",
```

```
    short:
```

"Не отсортированный список, а «мешок, из которого
formal:

"Полное бинарное дерево в массиве: дети узла i ле

complexity: "вставка/извлечение $O(\log n)$, «посмотре

demo: "heap",

simulator: "heap-beam-search",

},

{

id: "stack",

labels: ["стек", "стека", "стеке", "стеком", "LIFO"]

title: "Стек (LIFO)",

short: "Стопка тарелок: кладёшь сверху и берёшь свер

formal: "push / pop / top за $O(1)$. На стеке живёт р

demo: "stack",

simulator: "stack-dfs",

},

{

id: "queue",

labels: ["очередь", "очереди", "очередью", "FIFO"],

title: "Очередь (FIFO)",

short: "Очередь в столовой: кто первым встал, тот п

formal: "push_back / pop_front за $O(1)$. Основа BFS

demo: "queue",

simulator: "queue-bfs",

},

{

id: "dsu",

labels: ["DSU", "СНМ", "система непересекающихся мн

title: "DSU — система непересекающихся множеств",

short:

"«Кто твой староста?» Каждый элемент знает своего

formal:

"find со сжатием путей + union по рангу/размеру. I

complexity: "почти $O(1)$ — обратная функция Аккерман

demo: "dsu",

simulator: "mst",

},

{

```

    short:
        "«Куда откатиться, если следующая буква не подошла",
        строки.",
    formal: "link(v) = go(link(parent(v)), ch). Считает",
    demo: "suffix-link",
    simulator: "aho-corasick",
},
{
    id: "toposort",
    labels: ["топологическая сортировка", "топсорт", "топ",
    title: "Топологическая сортировка",
    short:
        "Порядок «сначала штаны, потом ботинки»: каждая с",
        циклов.",
    formal: "DFS + вершины в обратном порядке выхода, л",
    complexity: "O(V + E)",
    demo: "toposort",
    simulator: "top-sort",
},
{
    id: "relaxation",
    labels: ["релаксация", "релаксацию", "релаксации",
    title: "Релаксация ребра",
    short:
        "«А через соседа не быстрее?» Если известный путь",
        ,
    formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
        ан и Флойд.",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "prefix-sum",
    labels: ["префиксные суммы", "префиксная сумма", "пре",
    title: "Префиксные суммы",
    short:
        "Заранее посчитанные «сколько всего набежало от н",
    formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r] - P[l-1]"

```

```

    complexity: "O(V + E) одним DFS",
    demo: "bridge",
    simulator: "bridges-code",
},
{
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "то"],
    title: "Точка сочленения",
    short:
        "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
    formal: "Для не-корня: v — точка сочленения  Э ребро",
    demo: "articulation",
    simulator: "graph-articulation",
},
{
    id: "mst",
    labels: [
        "остовное дерево", "остовного дерева", "остовное",
        "минимальное остовное дерево", "MST",
    ],
    title: "Минимальное остовное дерево (MST)",
    short:
        "Связать все точки проводами так, чтобы суммарная",
    formal: "V-1 ребро, никаких циклов. Краскал (жадно и",
        "но).",
    demo: "mst",
    simulator: "mst",
},
{
    id: "amortized",
    labels: [
        "амортизированная", "амортизированный", "амортизир",
        "амортизированное", "амортизированного", "амортизир",
    ],
    title: "Амортизированная сложность",
    short:
        "Средняя цена операции «по абонементу»: одна опер",
    formal: "Метод потенциалов: дорогая операция оплачи
```

```

{
  id: "prefix-function",
  labels: [
    "префикс-функция", "префикс-функции", "префикс-фу",
    "префикс", "префикса", "суффиксом", "суффикс", "с",
    "подстроки", "подстроку", "подстрока",
  ],
  title: "Префикс-функция  $\pi$  (КМП)",
  short:
    "Для каждой позиции запоминаем: какой кусок начался",
    "от кусок.",
  formal:
    " $\pi[i]$  — длина наибольшего собственного префикса  $s$ ",
    "возвращая указатель по тексту.",
  complexity: " $O(n + m)$ ",
  demo: "prefix-function",
  simulator: "string-kmp",
},
{
  id: "z-function",
  labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
  title: "Z-функция",
  short:
    "Для каждой позиции: сколько символов подряд, начи",
    "аждой букве.",
  formal:
    " $z[i]$  = длина наибольшего общего префикса  $s$  и  $s[i$ ",
  complexity: " $O(n)$ ",
  demo: "prefix-function",
  simulator: "string-z-func",
},
{
  id: "dp",
  labels: [
    "динамическое программирование", "динамики", "дин",
    "мемоизация", "мемоизацию", "подзадач", "подзадач",
  ],
  title: "Динамическое программирование",

```

```

    short:
        "На каждом шаге хватаем то, что лучше прямо сейчас"
    formal:
        "Корректность обычно доказывается «леммой о разрезе»"
    demo: "mst",
    simulator: "mst",
},
{
    id: "euler",
    labels: ["эйлеров цикл", "эйлеров путь", "эйлерова теорема"],
    title: "Эйлеров путь и цикл",
    short:
        "Пройти по каждому ребру ровно один раз, не отрываясь"
    formal:
        "Связный граф: эйлеров цикл    все степени чётные; эйлеров путь    все степени нечётные"
    simulator: "euler-path-vs-cycle",
},
{
    id: "planarity",
    labels: ["планарный", "планарность", "планарного", "планаризация"],
    title: "Планарность",
    short:
        "Граф планарен, если его можно нарисовать на листе бумаги"
    formal:
        "V - E + F = 2 для связного планарного графа. Курсивом выделены вершины, рёбра и области"
    simulator: "planarity-euler-formula",
},
{
    id: "splay",
    labels: [
        "splay", "splay-дерево", "AVL", "AVL-дерево", "всесторонний поиск"
    ],
    title: "Splay и повороты",
    short:
        "Каждый раз, когда к узлу обратились, его «вытягивают» к корню"
    formal:

```



```

        "После k-й итерации гарантированно верны все кратчайшие пути, которые не содержат отрицательный цикл.",
        complexity: "O(V · E)",
        demo: "relax",
        simulator: "bellman-ford",
    },
    {
        id: "floyd",
        labels: [
            "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
            "промежуточная вершина", "промежуточную вершину",
        ],
        title: "Флойд-Уоршелл: разрешаем вершину k",
        short:
            "Внешний цикл — это не «шаг», а «какие пересадки сделать, чтобы пересадку в k?",
        formal:
            "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядок пересадки выбирается среди всех вершин.",
        complexity: "O(V³) времени, O(V²) памяти",
        demo: "floyd",
        simulator: "floyd",
    },
    {
        id: "prim",
        labels: ["Прима", "Прим", "Prim"],
        title: "Алгоритм Прима",
        short:
            "Остов растёт как плесень из одной точки: на каждом шаге добавляется ребро, которое не создаст цикла.",
        formal:
            "Множество посещённых + куча рёбер на границе. Добавляем ребро с минимальным весом, которое не создаст цикла.",
        complexity: "O(E log V)",
        demo: "mst",
        simulator: "mst",
    },
    {
        id: "boruvka",

```

```
{
  id: "graph",
  labels: [
    "граф", "графа", "графе", "графы", "графов", "граф",
    "вершины", "вершина", "вершин", "ребра с весом",
  ],
  title: "Граф",
  short:
    "Точки (вершины) и связи между ними (рёбра). Города и дороги.",
  formal:
    " $G = (V, E)$ . Ребро бывает ориентированным (улицы с односторонним движением) и неориентированным (двусторонняя улица).",
  demo: "graph",
  simulator: "graph-dfs-bfs",
},
{
  id: "coord-compress",
  labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
  title: "Сжатие координат",
  short:
    "Координаты бывают гигантские и дробные, а нам нужно хранить только порядковый номер.",
  formal:
    "sort + unique + binary search: значение → индекс",
  complexity: " $O(n \log n)$ ",
  demo: "coord-compress",
},
{
  id: "treap",
  labels: ["Treap", "декартово дерево", "декартова дерево"],
  title: "Treap = Tree + Heap",
  short:
    "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
  formal:
    "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить через них. Ожидаемая высота  $O(\log n)$ ).",
  complexity: "ожидаемо  $O(\log n)$ ",
}
```

```

    demo: "np",
  },
  {
    id: "range",
    labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
    title: "Запрос на отрезке",
    short:
      "Спрашиваем не про один элемент, а про кусок массива  

      предподсчёты.",
    formal:
      "Суммы — префиксные суммы  $O(1)$ . Минимум без изменений.",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
  },
  {
    id: "string",
    labels: ["строки", "строке", "строку", "строка", "строкам"],
    title: "Строка как массив символов",
    short:
      "Строковые алгоритмы держатся на трёх вопросах: где  

      (Z-функция) и где встречаются слова словаря (бордер-лист).",
    formal:
      "Наивный поиск подстроки —  $O(N \cdot M)$ . Префикс-функция —  $O(N)$ .",
    demo: "prefix-function",
    simulator: "string-kmp",
  },
  {
    id: "rotation",
    labels: ["поворот", "повороты", "поворота", "поворотом"],
    title: "Поворот (rotation) в дереве",
    short:
      "Локальная перевеска двух узлов: ребёнок встанет на  

      дившемся месту. Порядок ключей при этом не ломается.",
    formal:
      "Правый поворот вокруг p:  $x = p.left; p.left = x$ .  

      g и Zig-Zag.",
    complexity: "O(1)",
    demo: "zig",
  },

```

Универсальное пособие • полный исходный код

```
-----  
/** Быстрый доступ по id. */  
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {  
  
/** Все метки, отсортированные от длинных к коротким (ч  
export const GLOSSARY_LABELS: { label: string; id: string }[] =  
  e.labels.map((label) => ({ label, id: e.id })))  
).sort((a, b) => b.label.length - a.label.length);  
  
-----
```

ФАЙЛ 22/111: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРӨК: 411 | РАЗМЕР: 1.2 КБ

```
-----  
import { Chapter } from "../types";  
  
export const graphChapters: Chapter[] = [  
  {  
    id: "intro",  
    title: "Введение. Базовая база: куча, очередь, бинарное  
    type: "html",  
    category: "Введение",  
    content: `  
      <section id="intro-content" class="mb-12">  
        <div class="flex items-center mb-6 flex-wrap gap-2">  
          <span class="bg-indigo-600 text-white px-4 py-1">Введение</span>  
          <h2 class="text-3xl font-bold text-white">Базовая база</h2>  
        </div>  
  
        <div class="bg-slate-900 border border-indigo-500/40 p-6">  
          <p class="text-slate-200 text-sm leading-relaxed">  
            <b class="text-indigo-300">Базовая база</b> – это  
            дерево как массив, куча, очередь и стек, бинарное  
            Кодим по минимуму: там, где за нас уже написали.  
          </p>  
        </div>  
  
        <div class="space-y-8">
```

```
h = []
heapq.heappush(h, 5); heapq.heappush(h, 1); heapq.heappop(h)
h[0] # 1 – минимум виден без всякого обхода
heapq.heappop(h) # 1, куча починилась за  $O(\log n)$ 
```

<p class="text-slate-300 text-sm">Куча – движок, который поддерживает кучу, раз, и список с ...</p></div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4"><h3 class="text-xl font-bold text-sky-400 mb-4">Очередь</h3><p class="text-slate-300 text-sm mb-4">Очередь (BFS) отличается от стека (DFS) (первый вышел) отличаются одной операцией снятия элемента: оба конца работают за $O(1)$. Обычный список сдвигает весь хвост и стоит $O(n)$.</p><pre class="bg-slate-950 border border-slate-700">

```
from collections import deque
q = deque([1, 2])
q.append(3); q.popleft() # 1 – очередь (BFS)
s = [1, 2]; s.append(3); s.pop() # 3 – стек (DFS)</pre><p class="text-slate-300 text-sm">Вся разница между BFS (волнами), стек – DFS (вглубь).</p></div>
```

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4"><h3 class="text-xl font-bold text-amber-400 mb-4">Сортировка</h3><p class="text-slate-300 text-sm mb-4">Если массив отсортирован – понял, в какой половине жить, – и это 20 сравнений. В Python за это отвечает модуль bisect.</p><pre class="bg-slate-950 border border-slate-700">

```
from bisect import bisect_left
a = [10, 20, 30, 40, 50]
bisect_left(a, 30) # 2 – индекс первого элемента, не меньше 30
<p class="text-slate-300 text-sm">Без сортировки</p></div>
```

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4"><h3 class="text-xl font-bold text-rose-400 mb-4">Сортировка</h3><p class="text-slate-300 text-sm mb-4">Писать сортировку можно по-разному, но в Python есть встроенная функция sorted, которая сортирует список и возвращает новый список, не изменяя исходный.</p></div>

```

        left(x) = z          # z становится левым р
        высоты пересчитываются только у z и x — ост
    </div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
    <h3 class="text-xl font-bold text-teal-400 mb-4
    <p class="text-slate-300 text-sm mb-4">Оба обхо
        тару». Вся разница в таре: стек уходит вглуби
        ext-emerald-300">0(V + E)</span>: каждая вершн
        <pre class="bg-slate-950 border border-slate-700
def go(graph, start, tank):          # tank: список-стек
    seen, tank = {start}, tank([start])
    while tank:
        v = tank.pop() if isinstance(tank, list) else t
        for to in graph[v]:
            if to not in seen:
                seen.add(to); tank.append(to)
    return seen</pre>
    <p class="text-slate-300 text-sm">Дальше эта па
        надстройки над этими восемью строками.</p>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
    <h3 class="text-xl font-bold text-slate-300 mb-
    <div class="grid grid-cols-1 xl:grid-cols-2 gap
        <div class="bg-slate-800 p-6 rounded-lg bord
            <div class="absolute -top-3 left-4 bg-s
                border-indigo-500 shadow-md">
                    Аналогия 1: Схема кинозала
                </div>
                <p class="text-slate-300 text-sm">Дерев
                    азателей-«usher'ов» не нужно. Место n, дети
                    рху вниз и слева направо.</p>
                </div>
                <div class="bg-slate-800 p-6 rounded-lg bord
                    <div class="absolute -top-3 left-4 bg-s

```

```

title: "Билет 14. Дейкстра",
type: "html",
category: "Графы",
content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded
  <h2 class="text-3xl font-bold text-white">Дейкст
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border
    <h3 class="text-xl font-bold text-emerald-400">Аналогия 1: Позитивное планирование
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">Позитивное планирование — это когда вы знаете, что вы можете добраться до пункта назначения, но не знаете, как добраться.
      <p class="text-xl font-mono text-white">Позитивное планирование — это когда вы знаете, что вы можете добраться до пункта назначения, но не знаете, как добраться.
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
          Аналогия 1: Позитивное планирование
        </div>
        <p class="text-slate-300 text-sm mb-4">Позитивное планирование — это когда вы знаете, что вы можете добраться до пункта назначения, но не знаете, как добраться. (нельзя поехать и заработать). Он всегда работает.
        <div id="slot-chapter-image-dijkstra" class="h-400px">
        </div>

      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
          Ограничения
        </div>
        <p class="text-slate-300 text-sm mb-4">Ограничения — это когда вы знаете, что вы можете добраться до пункта назначения, но не знаете, как добраться. (нельзя поехать и заработать). Он всегда работает.
        <div id="slot-chapter-image-dijkstra" class="h-400px">
        </div>
      </div>
    </div>
  </div>
`

```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
  <!-- Аналогия 1 -->
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      ♂ Аналогия 1: Параноик
    </div>
    <p class="text-slate-300 text-sm mb-2">
      еряет ВСЕ возможные дороги V-1 раз подряд.
    <div id="slot-chapter-image-bellman"
    </div>

  <!-- Аналогия 2 -->
  <div class="bg-slate-900 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
      Отрицательный цикл
    </div>
    <p class="text-slate-300 text-sm mb-2">
      г станет ЕЩЕ короче – значит мы попали во в
    </div>
  </div>

  <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</section>`
},
{
  id: "floyd",
  title: "Билет 16. Флойд-Уоршелл",
  type: "html",
  category: "Графы",
  content: `<section id="floyd-content" class="mb-12">
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
  <h2 class="text-3xl font-bold text-white">Флойд

```



```

    },
    {
      id: "aho-corasick",
      title: "Билет 23. Алгоритм Ахо-Корасик",
      type: "html",
      category: "Строки",
      content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1 round
  <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-l
    <h3 class="text-xl font-bold text-blue-400 mb-4">

    <div class="bg-slate-900 p-4 rounded-lg mb-6 text
      <p class="text-sm text-blue-300 italic mb-2">0с
      <p class="text-lg font-mono text-white">Бор (Tr
    </div>
    <p class="text-slate-300 text-sm">Позволяет найти
      -mono text-amber-300 bg-black/30 px-1 rounded">
      y.</p>
    </div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      мы двигаемся по веткам. Если нужного продолж
      ("нити страховки") в самый длинный из извест
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-2

```

```

while (!q.empty()) {
    int v = q.front(); q.pop();
    for (auto const& [ch, child] : nodes[v].trie) {
        // Суффиксная ссылка ребёнка — это переход
        nodes[child].link = nodes[nodes[v].link].go

        // Если суффиксная ссылка сама является сло
        // Иначе наследуем terminal_link
        if (nodes[nodes[child].link].is_terminal) {
            nodes[child].term_link = nodes[child].l
        } else {
            nodes[child].term_link = nodes[nodes[ch
        }

        q.push(child);
    }
}
}
</div>
</div>
</details>
</div>
</section>`
},
{
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `<section id="alg-map-content" class="mb-1
<div class="flex items-center mb-6">
    <span class="bg-purple-600 text-white px-4 py-1
    <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

    options: [
        "Да: минимум и сумма считаются одним приёмом",
        "Нет: блоки перекрываются, а вычесть перекрытие",
        "Да, если заранее хранить суммы в нулевом уровне",
    ],
    correctIndex: 1,
    explanation: "Два (в 2D – четыре) блока накрывают
        обратная операция, но применить её к перекрытию
        ммы, билет 5."
},
{
    question: "Какие операции подходят разреженной та
    options: [
        "Любые ассоциативные",
        "Идемпотентные: min, max, gcd, побитовые И и ИЛИ",
        "Только сумма и произведение"
    ],
    correctIndex: 1,
    explanation: "Нужно  $op(x, x) = x$ : тогда дважды на
        приём и уходит в префиксные суммы."
},
{
    question: "Между запросами элементы матрицы меняю
    options: [
        "Разреженную таблицу: она перестраивается за  $O(n^2)$ ",
        "Префиксную матрицу: достаточно поправить одну
        "Дерево отрезков: и таблица, и префиксы статичны",
    ],
    correctIndex: 2,
    explanation: "Обе структуры статичны: изменение э
        ксной. С обновлениями работает дерево отрезков,
    }
},
"prefix-sums-2d": [
    {
        question: "Сколько памяти занимает двумерная преф
        options: [
            "N × M – как исходная матрица, плюс рамка из ну

```

```

        ной операции нет. Для min, max и gcd берут разро
    },
    {
        question: "Изменили один элемент матрицы. Что ста
        options: [
            "Поправится одна клетка за  $O(1)$ ",
            "Нужен пересчёт  $O(N \cdot M)$ : от этой клетки зависит
            "Ничего не станет: таблица остаётся верной"
        ],
        correctIndex: 1,
        explanation: "Каждая клетка  $S[i][j]$  включает в се
            я частые, это задача дерева отрезков (билет 1).
    }
],
"euler-path-vs-cycle": [
    {
        question: "В связном графе ровно 2 вершины с нечёт
        options: [
            "Есть эйлеров цикл (вернись домой!)",
            "Есть эйлеров путь (из одной нечётной в другую)
            "Ни пути, ни цикла нет — это полный провал"
        ],
        correctIndex: 1,
        explanation: "Если нечётных вершин ровно 2 — это
            ётной, заканчиваем в другой."
    },
    {
        question: "Почему гамильтонов цикл — это NP-полная
        options: [
            "Потому что Гамильтон — болезнь, и лечить её т
            "Потому что для эйлерова цикла есть простой кри
            "Потому что Эйлер был умнее Гамильтона"
        ],
        correctIndex: 1,
        explanation: "Для эйлерова цикла работает критери
            ого критерия. Это NP-полная задача, которую не
    },
    {

```

```

        "O(2^V) – полный перебор"
    ],
    correctIndex: 1,
    explanation: "DFS обходит каждую вершину и каждое
}
],
"planarity-euler-formula": [
{
    question: "Планарный граф имеет V=6, E=12. Возможн
    options: [
        "Да, если это двудольный граф",
        "Нет, нарушается неравенство  $E \leq 3V - 6$ ",
        "Да, если нет треугольных граней"
    ],
    correctIndex: 1,
    explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$  – на грани пр
        Так что такой граф непланарен."
    },
    {
        question: "Сколько граней у планарного графа с V=
        options: [
            "3 грани",
            "5 граней",
            "10 граней"
        ],
        correctIndex: 1,
        explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$ . Не
    },
    {
        question: "Почему K5 непланарен?",
        options: [
            "Потому что  $10 > 3*5 - 6 = 9$  – нарушение",
            "Потому что Эйлер был против",
            "Потому что 5 вершин – несчастливое число"
        ],
        correctIndex: 0,
        explanation: "У K5: V=5, E=10,  $3V - 6 = 9$ .  $10 > 9$ 
    }
}

```

```

/** Транслировать реальные переменные визуализациям тек
export function emitVizState(
  chapterId: string,
  snapshot: Omit<VizStateEventDetail, "chapterId">
) {
  if (typeof window === "undefined") return;
  const detail: VizStateEventDetail = { chapterId, ...s
  latestByChapter.set(chapterId, detail);
  window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Вернуть демонстрацию в ручной режим при закрытии/ре
export function clearVizState(chapterId: string) {
  if (typeof window === "undefined") return;
  latestByChapter.delete(chapterId);
  window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Текущий снимок компилятора для прямой отрисовки мас
export function useVizRuntime(): VizStateEventDetail | n
  const chapterId = useContext(VizChapterContext);
  const [runtime, setRuntime] = useState<VizStateEventD
    chapterId ? latestByChapter.get(chapterId) ?? null
  );

  useEffect(() => {
    setRuntime(chapterId ? latestByChapter.get(chapterId
    if (!chapterId) return;
    const handler = (e: Event) => {
      const detail = (e as CustomEvent<VizStateEventDet
      if (detail?.chapterId === chapterId) {
        setRuntime(detail.variables || detail.line !==
      }
    };
    window.addEventListener(CHANNEL, handler);
    return () => window.removeEventListener(CHANNEL, ha
  }, [chapterId]);

```

```

    }, [chapterId, currentStep, maxStep, goToStep, selectStep]
  }
}

```

```

/** Утилиты без небезопасных cast-ов для визуализаторов
export const vizNumber = (value: DebugValue | undefined) => {
  typeof value === "number" && Number.isFinite(value) ? value : null;
}

```

```

export const vizString = (value: DebugValue | undefined) => {
  typeof value === "string" ? value : null;
}

```

```

export const vizArray = (value: DebugValue | undefined) => {
  Array.isArray(value) ? value : null;
}

```

```

export const vizRecord = (value: DebugValue | undefined) => {
  value !== null && typeof value === "object" && !Array.isArray(value)
    ? (value as Record<string, DebugValue>)
    : null;
}

```

```

/* — Матрицы из компилятора: СВОЯ размерность, а не дефолтная

```

```

/**
 * Нормализованная матрица, пришедшая из выполненного Рунима
 *
 * Размерность берётся из самих данных: если пользователь ввёл
 * 5 вершин, визуализация рисует 5x5, а не подставляет его же
 * 7x7 (раньше ячейки «не той» размерности превращались в 0)
 *
 * Поддерживаются все учебные способы хранения:
 * list[list]      d = [[0, 4], [999, 0]]
 * list[dict]      d = [{0: 0, 1: 4}, {1: 0}]
 * dict[dict]      d = {0: {0: 0, 1: 4}}
 * dict[(i, j)]    d = {(0, 1): 4} — ключ после запятой
 */

```

```

export interface LiveMatrix {
  rows: number;
  cols: number;
  /** Значение ячейки; null — таких данных в коде пользователя нет
  at: (i: number, j: number) => number | null;
}

```

```

    if (n === null) return;
    cells.set(`${i}:${j}`, n);
    rows = Math.max(rows, i + 1);
    cols = Math.max(cols, j + 1);
};

const asArray = vizArray(value);
if (asArray) {
    asArray.slice(0, MAX_MATRIX_SIDE).forEach((row, i) => {
        const inner = vizArray(row);
        if (inner) {
            inner.slice(0, MAX_MATRIX_SIDE).forEach((cell, j) => {
                return;
            });
        }
        const record = vizRecord(row);
        if (record) {
            for (const [key, cell] of Object.entries(record)) {
                const j = Number(key);
                if (Number.isInteger(j)) put(i, j, cell);
            }
        }
    });
} else {
    const record = vizRecord(value);
    if (!record) return null;
    for (const [key, cell] of Object.entries(record)) {
        const pair = parsePairKey(key);
        if (pair) {
            put(pair[0], pair[1], cell);
            continue;
        }
        const i = Number(key);
        if (!Number.isInteger(i)) continue;
        const inner = vizArray(cell);
        if (inner) {
            inner.slice(0, MAX_MATRIX_SIDE).forEach((v, j) => {
                continue;
            });
        }
    }
}

```

/* — Выбор демонстрации внутри страницы (вкладки 1D / 2D)

```
const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

export interface VizDemoEventDetail {
  chapterId: string;
  /** base-запись страницы – demoId пустой; подписанные */
  demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта */
export function emitVizDemo(chapterId: string, demoId: string) {
  if (typeof window === "undefined") return;
  latestDemoByChapter.set(chapterId, demoId);
  window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>("viz-demo", {
    detail: { chapterId, demoId }
  }));
}

/**
 * Подписка: панель компилятора следует за активной вкладкой
 * Сразу отдаём последнее значение: компилятор часто отстает
 * как пользователь переключился с 1D на 2D.
 */
export function onVizDemo(chapterId: string, cb: (demoId: string) => void) {
  if (typeof window === "undefined") return () => {};
  const latest = latestDemoByChapter.get(chapterId);
  if (latest !== undefined) cb(latest);
  const handler = (e: Event) => {
    const detail = (e as CustomEvent<VizDemoEventDetail>).detail;
    if (detail && detail.chapterId === chapterId) cb(detail.demoId);
  };
  window.addEventListener(DEMO_CHANNEL, handler);
  return () => window.removeEventListener(DEMO_CHANNEL, handler);
}
```

Универсальное пособие • полный исходный код

```
-----
tree = [0] * (2 * n)
i = 0                                # лист: tree[n + i]

v, l, r = 1, 0, n - 1               # вершина и её отрезок
m = (l + r) // 2                     # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
    variables: [
        { name: "n", role: "длина исходного массива; листы"},
        { name: "i", role: "индекс элемента массива; его левый и правый дети"},
        { name: "v", role: "текущая вершина дерева отрезка"},
        { name: "l, r", role: "границы отрезка, за которыми лежат левый и правый дети"},
        { name: "m", role: "середина отрезка  $m = (l + r) // 2$ "},
    ],
},
treap: {
    vizTitle: "Декартово дерево: build → split → merge → erase",
    stepNote: "Шаг = один спуск в split/merge/erase (перезапись массива целиком.",
    code: `points = [[45, 8], [3, 6], [6, 5], [2, 4], [1, 2], [5, 7], [4, 3], [7, 1]]
n = len(points)                # сколько пар (x; y) – столько вершин
mid = 7                        # разрез для split:  $x \leq mid < y$ 
key = 9                        # какую точку удаляем в erase
step = 0                       # номер шага – листает кадры декартова дерева
height = 0                    # высота построенного дерева
root = None

# --- тело алгоритма ---
class Node:
    def __init__(s, x, y):
        s.x = x; s.y = y; s.l = None; s.r = None

def build(pts):
    """Декартово дерево из отсортированных по x точек:
    pts = sorted(pts, key=lambda p: p[0])
    stack = []
    for x, y in pts:
        cur = Node(x, y)
        last = None
```



```

while x.p:
    p = x.p; g = p.p
    step += 1
    if g is None:
        case = "zig" # родители
        rot(x)
    elif (g.l is p) == (p.l is x):
        case = "zig-zig" # x и p -
        rot(p); rot(x)
    else:
        case = "zig-zag" # змейка
        rot(x); rot(x)
    xk, pk, gk = x.k, p.k, (g.k if g else 0)
    print("шаг", step, "->", case, "| x =", xk, "p = ", pk)
    root = x
    print("    корень теперь", x.k, "| обход:", inorder(root))
    return x

```

1) Zig: родитель x – уже корень (дерево 40 -> 20)
 root = N(40); root.l = N(20); root.l.p = root
 splay(root.l)

2) Zig-Zig: бамбук 60 -> 40 -> 20, оба сына левые
 root = N(60); root.l = N(40); root.l.p = root; root.l.l = N(20)
 splay(root.l.l)

3) Zig-Zag: змейка 60 -> 20 -> 40 (40 – правый сын 20)
 root = N(60); root.l = N(20); root.l.p = root; root.l.r = N(40)
 splay(root.l.r),

```

variables: [
    { name: "case", role: "какой случай сработал: \"zig\", \"zig-zig\", \"zig-zag\" },
    { name: "step", role: "номер применения случая – 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689, 690, 691, 692, 693, 694, 695, 696, 697, 698, 699, 700, 701, 702, 703, 704, 705, 706, 707, 708, 709, 710, 711, 712, 713, 714, 715, 716, 717, 718, 719, 720, 721, 722, 723, 724, 725, 726, 727, 728, 729, 730, 731, 732, 733, 734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754, 755, 756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771, 772, 773, 774, 775, 776, 777, 778, 779, 780, 781, 782, 783, 784, 785, 786, 787, 788, 789, 790, 791, 792, 793, 794, 795, 796, 797, 798, 799, 800, 801, 802, 803, 804, 805, 806, 807, 808, 809, 810, 811, 812, 813, 814, 815, 816, 817, 818, 819, 820, 821, 822, 823, 824, 825, 826, 827, 828, 829, 830, 831, 832, 833, 834, 835, 836, 837, 838, 839, 840, 841, 842, 843, 844, 845, 846, 847, 848, 849, 850, 851, 852, 853, 854, 855, 856, 857, 858, 859, 860, 861, 862, 863, 864, 865, 866, 867, 868, 869, 870, 871, 872, 873, 874, 875, 876, 877, 878, 879, 880, 881, 882, 883, 884, 885, 886, 887, 888, 889, 890, 891, 892, 893, 894, 895, 896, 897, 898, 899, 900, 901, 902, 903, 904, 905, 906, 907, 908, 909, 910, 911, 912, 913, 914, 915, 916, 917, 918, 919, 920, 921, 922, 923, 924, 925, 926, 927, 928, 929, 930, 931, 932, 933, 934, 935, 936, 937, 938, 939, 940, 941, 942, 943, 944, 945, 946, 947, 948, 949, 950, 951, 952, 953, 954, 955, 956, 957, 958, 959, 960, 961, 962, 963, 964, 965, 966, 967, 968, 969, 970, 971, 972, 973, 974, 975, 976, 977, 978, 979, 980, 981, 982, 983, 984, 985, 986, 987, 988, 989, 990, 991, 992, 993, 994, 995, 996, 997, 998, 999, 1000, 1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009, 1010, 1011, 1012, 1013, 1014, 1015, 1016, 1017, 1018, 1019, 1020, 1021, 1022, 1023, 1024, 1025, 1026, 1027, 1028, 1029, 1030, 1031, 1032, 1033, 1034, 1035, 1036, 1037, 1038, 1039, 1040, 1041, 1042, 1043, 1044, 1045, 1046, 1047, 1048, 1049, 1050, 1051, 1052, 1053, 1054, 1055, 1056, 1057, 1058, 1059, 1060, 1061, 1062, 1063, 1064, 1065, 1066, 1067, 1068, 1069, 1070, 1071, 1072, 1073, 1074, 1075, 1076, 1077, 1078, 1079, 1080, 1081, 1082, 1083, 1084, 1085, 1086, 1087, 1088, 1089, 1090, 1091, 1092, 1093, 1094, 1095, 1096, 1097, 1098, 1099, 1100, 1101, 1102, 1103, 1104, 1105, 1106, 1107, 1108, 1109, 1110, 1111, 1112, 1113, 1114, 1115, 1116, 1117, 1118, 1119, 1120, 1121, 1122, 1123, 1124, 1125, 1126, 1127, 1128, 1129, 1130, 1131, 1132, 1133, 1134, 1135, 1136, 1137, 1138, 1139, 1140, 1141, 1142, 1143, 1144, 1145, 1146, 1147, 1148, 1149, 1150, 1151, 1152, 1153, 1154, 1155, 1156, 1157, 1158, 1159, 1160, 1161, 1162, 1163, 1164, 1165, 1166, 1167, 1168, 1169, 1170, 1171, 1172, 1173, 1174, 1175, 1176, 1177, 1178, 1179, 1180, 1181, 1182, 1183, 1184, 1185, 1186, 1187, 1188, 1189, 1190, 1191, 1192, 1193, 1194, 1195, 1196, 1197, 1198, 1199, 1200, 1201, 1202, 1203, 1204, 1205, 1206, 1207, 1208, 1209, 1210, 1211, 1212, 1213, 1214, 1215, 1216, 1217, 1218, 1219, 1220, 1221, 1222, 1223, 1224, 1225, 1226, 1227, 1228, 1229, 1230, 1231, 1232, 1233, 1234, 1235, 1236, 1237, 1238, 1239, 1240, 1241, 1242, 1243, 1244, 1245, 1246, 1247, 1248, 1249, 1250, 1251, 1252, 1253, 1254, 1255, 1256, 1257, 1258, 1259, 1260, 1261, 1262, 1263, 1264, 1265, 1266, 1267, 1268, 1269, 1270, 1271, 1272, 1273, 1274, 1275, 1276, 1277, 1278, 1279, 1280, 1281, 1282, 1283, 1284, 1285, 1286, 1287, 1288, 1289, 1290, 1291, 1292, 1293, 1294, 1295, 1296, 1297, 1298, 1299, 1300, 1301, 1302, 1303, 1304, 1305, 1306, 1307, 1308, 1309, 1310, 1311, 1312, 1313, 1314, 1315, 1316, 1317, 1318, 1319, 1320, 1321, 1322, 1323, 1324, 1325, 1326, 1327, 1328, 1329, 1330, 1331, 1332, 1333, 1334, 1335, 1336, 1337, 1338, 1339, 1340, 1341, 1342, 1343, 1344, 1345, 1346, 1347, 1348, 1349, 1350, 1351, 1352, 1353, 1354, 1355, 1356, 1357, 1358, 1359, 1360, 1361, 1362, 1363, 1364, 1365, 1366, 1367, 1368, 1369, 1370, 1371, 1372, 1373, 1374, 1375, 1376, 1377, 1378, 1379, 1380, 1381, 1382, 1383, 1384, 1385, 1386, 1387, 1388, 1389, 1390, 1391, 1392, 1393, 1394, 1395, 1396, 1397, 1398, 1399, 1400, 1401, 1402, 1403, 1404, 1405, 1406, 1407, 1408, 1409, 1410, 1411, 1412, 1413, 1414, 1415, 1416, 1417, 1418, 1419, 1420, 1421, 1422, 1423, 1424, 1425, 1426, 1427, 1428, 1429, 1430, 1431, 1432, 1433, 1434, 1435, 1436, 1437, 1438, 1439, 1440, 1441, 1442, 1443, 1444, 1445, 1446, 1447, 1448, 1449, 1450, 1451, 1452, 1453, 1454, 1455, 1456, 1457, 1458, 1459, 1460, 1461, 1462, 1463, 1464, 1465, 1466, 1467, 1468, 1469, 1470, 1471, 1472, 1473, 1474, 1475, 1476, 1477, 1478, 1479, 1480, 1481, 1482, 1483, 1484, 1485, 1486, 1487, 1488, 1489, 1490, 1491, 1492, 1493, 1494, 1495, 1496, 1497, 1498, 1499, 1500, 1501, 1502, 1503, 1504, 1505, 1506, 1507, 1508, 1509, 1510, 1511, 1512, 1513, 1514, 1515, 1516, 1517, 1518, 1519, 1520, 1521, 1522, 1523, 1524, 1525, 1526, 1527, 1528, 1529, 1530, 1531, 1532, 1533, 1534, 1535, 1536, 1537, 1538, 1539, 1540, 1541, 1542, 1543, 1544, 1545, 1546, 1547, 1548, 1549, 1550, 1551, 1552, 1553, 1554, 1555, 1556, 1557, 1558, 1559, 1560, 1561, 1562, 1563, 1564, 1565, 1566, 1567, 1568, 1569, 1570, 1571, 1572, 1573, 1574, 1575, 1576, 1577, 1578, 1579, 1580, 1581, 1582, 1583, 1584, 1585, 1586, 1587, 1588, 1589, 1590, 1591, 1592, 1593, 1594, 1595, 1596, 1597, 1598, 1599, 1600, 1601, 1602, 1603, 1604, 1605, 1606, 1607, 1608, 1609, 1610, 1611, 1612, 1613, 1614, 1615, 1616, 1617, 1618, 1619, 1620, 1621, 1622, 1623, 1624, 1625, 1626, 1627, 1628, 1629, 1630, 1631, 1632, 1633, 1634, 1635, 1636, 1637, 1638, 1639, 1640, 1641, 1642, 1643, 1644, 1645, 1646, 1647, 1648, 1649, 1650, 1651, 1652, 1653, 1654, 1655, 1656, 1657, 1658, 1659, 1660, 1661, 1662, 1663, 1664, 1665, 1666, 1667, 1668, 1669, 1670, 1671, 1672, 1673, 1674, 1675, 1676, 1677, 1678, 1679, 1680, 1681, 1682, 1683, 1684, 1685, 1686, 1687, 1688, 1689, 1690, 1691, 1692, 1693, 1694, 1695, 1696, 1697, 1698, 1699, 1700, 1701, 1702, 1703, 1704, 1705, 1706, 1707, 1708, 1709, 1710, 1711, 1712, 1713, 1714, 1715, 1716, 1717, 1718, 1719, 1720, 1721, 1722, 1723, 1724, 1725, 1726, 1727, 1728, 1729, 1730, 1731, 1732, 1733, 1734, 1735, 1736, 1737, 1738, 1739, 1740, 1741, 1742, 1743, 1744, 1745, 1746, 1747, 1748, 1749, 1750, 1751, 1752, 1753, 1754, 1755, 1756, 1757, 1758, 1759, 1760, 1761, 1762, 1763, 1764, 1765, 1766, 1767, 1768, 1769, 1770, 1771, 1772, 1773, 1774, 1775, 1776, 1777, 1778, 1779, 1780, 1781, 1782, 1783, 1784, 1785, 1786, 1787, 1788, 1789, 1790, 1791, 1792, 1793, 1794, 1795, 1796, 1797, 1798, 1799, 1800, 1801, 1802, 1803, 1804, 1805, 1806, 1807, 1808, 1809, 1810, 1811, 1812, 1813, 1814, 1815, 1816, 1817, 1818, 1819, 1820, 1821, 1822, 1823, 1824, 1825, 1826, 1827, 1828, 1829, 1830, 1831, 1832, 1833, 1834, 1835, 1836, 1837, 1838, 1839, 1840, 1841, 1842, 1843, 1844, 1845, 1846, 1847, 1848, 1849, 1850, 1851, 1852, 1853, 1854, 1855, 1856, 1857, 1858, 1859, 1860, 1861, 1862, 1863, 1864, 1865, 1866, 1867, 1868, 1869, 1870, 1871, 1872, 1873, 1874, 1875, 1876, 1877, 1878, 1879, 1880, 1881, 1882, 1883, 1884, 1885, 1886, 1887, 1888, 1889, 1890, 1891, 1892, 1893, 1894, 1895, 1896, 1897, 1898, 1899, 1900, 1901, 1902, 1903, 1904, 1905, 1906, 1907, 1908, 1909, 1910, 1911, 1912, 1913, 1914, 1915, 1916, 1917, 1918, 1919, 1920, 1921, 1922, 1923, 1924, 1925, 1926, 1927, 1928, 1929, 1930, 1931, 1932, 1933, 1934, 1935, 1936, 1937, 1938, 1939, 1940, 1941, 1942, 1943, 1944, 1945, 1946, 1947, 1948, 1949, 1950, 1951, 1952, 1953, 1954, 1955, 1956, 1957, 1958, 1959, 1960, 1961, 1962, 1963, 1964, 1965, 1966, 1967, 1968, 1969, 1970, 1971, 1972, 1973, 1974, 1975, 1976, 1977, 1978, 1979, 1980, 1981, 1982, 1983, 1984, 1985, 1986, 1987, 1988, 1989, 1990, 1991, 1992, 1993, 1994, 1995, 1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027, 2028, 2029, 2030, 2031, 2032, 2033, 2034, 2035, 2036, 2037, 2038, 2039, 2040, 2041, 2042, 2043, 2044, 2045, 2046, 2047, 2048, 2049, 2050, 2051, 2052, 2053, 2054, 2055, 2056, 2057, 2058, 2059, 2060, 2061, 2062, 2063, 2064, 2065, 2066, 2067, 2068, 2069, 2070, 2071, 2072, 2073, 2074, 2075, 2076, 2077, 2078, 2079, 2080, 2081, 2082, 2083, 2084, 2085, 2086, 2087, 2088, 2089, 2090, 2091, 2092, 2093, 2094, 2095, 2096, 2097, 2098, 2099, 2100, 2101, 2102, 2103, 2104, 2105, 2106, 2107, 2108, 2109, 2110, 2111, 2112, 2113, 2114, 2115, 2116, 2117, 2118, 2119, 2120, 2121, 2122, 2123, 2124, 2125, 2126, 2127, 2128, 2129, 2130, 2131, 2132, 2133, 2134, 213
```

```

        stepNote: "Шаг = заполнение очередного  $P[i] = P[i-1]$ 
code: `a = [3, 1, 4, 1, 5, 9, 2, 6]` # исходный массив
n = len(a) # длина: префиксов будет n
i = 0 # текущая граница префикса

P = [0] #  $P[0] = 0$  – пустой префикс
for i in range(1, n + 1):
    P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)`
variables: [
    { name: "a", role: "исходный массив 1D-демонстрация"
    { name: "n", role: "длина массива a; префиксов будет n"
    { name: "P", role: "префиксные суммы:  $P[i] = a[0] + \dots + a[i-1]$ "
    { name: "i", role: "граница префикса: на шаге считаем до i"
],
},
"dynamic-programming": {
    vizTitle: "Динамическое программирование (мемоизация)"
    stepNote: "Шаг = событие внутри fibМемо(k): вызов,
code: `n = 5`
memo = {}

def fib(k):
    if k in memo:
        return memo[k]
    if k <= 2:
        return 1
    print(f"считаем fib({k-1}) + fib({k-2})")
    r1 = fib(k - 1)
    r2 = fib(k - 2)
    memo[k] = r1 + r2
    return memo[k]

print("итог:", fib(n))`
variables: [
    { name: "n", role: "целевой номер числа Фибоначчи"
    { name: "k", role: "аргумент текущего рекурсивного вызова"
    { name: "memo", role: "кэш уже посчитанных значений"

```

```

code: `from collections import deque

q = deque(["A"])          # очередь BFS
u = "B"                   # сосед, найденный из текущ
q.append(u)                # enqueue соседа – в конец
front = q[0]              # голова очереди: её обрабо
v = q.popleft()           # шаг: достали голову и обр
print("front =", front, "| v =", v, "| очередь:", list(
    variables: [
        { name: "front", role: "голова очереди – вершина,
        { name: "v", role: "вершина, извлечённая из очере
        { name: "u", role: "сосед вершины v, которого кла
    ],
},
"everyday-basics": {
    vizTitle: "Бытовой тренажёр: стек, очередь, куча",
    stepNote: "Шаг = одна бытовая операция: тарелка на
    code: `from collections import deque

st, q = [], deque()       # тарелки (стек) и очередь
st.append(" "); top = st.pop()
q.append(" "); q.append(" "); first = q.popleft()

i = 0                     # счётчик выполненных опера
for op in ("push", "pop", "enqueue", "dequeue"):
    i += 1
print("тарелка:", top, "| первый из очереди:", first, "
    variables: [
        { name: "top / front", role: "верх стопки (стек)
        { name: "i", role: "индекс в массиве кучи; родите
    ],
},
"graph-dfs-bfs": {
    vizTitle: "DFS и BFS на графе",
    stepNote: "Шаг = переход из текущей вершины v к неп
    code: `from collections import deque

adj = {"A": ["B", "C"], "B": ["D"], "C": [], "D": []}
```

Универсальное пособие • полный исходный код

```
-----
v = -1                                # вершина, в которой обход начался
stack = []                             # фронт обхода (стек для DFS)
i = 0                                  # номер шага — двигает по какому-то ребру

# --- тело алгоритма ---
for start in range(n):
    if comp[start] != -1:
        continue
    comps += 1                          # непокрашенная вершина => новая компонента
    c = comps
    stack = [start]
    comp[start] = c
    used[start] = True
    v = start
    while stack:
        v = stack.pop()                # для BFS: v = stack.pop(0)
        i += 1
        for to in adj[v]:
            if comp[to] == -1:
                comp[to] = c
                used[to] = True
                stack.append(to)
print("компонент:", comps)
print("разметка comp:", comp)`
variables: [
    { name: "n", role: "число вершин; в демонстрации — количество узлов графа" },
    { name: "adj", role: "списки смежности — тот же граф, что и g" },
    { name: "comp", role: "разметка: номер компоненты", range: "-1 ... comps" },
    { name: "used", role: "булев вариант разметки — для BFS" },
    { name: "comps", role: "счётчик найденных компонент" },
    { name: "c", role: "номер компоненты, которую красим сейчас" },
    { name: "v", role: "вершина, где обход находится" },
    { name: "stack", role: "фронт обхода: стек (DFS) или очередь (BFS)" },
    { name: "i / step", role: "номер шага — перематываем кадр" },
],
},
"top-sort": {
```

```
def dfs1(v):
    used[v] = True
    for to in adj[v]:
        if not used[to]:
            dfs1(to)
    order.append(v)          # выходим из вершины — она

for v in adj:
    if not used[v]:
        dfs1(v)
        print("DFS из", v, "закончен | order:", order)

# транспонирование: каждое ребро разворачиваем
adjT = {}
for v in adj:
    adjT[v] = []
for v in adj:
    for to in adj[v]:
        adjT[to].append(v)
print("обратный граф:", adjT)

# фаза 2: идём по order С КОНЦА, DFS по adjT красит одн
comp = {}
c = 0
for i in range(len(order) - 1, -1, -1):
    v = order[i]
    if v in comp:
        continue
    stack = [v]
    comp[v] = c
    while stack:
        u = stack.pop()
        for to in adjT[u]:
            if to not in comp:
                comp[to] = c
                stack.append(to)
    print("SCC", c, "собрана из", v, ":", comp)
```



```
},  
    "bridges-code": {  
        vizTitle: "Мосты: DFS + tin/low",  
        stepNote: "Шаг = строка псевдокода слева; ребро (v,  
            code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1  
timer = 3                                # как на демо  
  
v, to = "B", "G"                        # шаг: возврат из G  
is_bridge = low[to] > tin[v]             # 3 > 2 → мост!  
print(f"ребро {v}-{to}: мост? {is_bridge}")`,  
        variables: [  
            { name: "v", role: "текущая вершина DFS", range:  
            { name: "to", role: "сосед; ребро (v, to) проверяе  
            { name: "tin[v]", role: "время входа в вершину – m  
            { name: "low[v]", role: "минимум tin, куда можно ,  
            { name: "timer", role: "глобальный счётчик времени  
        ],  
    },  
    "euler-path-vs-cycle": {  
        vizTitle: "Эйлеров маршрут: алгоритм Иера",  
        stepNote: "Шаг = переход по неиспользованному ребру  
            идти нельзя.",  
        code: `n = 5                      # вершин: 0...4 (на  
edges = [(0, 1), (0, 2), (1, 3), (2, 4), (1, 2), (3, 4)]  
adj = {v: [] for v in range(n)}  
for i, (u, v) in enumerate(edges):  
    adj[u].append((v, i))  
    adj[v].append((u, i))  
deg = {v: len(adj[v]) for v in range(n)}      # степени ве  
odd = [v for v in range(n) if deg[v] % 2 == 1]   # нечёт  
start = odd[0] if odd else 0                    # путь начинае  
used = [False] * len(edges)                     # какие рёбра :  
path = [start]                                   # маршрут: пос.  
last = start                                     # где мы сейча  
stack = [start]                                  # стек для отк  
v = start  
step = 0
```

```

V = n                                # вершин для формулы
E = sum(len(a) for a in adj) // 2    # рёбер: в списке смежн
F = 2 - V + E                        # граней: из  $V - E + F = 2$ 

# --- тело алгоритма ---
print("V =", V, "| E =", E, "| F =", F, "|  $V - E + F = 2$ ")
for v in range(n):
    used = set()
    for u in adj[v]:
        if color[u] != 0:
            used.add(color[u])
    c = 1
    while c in used:                  # наименьший свободный цвет
        c += 1
    color[v] = c
    chi = max(chi, c)
    i += 1
    print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
variables: [
    { name: "n", role: "число вершин графа на демонстрации" },
    { name: "adj", role: "списки смежности — тот же граф, что и n" },
    { name: "color", role: "массив цветов вершин (0 = не покрашено)" },
    { name: "v", role: "вершина, которую красим сейчас" },
    { name: "u", role: "сосед вершины v, чей цвет уже есть в used" },
    { name: "delta", role: "максимальная степень  $\Delta = \max \deg(v)$ " },
    { name: "chi", role: "сколько цветов реально понадобится" },
    { name: "i / step", role: "номер шага — перематываем видео" },
    { name: "V, E, F", role: "для вкладки «Планарность»" },
]
},
dijkstra: {
    vizTitle: "Дейкстра",
    stepNote: "Шаг = старт, извлечение ближайшей вершины",
    code: `import heapq

```

Универсальное пособие • полный исходный код

```
-----
nodes = ["S", "A", "B", "C", "D", "E", "F"]
n = len(nodes)          # 7 вершин => n - 1 = 6 работ
m = len(E)               # 9 рёбер
```

```
dist = {v: INF for v in nodes}
dist["S"] = 0            # старт – База
```

```
# --- тело алгоритма ---
```

```
last_changed = ""
for i in range(1, n):    # итерация i = 1 ... n-1
    last_changed = ""
    for u, v, w in E:    # шаг: релаксация ребра (u, v)
        if dist[u] != INF and dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            last_changed = v
    if last_changed == "":
        print(f"итерация {i}: изменений нет – сошлось для всех вершин")
        break
    print(f"итерация {i}: " + ", ".join(f"{k}={dist[k]}:" for k in nodes))
```

```
# контрольный (n-й) проход: улучшение здесь означает отрицательный цикл
cycle_edge = None
for u, v, w in E:
    if dist[u] != INF and dist[u] + w < dist[v]:
        cycle_edge = (u, v, w)
        break
```

```
if cycle_edge:
    print("отрицательный цикл: улучшается ребро", cycle_edge)
else:
```

```
    print("отрицательных циклов нет:", {k: dist[k] for k in nodes})
    variables = [
        { name: "n", role: "число вершин графа – ровно n-1"},
        { name: "m", role: "число рёбер, по которым проходят"},
        { name: "i", role: "номер итерации (после i итераций)"},
        { name: "u, v, w", role: "текущее ребро: откуда, куда"},
        { name: "dist[v]", role: "расстояние; если обновилось"},
    ],
```

```

        на-Форда" },
        { name: "u, v", role: "ребро (u, v) перевзвешивает",
        { name: "d[v]", role: "расстояние Дейкстры в новом графе"
        },
    ],
},
mst: {
    vizTitle: "Остовное дерево: Краскал и DSU",
    stepNote: "Шаг = следующее ребро из отсортированных",
    code: `n = 9 # вершин: A..I — те, что в графе
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "A", "C"),
        (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "D", "F"),
        (12, "H", "I"), (13, "F", "I")]
edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGH I"} # DSU: представителю компонента
rank = {v: 0 for v in "ABCDEFGH I"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое рассматриваем

# --- тело алгоритма ---
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]] # сжатие путей
        x = parent[x]
    return x

for i, (w, a, b) in enumerate(edges):
    u, v = a, b
    ra, rb = find(a), find(b)
    if ra != rb: # разные компоненты
        parent[ra] = rb
        if rank[ra] == rank[rb]:
            rank[rb] += 1
        mst.append((w, a, b))
        total += w
print("вес остова:", total, "| рёбер:", len(mst))

```

```

vizTitle: "Z-функция",
stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]",
code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0          # правый Z-блок [l, r]
z = [None] * n            # пустые клетки: значения по умолчанию
z[0] = 0

for i in range(1, n):     # шаг: вычисляем z[i]
    if i <= r:
        z[i] = min(r - i + 1, z[i - l]) # из блока
    else:
        z[i] = 0           # свежая клетка: с нуля
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1       # досчёт в лоб
        if i + z[i] - 1 > r:
            l, r = i, i + z[i] - 1
print(z)`
variables: [
    { name: "n", role: "длина строки s", range: "n = len(s)" },
    { name: "i", role: "позиция, для которой считаем z[i]" },
    { name: "l, r", role: "правый крайний Z-блок: отрезок [l, r]" },
    { name: "z[i]", role: "длина наибольшего общего префикса s[i:] и s[0:z[i]]" }
],
},
"aho-corasick": {
    vizTitle: "Ахо-Корасик: бор, fail-ссылки и автомат",
    stepNote: "Шаг = один символ текста: state = go(state, c). Если не найдено, то переходят в корень. Если совпадают с картинкой «водопада».",
    code: `words = ["HERS", "HIS", "SHE", "HE"] # образцы
text = "USHERS" # текст, в котором ищем
n = len(text) # длина текста
state = 0 # текущее состояние автомата
i = 0 # индекс текущего символа
c = "" # сам символ
nodes = 1 # сколько вершин в автомате
queue = [] # очередь BFS, к которой добавляем новые вершины
fail = {} # fail[v] = вершина, в которую переходим, если не найдено

```

```

        queue.append(u)
print("вершин в боре:", nodes)
print("fail-ссылки:", fail)

def go(v, ch):
    """Переход автомата: сначала по бору, иначе спускаем
    while v != 0 and (v, ch) not in nxt:
        v = fail[v]
    return nxt.get((v, ch), 0)

for i, c in enumerate(text):
    state = go(state, c)
    v = state
    while v != 0:                                # собираем все образцы
        if v in term:
            matches.append((i, term[v]))
        v = fail[v]
    print(f"i={i} c='{c}' state={state} найдено={matches}")

print("всего вхождений:", len(matches), matches),
variables: [
    { name: "words", role: "набор образцов – по нему",
      },
    { name: "text", role: "сканируемый текст – подста",
    { name: "state", role: "текущее состояние автомата",
    { name: "i / c", role: "индекс и сам символ текста",
    { name: "nxt", role: "переходы бора: nxt[(v, ch)]",
    { name: "fail", role: "fail[v] – вершина наибольшей",
    { name: "term", role: "в каких вершинах заканчивае",
    { name: "queue", role: "очередь BFS, которой натя",
    { name: "matches", role: "найденные вхождения: (ин",
    { name: "nodes", role: "сколько вершин в боре (ко",
],
},
"complexity-classes": {
    vizTitle: "Классы сложности: рост по n и ops",
    stepNote: "Шаг = одна точка (n, ops): визуализация",
    code: `n = 1                                # размер входа

```

```

    },
    intro: {
        // мнемодкарточки без шагов алгоритма
        variables: [],
    },

    /* — Дополнительные демонстрации внутри страниц (вкл.)

    "planarity-euler-formula#greedy": {
        vizTitle: "Жадная покраска вершин ( $\chi \leq \Delta + 1$ )",
        stepNote: "Шаг = вершина v получает наименьший цвет",
        code: `n = 8                                # вершин в графе
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj)    # максимальная степень
color = [0] * n                     # 0 = ещё не покрашена
chi = 0                             # сколько цветов понадобилось
v = -1                             # вершина, которую красим сейчас
i = 0                               # номер шага

# --- тело алгоритма ---
for v in range(n):
    used = set()
    for u in adj[v]:
        if color[u] != 0:
            used.add(color[u])
    c = 1
    while c in used:                 # наименьший свободный цвет
        c += 1
    color[v] = c
    chi = max(chi, c)
    i += 1
    print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
        variables: [
            { name: "color", role: "цвета вершин (0 = не покрашено)", range: "0 ... n" },
            { name: "v", role: "текущая вершина", range: "0 ... n-1" },
            { name: "delta / chi", role: "максимальная степень / количество цветов", range: "0 ... n" },
            { name: "i / step", role: "номер кадра", range: "0 ... n-1" },
        ]
    },

```

```

    ],
},
"planarity-euler-formula#edges": {
    vizTitle: "Покраска рёбер (Визинг:  $\Delta \leq \chi' \leq \Delta + 1$ )",
    stepNote: "Шаг = ребро e получает наименьший цвет, <= \Delta",
    code: `n = 8
edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 4), (3, 4)]
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj)
ecolor = [0] * len(edges) # цвет каждого ребра
inc = [dict() for _ in range(n)] # inc[v][цвет] = ребро
e = -1 # номер текущего ребра
v = u = -1 # его концы
chi_e = 0

# --- тело алгоритма ---
for e, (a, b) in enumerate(edges):
    v, u = a, b
    used = set(inc[a].values()) | set(inc[b].values())
    c = 1
    while c in used:
        c += 1
    ecolor[e] = c
    inc[a][e] = c
    inc[b][e] = c
    chi_e = max(chi_e, c)
    print("ребро", (a, b), "занято:", sorted(used), "-> ", c)
print("chi' =", chi_e, "| delta =", delta, "| delta <= \Delta")
variables: [
    { name: "ecolor", role: "цвета рёбер – рёбра на графе", range: "0..delta" },
    { name: "e", role: "номер текущего ребра", range: "0..len(edges)-1" },
    { name: "v, u", role: "концы текущего ребра – подмножества V", range: "0..n-1" },
    { name: "inc", role: "для каждой вершины: какие цвета имеют инцидентные рёбра", range: "0..delta" },
    { name: "chi_e / delta", role: "χ' – сколько цветов", range: "0..delta+1" }
],
},
"graph-components#dfs": {
    vizTitle: "Компоненты связности: DFS-покраска",

```



```

    },
    "graph-components#dsu": {
        vizTitle: "Компоненты связности: DSU по рёбрам",
        stepNote: "Шаг = одно ребро (u, v): union склеивает",
        code: `n = 10                                # вершин в графе
edges = [(0, 1), (1, 2), (2, 3), (0, 3), (1, 3), (4, 5)]
parent = list(range(n))    # DSU: parent[v] = представи
rank = [0] * n             # высота дерева – по ней реш
comps = n                  # сначала каждая вершина – о
e = -1                     # номер обрабатываемого ребра
u, v = -1, -1              # концы текущего ребра
step = 0                   # номер шага – двигает по ка

# --- тело алгоритма ---
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]    # сжатие путей
        x = parent[x]
    return x

def union(a, b):
    global comps
    ra, rb = find(a), find(b)
    if ra == rb:
        return False                    # ребро внутри
    if rank[ra] < rank[rb]:
        ra, rb = rb, ra
    parent[rb] = ra
    if rank[ra] == rank[rb]:
        rank[ra] += 1
    comps -= 1
    return True

for e, (a, b) in enumerate(edges):
    u, v = a, b
    step = e + 1
    union(a, b)
print("компонент:", comps)

```

```

-----
k = 3                                # размер клики, которую ищем

# --- тело алгоритма ---
# Сведение 3-SAT <=p КЛИКА: вершина = {номер дизъюнкта,
# ребро = два литерала из РАЗНЫХ дизъюнктов, не противоречивые
clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
V = [(i, l) for i, cl in enumerate(clauses) for l in cl]
E = [(a, b) for a in V for b in V if a[0] < b[0] and a[1] != b[1]]
for step in range(7):
    print("кадр", step, "| вершин:", len(V), "| рёбер:", len(E))
print("формула выполнима <=> в графе есть клика размера k")
variables: [
    { name: "step", role: "номер кадра сведения (0..6)" },
    { name: "cls", role: "класс задачи A, которую сводят к КЛИКА" },
    { name: "n", role: "размер входа x; после сведения к КЛИКА" },
    { name: "k", role: "размер клики, которую ищет рекурсия" },
],
},
"complexity-classes#verify": {
    vizTitle: "Верификатор 3-SAT: проверка сертификата",
    stepNote: "Шаг = один дизъюнкт: верификатор подставляет значения в формулу",
    code: `clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
cert = [1, 0, 1, 0]          # сертификат: x1=1, x2=0, x3=1
k = 0                        # номер проверяемого дизъюнкта
ok = True                    # вердикт верификатора

# --- тело алгоритма ---
for k, cl in enumerate(clauses):
    hit = False
    for l in cl:
        val = cert[l - 1] if l > 0 else 1 - cert[-l - 1]
        if val == 1:
            hit = True
    ok = ok and hit
    print("дизъюнкт", k + 1, cl, "выполнен" if hit else "не выполнен")
print("сертификат", "принят" if ok else "отвергнут")`,
    variables: [
        { name: "cert", role: "сертификат – назначение переменных" },
    ],
},

```

```

-----
edges.sort()                # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGHI"} # DSU: представителем компонента
rank = {v: 0 for v in "ABCDEFGHI"}   # высота дерева
mst = []                          # рёбра, взятые в остов
total = 0                         # вес остова (на этом графе = 51)
i = 0                             # номер текущего ребра
u, v, w = None, None, None        # концы и вес ребра, которое рассматриваем

# --- тело алгоритма ---
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]] # сжатие путей
        x = parent[x]
    return x

for i, (w, a, b) in enumerate(edges):
    u, v = a, b
    ra, rb = find(a), find(b)
    if ra != rb:                    # разные компоненты
        parent[ra] = rb
        if rank[ra] == rank[rb]:
            rank[rb] += 1
        mst.append((w, a, b))
        total += w
print("вес остова:", total, "| рёбер:", len(mst))
print(mst)`
    variables: [
        { name: "i", role: "номер текущего ребра – перебираем"},
        { name: "u, v, w", role: "концы и вес рассматриваемого ребра"},
        { name: "parent", role: "DSU-массив: цвета компонентов"},
        { name: "mst", role: "рёбра, уже взятые в остов", "51"},
        { name: "total", role: "вес остова (51 на этом графе)"},
    ],
},
"mst#prim": {
    vizTitle: "Остовное дерево: Прим (рост из вершины E)",
    stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
    code: `import heapq

```

```

-----
vizTitle: "Остовное дерево: Борувка (фазы-«пятiletки»)"
stepNote: "Шаг = одна фаза: каждая компонента ОДНОВЕРХОВА"
code: `n = 9                                # вершин: A...I
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "C", "D"),
         (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "F", "G"),
         (12, "H", "I"), (13, "F", "I")]
comp = {v: v for v in "ABCDEFGH I"}         # DSU: корень компоненты
cheapest = {}                               # корень компоненты -> самое дешёвое ребро
mst = []                                    # рёбра остова
total = 0                                   # вес остова (= 51)
phase = 0                                   # номер фазы («пятiletки») — от 0 до 4
u, v, w = None, None, None                # ребро-кандидат

# --- тело алгоритма ---
def find(x):
    while comp[x] != x:
        comp[x] = comp[comp[x]]
    x = comp[x]
    return x

while len(mst) < n - 1:
    phase += 1
    cheapest = {}
    for (w, a, b) in edges:                 # КАЖДАЯ компонента ОДНОВЕРХОВА
        ra, rb = find(a), find(b)
        if ra == rb:
            continue
        u, v = a, b
        if ra not in cheapest or w < cheapest[ra][0]:
            cheapest[ra] = (w, a, b)
        if rb not in cheapest or w < cheapest[rb][0]:
            cheapest[rb] = (w, a, b)
    if not cheapest:
        break                               # граф несвязен
    for (w, a, b) in cheapest.values():
        ra, rb = find(a), find(b)
        if ra == rb:
            continue                         # две компоненты

```

```

        { name: "k", role: "уровень: блоки  $2^k \times 2^k$  (шаг",
        { name: "st2[{r,c,k,k}]", role: "минимум квадратн
    },
},
"sparse-table#2d-query": {
    vizTitle: "Разреженная таблица 2D: запрос",
    stepNote: "Демонстрация кликабельна вручную: выбери
    code: `# st2[{r, c, kx, ky}] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
    variables: [
        { name: "r1, c1, r2, c2", role: "углы запрашиваем
        { name: "kx, ky", role: "уровень самого крупного
    ],
},
"prefix-sums-2d#2d": {
    vizTitle: "Префиксные суммы 2D (прямоугольники)",
    stepNote: "Демонстрация кликабельна вручную: наведи
    code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
    S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`
    variables: [
        { name: "i, j", role: "текущая ячейка таблицы S (

```

```
-----
const keep: string[] = [];
for (const line of code.split("\n")) {
  const t = line.trim();
  if (t === "# --- тело алгоритма ---" || /^(for|while)/.test(t)) {
    keep.push(line);
  }
}
// убрать пустые строки с конца
while (keep.length > 0 && keep[keep.length - 1].trim() === "") {
  keep.pop();
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, chapterTitle: string, sync: Sync) {
  const sync = getPageSync(chapterId, demoId);

  const head = sync.vizTitle
    ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#`
    : `# «${chapterTitle}»\n# демо на странице нет – св

  if (!sync.code) return head;
  const init = extractInit(sync.code);
  return `${head}\n${init}\n`;
}
-----
```

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

=====

ФАЙЛ 26/111: src/data/tickets/ticket1_segtree.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 217 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from '../../../types';
```

```

span class="font-mono text-emerald-300">[0, 1
тям <span class="font-mono text-emerald-300
– это одиночные элементы массива. В узле л
чество единиц, И/ИЛИ – любая операция, кото
>).</p>

```

```

<p class="text-slate-300 text-sm mb-4">Отсю
а каждом уровне встречает один из трёх случ
не пересекается – возвращаем нейтральный э
склеиваем. Частичных пересечений на уровне
одного элемента – спуск к листу и пересчёт

```

```

<div class="grid grid-cols-1 sm:grid-cols-3
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2
  <div class="bg-slate-800 p-6 rounded-lg
    <div class="absolute -top-3 left-4 l
  <div class="border border-emerald-500 shadow-md">

```

Аналогия 1: Корпоративная пре

```

  </div>
  <p class="text-slate-300 text-sm">Б
икам департаментов: «внутри всем +5 %». Нач
nt-mono text-emerald-300">lazy[v]</span>)</div>
свой баланс. Это и есть <b>отложенное обнов
</div>

```

```
<p class="text-slate-500 text-xs mt-3">
  кивают в строгом порядке: сначала присваива
</div>
```

```
<div class="bg-slate-900/40 rounded-lg border
  <p class="text-slate-300 text-sm mb-2">
  <p class="text-slate-400 text-sm mb-1">
за  $O(1)$ , но строятся один раз и умирают при
ми.</p>
```

```
  <p class="text-slate-400 text-sm mb-1">
идемпотентные запросы (min, gcd, И) по ста
  <p class="text-slate-400 text-sm mb-1">
n ячеек (не  $2n!$ ) – иначе на  $n = 6$  поймать п
n ...  $2n-1$ ] и  $n$  – степень двойки (или использ
  <p class="text-slate-400 text-sm">• <b>
ть лень во всё дерево – это  $O(n)$  и весь смы
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
-2 py-0.5 rounded text-xs font-mono text-em
ся один раз), запрос на отрезке <code class=
-slate-800"> $O(\log n)$ </code>, одиночное обно
-300 border border-slate-800"> $O(\log n)$ </code>
s font-mono text-emerald-300 border border-
xt-xs font-mono text-emerald-300 border bord
ounded text-xs font-mono text-emerald-300 b
```

```
<p class="text-slate-500 text-xs mt-3 leading
  в редакторах и базах, сжатие координат + «
оить», дерево отрезков по дереву (HLD), пер
</p>
```

```
<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```

```
  Код: построение и запрос суммы (C
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```



```

        y[4 * MAXN];           // «обещание»: прибавит

void push(int v, int l, int r) {
    if (lazy[v] == 0) return;
    int m = (l + r) / 2;
    tree[2 * v]      += lazy[v] * (m - l + 1);    // п
    lazy[2 * v]      += lazy[v];
    tree[2 * v + 1] += lazy[v] * (r - m);        // и
    lazy[2 * v + 1] += lazy[v];
    lazy[v] = 0;                                // о
}

void add(int v, int l, int r, int L, int R, int x) { /
    if (r < L || R < l) return;
    if (L <= l && r <= R) {
        tree[v] += x * (r - l + 1);
        lazy[v] += x;                            // д
        return;
    }
    push(v, l, r);                                // о
    int m = (l + r) / 2;
    add(2 * v, l, m, L, R, x);
    add(2 * v + 1, m + 1, r, L, R, x);
    tree[v] = tree[2 * v] + tree[2 * v + 1];
}

```

<p class="text-slate-400 text-xs">Пр
 г накопиться. Забыл push – получил правильн
 </div>
 </details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: итеративное дерево (2n, снизу)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 # для удобств

```
</section>
```

```
`  
,  
  },  
];
```

ФАЙЛ 27/111: src/data/tickets/ticket14_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТР0К: 1235 | РАЗМЕР: 10000 Б

```
import { Chapter } from '../types';
```

```
export const tickets14to20: Chapter[] = [  
  {
```

```
    id: "dijkstra",  
    title: "14. Графы. Поиск кратчайшего пути. Дейкстра",  
    type: "html",  
    category: "Графы. Пути",  
    content: `
```

```
<section id="dijkstra-content" class="mb-12 scroll-mt-12">  
  <div class="flex items-center mb-6 flex-wrap gap-3">  
    <span class="bg-blue-600 text-white px-4 py-1 rounded">14. Графы. Поиск кратчайшего пути. Дейкстра</span>  
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">14. Графы. Поиск кратчайшего пути. Дейкстра</h2>  
  </div>
```

```
  <div class="space-y-8">  
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">  
      <h3 class="text-xl font-bold text-emerald-400">14.1. Поиск кратчайшего пути. Дейкстра</h3>  
  
      <div class="bg-slate-900 p-4 rounded-lg mb-4">  
        <p class="text-lg text-emerald-300 italic">Дейкстра — алгоритм поиска кратчайшего пути в графе с неотрицательными весами.</p>  
        <p class="text-xl font-mono text-white">Дейкстра</p>  
      </div>
```

```
      <p class="text-slate-300 text-sm mb-4">Дейкстра — алгоритм поиска кратчайшего пути в графе с неотрицательными весами.</p>  
      <p>Дейкстра — алгоритм поиска кратчайшего пути в графе с неотрицательными весами. Он работает следующим образом: выбирается начальная вершина, и для нее устанавливается расстояние 0. Затем для каждой соседней вершины вычисляется расстояние от начальной вершины до нее. Если это расстояние меньше, чем текущее расстояние, то оно заменяется на новое. Этот процесс повторяется до тех пор, пока не будут обработаны все вершины. В результате получается кратчайший путь от начальной вершины до каждой вершины графа. Требуемый кратчайший путь от начальной вершины до целевой вершины получается, проследив за расстояниями от начальной вершины до каждой вершины графа. Требуемый кратчайший путь от начальной вершины до целевой вершины получается, проследив за расстояниями от начальной вершины до каждой вершины графа. Требуемый кратчайший путь от начальной вершины до целевой вершины получается, проследив за расстояниями от начальной вершины до каждой вершины графа.</p>
```

```
ext-emerald-300">w ≥ 0</span>}, зафиксирова
ию расстояния!</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg
  <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
```

Ограничения: минус ломает жадно

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4"
lass="font-mono text-emerald-300">s→a = 2</
ono text-emerald-300">b→a = -2</span>. Верн
начала вытащит <span class="font-mono text-
o text-emerald-300">3 - 2 = 1</span> придёт
Джонсона (билет 17).</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-blue-400 m
```

```
<p class="text-slate-300 text-sm mb-4">Демон
D, Дом 2 E, ТЦ F, Метро G) и 11 <b>одностор
ршины <span class="font-mono text-emerald-3
(u → v)</span>: переменные <span class="fon
>, <span class="font-mono text-emerald-300"
ают картинку напрямую.</p>
```

```
<ol class="list-decimal pl-5 space-y-2 text
  <li><b>Инициализация.</b> <span class="
text-emerald-300">∞</span>; в кучу кладём <
emerald-300">prev</span> пуст.</li>
```

```
<li><b>Извлечь минимум.</b> Достаём пар
"font-mono text-emerald-300">du</span>. Если
xt-emerald-300">done</span> – пара устарела
```

```
<li><b>Зафиксировать.</b> <span class="
меняется.</li>
```

```
</div>
```

```
<div class="bg-slate-800/70 p-5 rounded-lg">
  <p class="text-sm font-bold text-white">
    <table class="w-full text-xs text-left">
      <thead>
        <tr class="text-slate-500 border">
          <th class="py-1.5 pr-2 font">
          <th class="py-1.5 pr-2 font">
          <th class="py-1.5 font-bold">
        </tr>
      </thead>
      <tbody class="text-slate-300">
        <tr class="border-b border-slate-
y-2 pr-2">лучший известный путь  $s \rightarrow v$ </td><td>
        <tr class="border-b border-slate-
y-2 pr-2">предок  $v$  на текущем лучшем пути</td><td>
        <tr class="border-b border-slate-
lass="py-2 pr-2">вершины с окончательным  $dist[v]$ </td></tr>
        <tr class="align-top"><td class="py-2">исходная вершина, вершина)</td><td class="py-2">извлечение вершины с минимальным  $dist[v]$ </td></tr>
      </tbody>
    </table>
  </div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border">
  <h3 class="text-xl font-bold text-indigo-400">Извлечение вершины с минимальным  $dist[v]$ 

  <p class="text-slate-300 text-sm mb-4"><b>Инициализация.</b> Для каждой вершины  $v \in V$  извлекается из кучи, <span class="font-mono text-emerald-300"> $s \rightarrow u$ </span>. Отсюда <span class="text-emerald-300"> $dist[v] = \delta(s, v)$ </span> для всех достижимых вершин  $v$  и <span class="text-emerald-300"> $dist[v] = \infty$ </span> для недостижимых вершин  $v$ .</p>
```

```
<details class="bg-slate-900/40 rounded-lg">
  <summary class="p-4 font-bold text-slate-300">Алгоритм Дейкстры</summary>
  <div class="p-4 text-slate-300">
    <p>Алгоритм Дейкстры находит кратчайшие пути от источника  $s$  к каждой вершине  $v \in V$ .</p>
    <p>Алгоритм работает следующим образом:</p>
    <ol>
      <li>Инициализация.</li>
      <li>Извлечение вершины с минимальным  $dist[v]$ .</li>
      <li>Обновление расстояний.</li>
    </ol>
  </div>
</details>
```

```

        </div>
    </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-slate-800">
    <h3 class="text-xl font-bold text-emerald-400">Дейкстра на графе
    <div class="overflow-x-auto my-4">
        <table class="w-full text-xs text-left">
            <thead>
                <tr class="text-slate-500 border-b">
                    <th class="py-2 pr-3 font-b">
                    <th class="py-2 pr-3 font-b">
                    <th class="py-2 font-bold">
                </tr>
            </thead>
            <tbody class="text-slate-300">
                <tr class="border-b border-slate-800">
                    <td class="py-2 pr-3 font-mono">priority_queue (C++)</td><td class="py-2 pr-3 font-mono">й выбор</td></tr>
                <tr class="border-b border-slate-800">
                    <td class="py-2 pr-3 font-mono">ease-key</td><td class="py-2 pr-3 font-mono">очереди</td></tr>
                <tr class="border-b border-slate-800">
                    <td class="py-2 pr-3 font-mono">поиск минимума</td><td class="py-2 pr-3 font-mono"></td></tr>
                <tr class="border-b border-slate-800">
                    <td class="py-2 pr-3 font-mono">><td class="py-2 pr-3 font-mono">0(V + E)</td></tr>
                <tr class="align-top">
                    <td class="py-2 pr-3 font-mono">t-mono">0(E + V · C)</td><td class="py-2">ц
                </tr>
            </tbody>
        </table>
    </div>

<details class="bg-slate-900/40 rounded-lg border-2 border-slate-800">
    <summary class="p-4 font-bold text-slate-400 border-b border-slate-700/50">» Код: Дейкстра на
    <div class="p-5 text-sm text-slate-300"

```

```

    "E": [{"F", 1}],
    "F": [],
    "G": [{"D", 1}, {"E", 2}],
}
dist, prev = dijkstra(g, "A")
print(dist) # A 0, B 3, C 2, D 6, E 7,
print(restore(prev, "A", "F")) # ['A', 'C', 'G', 'E',
    <p class="text-slate-400 text-xs">П
        же вершины, потому что читает ваши <span c
nt-mono">done</span>.</p>
    </div>
</details>

    <details class="bg-slate-900/40 rounded-lg l
        <summary class="p-4 font-bold text-slate
-b border-slate-700/50">> Код: C++ (priority
    <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
    t;
#include <queue>;
using namespace std;
const long long INF = 4e18; // не INT_MAX:

int n;
vector<vector<pair<int,int>>>> g;
vector<long long> d;
vector<int> p;

void dijkstra(int s) {
    d.assign(n, INF); p.assign(n, -1); d[s] = 0;
    priority_queue<pair<long long,int>;,
        vector<pair<long long,int>>&
        greater<pair<long long,int>>;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du > d[u]) continue; // устарев
        for (auto [v, w] : g[u]) {

```



```

<div class="grid grid-cols-1 sm:grid-cols-3">
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
  </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      Аналогия 1: Патруль Бэтмена на м
    </div>
    <div class="my-3 rounded-lg overflow">
      все ули
овно <b>1 выходной в году</b>, когда он отдр
-ю ночь Бэтмен делает контрольный проезд: е
о-то района снова уменьшилось – значит, Джо
</div>

```

```

  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
      Отрицательный цикл: временная п
    </div>

```



```

    </table>
    <p class="text-slate-400 text-xs mt-3">
бном» порядке: <span class="font-mono">S→A,
порядок рёбер местами – и понадобится до ше
</div>

<div class="bg-slate-800/70 p-5 rounded-lg"
    <p class="text-sm font-bold text-white"
    <p class="text-slate-300 text-sm mb-3">
тава A</span> имеет суммарный вес <span cla
веет» на 2.</p>
    <table class="w-full text-xs text-left"
        <thead>
            <tr class="text-slate-500 borde
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 font-bold
            </tr>
        </thead>
        <tbody class="text-slate-300">
            <tr class="border-b border-slate
-2 font-mono">0</td><td class="py-2 pr-2 fo
td></tr>
            <tr class="border-b border-slate
-2 font-mono">-2</td><td class="py-2 pr-2 f
/td></tr>
            <tr class="border-b border-slate
-2 font-mono">-6</td><td class="py-2 pr-2 f
</td></tr>
            <tr class="border-b border-slate
"py-2 pr-2 font-mono">-10</td><td class="py
nt-mono">-2</td></tr>
            <tr class="align-top"><td class
12</td><td class="py-2 pr-2 font-mono">-9</
ont-bold">всё ещё уменьшается → отрицательны
        </tbody>

```

```

<div class="overflow-x-auto">
  <table class="w-full text-xs text-l
    <thead>
      <tr class="text-slate-500 b
        <th class="py-1.5 pr-2
        <th class="py-1.5 pr-2
        <th class="py-1.5 pr-2
        <th class="py-1.5 font-l
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-
2">одна вершина → все</td><td class="py-2 p
      <tr class="border-b border-
font-mono">0((V+E) log V)</td><td class="p
      <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">нет</td><td c
    </tr>
      <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">молча ошибётс
emerald-300">обнаруживает (d[i][i] &lt; 0)<
      <tr class="border-b border-
ass="py-2 pr-2">список смежности</td><td cl
      <tr class="align-top"><td c
φ, веса ≥ 0</td><td class="py-2 pr-2">минус
    </tbody>
  </table>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-emerald-4

  <div class="bg-slate-900/60 p-4 rounded-lg l
    <p class="text-xs font-bold text-amber-:
    :</p>
  <pre class="bg-slate-950 p-3 rounded text-xs font-mono

```

```

if changed_vertex == -1:
    return dist, prev, None # отрицательны

# changed_vertex улучшен на n-й итерации:
# поднимаемся по prev на n шагов – гарантированно з
y = changed_vertex
for _ in range(n):
    y = prev[y]

cycle = [y]
v = prev[y]
while v != y:
    cycle.append(v)
    v = prev[v]
cycle.append(y)
cycle.reverse()
return dist, prev, cycle

# демо-граф страницы (9 рёбер, режим с отрицательным ци
nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [{"S", "A", 4}, {"S", "B", 5}, {"B", "C", -2}, {"A", "C", 1},
      {"C", "D", 3}, {"C", "E", 4}, {"D", "F", 2}, {"E", "F", 1}]
E = [(idx[u], idx[v], w) for u, v, w in E]

dist, prev, cyc = bellman_ford(len(nodes), E + [(idx["D",
print([dist[i] for i in range(len(nodes))]) # [0, 4,
print(cyc) # None –

dist2, _, cyc2 = bellman_ford(len(nodes), E + [(idx["D",
print([nodes[i] for i in cyc2]) # ['D',
print([dist2[i] for i in range(len(nodes))]) # все
    <p class="text-slate-400 text-xs">П
        имая вершина с <span class="font-mono">INF<
        ля <span class="font-mono">float('inf')</sp
    </div>
</details>

```

```

        vector<int> cycle;
        for (int v = y;; v = p[v]) {
            cycle.push_back(v);
            if (v == y && cycle.size() > 1) break;
        }
        reverse(cycle.begin(), cycle.end());
        return cycle;
    }

```

```

int main() {
    int s = 0;
    int x = bellman_ford(s);
    if (x == -1) cout <<< "отрицательных циклов не  

    else {
        cout <<< "цикл: ";
        for (int v : negative_cycle(x)) cout <<< v  

        cout <<< "\n";
    }
}
</code></pre>

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg

<summary class="p-4 font-bold text-slate

-b border-slate-700/50">» Оптимизации: ранни

<div class="p-5 text-sm text-slate-300

```

<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
    rt deque

```

```

def spfa(n, adj, s):
    """adj[u] = [(v, w), ...]. Релаксируем только «живые»
    INF = float("inf")
    dist = [INF] * n
    in_queue = [False] * n
    counter = [0] * n # сколько раз вершина
    dist[s] = 0
    q = deque([s])
    in_queue[s] = True

```

```

отрицательным циклом <span class="font-mono" style="font-family: monospace;">
    <li><b>Переполнение INF.</b> <span class="font-mono" style="font-family: monospace;">
        INT_MAX</span> превращается в отрицательное значение, что приводит к переполнению
        запасом и <span class="font-mono" style="font-family: monospace;">long long</span>
    <li><b>Неориентированный граф с минимальным циклом</b> <span class="font-mono" style="font-family: monospace;">-3</span> – это цикл длины 3 в
        отрицательном цикле. Минусы в неориентированном графе.
    <li><b>Цикл есть, но не от старта.</b> <span class="font-mono" style="font-family: monospace;">-3</span>
</span>. Нужна проверка всего графа – добавление ребра (17) и запустите от неё.</li>
    <li><b>«Ответ  $-\infty$ ».</b> Если отрицательный цикл: <span class="font-mono" style="font-family: monospace;">dist[v] = -\infty</span>: печатать ответ
        ходом в обратном графе.</li>
    <li><b>SPFA как «быстрый Форд–Беллман»</b> в условии <span class="font-mono" style="font-family: monospace;">V, E \leq 10^5</span>
</ul>
</div>

<p class="text-slate-300 text-sm mt-4"><b>Сложные задачи</b>
    потенциалы <span class="font-mono text-emerald-500" style="font-family: monospace;"></span>
    Дейкстры. Флойд–Уоршелл (16) решает все задачи на кратчайшие пути.
    порядок для DAG – билет 9.</p>
</div>
</div>
</section>`
},
{
    id: "floyd",
    title: "16. Графы. Поиск кратчайшего пути. Флойд",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
                16. Графы. Поиск кратчайшего пути. Флойд
            </h2>
        </span>
    </div>
`

```

```

        Суть фокуса (по шагам)
    </div>
    <p class="text-slate-300 text-sm mb-4 lg:mb-6">
        Главный герой – внешний цикл
        <span class="font-mono text-emerald-300">k</span> т.е.
        </span>, если в качестве промежуточных
        , ..., k}</span>?». Каждый шаг добавляет в ра
    </p>
    <ul class="text-sm text-slate-300 space-y-4">
        <li><b>Шаг k = -1 (старт):</b> разрешены пер
        <li><b>Шаг k = 0:</b> разрешаем пер
        <span class="font-mono text-emerald-300">i → 0 → j</span>, чем текущий <span class="font-mono text-emerald-300">i
        <li><b>Шаг k = 1:</b> разрешены пер
        <span class="font-mono text-emerald-300">l → j</span> уже может стоять
        <li><b>Шаг k = V - 1:</b> разрешены
    </ul>
</div>

<div class="grid grid-cols-1 sm:grid-cols-3 lg:grid-cols-4">
    <div class="bg-slate-900 border border-emerald-500 shadow-md">
        <p class="text-[10px] uppercase tracking-wider">Шаг k = -1 (старт):</p>
        <p class="font-mono text-emerald-300">i → 0 → j</p>
    </div>
    <div class="bg-slate-900 border border-emerald-500 shadow-md">
        <p class="text-[10px] uppercase tracking-wider">Шаг k = 0:</p>
        <p class="font-mono text-emerald-300">i → 0 → j</p>
    </div>
    <div class="bg-slate-900 border border-emerald-500 shadow-md">
        <p class="text-[10px] uppercase tracking-wider">Шаг k = 1:</p>
        <p class="font-mono text-emerald-300">l → j</p>
    </div>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Таблица пересадок в

```

```

                <th class="py-1.5 pr-2 font-
                <th class="py-1.5 font-bold
            </tr>
        </thead>
        <tbody class="text-slate-300">
            <tr class="border-b border-slate-
-2 font-mono">0</td><td class="py-2">в вершн
            <tr class="border-b border-slate-
-2 font-mono">4</td><td class="py-2">открыва
            <tr class="border-b border-slate-
-2 font-mono">3</td><td class="py-2">2 – вх
            <tr class="border-b border-slate-
-2 font-mono">2</td><td class="py-2">3 – вх
            <tr class="border-b border-slate-
-2 font-mono">4</td><td class="py-2">через
            <tr class="border-b border-slate-
-2 font-mono">0</td><td class="py-2">из 5 в
            <tr class="align-top"><td class="
300">10</td><td class="py-2">ребро 6 → 1 за
        </tbody>
    </table>
    <p class="text-slate-400 text-xs mt-3">
mono text-emerald-300">0 → {0, 4, 7, 2, 6,
"font-mono">0 → 3 → 4 → 6</span> (2 + 4 + 1
всего в финальной матрице 11 таких пар. Диа
</div>

<div class="bg-slate-800/70 p-5 rounded-lg
    <p class="text-sm font-bold text-white
    <table class="w-full text-xs text-left
        <thead>
            <tr class="text-slate-500 borde
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 pr-2 font
                <th class="py-1.5 font-bold
            </tr>
        </thead>
        <tbody class="text-slate-300">

```

```

- mono">d[k][k] < 0 и рассуждение
</div>
</details>

```

```

<details class="bg-slate-900/40 rounded-lg
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">> Эксперимент: пере
  <div class="p-5 text-sm text-slate-300

```

```

<pre class="bg-slate-950 p-4 rounded overflow-x-auto te

```

```

M = [[0,4,INF,2,INF,INF,INF],
      [INF,0,3,INF,INF,3,INF],
      [INF,INF,0,INF,2,INF,INF],
      [INF,INF,INF,0,4,2,INF],
      [INF,INF,INF,INF,0,INF,1],
      [INF,INF,INF,INF,INF,0,5],
      [INF,1,INF,INF,INF,INF,0]]

```

```

def floyd(M, order="kij"):
    n = len(M)
    d = [row[:] for row in M]
    for a in range(n):
        for b in range(n):
            for c in range(n):
                k, i, j = {"kij": (a, b, c),
                           "ijk": (c, a, b),
                           "ikj": (c, a, b)}[order]
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
    return d

```

```

good = floyd(M, "kij")
bad = floyd(M, "ijk")
print(good[2][1], bad[2][1])    # 4 и 999 – путь 2→4→6→1
print(sum(1 for i in range(7) for j in range(7) if good
# 4 клетки отличаются; при порядке (i,k,j) их 6
<p class="text-slate-400 text-xs">В
угих «почти работают» (большинство клеток с
</div>

```



```

        return d, nxt, negative

def path(nxt, i, j):
    if nxt[i][j] == -1:
        return None # j недостижим
    p = [i]
    while p[-1] != j:
        p.append(nxt[p[-1]][j])
    return p

nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [ ("S","A",4), ("S","B",5), ("B","C",-2), ("A","C",1),
      ("C","D",3), ("C","E",4), ("D","F",2), ("E","F",1) ]
E = [(idx[u], idx[v], w) for u, v, w in E]

d, nxt, neg = floyd_warshall(len(nodes), E)
print([d[0][j] for j in range(len(nodes))]) # [0, 4, 5, 3, 2, 1]
print(path(nxt, idx["S"], idx["F"])) # [0, 2, 1]
print(neg) # [] — отрицательного цикла нет

```

</div>
</details>

```

<details class="bg-slate-900/40 rounded-lg p-4">
  <summary class="p-4 font-bold text-slate-300">
    -b border-slate-700/50">» Код: C++ (матрица
    <div class="p-5 text-sm text-slate-300">
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
    t;
#include <algorithm>
using namespace std;
const long long INF = 4e18;

int n;
vector<vector<long long>>> d(n, vector<long long>(n, INF));

void floyd() {
    for (int i = 0; i < n; ++i) d[i][i] = 0;

```

```

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">» Та же динамика дл
    <div class="p-5 text-sm text-slate-300"
      <table class="w-full text-xs text-l
        <thead>
          <tr class="text-slate-500 b
            <th class="py-1.5 pr-2
            <th class="py-1.5 pr-2
            <th class="py-1.5 font-l
          </tr>
        </thead>
        <tbody class="text-slate-300">
          <tr class="border-b border-
имость)</td><td class="py-2 pr-2 font-mono">
ь ребро; r[i][i] = true</td></tr>
          <tr class="border-b border-
пути)</td><td class="py-2 pr-2 font-mono">d
= w(i,j); d[i][i] = 0</td></tr>
          <tr class="border-b border-
ой сети</td><td class="py-2 pr-2 font-mono">
и отсутствии положительных циклов</td></tr>
          <tr class="align-top"><td c
o">if d[i][k]+d[k][j] == d[i][j]: cnt[i][j]
а переполнением</td></tr>
        </tbody>
      </table>
      <p class="text-slate-400 text-xs mt
> при <span class="font-mono">r[i][k]</span>
V³)</span>.</p>
    </div>
  </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-amber-400

  <div class="bg-rose-950/30 p-5 rounded-lg b

```

```

title: "17. Графы. Поиск кратчайшего пути. Ускорени
type: "html",
description:
  "Джонсон (перевзвешивание потенциалов), встречный
category: "Графы. Пути",
content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-amber-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-amber-300 italic">
        <p class="text-xl font-mono text-white">
        е посещённых вершин (встречный поиск, ранний
        contraction hierarchies).</p>
      </div>

      <p class="text-slate-300 text-sm mb-4">Дейк
      в лоб. На экзамене спрашивают, <b>что сдела
      строки перевзвешивания» до промышленных у

      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Такси с навигатором
          </div>
          <p class="text-slate-300 text-sm mb-4">
            ы одинаково. А* — тот же таксист, но с нави
            /span> прибавляется оценка остатка пути по
            т туда, где сумма меньше. Навигатор обязан
            /span>): занижать можно, завышать нельзя —
            0">h = 0</span> это снова Дейкстра.</p>
          </div>

```

```

        лько 0 и 1</td></tr>
        <tr class="border-b border-slate-200">
<td class="py-2 pr-3">куча → массив корзинок<td class="py-2 pr-3">C небольшое</td></tr>
        <tr class="border-b border-slate-200">
<td class="py-2 pr-3">decrease-key за  $O(1)$  <td class="py-2 pr-3">теоретически; на практике двоичная/d-ар<td class="py-2 pr-3">
        <tr class="border-b border-slate-200">
<td class="py-2 pr-3">не релаксируем все рё<td class="py-2 pr-3">
, на практике быстро</td><td class="py-2 pr-3">о<td class="py-2 pr-3">
        <tr class="align-top"><td class="py-2 pr-3">3">переносит работу в предобработку</td><td class="py-2 pr-3">
миллионы запросов (карты)</td></tr>
    </tbody>
</table>
</div>

```

```

<h3 class="text-lg font-bold text-white mt-6">
<div class="bg-slate-900 p-4 rounded-lg mb-6">
    <p class="text-xl font-mono text-white">
</div>
<div class="grid grid-cols-1 gap-6 mb-6">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-amber-500 shadow-md">
            ☞ Суть перевзвешивания
        </div>
        <p class="text-slate-300 text-sm mb-6">
            Дейкстра (билет 14) быстрая, но не оптимальный. Алгоритм Джонсона делает финт: пере-
            ь теми же.
        </p>
        <ol class="text-sm text-slate-300 space-y-2">
            <li>Добавляем <b>фиктивную вершину</b>
            <li>Запускаем от s <b>Форда-Беллмана</b>
            </ol>
            <p class="text-slate-300 text-sm">важный факт:</i> Джонсон НЕ «удаляет» отрица-
            кой и прекращает работу, так как при отрица-
            <li>Перевзвешиваем: <span class="text-amber-500 font-weight-normal">

```

оторых расстояние только что уменьшилось

ческого прогона, но асимптотика та же, $O(V \cdot E)$.
пользуют осторожно, с эвристиками SLF/LLL).
) уменьшает высоту и на практике быстрее, к

Навигация по графу

в **предобработку**. Сначала – **сжатие** индексами $0 \dots N$, и уже по ним строят граф и сетевые потоки. **Contraction hierarchies** (так и называются) и запрос идёт двунаправленным поиском вверх и вниз. В вершине заранее приписывают «метки», и запрос

тро, но не кратчайший. Переоценка = неверный ответ.

именимой к отрицательным весам и даёт все возможные пути.

$\text{in}_f + \text{min}_b \geq \mu$. Остановка по встрече даёт ответ.

ок сломается. Там нужен Dial.

практике меньше бесполезных релаксаций.

Алгоритмы поиска кратчайшего пути

м числом раскрытий, 0-1 BFS $O(V + E)$, Дейкстра с кучей $O(E + V \cdot \log V)$, Dial $O(E + V \cdot C)$, где C – максимальная длина ребра.

```

        for to, w in g[v]:
            if d[s][to] > dist + w:
                d[s][to] = dist + w
                heapq.heappush(heap, (d[s][to], to))
    for v in range(n):
        if d[s][v] < INF:
            d[s][v] = d[s][v] - h[s] + h[v]    # откорректировать
    return d

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg p-4">

<summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">

Код: встречный Дейкстра, A*, 0-1

</summary>

<div class="p-5 text-sm text-slate-300">

<pre class="bg-slate-950 p-4 rounded">

встречный Дейкстра: остановка по min_f + min_g

def bidirectional(g, rg, s, t): # rg – граф с перестройкой

INF = float("inf")

df = {s: 0}; db = {t: 0}

qf = [(0, s)]; qb = [(0, t)]

mu = INF

while qf and qb:

if qf[0][0] + qb[0][0] >= mu: # вот правый

return mu

dist, v = heapq.heappop(qf)

if dist > df.get(v, INF): continue

for to, w in g[v]:

nd = dist + w

if nd < df.get(to, INF):

df[to] = nd; heapq.heappush(qf, (nd, to))

if to in db:

mu = min(mu, nd + db[to]) # встреча:

dist, v = heapq.heappop(qb)

if dist > db.get(v, INF): continue

for to, w in rg[v]:

```

        (dq.appendleft(to) if w == 0 else dq.append(to))
    return d</pre>
</div>
</details>

<details class="bg-slate-900/40 rounded-lg border-1
  <summary class="p-4 font-bold text-slate-300
    -b border-slate-700/50">
    Вопросы, которые любят задавать (и на которые
  </summary>
  <div class="p-5 text-sm text-slate-300">
    <p>• Почему после перевзвешивания в узлах
    есть условие отсутствия улучшающей релаксации?
    <p>• Почему кратчайшие пути не меняются?
    иш) и одинаков для всех путей между этой парой
    <p>• Что лучше для всех пар: Флойд-Уоршелл или
    g V) выигрывает на разреженных.</p>
    <p>• Какая эвристика A* допустима на графах с
    ии скорости можно делить на максимальную скорость
    <p>• Зачем нужен ранний выход в Дейкстре?
    смысла нет.</p>
    <p>• Почему 0-1 BFS работает без кучи?
    же отсортирована.</p>
  </div>
</details>
</div>
</div>
</section>`
  },
];

```

ФАЙЛ 28/111: src/data/tickets/ticket18_20_mst.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 231 | РАЗМЕР: 1.2 KB

```
import { Chapter } from '../../../types';
```

MST. Алгоритмы отличаются только тем, **к** какой букве принадлежит ребро. Прим – «выросшее дерево против остального графа».

```
<div class="bg-slate-800 p-5 rounded-lg border">
  <h4 class="text-amber-400 font-bold text-sm">Пример
  <p class="text-slate-200 text-sm font-sans">
    <p class="text-slate-300 text-sm mb-3">
      Если удалить букву Р – остаётся остов.</p>
      <p class="text-slate-300 text-sm">Остов – это связный
      подграф: минимум костей, чтобы всё держалось.
    </p>
  </div>
```

Второй вариант – алгоритм Крускала. Он никогда не входит в MST. Именно его использовали в цикле (рёбра-то мы берём по возрастанию веса).

```
<div class="overflow-x-auto my-6">
  <table class="w-full text-xs text-left">
    <thead>
      <tr class="text-slate-500 border-b">
        <th class="py-2 pr-3 font-bold">№
        <th class="py-2 pr-3 font-bold">Веса
        <th class="py-2 pr-3 font-bold">Вид
        <th class="py-2 pr-3 font-bold">Статус
        <th class="py-2 font-bold">Действие
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate-700">
        <td class="py-2 pr-3 font-bold">1
        <td class="py-2 pr-3">Рёбра
        <td class="py-2 pr-3 font-medium">
          (1,2) (1,3) (1,4) (2,3) (2,4) (3,4)
        <td class="py-2 pr-3 font-medium">
          (1,2) (1,3) (1,4) (2,3) (2,4) (3,4)
        <td class="py-2">рёбра даны
      </tr>
      <tr class="border-b border-slate-700">
        <td class="py-2 pr-3 font-bold">2
        <td class="py-2 pr-3">Растёр
```



```
<div class="absolute -top-3 left-4 l
r border-rose-500 shadow-md">
```

Аналогия к билету 20 • Боровк

```
</div>
```

```
<p class="text-slate-300 text-sm">С
айшему чужому племени: к вечеру компонент в,
<b>log2 V</b> дней образуется единая федер
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-900/40 rounded-lg bord
```

```
<p class="text-slate-300 text-sm mb-2">
```

```
<p class="text-slate-400 text-sm mb-1">
```

состояние от одной вершины до каждой; MST ми
куче другой: у Прима – вес ребра к дереву,
тся.</p>

```
<p class="text-slate-400 text-sm mb-1">
```

вных весах деревьев может быть несколько, н

```
<p class="text-slate-400 text-sm mb-1">
```

дом вместо почти-константного find/union.</p>

```
<p class="text-slate-400 text-sm mb-1">
```

и Краскал построил минимальный остовный <i>

```
<p class="text-slate-400 text-sm">• <b>P
```

ин раз, DSU спасает от дубля (find(u) == fi
</div>

```
<p class="text-slate-300 text-sm mt-4"><b>с
py-0.5 rounded text-xs font-mono text-emera
```

бесплатный); Прим с двоичной кучей <code>с
rder-slate-800">O(E log V)</code>, на плотн
text-xs font-mono text-emerald-300 border b
d text-xs font-mono text-emerald-300 border
ex <code class="bg-slate-950 px-2 py-0.5 ro
</p>

```
<details class="bg-slate-900/40 rounded-lg l
```

```
<summary class="p-4 font-bold text-slate
```

```
-b border-slate-700/50">
```

```

key = [INF] * n          # минимальный вес ребра, ведущ
used = [False] * n
key[0] = 0
heap = [(0, 0)]          # (вес ребра, вершина) ← вот
total = 0

```

```

while heap:
    w, v = heapq.heappop(heap)
    if used[v]:
        continue        # устаревшая запись в куче
    used[v] = True
    total += w
    for to, weight in graph[v]:
        if not used[to] and weight < key[to]:
            key[to] = weight
            heapq.heappush(heap, (weight, to))
print(total)

```

(накапливаем путь от старта), здесь
 а – другая задача.

Код 20: Боровка (Скрыто)

Код 20: Боровка (Скрыто)

```

def find(v):
    while parent[v] != v:
        parent[v] = parent[parent[v]]
        v = parent[v]
    return v

comps = n

```

```
        <p>• Что если граф несвязный? — все  
    <p>  
        <p>• Как найти второе по величине о  
ребро на образовавшемся цикле (LCA + максим  
    </div>  
    </details>  
  </div>  
</div>  
</section>`,  
  },  
];
```

ФАЙЛ 29/111: src/data/tickets/ticket21_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 444 | РАЗМ

```
import { Chapter } from '../../../types';  
  
export const tickets21to24: Chapter[] = [  
  {  
    id: "string-kmp",  
    title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",  
    type: "html",  
    category: "Строки",  
    content: `  
<section id="string-kmp" class="mb-12 scroll-mt-10">  
  <div class="flex items-center mb-6 flex-wrap gap-3">  
    <span class="bg-indigo-600 text-white px-4 py-1">  
    <h2 class="text-2xl sm:text-3xl font-bold text-  
  </div>  
  <div class="space-y-8">  
    <div class="bg-slate-700/50 p-6 rounded-xl bord  
      <h3 class="text-xl font-bold text-blue-400 r  
      <div class="bg-slate-900 p-4 rounded-lg mb-  
        <p class="text-lg text-blue-300 italic r  
        <p class="text-xl font-mono text-white">
```

```

<p>Тот же пример: <b>abcabcd</b></p>
<ul class="list-disc pl-5 space-y-2">
  <li><b>a</b>: Префиксов нет. <code>
  <li><b>ab</b>: Из начала ("a")
>
  <li><b>abc</b>: Опять мимо. <code>
  <li><b>abca</b>: 0! Начало ("a"
</code></li>
  <li><b>abcab</b>: Начало ("ab")
/li>
  <li><b>abcabc</b>: Начало ("abc
e></li>
  <li><b>abcabcd</b>: «d» всё исп
последняя буква). Длина 3: <b>abc</b> проти
digo-300">π[6] = 0</code></li>
</ul>
</div>
</details>
<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
  Код C++ (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300
  <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
  int j = pi[i-1];
  while (j > 0 && s[i] != s[j]) j = pi[j-1];
  if (s[i] == s[j]) j++;
  pi[i] = j;
}</pre>
</div>
</details>

<h3 class="text-lg font-bold text-white mt-
<div class="bg-slate-900 p-4 rounded-lg mb-
  <p class="text-xl font-mono text-white">

```



```

        </p>
    </div>
</div>

```

```

<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

Пример вычисления (Скрыто)

```

    </summary>
    <div class="p-5 text-sm text-slate-300
        <p>Тот же пример: <b>abcabcd</b></p>
        <ul class="list-disc pl-5 space-y-2
            <li><code>Z[0]</code> – не счита
            <li><code>Z[1]</code> (<b>bcabcd
[1] = 0</code></li>
            <li><code>Z[2]</code> (<b>cabcd
</li>
            <li><code>Z[3]</code> (<b>abcd<
а дальше s[6]='d' против s[3]='a' – мимо.
            <li><code>Z[4], Z[5], Z[6]</code>
        </ul>
    </div>
</details>

```

```

<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

Код C++ (Скрыто)

```

    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounde
int l = 0, r = 0;
for (int i = 1; i &lt; n; i++) {
    if (i &lt;= r) z[i] = min(r - i + 1, z[i - 1]);
    while (i + z[i] &lt; n && s[z[i]] == s[i + z[i]]) z
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
    </div>
</details>

```

```

    <div class="bg-slate-900 p-6 rounded-lg border-2"
      <div class="absolute -top-3 left-4 bg-rose-900
        -500 shadow-md">
        Аналогия 2: Матёшка (Терминальные)
      </div>
      <p class="text-slate-300 text-sm mb-4">Представ
        нашли и "he", потому что оно спрятано внутри
      </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg border b
      <summary class="p-4 font-bold text-slate-400 ou
        r-slate-700/50">
        Код (Скрыто)
      </summary>
      <div class="p-5 text-sm text-slate-300 space-y-
        <pre class="bg-slate-950 p-4 rounded overfl

int get_link(int v) {
  if (t[v].link == -1) {
    if (v == 0 || t[v].p == 0) t[v].link = 0;
    else t[v].link = go(get_link(t[v].p), t[v].pch)
  }
  return t[v].link;
}

int go(int v, char c) {
  if (t[v].go[c] == -1) {
    if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
    else t[v].go[c] = v == 0 ? 0 : go(get_link(v),
  }
  return t[v].go[c];
}</pre>
    </div>
  </details>
</div>
</section>`
},

```

```

    </div>
    <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #4b4b9b; border-radius: 10px; padding: 10px;">
      <div class="absolute -top-3 left-4 right-4 bottom-4 border border-purple-500 shadow-md" style="border: 1px solid #8080ff; border-radius: 5px; padding: 5px;">
        Аналогия 2: Переводчик (сведение)
      </div>
      <p class="text-slate-300 text-sm">Сведение с русского на английский (сведение за полином) и друг-англичанин. Отсюда главный инструмент: если к В сводится А, то А не стоит.</p>
    </div>
  </div>

  <h3 class="text-lg font-bold text-white mt-4">Иерархия сложности
  <p class="text-slate-300 text-sm mb-3">Известно, что сложность в цепочке доказывається теоремой об иерархии сложности.
  <div class="bg-slate-950 border border-slate-800 padding-4" style="background-color: #1a202c; border: 1px solid #2d3748; padding: 10px;">
    <p class="font-mono text-sm sm:text-base">
       $P \subseteq NP \subseteq PSPACE \subseteq EXP \subseteq NEXP$   

      <span class="text-fuchsia-300">NP-полнота (не доказано)</span>
    </p>
  </div>
  <div class="overflow-x-auto">
    <table class="w-full text-xs text-left" style="width: 100%; text-align: left;">
      <thead>
        <tr class="text-slate-500 border-b" style="border-bottom: 1px solid #4b4b9b;">
          <th class="py-2 pr-3 font-bold">Сложность
          <th class="py-2 pr-3 font-bold">Описание
          <th class="py-2 font-bold">Примечания
        </tr>
      </thead>
      <tbody class="text-slate-300">
        <tr class="border-b border-slate-800" style="border-bottom: 1px solid #2d3748;">
          <td class="py-2 pr-3">детерминированно за  $O(n^k)$ 
          <td class="py-2 pr-3">удольности, максимальный поток
        </tr>
        <tr class="border-b border-slate-800" style="border-bottom: 1px solid #2d3748;">
          <td class="py-2 pr-3">сертификат проверяется за полином
        </tr>
      </tbody>
    </table>
  </div>

```



```

        ke.</li>
        <li><b>Параметризованная сложность
        n).</li>
        <li><b>Специальные классы входов</b>
полиномиальными (независимое множество на д
        <li><b>Готовые решатели</b>: SAT/ILP
всего алгоритма.</li>
    </ul>
</div>

```

```

<div class="bg-slate-900/40 rounded-lg border
    <p class="text-slate-300 text-sm mb-2">
    <p class="text-slate-400 text-sm mb-1">
NP: любая задача из P автоматически в NP.
    <p class="text-slate-400 text-sm mb-1">
становки – неразрешима, зато NP-трудна). NP
    <p class="text-slate-400 text-sm mb-1">
стной трудной A <b>к</b> B ( $A \leq_p B$ ), а не н
    <p class="text-slate-400 text-sm mb-1">
    <p class="text-slate-400 text-sm mb-1">
дит по времени на  $n = 40$ . NP-полнота запрещ
    <p class="text-slate-400 text-sm">• <b>
ит RSA. «Раз она в NP, значит NP-полная» –
</div>

```

```

<p class="text-slate-300 text-sm mt-4"><b>с</b>
ass="bg-slate-950 px-2 py-0.5 rounded text-
еребор сертификатов <code class="bg-slate-9
^poly(n))</code>; сведение обязано работать
0 border border-slate-800">0(poly(n))</code>

```

```

<details class="bg-slate-900/40 rounded-lg
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

Код: верификатор сертификата и пр

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
<pre class="bg-slate-950 p-4 rounded
```

```

        if col[to] == -1:
            col[to] = col[v] ^ 1
            dq.append(to)
        elif col[to] == col[v]:
            return False
    return True</pre>

```

```

        <p class="text-slate-400 text-xs">0
товый ответ, но не находить его; для задачи
        </div>
</details>

```

```

<details class="bg-slate-900/40 rounded-lg
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

```

        Вопросы, которые любят задавать (
    </summary>

```

```

    <div class="p-5 text-sm text-slate-300
        <p>• Дайте определение NP через вер
нтны.</p>
        <p>• Что такое NP-полная задача и к
    </p>

```

```

        <p>• Почему первая NP-полная задача
        <p>• Если  $P = NP$ , что станет с крип
их задач исчезнет граница approximability)<
        <p>• Пример задачи в NP и co-NP, пр
– затем AKS доказал  $P$ ></p>
        <p>• Чем NP-трудная задача отличаетс
облема остановки)</p>
    </div>
</details>

```

```

        </div>
    </div>
</section>`
    }
};

```

```
        <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            Аналогия: Списки = Контакты
        </div>
        <p class="text-slate-300 text-sm mb-1">
            т. Оптимально для почти всех задач.</p>
        </div>
    </div>
</div>

{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
    <h3 class="text-xl font-bold text-indigo-400">DFS vs BFS</h3>

    <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <!-- Аналогия DFS -->
        <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-indigo-500 shadow-md">
                ♂ DFS (Поиск в глубину) = 0
            </div>
            <p class="text-slate-300 text-sm mb-1">
                <b>Что делает:</b> Поиск в глубину — это алгоритм, который
                да попадает в тупик — делает шаг назад по следу.
            </p>
            <ul class="text-xs text-indigo-300">
                <li><b>Где используется:</b></li>
                <li>Топологическая сортировка (линейное время)</li>
                <li>Поиск мостов и точек сочленения</li>
                <li>Компоненты сильной связности</li>
                <li>Детектирование циклов (через DFS)</li>
            </ul>
        </div>

        <!-- Аналогия BFS -->
        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
```

```
vis[start_node] = true;
```

```
while (!q.empty()) {
    int v = q.front(); q.pop();
    for (int u : adj[v]) {
        if (!vis[u]) {
            vis[u] = true;
            q.push(u);
        }
    }
}
```

```
</pre>
</div>
</details>
</div>
</div>
```

```
</section>`,
```

```
{,
```

```
{
    id: "graph-planar-colors",
    title: "7. Графы. Планарные. Покраска",
    type: "html",
    description: "Планарные графы: формула Эйлера, K5 и",
    category: "Графы. Теория",
    content: `
```

```
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">7</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">Графы. Планарные. Покраска</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">Планарные графы</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Планарный граф — граф, который можно нарисовать в плоскости так, чтобы никакие две его ребра не пересекались.</p>
        <p class="text-2xl font-mono text-white">Графы, которые можно нарисовать в плоскости так, чтобы никакие две его ребра не пересекались, называются планарными.</p>
        <p class="text-xs text-slate-400 mt-1">Планарные графы имеют много интересных свойств. Например, для любого планарного графа справедлива формула Эйлера:</p>
      </div>
    </div>
  </div>
</div>
```

```

<div class="space-y-3 text-slate-300 text-slate-300">
  <p><b>Сложность задачи раскраски:</b></p>
  <ul class="list-disc list-inside space-y-3">
    <li><b> $k = 2$  (2-раскрашиваемость):</b> Обычный BFS/DFS обходом.</li>
    <li><b> $k \geq 3$  (k-colorability):</b> NP-полная задача.</li>
  </ul>

  <p class="mt-3"><b>Теоремы о раскраске:</b></p>
  <ul class="list-disc list-inside space-y-3">
    <li><b>Жадная раскраска:</b> дает вершину максимальной степени вершины.</li>
    <li><b>Теорема Брукса:</b> Для простых графов  $K_n$  и нечётных циклов  $C_{2k+1}$ .</li>
    <li><b>Теорема о 4 красках:</b> Любые плоские графы раскрашиваются в 4 цвета.</li>
  </ul>
</div>
</div>
</div>
</section>`,
  },
  {
    id: "graph-components",
    title: "8. Графы. Компоненты связности",
    type: "html",
    description:
      "Компоненты связности: обходами DFS/BFS за  $O(V + E)$ .",
    category: "Графы. База",
    content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">8. Графы. Компоненты связности</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">8. Графы. Компоненты связности</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <p>Компоненты связности графа — это максимальные по включению подграфы, в которых любые две вершины соединены путём из вершин этого подграфа.</p>
    </div>
  </div>
</section>
`
  }
]

```

```

<div class="overflow-x-auto">
  <table class="w-full text-xs text-left">
    <thead>
      <tr class="text-slate-500 border-b">
        <th class="py-2 pr-3 font-b">
        <th class="py-2 pr-3 font-b">
        <th class="py-2 pr-3 font-b">
        <th class="py-2 font-bold">
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate-200">
        <td class="py-2 pr-3">забыли про стрелку
        <td class="py-2 font-mono">0(V +
        <tr class="border-b border-slate-200">
          <td class="py-2 pr-3">(SCC)
          <td class="py-2 pr-3">из любой в
        <td class="py-2 font-mono">0(V + E)
        <tr class="align-top">
          <td class="py-2 pr-3">до кого дойдёт старт
          <td class="py-2 pr-3">V + E)
        </tr>
      </tbody>
    </table>
  </div>
  <p class="text-slate-400 text-xs mt-3">Клас
  ультат SCC. Обход из s даёт только вершины,
</div>
</div>
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Топологическая сортировка ДАГ и детект
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

        <br>* <span class="font-mono text-amber-500">
        <br>* <span class="font-mono text-emerald-500">
        <br>Если во время обхода из серы вершины
и <b>гарантированный цикл</b>!
    </p>
</div>

```

```

<details class="bg-slate-900/40 rounded-lg border-1
    <summary class="p-4 font-bold text-slate-300
    -b border-slate-700/50">
    Корректный код C++ с детекцией цикла
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounded"

```

```

vector<int> adj[MAXN];
int color[MAXN]; // 0=белый, 1=серый (в стеке), 2=чёрный
vector<int> ans;
bool has_cycle = false;

```

```

void dfs(int v) {
    color[v] = 1; // Зашли – вершина серая
    for (int to : adj[v]) {
        if (color[to] == 0) {
            dfs(to);
            if (has_cycle) return;
        } else if (color[to] == 1) {
            // Переход в серую вершину = НАЙДЕН ЦИКЛ!
            has_cycle = true;
            return;
        }
    }
    color[v] = 2; // Вышли – вершина чёрная
    ans.push_back(v); // Сохраняем в момент ВЫХОДА (post-order)
}

```

```

bool topological_sort(int n) {
    fill(color, color + n + 1, 0);
    has_cycle = false;

```

```

    <p class="text-slate-300 text-sm mb-4">
      <strong>«Это просто цикл?» – Нет!</strong>
    </p>

    <details class="bg-slate-900/40 rounded-lg" >
      <summary class="p-4 font-bold text-slate-300 border-slate-700/50">
        Код: Алгоритм Косарайю (Скрыто)
      </summary>
      <div class="p-5 text-sm text-slate-300">
        <p class="text-emerald-400 font-bold">
          <ol class="list-decimal list-inside">
            <li>Запускаем DFS на исходном графе
            </li>
            <li>Разворачиваем этот список (и делаем его <b>не</b> топологическая сортировка самого графа AG конденсации.</li>
            <li>Переворачиваем все ребра в графе
            <li>Идем по <code>order</code>: делаем тот запуск – образуют очередную SCC!</li>
          </ol>
        </div>
      </details>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-bridges",
    title: "11. Графы. Мосты",
    type: "html",
    description: "Рёбра, удаление которых увеличивает количество компонент связности графа",
    category: "Графы. Продвинутое",
    content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">

```



```

    }
  }</pre>
</div>
</details>
</div>
</div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров путь и цикл",
    type: "html",
    description: "Условия существования и построение Эйлерова",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">13. Графы. Эйлеров путь и цикл
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
      <h3 class="text-xl font-bold text-indigo-400">Условия существования
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">Теорема 13.1 (Эйлера). Пусть  $G$  — связный граф. Тогда в  $G$  существует
        <p class="text-base sm:text-lg font-monospace">Эйлеров цикл тогда и только тогда, когда каждая вершина имеет чётную степень.
      </p>
      </div>
      <div class="bg-slate-900 p-4 rounded-lg">
        <div class="grid grid-cols-1 md:grid-cols-2 gap-4">
          <div class="p-4">
            <!-- Неориентированные графы -->
            <div class="bg-slate-800 p-5 rounded-lg">
              <p class="text-indigo-300 font-bold">Теорема 13.2 (Эйлера). Пусть  $G$  — связный граф. Тогда в  $G$  существует
              <ul class="list-disc list-inside space-y-2">
                <li><b>Эйлеров цикл:</b> связан с условием  $\deg[v] \equiv 0 \pmod{2}$  для всех вершин  $v$ .</li>
                <li><b>Эйлеров путь:</b> связан с условием, что граф связен и не более чем одна вершина имеет нечётную степень.
              </ul>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```
        path.push_back(v); // Добавляем в путь на выходе!
    }</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
];
```

РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

ФАЙЛ 31/111: src/components/ArticulationPointsViz.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizNumber } from '../data/vizS
import { Play, Pause, RotateCcw, Info } from "lucide-re
```

```
interface Node {
  id: number;
  x: number;
  y: number;
  label: string;
}
```

```
interface Edge {
  u: number;
  v: number;
}
```

```
const nodes: Node[] = [
  { id: 0, x: 250, y: 50, label: "0" },
```

```

-----
export const ArticulationPointsViz: React.FC = () => {
  const [steps, setSteps] = useState<StepInfo[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  useVizStepSync(currentStepIndex, setCurrentStepIndex, () => {
    const v = vizNumber(vars.v);
    if (v === null) return null;
    return steps.map((step, index) => ({ step, index }));
  });
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const newSteps: StepInfo[] = [];
    let timer = 0;
    const visited = new Set<number>();
    const ap = new Set<number>();
    const currentTin = Array(nodes.length).fill(-1);
    const currentLow = Array(nodes.length).fill(-1);

    const recordStep = (desc: string, u: number | null) => {
      newSteps.push({
        desc,
        visited: new Set(visited),
        ap: new Set(ap),
        currentNode: u,
        tin: [...currentTin],
        low: [...currentLow],
      });
    };

    recordStep("Начало DFS (Тарьян) для поиска точек соarticulation", null);

    const dfs = (v: number, p: number = -1) => {
      visited.add(v);
      currentTin[v] = currentLow[v] = timer++;
      let children = 0;

      recordStep(`Заходим в вершину ${v}. tin[${v}]=${currentTin[v]}`, v);
    };
  }, []);
};

```

```

        v,
      });
    }
  };

  for (let i = 0; i < nodes.length; i++) {
    if (!visited.has(i)) {
      dfs(i);
    }
  }

  recordStep("Алгоритм завершен. Найдены все точки со
  setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
  desc: "",
  visited: new Set(),
  ap: new Set(),
  currentNode: null,
  tin: [],
  low: [],
};

useEffect(() => {
  let interval: NodeJS.Timeout;
  if (isPlaying && currentStepIndex < steps.length -
    interval = setInterval(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {

```

```

<div className="grid grid-cols-1 lg:grid-cols-3 g
  <div className="lg:col-span-2 relative h-96 lg:l
    <svg className="w-full h-full" viewBox="0 0 8
      {/* Edges */}
      {edges.map((edge, i) => {
        const u = nodes.find((n) => n.id === edge
        const v = nodes.find((n) => n.id === edge

        const isBridge =
          (edge.u === 3 && edge.v === 4) ||
          (edge.u === 4 && edge.v === 3);

        return (
          <line
            key={"edge-" + i}
            x1={u.x}
            y1={u.y}
            x2={v.x}
            y2={v.y}
            stroke={isBridge ? "#f43f5e" : "#475568"}
            strokeWidth={isBridge ? "4" : "2"}
            strokeDasharray={isBridge ? "8,4" : "1"}
          />
        );
      })}

      {/* Nodes */}
      {nodes.map((node) => {
        const isCurrent = currentStep.currentNode
        const isVisited = currentStep.visited.has
        const isAp = currentStep.ap.has(node.id);

        const fill = isCurrent
          ? "#3b82f6"
          : isAp
            ? "#e11d48"
            : isVisited
              ? "#10b981"

```

```

        y={node.y - r - 15}
        textAnchor="middle"
        fill="#94a3b8"
        fontSize="10px"
      >
        tin:{" "}
        {currentStep.tin[node.id] >= 0
          ? currentStep.tin[node.id]
          : "-"}
      </text>
      <text
        x={node.x}
        y={node.y + r + 20}
        textAnchor="middle"
        fill="#94a3b8"
        fontSize="10px"
      >
        low:{" "}
        {currentStep.low[node.id] >= 0
          ? currentStep.low[node.id]
          : "-"}
      </text>
    </>
  })
</g>
);
}}
</svg>
</div>

```

```

<div className="bg-slate-950 p-6 border-t lg:bo
  <h4 className="text-sm font-bold text-slate-4
    Статус алгоритма
  </h4>

```

```

<div className="bg-slate-900 border border-sl
  <p className="text-white font-medium min-h-
    {currentStep.desc}

```

```

                {step.desc}
            </div>
        )}}
    </div>
</div>
<div className="mt-4 shrink-0">
  <div className="w-full bg-slate-800 h-2 rounded">
    <div
      className="bg-rose-500 h-full transition"
      style={{
        width: `${(currentStepIndex / Math.max(
          />
        </div>
      </div>
    </div>
  </div>
</div>
);
};

```

ФАЙЛ 32/111: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord, vizString } from 'viz';
import { LiveDistTable } from './LiveDistTable';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  iteration: number;
  checkingEdge: { u: string; v: string; w: number } | null;
  distances: { [key: string]: number };
}

```



```

    if (cycle) {
      return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
    } else {
      return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
    }
  };

  const edges = getEdges(hasCycle);

  const getAdjList = (ed: Edge[]) => {
    const list: { [key: string]: { v: string; w: number } } = {};
    nodes.forEach(n => (list[n.id] = []));
    ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
    return list;
  };
  const adjList = getAdjList(edges);

  const codeLines = [
    { line: 1, text: 'function bellman_ford(graph, start)' },
    { line: 2, text: '  dist = {v: ∞}; dist[start] = 0;' },
    { line: 3, text: '  for i from 1 to V - 1: // |V| - 1 iterations' },
    { line: 4, text: '    for (u, v, weight) in E:' },
    { line: 5, text: '      if dist[u] + weight < dist[v]' },
    { line: 6, text: '        dist[v] = dist[u] + weight;' },
    { line: 7, text: '  for (u, v, weight) in E: // Проверка на отрицательный цикл' },
    { line: 8, text: '    if dist[u] + weight < dist[v]' },
    { line: 9, text: '      return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ"' },
    { line: 10, text: '  return dist' },
  ];

  useEffect(() => {
    const simulationSteps: Step[] = [];
    let dist: { [key: string]: number } = { S: 0, A: 99 };
    let prev: { [key: string]: string | null } = { S: null };

    simulationSteps.push({
      iteration: 0, checkingEdge: null,
      distances: { ...dist }, previous: { ...prev },
    });
  });

```

```

    if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
      let cycleFound = false;
      for (const edge of edges) {
        if (dist[edge.u] !== 999 && dist[edge.u] + edge.weight < dist[edge.v]) {
          cycleFound = true;
          simulationSteps.push({
            iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
            distances: { ...dist }, previous: { ...prevDist },
            log: `    ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v}`,
            activeCodeLine: 9,
          });
          break;
        }
      }
    }

    if (!cycleFound) {
      simulationSteps.push({
        iteration: vCount, checkingEdge: null,
        distances: { ...dist }, previous: { ...prevDist },
        log: '    Проверка завершена. Ни одно ребро не попало в цикл.',
        activeCodeLine: 10,
      });
    }
  }

  setSteps(simulationSteps);
  setCurrentStepIndex(0);
  setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
}, [isPlaying, currentStepIndex, steps.length]);

```



```

    {nodes.map((node) => {
      const dist = currentStep.distances[node.id]
      const isCheckingNode = currentStep.checkingNode ===
        node.id);

      return (
        <g key={node.id} className="transition-
          <circle cx={node.x} cy={node.y} r={is
        ckingNode ? '#7dd3fc' : '#64748b'} strokeWid
          <text x={node.x} y={node.y + 5} fill=
          <text x={node.x} y={node.y - 28} fill
        ight="bold" textAnchor="middle" className="
          <text x={node.x} y={node.y + 35} fill
        split(' ')[0]}</text>
          </g>
        );
      })}
    </svg>

    <div className="w-full bg-slate-950/80 p-4 ro
      <p className="font-semibold text-amber-400
      <p className="min-h-[40px] flex items-cente
    </div>
  </div>

  {/* Список смежности */}
  <div className="w-full lg:w-1/3 bg-emerald-950/
    <h4 className="text-lg font-bold text-emerald
    <div className="space-y-2 text-sm font-mono t
      {Object.keys(adjList).map(u => {
        <div key={u} className="flex flex-wrap g
        -colors">
          <span className="font-bold text-white
            {adjList[u].map((edge, idx) => {
              const isChk = currentStep.checkingE
              return (
                <span key={idx} className={`px-2

```

```
      {codeLines.map(item => (  
        <div key={item.line} className={`py-1.5  
e === item.line ? 'bg-blue-900/80 text-amber  
        <span className="text-slate-600 w-5 s  
        <span className="whitespace-pre flex-  
        </div>  
      )}}  
    </div>  
  </div>  
</div>  
</div>  
</div>  
);  
}
```

ФАЙЛ 33/111: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect, useCallback } from 'react'  
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react'
```

```
const NODES = [  
  { id: 0, label: '0', x: 300, y: 50 },  
  { id: 1, label: '1', x: 150, y: 150 },  
  { id: 2, label: '2', x: 450, y: 150 },  
  { id: 3, label: '3', x: 75, y: 250 },  
  { id: 4, label: '4', x: 225, y: 250 },  
  { id: 5, label: '5', x: 375, y: 250 },  
  { id: 6, label: '6', x: 525, y: 250 },  
];
```

```
const EDGES = [  
  { from: 0, to: 1 }, { from: 0, to: 2 },  
  { from: 1, to: 3 }, { from: 1, to: 4 },  
  { from: 2, to: 5 }, { from: 2, to: 6 }  
];
```

```
while (queue.length > 0) {
  steps.push({
    activeLine: 4,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: null,
    logs: `Очередь не пуста, элементов: ${queue.length}
  });
```

```
const curr = queue.shift()!;
steps.push({
  activeLine: 5,
  queue: [...queue],
  visited: Array.from(visited),
  currentNode: curr,
  logs: `Достаем из очереди (${curr}).`
});
```

```
const neighbors = ADJ[curr];
if (neighbors.length > 0) {
  for (const n of neighbors) {
    if (!visited.has(n)) {
      steps.push({
        activeLine: 7,
        queue: [...queue],
        visited: Array.from(visited),
        currentNode: curr,
        logs: `Рассматриваем соседа: ${n}. Он еще не
      });
```

```
visited.add(n);
queue.push(n);
```

```
steps.push({
  activeLine: 9,
  queue: [...queue],
  visited: Array.from(visited),
  currentNode: curr,
```

```

// @ts-expect-error зарезервировано под кнопку «назад»
const handlePrev = useCallback(() => {
  setStepIdx(prev => Math.max(prev - 1, 0));
}, []);

useEffect(() => {
  if (!isPlaying) return;
  if (stepIdx >= STEPS.length - 1) {
    setIsPlaying(false);
    return;
  }
  const timer = setTimeout(handleNext, 1200);
  return () => clearTimeout(timer);
}, [isPlaying, stepIdx, handleNext]);

return (
  <div className="bg-slate-900 border-2 border-slate-100" >
    <div className="flex flex-col md:flex-row items-center justify-between" >
      <div>
        <h3 className="text-2xl font-bold text-white" >
          <span className="text-blue-400" > </span> Ал
        </h3>
        <p className="text-slate-400 text-sm mt-1">Ви
      </div>
      <div className="flex bg-slate-800 p-1.5 rounded" >
        <button
          onClick={() => { setIsPlaying(false); setSt
          className="p-2 text-slate-400 hover:text-wh
          title="Сброс"
        >
          <RotateCcw className="w-5 h-5" />
        </button>
        <button
          onClick={() => setIsPlaying(p => !p)}
          className="p-2 text-blue-400 hover:text-bl
          title={isPlaying ? 'Пауза' : 'Воспроизведен
        >
        {isPlaying ? <Pause className="w-5 h-5" />

```

```

let fill = '#1e293b'; // slate-800
let stroke = '#475569'; // slate-600
let scale = 1;
void scale;

if (isCurrent) {
  fill = '#3b82f6'; // blue-500
  stroke = '#60a5fa'; // blue-400
  scale = 1.3;
} else if (isInQueue) {
  fill = '#059669'; // emerald-600
  stroke = '#34d399'; // emerald-400
  scale = 1.1;
} else if (isVisited) {
  fill = '#0f172a'; // slate-900
  stroke = '#3b82f6'; // blue-500
}

return (
  <g key={n.id} className="transition-tran
    {isCurrent && {
      <circle cx={n.x} cy={n.y} r="30" fi
    }}
    <circle
      cx={n.x} cy={n.y}
      r="20"
      fill={fill}
      stroke={stroke}
      strokeWidth="3"
      className="transition-colors durati
  />
  <text
    x={n.x} y={n.y}
    textAnchor="middle"
    dy=".3em"
    className="fill-white font-bold tex
  >

```



```

        </div>
      </div>

      <div className="bg-slate-800/80 border border
        <div className="text-sm font-bold text-eme
          <span>Состояние Очереди (Queue):</span>
          <span className="text-xs text-slate-500
        </div>
        <div className="flex items-center gap-2 mi
          {step.queue.length === 0 ? (
            <span className="text-slate-500 text-x
          ) : (
            step.queue.map(({qId, i}) => {
              <span key={`${qId}-${i}`} className=
            enter justify-center rounded-md shadow-lg s
              {qId}
            </span>
          ))
        )}
      </div>
      <div className="mt-2 text-sm text-amber-300
        ms-center justify-center">
        "{step.logs}"
      </div>
    </div>
  </div>
</div>
</div>
);
}

```

ФАЙЛ 34/111: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import dijkstraImg from '../assets/dijkstra-freakazoid.
```

```
    });  
  }  
}
```

ФАЙЛ 35/111: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";  
import { ChevronLeft, ChevronRight } from "lucide-react";  
import type { Chapter } from "../types";
```

```
interface Props {  
  prev?: Chapter;  
  next?: Chapter;  
  index: number;  
  total: number;  
  onGo: (id: string) => void;  
  compact?: boolean;  
}
```

```
/**
```

```
 * Переходы «предыдущая / следующая тема» – как на CodeSandbox
```

```
 * Компактный вариант живёт в шапке главы, крупный – по умолчанию
```

```
 */
```

```
export const ChapterNav: React.FC<Props> = ({ prev, next, onGo, compact }) => {  
  if (compact) {  
    return (  
      <div className="flex items-center gap-2 text-xs">  
        <button  
          disabled={!prev}  
          onClick={() => prev && onGo(prev.id)}  
          title={prev ? prev.title : "Это первая тема"}  
          className="flex items-center gap-1 px-2.5 py-1.5  
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50">  
          <ChevronLeft className="w-4 h-4" />  
        <span>←</span>  
        <button  
          disabled={!next}  
          onClick={() => next && onGo(next.id)}  
          title={next ? next.title : "Это последняя тема"}  
          className="flex items-center gap-1 px-2.5 py-1.5  
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50">  
          <ChevronRight className="w-4 h-4" />  
        <span>→</span>  
      </div>  
    );  
  }  
  return (  
    <div className="flex items-center gap-2 text-sm">  
      <button  
        disabled={!prev}  
        onClick={() => prev && onGo(prev.id)}  
        title={prev ? prev.title : "Это первая тема"}  
        className="flex items-center gap-1 px-3 py-2  
          :bg-slate-700 enabled:hover:text-white disabled:opacity-50">  
        <ChevronLeft className="w-5 h-5" />  
        <span>←</span>  
      <button  
        disabled={!next}  
        onClick={() => next && onGo(next.id)}  
        title={next ? next.title : "Это последняя тема"}  
        className="flex items-center gap-1 px-3 py-2  
          :bg-slate-700 enabled:hover:text-white disabled:opacity-50">  
        <ChevronRight className="w-5 h-5" />  
        <span>→</span>  
      </div>  
    );  
  }  
};
```

```
      <button
        onClick={() => onGo(next.id)}
        className="group text-right p-4 rounded-xl bg-
          transition-all sm:col-start-2"
      >
        <div className="flex items-center justify-end
          text-indigo-400 mb-1">
          Следующая тема <ChevronRight className="w-3
        </div>
        <div className="text-sm font-bold text-slate-1
      </button>
    })
  </nav>
);
};
```

ФАЙЛ 36/111: src/components/ChapterQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИ

```
import React, { useEffect, useState } from "react";
import { ArrowRight, CheckCircle2, RotateCcw, XCircle,
import { quizzes } from "../data/quizzes";
```

```
interface Props {
  chapterId: string;
}
```

```
/**
```

```
 * Блиц по прочитанной теме: 3–5 коротких вопросов с не
 *
```

```
 * Вопросы живут в `src/data/quizzes.ts` (по id главы).
```

```
 * компонент-викторина существовали, но не были подклю
```

```
 * контент был, а добраться до него было нельзя. Теперь
```

```
 * демонстрации: прочитал → посмотрел, как работает → пр
```

```
 *
```

```

const restart = () => {
  setIdx(0);
  setPicked(null);
  setScore(0);
  setFinished(false);
};

return (
  <section aria-label="Блиц по теме" className="mt-8">
    <div className="bg-slate-700/50 p-5 rounded-xl border">
      <div className="flex items-center justify-between">
        <h3 className="flex items-center gap-2 text-white">
          <Zap className="w-4 h-4 text-indigo-400" />
          Блиц: {list.length} вопроса по этой теме
        </h3>
        <span className="text-[11px] font-mono text-white">
          {finished ? `итог ${score}/${list.length}` : ''}
        </span>
      </div>

      {finished ? (
        <div className="text-center py-3">
          <p className="text-sm text-slate-300 mb-1">
            {score === list.length
              ? "Все верно: тему можно закрывать."
              : score * 2 >= list.length
                ? "Половина и больше верно – стоит пересмотреть."
                : "Стоит вернуться к тексту страницы:"
            }
          </p>
          <p className="text-2xl font-bold text-white">
            {score} / {list.length}
          </p>
          <button
            type="button"
            onClick={restart}
            className="inline-flex items-center gap-1 font-medium font-sans text-white text-sm font-bold text-indigo-200 hover:bg-indigo-500/20"
          >
            Пересмотреть
          </button>
        </div>
      ) : (
        <div className="text-center py-3">
          <p className="text-sm text-slate-300 mb-1">
            {score === list.length
              ? "Все верно: тему можно закрывать."
              : score * 2 >= list.length
                ? "Половина и больше верно – стоит пересмотреть."
                : "Стоит вернуться к тексту страницы:"
            }
          </p>
          <p className="text-2xl font-bold text-white">
            {score} / {list.length}
          </p>
          <button
            type="button"
            onClick={restart}
            className="inline-flex items-center gap-1 font-medium font-sans text-white text-sm font-bold text-indigo-200 hover:bg-indigo-500/20"
          >
            Пересмотреть
          </button>
        </div>
      )}
    </div>
  </section>
);

```

```
        {correct ? "Верно" : "Неверно"}
      </p>
      <p className="text-[13px] leading-relaxed">
    </div>
  )}

  <div className="mt-3 flex justify-end">
    <button
      type="button"
      onClick={next}
      disabled={!answered}
      className="inline-flex items-center gap-2 font-medium text-indigo-200 transition-colors hover:text-indigo-500/15"
    >
      {idx + 1 < list.length ? "Следующий вопрос" : "Завершить"}
      <ArrowRight className="w-3.5 h-3.5" />
    </button>
  </div>
</>
  )}
</div>
</section>
);
};

export default ChapterQuiz;
```

ФАЙЛ 37/111: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, FileText, Loader2, Maximize2, Minimize2 } from "lucide-react";
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
```

```

const cancelHide = useCallback(() => {
  if (hideTimer.current) {
    window.clearTimeout(hideTimer.current);
    hideTimer.current = null;
  }
}, []);

const scheduleHide = useCallback(() => {
  cancelHide();
  hideTimer.current = window.setTimeout(() => setAnchor(
), [cancelHide]);

useEffect(() => {
  setAnchor(null);
  setSaving("idle");
}, [chapter.id]);

/**
 * Печать/«сохранить в PDF» этой темы: раскрываем все
 * чтобы на бумагу попал весь текст (доказательства,
 * а после печати возвращаем их в исходное состояние.
 */
useEffect(() => {
  const each = () => Array.from(contentRef.current?.q
  const openAll = () => {
    each().forEach((d) => {
      d.dataset.wasOpen = d.open ? "1" : "0";
      d.open = true;
    });
  };
  const restore = () => {
    each().forEach((d) => {
      if (d.dataset.wasOpen === "0") d.open = false;
      delete d.dataset.wasOpen;
    });
  };
  window.addEventListener("beforeprint", openAll);
  window.addEventListener("afterprint", restore);

```

```

const handleClick = (e: React.MouseEvent) => {
  // ссылка на другой билет: переходим внутри приложения
  const link = (e.target as HTMLElement).closest<HTMLAnchorElement>('a')
  if (link) {
    e.preventDefault();
    onGo(link.dataset.goto ?? "");
    return;
  }
  const el = (e.target as HTMLElement).closest<HTMLAnchorElement>('a')
  if (!el) return;
  e.preventDefault();
  if (anchor && anchor.termId === el.dataset.termId)
    else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
  const el = (e.target as HTMLElement).closest<HTMLAnchorElement>('a')
  if (el) open(el);
};

const viz = getViz(chapter.id);

return (
  <>
    <article className="chapter-body">
      {/* панель загрузки: сохранить именно эту страницу */}
      <div className="flex flex-wrap items-center gap-2">
        <button
          onClick={saveHtml}
          disabled={saving === "work"}
          title="Сохранить эту тему одним автономным файлом"
          className="flex items-center gap-1.5 text-x-small border border-gray-300 hover:text-white hover:border-emerald-500"
        >
          {saving === "work" ? (
            <Loader2 className="w-3.5 h-3.5 animate-spin" />
          ) : saving === "done" ? (

```

```

{chapter.id === "intro" && {
  <div className="my-8 print:hidden">
    <h3 className="text-xl font-bold text-white">
      <span> </span> Мнемонические карточки кра
    </h3>
    <MnemonicCards />
  </div>
}}

[["dijkstra", "bellman-ford", "floyd"].includes
  <div className="my-6 print:hidden">
    <ChapterImage vizType={chapter.id} />
  </div>
})

<p className="mt-6 flex items-start gap-2 text-
ed-lg px-3 py-2 print:hidden">
  <MousePointerClick className="w-4 h-4 shrink-1">
    <span>
      Подчёркнутые термины – интерактивные: навед
      объяснение на пальцах, мини-анимацию и кноп
    </span>
  </p>

{viz && {
  <section id="chapter-viz" className="mt-8 scr
    <div className="flex items-center justify-b
      <h3 className="flex items-center gap-2 te
        <Sparkles className="w-4 h-4 text-indigo
          Демонстрация: {viz.title}
        </h3>
        <div className="flex items-center gap-2">
          <button
            onClick={() => onOpenSimulator(chapte
              className="flex items-center gap-1.5
            slate-300 hover:text-white hover:border-ind
              >

```



```

        />
      </>
    );
  };
};

```

ФАЙЛ 38/111: src/components/ComplexityViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useContext, useEffect, useMemo, useState } from 'react';
import { ChevronLeft, ChevronRight, Play, Pause, Rotate } from 'react-feather';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizStep } from 'viz';

```

```
/**
```

```
 * Билет 24 – «Классы сложности, сведение задач».
```

```
 *
```

```
 * Четыре вкладки:
```

```
 * 1. «Иерархия» –  $P \subseteq NP \subseteq PSPACE \subseteq EXP$  и NP-полные
```

```
 * 2. «Сведение» –  $A \leq_p B$  по шагам: преобразовал вход
```

```
 * 3. «Верификатор» – проверка сертификата для 3-SAT по
```

```
 * 4. «Рост» – живой график по переменным n и ops
```

```
 * сколько операций насчитал твой алгоритм
```

```
 * похоже (полином или экспонента).
```

```
 *
```

```
 * Синхронизация с компилятором (PAGE_SYNC["complexity-"]
```

```
 * cls – имя класса (подсветит нужный овал), step/i –
```

```
 * k – номер проверяемого дизъюнкта, cert/assign – сер
```

```
 * n и ops – точки на графике роста, ok/ans – вердикт
```

```
 */
```

```
type Tab = "hierarchy" | "reduce" | "verify" | "growth"
```

```
const TABS: { id: Tab; label: string; icon: React.ReactElement } =
```

```
  { id: "hierarchy", label: "1 · Иерархия классов", icon: <ChevronLeft /> },
```

```
  { id: "reduce", label: "2 · Сведение  $A \leq_p B$ ", icon: <ChevronRight /> },
```

```
  { id: "verify", label: "3 · Верификатор (NP)", icon: <Play /> },
```

```
-----
const BOX_LABEL: Record<string, { t: string; c: string
  A: { t: "задача A\n(NP-полная)", c: "#f43f5e" },
  x: { t: "вход x\n|x| = n", c: "#64748b" },
  f: { t: "f: сведение\nO(poly(n))", c: "#f59e0b" },
  y: { t: "вход y\n|y| = poly(n)", c: "#64748b" },
  B: { t: "решатель B\n(гипотеза: быстрый)", c: "#6366f1" },
  ans: { t: "ответ\nA(x) = B(f(x))", c: "#10b981" },
};
```

```
/* ----- Верификатор -----
```

```
/** 3-SAT формула: 4 дизъюнкта по 3 литерала, переменные
```

```
const CLAUSES: [number, number, number][] = [
  [1, 2, -3],
  [-1, 3, 4],
  [2, -3, -4],
  [-2, 1, 3],
];
```

```
/** Сертификат, который проверяем: x = (1, 0, 1, 0) — ok
```

```
const CERT = [1, 0, 1, 0];
```

```
const VERIFY_STEPS = CLAUSES.map((cl, i) => ({
  title: `Дизъюнкт ${i + 1}: (${cl.map((l) => (l > 0 ?
  note: `Подставляем сертификат x = (${CERT.join(", ")}
    .map((l) => (l > 0 ? `x${l}=${CERT[l - 1]}` : `¬x${
    .join(", ")}). ${cl.some((l) => (l > 0 ? CERT[l - 1]
      : "Все литералы ложны — дизъюнкт НЕ выполнен.")`
  ok: cl.some((l) => (l > 0 ? CERT[l - 1] === 1 : CERT[
})));
```

```
/* ----- Рост -----
```

```
interface Sample { n: number; ops: number }
```

```
const GROWTH_LINES = [
  { id: "n", label: "n", fn: (n: number) => n, color: "#f43f5e" },
  { id: "n log n", label: "n · log2 n", fn: (n: number) => n * Math.log2(n), color: "#64748b" },
  { id: "n2", label: "n2", fn: (n: number) => n * n, color: "#6366f1" },
];
```

```
const liveStep = vars ? (vizNumber(vars.step) ?? vizNum
useEffect(() => {
  if (liveStep === null || !Number.isFinite(liveStep))
    setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);
```

```
const liveCls = vars ? vizString(vars.cls) : null;
const liveK = vars ? vizNumber(vars.k) : null;
const liveCert = vars ? vizArray(vars.cert) ?? vizArr
const liveOk = vars ? (vars.ok === undefined ? null :
const liveN = vars ? vizNumber(vars.n) : null;
const liveOps = vars ? (vizNumber(vars.ops) ?? vizNum
```

```
/** Копим точки (n, ops): код пользователя крутит цикл
useEffect(() => {
  if (liveN === null || liveOps === null) return;
  if (!Number.isFinite(liveN) || !Number.isFinite(live
  setSamples((prev) => {
    const last = prev[prev.length - 1];
    if (last && last.n === liveN && last.ops === live
    const withoutN = prev.filter((s) => s.n !== liveN
    return [...withoutN, { n: liveN, ops: liveOps }].
  });
}, [liveN, liveOps]);
```

```
/** Оценка показателя роста по последним точкам (лог-
const verdict = useMemo(() => {
  const pts = samples.filter((s) => s.ops > 0 && s.n > 0);
  if (pts.length < 3) return null;
  const xs = pts.map((p) => Math.log(p.n));
  const ys = pts.map((p) => Math.log(p.ops));
  const mx = xs.reduce((a, b) => a + b, 0) / xs.length;
  const my = ys.reduce((a, b) => a + b, 0) / ys.length;
  let num = 0;
  let den = 0;
  for (let i = 0; i < xs.length; i++) {
    num += (xs[i] - mx) * (ys[i] - my);
    den += (xs[i] - mx) ** 2;
```

```

const allOk = VERIFY_STEPS.every((s) => s.ok);

return (
  <div className="bg-slate-800/90 p-5 sm:p-6 rounded-
    <div className="flex flex-wrap items-center justifi
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-purple-600 text-white
          <Boxes className="w-5 h-5" />
        </div>
        <div>
          <h3 className="text-lg sm:text-xl font-bold
            <p className="text-xs text-purple-300">P · I
          </div>
        </div>
      </div>
    </div>

    <div className="flex flex-wrap gap-2 mb-4">
      {TABS.map((t) => {
        <button
          key={t.id}
          onClick={() => setTab(t.id)}
          className={"flex items-center gap-1.5 px-3 p-1
            bg-purple-600 text-white" : "bg-slate-900/60 text
          >
            {t.icon} {t.label}
          </button>
        </div>
      </div>

      {(tab === "reduce" || tab === "verify") && (
        <div className="flex flex-wrap items-center gap
          <button onClick={() => { setStep(0); setPlayin
            late-300 hover:text-white"><RotateCcw class
          <button onClick={() => { setPlaying(false); s
            lg bg-slate-900 border border-slate-700 tex
            /></button>
          <button onClick={() => setPlaying((p) => !p)}
            text-xs font-bold hover:bg-purple-500">

```

```

        </svg>
      </div>
      <p className="text-xs text-slate-400 leading-4">
        <p className="text-[11px] text-slate-500">
          Подсказка: выполни в компиляторе <code>clas
        </p>
      </div>
    })
  }

```

```

  { /* _____ СВЕДЕНИЕ _____
    {tab === "reduce" && (
      <div className="space-y-3">
        <div className="bg-slate-950 rounded-xl p-4 b
          <div className="flex flex-wrap items-center
            {reduceFrame.bboxes.map((b, i) => {
              const meta = BOX_LABEL[b];
              const active = i === reduceFrame.active
              return (
                <React.Fragment key={b + i}>
                  <div
                    className="rounded-lg border px-3
                    style={{
                      borderColor: active ? meta.c :
                      background: active ? meta.c + "
                      opacity: active ? 1 : 0.45,
                      boxShadow: active ? `0 0 18px $
                      minWidth: 96,
                    }}
                  >
                    <span className="text-[11px] font
                  </div>
                    {i < reduceFrame.bboxes.length - 1 &
                  </React.Fragment>
                );
              })
            })
          </div>
        </div>
      <div className="bg-slate-900/70 border border
    }
  }

```

```

        "}}</span>
        <span className={"text-[11px] font-1
0"}}>
            {checked ? (st.ok ? "✓ истинно" :
            </span>
        </div>
    );
    }}}
</div>
{{(verifyDone || liveOk !== null) && (
    <div className={"mt-3 rounded-lg border p
/30 text-emerald-300" : "border-rose-600/40
    {(liveOk ?? allOk) ? "Сертификат принят
вергнут: хотя бы один дизъюнкт ложен."}
    </div>
    )}
</div>
<div className="bg-slate-900/70 border border
    <p className="text-sm font-bold text-white
    <p className="text-xs text-slate-400 leading
определение NP: короткий сертификат + быстр
</div>
</div>
)}}

{/* _____ ПОСТ _____
{tab === "growth" && (
    <div className="space-y-3">
        <div className="bg-slate-950 rounded-xl p-4 b
        <div className="flex flex-wrap items-center
            <span className="text-[10px] uppercase tr
            <span className="font-mono text-xs bg-sla
            ?? "-"></span>
            <span className="font-mono text-xs bg-sla
e0ps ?? "-"></span>
            <button onClick={() => setSamples([])} cl
0 rounded-lg px-2 py-1">очистить график</bu
        </div>

```

```

    <div className={"rounded-xl border p-3.5 " +
      /60"}>
      <p className="text-sm font-bold text-white"
        {verdict ? `Похоже на ${verdict.name}` :
      </p>
      <p className="text-xs text-slate-400 leading-5"
        {verdict
          ? `Наклон в лог-лог координатах  $\approx$  ${verdict.slopeTail.toFixed(2)} – кривая загибается`
          : "Как только в коде появятся n и ops, появятся c n)."}
        </p>
      </div>
    </div>
  )}

  {/* живые переменные */}
  <div className="mt-3 flex flex-wrap items-center"
    <span className="text-[10px] uppercase tracking-wider"
      {vars ? (
        ["n", "ops", "cls", "k", "cert", "ok"].map((name) => {
          vars[name] === undefined ? null : (
            <span key={name} className="text-[11px] font-mono"
              {name} = {JSON.stringify(vars[name]).slice(0, 10)}
            </span>
          )
        })
      ) : (
        <span className="text-[11px] text-slate-500">
          открой панель Python: переменные <code className="font-mono text-slate-300">k</code>, <code className="font-mono text-slate-300">n</code> управляют этой демонстрацией
        </span>
      )}
    </div>
  </div>
);

```

```

    { line: "}", highlight: "normal" },
];

interface DfsStep {
    currentNode: number | null;
    visitedIndices: number[];
    tin: Record<number, number>;
    low: Record<number, number>;
    stack: number[];
    bridges: string[];
    log: string;
    activeEdge: [number, number] | null;
    backEdge: [number, number] | null;
    activeCodeLine: number[];
}

const GRAPH_NODES = [
    { id: 0, label: "A", x: 100, y: 120 },
    { id: 1, label: "B", x: 260, y: 120 },
    { id: 2, label: "C", x: 180, y: 200 },
    { id: 3, label: "D", x: 180, y: 300 },
    { id: 4, label: "E", x: 100, y: 370 },
    { id: 5, label: "F", x: 260, y: 370 },
    { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
    { u: 0, v: 1, key: "0-1" },
    { u: 1, v: 2, key: "1-2" },
    { u: 2, v: 0, key: "0-2" },
    { u: 2, v: 3, key: "2-3" },
    { u: 3, v: 4, key: "3-4" },
    { u: 4, v: 5, key: "4-5" },
    { u: 5, v: 3, key: "3-5" },
    { u: 1, v: 6, key: "1-6" }
];

const DFS STEPS: DfsStep[] = [

```



```

    activeEdge: null, backEdge: [2,0],
    log: "Видим соседа A (уже посещён)! Обратное ребро
    activeCodeLine: [7,8,9]
},
{
    currentNode: 3, visitedIndices: [0,1,6,2,3],
    tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3:2},
    activeEdge: [2,3], backEdge: null,
    log: "Идём C→D. tin[D]=5, low[D]=5.",
    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 4, visitedIndices: [0,1,6,2,3,4],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2:1,3:2,4:3},
    activeEdge: [3,4], backEdge: null,
    log: "D→E. tin[E]=6, low[E]=6.",
    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:1,3:2,4:3,5:4},
    activeEdge: [4,5], backEdge: null,
    log: "E→F. tin[F]=7, low[F]=7.",
    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:1,3:2,4:3,5:4},
    activeEdge: null, backEdge: [5,3],
    log: "Из F видим D (посещён). Обратное ребро (F,D).",
    activeCodeLine: [7,8,9]
},
{
    currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:1,3:2,4:3,5:4},
    activeEdge: null, backEdge: null,
    log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо
    activeCodeLine: [11,13,16,17]
}

```

```

export function DfsBridgesSimulator() {
  const [dfsIdx, setDfsIdx] = useState(0);
  useVizStepSync(dfsIdx, setDfsIdx, DFS_STEPS.length - 1);
  const rawV = vizString(vars.v) ?? (vizNumber(vars.v) ?? null);
  const rawTo = vizString(vars.to) ?? (vizNumber(vars.to) ?? null);
  const toId = (value: string | null) => {
    if (value === null) return null;
    const byLabel = GRAPH_NODES.find((node) => node.label === value);
    return byLabel ?? (Number.isFinite(Number(value)) ? Number(value) : null);
  };
  const v = toId(rawV);
  const to = toId(rawTo);
  if (v === null) return null;
  const candidates = DFS_STEPS
    .map((step, index) => ({ step, index }))
    .filter(({ step }) => step.currentNode === v && (step.to === to || step.to === null));
  return candidates.at(-1)?.index ?? null;
});
const [dfsAuto, setDfsAuto] = useState(false);
const dfsTimer = useRef<NodeJS.Timeout | null>(null);

useEffect(() => {
  if (dfsAuto) {
    dfsTimer.current = setInterval(() => {
      setDfsIdx(prev => {
        if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false); }
        return prev + 1;
      });
    }, 2500);
  }
  return () => { if (dfsTimer.current) clearInterval(dfsTimer.current); };
}, [dfsAuto]);

const step = DFS_STEPS[dfsIdx];

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">

```

```
const activeTree = step.activeEdge && {
  (step.activeEdge[0]===e.u && step.activeEdge[1]===e.v) ||
  (step.activeEdge[0]===e.v && step.activeEdge[1]===e.u);
};
const activeBack = step.backEdge && {
  (step.backEdge[0]===e.u && step.backEdge[1]===e.v) ||
  (step.backEdge[0]===e.v && step.backEdge[1]===e.u);
};
let sc = "#475569", sw = 2, dash = "none";
if (bridge) { sc = "#eab308"; sw = 4; }
else if (activeTree) { sc = "#10b981"; sw = 2; }
else if (activeBack) { sc = "#f43f5e"; sw = 2; }
else if (step.visitedIndices.includes(e.u) || step.visitedIndices.includes(e.v)) {
  return <line key={e.key} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
    stroke={sc} strokeWidth={sw} strokeDash={dash} className="transition-all duration-500"/>
}
{step.currentNode !== null && step.currentNode.id === n.id ? (
  const n = GRAPH_NODES.find(x => x.id === n.id);
  if (!n) return null;
  return <circle cx={n.x} cy={n.y} r={18} fill={cur} stroke={st} strokeWidth={2}/>
) : null}
)}()
GRAPH_NODES.map(n => {
  const cur = step.currentNode === n.id;
  const vis = step.visitedIndices.includes(n.id);
  const st = step.stack.includes(n.id);
  return <g key={n.id}>
    <circle cx={n.x} cy={n.y} r={12} fill={cur ? "#10b981" : st ? "#064e3b" : "#fff"}
      stroke={cur ? "#fff" : st ? "#34d399" : "#475569"} strokeWidth={2.5}
      className="transition-all duration-300"/>
    <text x={n.x} y={n.y+3} textAnchor="middle">label</text>
    {vis && (
      <rect x={n.x-18} y={n.y-28} width={36} height={56} fill="#fff" stroke="#475569"
        className="transition-all duration-300"/>
    )}
  </g>
)}
```

```

    { /* Stack */ }
    <div className="bg-slate-900/50 rounded-xl bo
      <div className="text-[10px] font-bold text-
        <span>Стек DFS</span>
        <span className="text-indigo-400">{step.s
      </div>
      <div className="flex flex-col-reverse gap-1
        {step.stack.length === 0
          ? <div className="text-[10px] text-slate
            : step.stack.map((id, i) => {
              <div key={id} className={`px-2 py-0
                i === step.stack.length-1
                  ? "bg-emerald-600/20 text-emera
                  : "bg-slate-900 text-slate-400
              }`}>Вершина {GRAPH_NODES.find(n=>n.
                <span className="text-slate-500 m
              </div>
            })
          }
        </div>
      </div>
    </div>
  </div>

```

```

    { /* Log */ }
    <div className="bg-indigo-950/10 border borde
      <p className="text-[11px] text-slate-300 le
      <div className="mt-2 pt-2 border-t border-s
        <span className="text-slate-400">Мосты:</
        <span className="font-mono font-bold text
          {step.bridges.length > 0 ? step.bridges
        </span>
      </div>
    </div>
  </div>
</div>
</div>
</div>
);
}

```

```

const edges: Edge[] = [
  { u: 'A', v: 'B', w: 4 },
  { u: 'A', v: 'C', w: 2 },
  { u: 'C', v: 'B', w: 1 },
  { u: 'C', v: 'G', w: 3 },
  { u: 'C', v: 'E', w: 8 },
  { u: 'B', v: 'G', w: 4 },
  { u: 'B', v: 'D', w: 5 },
  { u: 'G', v: 'D', w: 1 },
  { u: 'G', v: 'E', w: 2 },
  { u: 'D', v: 'F', w: 3 },
  { u: 'E', v: 'F', w: 1 },
];

const adjList: { [key: string]: { v: string; w: number } } = {};
nodes.forEach(n => (adjList[n.id] = []));
edges.forEach(e => {
  adjList[e.u].push({ v: e.v, w: e.w });
});

const codeLines = [
  { line: 1, text: 'function dijkstra(graph, start):' },
  { line: 2, text: '    dist = {v: ∞}; dist[start] = 0' },
  { line: 3, text: '    pq = PriorityQueue(); pq.push(start)' },
  { line: 4, text: '    while not pq.empty():' },
  { line: 5, text: '        d, u = pq.pop() // Извлечение минимума' },
  { line: 6, text: '        if d > dist[u]: continue' },
  { line: 7, text: '        for v, weight in graph.neighbors(u):' },
  { line: 8, text: '            if dist[u] + weight < dist[v]:' },
  { line: 9, text: '                dist[v] = dist[u] + weight' },
  { line: 10, text: '                pq.push((dist[v], v))' },
  { line: 11, text: '    return dist // Все кратчайшие пути' },
];

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState(false);

```

```

simulationSteps.push({
  currentNode: u, checkingEdge: { u, v },
  distances: { ...dist }, visited: { ...visited },
  log: `  Смотрим соседа ${v}. Текущее расстояние
        ${dist[u] + edge.w}.`,
  activeCodeLine: 8,
});

if (dist[u] + edge.w < dist[v]) {
  dist[v] = dist[u] + edge.w;
  prev[v] = u;
  simulationSteps.push({
    currentNode: u, checkingEdge: { u, v },
    distances: { ...dist }, visited: { ...visited },
    log: `  Улучшение (Релаксация)! Новое кратчайшее
          расстояние до ${v} равно ${dist[v]}.`,
    activeCodeLine: 9,
  });
}
}
}
}

```

```

simulationSteps.push({
  currentNode: null, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited },
  log: ` Дейкстра всё доставил! Все возможные кратчайшие
        расстояния найдены.`,
  activeCodeLine: 11,
});

```

```

setSteps(simulationSteps);
}, []));

```

```

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  }
}, [isPlaying, currentStepIndex]);

```

```

        <p className="text-sm text-indigo-300">Полн
    </div>
</div>

<div className="flex flex-wrap items-center jus
    <button
        onClick={() => { setIsPlaying(false); setCu
        disabled={compilerLinked}
        title={compilerLinked ? HINT_TEXT : "Вернуть
        className="flex items-center gap-1 bg-slate
        ver:bg-slate-700 text-white px-3 py-2 round
    >
        Сброс
    </button>
    <button
        onClick={() => setIsPlaying(!isPlaying)}
        disabled={compilerLinked}
        title={compilerLinked ? HINT_TEXT : "Проигр
        className={`flex items-center gap-1 px-4 py
        disabled:cursor-not-allowed ${
            isPlaying ? 'bg-amber-600 hover:bg-amber-!
        }`}
    >
        {isPlaying ? 'Пауза' : 'Пуск'}
    </button>
    <button
        onClick={() => { setIsPlaying(false); if (c
        disabled={compilerLinked || currentStepInde
        title={compilerLinked ? HINT_TEXT : "Следую
        className="flex items-center gap-1 bg-indigo
        rsor-not-allowed text-white px-3 py-2 round
    >
        War
    </button>
</div>
</div>

<LiveDistTable

```

```

        let marker = 'url(#arrow-dh-normal)';
        if (isChecked) marker = 'url(#arrow-dh-a
        else if (isOptimal) marker = 'url(#arrow-

    return (
      <g key={idx}>
        <line
          x1={uNode.x} y1={uNode.y}
          x2={vNode.x} y2={vNode.y}
          stroke={isChecked ? '#f59e0b' : is
          strokeWidth={isChecked ? 5 : isOpt
          markerEnd={marker}
          className="transition-all duration-
          strokeDasharray={isChecked ? '4,4'
        />
        <circle
          cx={({uNode.x + vNode.x} / 2} cy={({u
          r="12" fill="#1e293b"
          stroke={isChecked ? '#f59e0b' : '#
        />
        <text
          x={({uNode.x + vNode.x} / 2} y={({uNo
          fill={isChecked ? '#f59e0b' : '#f8
          fontSize="11" fontWeight="bold" tex
        >
          {edge.w}
        </text>
      </g>
    );
  }}}

  {/* Вершины */}
  {nodes.map((node) => {
    const isCurrent = currentStep.currentNode
    const isVisited = currentStep.visited[node
    const dist = currentStep.distances[node.i

    return (

```



```

        t-slate-300'
        }`}>
        <div className="flex items-center gap-2">
          <span className={`px-2 py-0.5 rounded-sm text-slate-300'}`}>
            {u}
          </span>
          <span className="text-slate-400">→</span>
        </div>
        <div className="flex flex-wrap gap-2">
          {adjList[u].map((edge, idx) => {
            const isCheckingThis = currentStep === idx
            return (
              <span key={idx} className={`px-2 py-0.5 rounded-sm
                isCheckingThis ? 'bg-amber-500 text-white' : 'text-slate-300'
              }`}>
                <span>{edge.v}</span><span class="font-size-0.8em"> {edge.w}</span>
                <span className={isCheckingThis ? 'font-weight-bold' : ''}>
                  {edge.c}
                </span>
              </span>
            );
          })}
        </div>
      </div>
    </div>
  </div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10 p-6">
    <div>
      <h4 className="text-lg font-bold text-rose-100">Таблица расстояний
      </h4>
      <div className="overflow-x-auto">
        <table className="w-full text-left border-collapse">

```

```
    <div className="w-full lg:w-1/2 bg-slate-900/60">
      <div>
        <h4 className="text-lg font-bold text-slate-400">
          <div className="bg-slate-950/80 rounded-lg p-4">
            {codeLines.map((item) => {
              const isActive = currentStep.activeCodeLine === item.line
              return (
                <div key={item.line} className={`py-1 px-2 ${
                  isActive ? 'bg-indigo-900/80 text-white' : 'text-slate-400'
                }`} >
                  <span className="text-slate-600 select-none">{item.line}</span>
                  <span className="whitespace-pre font-family: monospace; font-size: 0.9em; margin-left: 0.5em;">{item.code}</span>
                </div>
              )
            })}
          </div>
        </div>
      </div>
    </div>
  </div>
</div>
);
}
```

ФАЙЛ 41/111: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord } from '../vizRuntime';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
```

```
interface DPStep {
  id: number;
  n: number;
```

```

    if (n in memoMap) {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'memo_hit',
        desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано`,
        codeLine: 2,
        memoState: { ...memoMap }
      });
      return memoMap[n];
    }

    if (n <= 2) {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'base_case',
        desc: `Базовый случай: n <= 2 (n=${n}). Возвр`,
        codeLine: 3,
        memoState: { ...memoMap }
      });
      return 1;
    }

    generatedSteps.push({
      id: stepId++,
      n,
      action: 'call',
      desc: `Вычисляем рекурсивно: fibМемо(${n - 1})`,
      codeLine: 4,
      memoState: { ...memoMap }
    });

    const res1 = simulateFib(n - 1);
    const res2 = simulateFib(n - 2);
    memoMap[n] = res1 + res2;

```

```

    }, [isPlaying, currentStepIndex, steps, speed]));

    const baseStep = steps[currentStepIndex] || null;
    const vars = runtime?.variables;
    const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n);
    const liveMemo = vizRecord(vars?.memo);
    const compilerLinked = liveK !== null || !!liveMemo;
    const memoState = liveMemo
      ? Object.fromEntries(Object.entries(liveMemo).flatMap(
        (value) => {
          const number = vizNumber(value);
          return number === null ? [] : [[Number(key), number];]
        }
      ))
      : baseStep?.memoState || {};
    const currentStep: DPStep | null = baseStep && compilerLinked
      ? {
        ...baseStep,
        n: liveK ?? baseStep.n,
        memoState,
        desc: `Компилятор: k=${liveK ?? "-"}; таблица memo: ${memoState}`
      }
      : baseStep;

    return (
      <div className="bg-slate-900 rounded-2xl border border-slate-800" >
        { /* Header & Controls */ }
        <div className="flex flex-col lg:flex-row lg:items-center" >
          <div>
            <div className="flex items-center gap-3 mb-2" >
              <div className="p-2 bg-emerald-500/10 border border-emerald-500" >
                <RefreshCw className="w-6 h-6" />
              </div>
              <h3 className="text-2xl font-extrabold text-white" >
                Компилятор
              </h3>
            </div>
            <p className="text-sm text-slate-400" >
              Смотри, как таблица DP кэширует результаты
            </p>
          </div>
        </div>
      </div>
    );
  }
}

```

```

        }` }
      >
      {isPlaying ? <Pause className="w-4 h-4" />
      {isPlaying ? 'Пауза' : 'Пуск'}}
    </button>
    <button
      onClick={() => { setIsPlaying(false); set
      disabled={currentStepIndex === steps.length
      className="p-2 text-slate-400 hover:text-
      title="Шаг вперед"
    >
      <ChevronRight className="w-5 h-5" />
    </button>
  </div>
</div>
</div>

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded
  <h4 className="text-xs font-bold uppercase trac
  <div className="flex flex-wrap items-center gap
    {Array.from({ length: targetN }, (_, i) => i
    const isCached = num in memoState || num <=
    const val = num <= 2 ? 1 : memoState[num];
    const isCurrent = currentStep?.n === num;

    return (
      <div
        key={num}
        className={`flex flex-col items-center p
          isCurrent
            ? 'bg-emerald-950 border-emerald-500
            : isCached
            ? 'bg-slate-900 border-emerald-500/
            : 'bg-slate-900/50 border-slate-800
          }` }
      >
        <span className="text-xs text-slate-400

```

```

        activeTab === 'code'
          ? 'border-emerald-500 text-emerald-400 bg-slate-900'
          : 'border-transparent text-slate-400 hover:text-emerald-400'
      }` }
    >
    <Code className="w-4 h-4" />
    Интерактивный код
  </button>
</div>

{/* Tab Content */}
<div className="min-h-[320px] flex flex-col justify-between" >
  {activeTab === 'table' && (
    <div className="bg-slate-950 p-6 rounded-2xl border border-slate-800" >
      <h5 className="font-bold text-white text-sm" >Таблица
        <span className="text-emerald-400" > {steps.length} шагов
      </h5>
      <p className="text-sm text-slate-400" >
        В классической рекурсии для  $N=\{targetN\}$  по
        вычисленному значению в таблице, мы моментами
      </p>
      <div className="p-4 bg-slate-900 rounded-xl border border-slate-800" >
        <span>Всего шагов симуляции: {steps.length}</span>
        <span className="text-emerald-400 font-bold" > {steps[steps.length-1]}
      </div>
    </div>
  )}

  {activeTab === 'code' && (
    <div className="bg-slate-950 rounded-2xl border border-slate-800" >
      <div className="bg-slate-900 px-6 py-3 border border-slate-800" >
        <span className="text-xs font-bold font-mono" >DP_CODE_LINES
        <span className="text-xs text-emerald-400" > {steps.length}
      </div>
      <pre className="p-6 font-mono text-sm space-pre" >
        {DP_CODE_LINES.map((line, idx) => {
          const lineNum = idx + 1;

```

```
    });  
  }  
}
```

ФАЙЛ 42/111: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useContext, useEffect, useMemo, useState } from "react";  
import { Undo, Terminal, Route } from "lucide-react";  
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from
```

```
const C1_NODES = [  
  { id: 0, label: "A", x: 190, y: 60 },  
  { id: 1, label: "B", x: 280, y: 140 },  
  { id: 2, label: "C", x: 100, y: 140 },  
  { id: 3, label: "D", x: 280, y: 250 },  
  { id: 4, label: "E", x: 100, y: 250 }  
];
```

```
const C1_EDGES = [  
  { u: 0, v: 1 }, { u: 0, v: 2 },  
  { u: 1, v: 3 }, { u: 2, v: 4 },  
  { u: 1, v: 2 },  
  { u: 3, v: 4 }  
];
```

```
const ekey = (a: number, b: number) => `${Math.min(a, b)}-${Math.max(a, b)}`;  
const DEG = C1_NODES.map((n) => C1_EDGES.filter((e) => e.u === n.id || e.v === n.id));  
const ODD = C1_NODES.map((n) => n.id).filter((id) => DEG[id].length % 2 !== 0);
```

```
/**
```

```
 * Билет 13 – «Графы. Эйлеров цикл».
```

```
 *
```

```
 * Две части:
```

```
 * 1. игра «пройди все рёбра» кликами (Эйлер не прощает ошибок);
```

```
 * 2. критерий: степени вершин и число нечётных (0 → цикл, 2 → путь, >2 → не существует).
```

```

    if (c1VisitedEdges.includes(key)) {
      setClMsg("Это ребро уже пройдено! Эйлер не прощает
      setClDone(true); return;
    }
    const nextEdges = [...c1VisitedEdges, key];
    const nextPath = [...c1Path, vId];
    setClVisitedEdges(nextEdges);
    setClPath(nextPath);

    if (nextEdges.length === C1_EDGES.length) {
      if (nextPath[0] === vId) {
        setClMsg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф
      } else {
        setClMsg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С
          не цикл.`);
      }
      setClDone(true);
    } else {
      setClMsg(`Пройдено рёбер: ${nextEdges.length} / $
    }
  };

/* — что говорит компилятор — */
const live = useMemo(() => {
  if (!vars) return null;
  const rawPath = vizArray(vars.path) ?? vizArray(var
  const path = (rawPath ?? [])
    .map((x) => (typeof x === "number" ? x : vizNumbe
    .filter((x): x is number => x !== null && Number.
  const usedRaw = vizArray(vars.used) ?? vizArray(var
  const usedFlags = (usedRaw ?? []).map((x) => Boolean
  const degRec = vizRecord(vars.deg) ?? vizRecord(var
  const deg = degRec
    ? C1_NODES.map((n) => {
      const val = degRec[String(n.id)] ?? degRec[n.
      const num = typeof val === "number" ? val : v
      return num ?? DEG[n.id];
    })
  })

```



```

    odd.length === 0
      ? { text: "0 нечётных → ЭЙЛЕРОВ ЦИКЛ есть (старт и финиш в одной)" }
      : odd.length === 2
      ? { text: `2 нечётные (${odd.map((id) => C1_NODES[id].label).join(" и ")}) в другой`, cls: "bg-amber-950/40 border-amber-950/40" }
      : { text: `${odd.length} нечётных → эйлерова маршрута нет` };
  }

  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between">
        <h4 className="text-base font-bold text-white">
          Все рёбра ровно по одному разу</h4>
        <button onClick={resetC1} className="flex items-center justify-center gap-2 border border-slate-700 transition-all">
          <Undo className="h-3.5 w-3.5" /> Сброс
        </button>
      </div>

      <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
        <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate_1px)] border border-transparent border-dashed border-[3px] z-10">
          <svg className="w-full h-[260px] z-10 relative">
            {C1_EDGES.map((e, i) => {
              const u = C1_NODES.find(n => n.id === e.u)!;
              const v = C1_NODES.find(n => n.id === e.v)!;
              const key = ekey(e.u, e.v);
              const used = visitedKeys.includes(key);
              // номер этого ребра в маршруте – подписываем
              const order = path.length > 1 ? (() => {
                for (let s = 0; s + 1 < path.length; s++) {
                  if (path[s].u === u && path[s].v === v) return s + 1;
                }
                return null;
              })() : null;
              const isLastEdge = order !== null && order === C1_EDGES.length - 1;
              return <g key={i}>
                <line x1={u.x} y1={u.y} x2={v.x} y2={v.y}
                  stroke={isLastEdge ? "#a5b4fc" : used ? "#fcd34d" : "#444"}
                  strokeWidth={isLastEdge ? 5 : used ? 4 : 2} />
              </g>
            })}
          </svg>
        </div>
      </div>
    </div>
  );
}

```

```

    </div>

    { /* Критерий Эйлера */
    <div className={`p-3 rounded-xl border text-xs font-mono`} >
      Критерий Эйлера (для связного графа): {verdict}
      <span className="block font-normal opacity-80 m-1">
        Степени: {C1_NODES.map((n) => `${n.label}=${n.degree}`)}
        две нечётны. Алгоритм Иера строит маршрут за
      </span>
    </div>

    <div className="flex flex-wrap items-center gap-2">
      <span className="text-[10px] uppercase tracking-wider">
        /> из компилятора:</span>
      {live ? (
        <>
          <span className="text-[11px] font-mono bg-slate-500 p-1">
            {live.path.map((id) => C1_NODES[id].label).join('→')}
          <span className="text-[11px] font-mono bg-slate-500 p-1">
            пройдено: {live.keys.length}/{C1_EDGES.length}
          <span className="text-[11px] font-mono bg-slate-500 p-1">
            {odd.length}</span>
            {live.step !== null && <span className="text-[11px] font-mono bg-slate-500 p-1">
              x-2 py-1">step = {live.step}</span>}
          </>
        ) : (
          <span className="text-[11px] text-slate-500">
            открой панель Python и запусти код: массив C1_NODES —
            том графе, а <code className="font-mono text-[11px] text-slate-500">
              есчитают критерий
            </span>
          </div>
        )}
      </div>
    </div>
  );
}

```

```

    [999, 0, 3, 999, 999, 3, 999],
    [999, 999, 0, 999, 2, 999, 999],
    [999, 999, 999, 0, 4, 2, 999],
    [999, 999, 999, 999, 0, 999, 1],
    [999, 999, 999, 999, 999, 0, 5],
    [999, 1, 999, 999, 999, 999, 0]
  ];

  const adjList: { [key: number]: { v: number; w: number }[] } = {
    0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
    1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
    2: [{ v: 4, w: 2 }],
    3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
    4: [{ v: 6, w: 1 }],
    5: [{ v: 6, w: 5 }],
    6: [{ v: 1, w: 1 }],
  };

  const codeLines = [
    { line: 1, text: 'function floyd_warshall(matrix, V' },
    { line: 2, text: '    dist = copy(matrix)' },
    { line: 3, text: '    for k from 0 to V - 1: // Пос' },
    { line: 4, text: '        for i from 0 to V - 1:' },
    { line: 5, text: '            for j from 0 to V - 1' },
    { line: 6, text: '                if dist[i][k] + d' },
    { line: 7, text: '                    dist[i][j] = ' },
    { line: 8, text: '    return dist' },
  ];

  useEffect(() => {
    const simulationSteps: Step[] = [];
    const dist = initialMatrix.map(row => [...row]);

    simulationSteps.push({
      k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
      log: '    Инициализация матрицы прямых путей, ∞ (999)',
      activeCodeLine: 2,
    });
  });

```

```

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;

const vars = runtime?.variables;
const liveK = vizNumber(vars?.k);
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
/**
 * Матрица из кода пользователя – СВОЕЙ размерности.
 * протискивались в демо-сетку 7×7: при n < 7 появлялись нули
 * при n > 7 компонент падал на nodeNames[i].split(...).length
 */
const liveSource = pickLiveVar(vars, ["d", "dist", "dist2"]);
const liveMat = liveMatrix(liveSource?.value);
const compilerLinked = liveK !== null || liveI !== null || liveJ !== null;

const rows = liveMat?.rows ?? baseStep.matrix.length;
const cols = liveMat?.cols ?? (baseStep.matrix[0]?.length ?? 0);
const shownMatrix: (number | null)[][] = liveMat
  ? Array.from({ length: rows }, (_, i) => Array.from({ length: cols }, (_, j) => liveMat[i][j] || baseStep.matrix[i][j]))
  : baseStep.matrix;

const demoSize = baseStep.matrix.length;
const sizeMismatch = compilerLinked && liveMat && (liveMat.length !== demoSize)
  ? `Демо-схема графа нарисована для ${demoSize} вершин. Если размерности не совпадают, то схема графа остаётся демонстрационной.`
  : null;

```

```

    </div>
  </div>

  <div className="flex flex-col lg:flex-row gap-6 m
    { /* Граф */ }
    <div className="w-full lg:w-2/3 bg-indigo-950/2
      stify-center min-h-[400px]">
      <div className="absolute top-3 left-3 bg-indi
        ont-bold shadow-sm">
        {currentStep.k >= 0 && currentStep.k < 7 ?
      </div>

      <svg className="w-full h-auto max-h-[500px]"
        <defs>
          <marker id="arrow-fw-normal" markerWidth=
            <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
          <marker id="arrow-fw-active" markerWidth=
            <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
          <marker id="arrow-fw-via" markerWidth="6"
            th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
        </defs>
        {Object.keys(adjList).flatMap((uStr) => {
          const u = parseInt(uStr);
          return adjList[u].map((edge) => {
            const uNode = nodesSvg.find((n) => n.id
            const vNode = nodesSvg.find((n) => n.id
            const isTarget = currentStep.i === u &&
            const isVia = (currentStep.i === u && c
            let strokeColor = '#475569'; let marker
            if (isTarget) { strokeColor = '#10b981'
            else if (isVia) { strokeColor = '#818cf8
            return (
              <g key={` ${u} - ${edge.v} `}>
                <line x1={uNode.x} y1={uNode.y} x2=
                markerEnd={marker} strokeDasharray={isTarg
                <circle cx={({uNode.x + vNode.x} / 2
                isVia ? '#818cf8' : '#64748b'} strokeWidth
                <text x={({uNode.x + vNode.x} / 2} y

```

```

const isK = currentStep.k === u;
return (
  <div key={u} className={`flex flex-wrap
    isI ? 'bg-amber-900/40 border-amber-
    isK ? 'bg-indigo-900/40 border-ind
  }`}>
    <span className="font-bold text-white
e-400">→</span>
    {adjList[u].map((edge, idx) => {
      const isUpd = currentStep.i === u
      const isVia = currentStep.k !== -
rentStep.j === edge.v);
      return (
        <span key={idx} className={`px-1
dow' : isVia ? 'bg-indigo-500 text-white bo
        {edge.v} ({edge.w})
      </span>
      )
    })}
  </div>
);
}}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  /* Матрица расстояний */
  <div className="w-full lg:w-1/2 bg-rose-950/10
    <div className="flex justify-between items-center
      <h4 className="text-lg font-bold text-rose-1
    </div>
    {sizeMismatch ? (
      <p className="mb-3 rounded-lg border border
        {sizeMismatch}
      </p>
    ) : null}
  <div className="overflow-x-auto">

```

```
-----
    <h4 className="text-lg font-bold text-slate-600">
    <div className="bg-slate-950/80 rounded-lg p-4">
      {codeLines.map(item => (
        <div key={item.line} className={`py-1.5 px-4
e === item.line ? 'bg-blue-900/80 text-amber-400' : 'text-slate-600'}`>
        <span className="text-slate-600 w-5 shrink-0">{item.line}</span>
        <span className="whitespace-pre flex-grow-1">{item.code}</span>
      </div>
      )
    )
  </div>
</div>
</div>
</div>
</div>
);
}
```

ФАЙЛ 44/111: src/components/GraphComponentsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 100

```
-----
import React, { useCallback, useContext, useEffect, useLayoutEffect } from 'react';
import { ChevronLeft, ChevronRight, Play, Pause, RotateCcw } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from './VizChapterContext';
```

```
/**
```

```
 * Билет 8 – «Графы. Компоненты связности».
```

```
 *
```

```
 * Две вкладки:
```

```
 * 1. «Обход» – DFS/BFS-покраска: внешний цикл по непокрашенным вершинам.
```

```
 *    каждый новый запуск = +1 к счётчику компонент.
```

```
 * 2. «DSU» – рёбра приходят по одному, union склеивает компоненты.
```

```
 *    comps уменьшается.
```

```
 *
```

```
 * Синхронизация с Python-компилятором (PAGE_SYNC["graphviz"])
```

```
 *   - comp / color / used – массив разметки: вершины к раскраске / окрашены /
```

```

    ADJ[u].push(v);
    ADJ[v].push(u);
  });
  ADJ.forEach((a) => a.sort((x, y) => x - y));

  /** Палитра компонент. */
  const COLORS = ["#6366f1", "#10b981", "#f43f5e", "#f59e0b"];

  interface DfsFrame {
    title: string;
    note: string;
    comp: number[];           // 0 = не покрашена, иначе нет
    cur: number | null;       // текущая вершина
    frontier: number[];       // стек/очередь
    comps: number;            // счётчик компонент
    edge: [number, number] | null;
  }

  /** Покадровая раскладка DFS-обхода: ровно тот порядок,
  function buildDfsFrames(): DfsFrame[] {
    const frames: DfsFrame[] = [];
    const comp = new Array(NODES.length).fill(0);
    const push = (cur: number | null, frontier: number[],
      => {
      frames.push({ title, note, comp: [...comp], cur, frontier });
    });
    let c = 0;
    push(null, [], c, "Старт: все вершины серые", "comp = 0, все вершины дой непокрашенной.");
    for (let start = 0; start < NODES.length; start++) {
      if (comp[start] !== 0) continue;
      c += 1;
      const stack = [start];
      comp[start] = c;
      push(start, stack, c, `Новый запуск: c = ${c}, стартовый
        цвет ${c} и кладём в стек.`);
      while (stack.length) {
        const v = stack[stack.length - 1];

```



```

    EDGES.forEach(([u, v], i) => {
      const ru = find(u);
      const rv = find(v);
      if (ru === rv) {
        push(`Ребро ${i}: (${u}, ${v}) — корни совпали`,
          кл, union ничего не меняет, comps остается ${comps},
          return;
        }
      parent[rv] = ru;
      comps -= 1;
      push(`Ребро ${i}: union(${u}, ${v})`, `find(${u}) =
        ${ru}, ${rv} стало ${comps}.`, [u, v], true);
    });
    push(`Готово: comps = ${comps}`, `Все ${EDGES.length}
      ребра.`, null, false);
    return frames;
  }

const DFS_FRAMES = buildDfsFrames();
const DSU_FRAMES = buildDsuFrames();

/** Корень DSU-множества с полным сжатием пути. */
const dsuRoot = (parent: number[], v: number): number => {
  let cur = v;
  const seen = new Set<number>();
  while (parent[cur] !== cur && !seen.has(cur)) {
    seen.add(cur);
    cur = parent[cur];
  }
  return cur;
};

export const GraphComponentsViz: React.FC = () => {
  const chapterId = useContext(VizChapterContext);
  const runtime = useVizRuntime();
  const [mode, setMode] = useState<Mode>("dfs");
  const [step, setStep] = useState(0);
  const [playing, setPlaying] = useState(false);

```

```

/** parent / p из кода → DSU-множества по корням. */
const liveParent = useMemo(() => {
  const arr = vars ? (vizArray(vars.parent) ?? vizArr
  if (!arr || arr.length < NODES.length) return null;
  return arr.slice(0, NODES.length).map((v) => vizNum
}, [vars]);
const liveCur = vars ? (vizNumber(vars.v) ?? vizNumbe
const liveComps = vars ? (vizNumber(vars.comps) ?? vi
const liveFrontier = useMemo(() => {
  if (!vars) return null;
  const arr = vizArray(vars.stack) ?? vizArray(vars.d
  if (!arr) return null;
  return arr.map((v) => vizNumber(v)).filter((v): v i
}, [vars]);
/** Шаг из кода: i / e / step — двигает покадровую де
const liveStep = vars ? (vizNumber(vars.step) ?? vizN

useEffect(() => {
  if (liveStep === null || !Number.isFinite(liveStep)
  goTo(liveStep);
}, [liveStep, goTo]);

/* — какая разметка сейчас в силе: код пользователя
/** Массив parent, который сейчас рисуем: из кода пол
const parentNow = liveParent ?? (mode === "dsu" ? fra
/** Номер множества по parent[]: корни нумеруются в п
const setOf = useCallback(
  (parent: number[], id: number): number => {
    const roots = Array.from(new Set(parent.map((_, i
    return roots.indexOf(dsuRoot(parent, id)) + 1;
  },
  []
);

const compOf = (id: number): number => {
  if (liveComp && liveComp[id] !== undefined && liveC
  if (parentNow) return setOf(parentNow, id);
  return frame.comp?.[id] ?? 0;

```

```

        >
        <Network className="w-3.5 h-3.5" /> DSU (pē
    </button>
  </div>
</div>

{/* панель управления */}
<div className="flex flex-wrap items-center gap-2"
  <button onClick={() => { setStep(0); setPlaying
    late-700 text-slate-300 hover:text-white tran
    <RotateCcw className="w-4 h-4" />
  </button>
  <button onClick={() => { setPlaying(false); goT
    g-slate-900 border border-slate-700 text-slate
    <ChevronLeft className="w-4 h-4" />
  </button>
  <button
    onClick={() => setPlaying((p) => !p)}
    className="flex items-center gap-1.5 px-3 py-1
      ition-colors"
  >
    {playing ? <Pause className="w-4 h-4" /> : <P
  </button>
  <button onClick={() => { setPlaying(false); goT
    d-lg bg-slate-900 border border-slate-700 tex
    <ChevronRight className="w-4 h-4" />
  </button>
  <span className="text-[11px] text-slate-500 ml-
  <span className="ml-auto flex items-center gap-
    700/40 rounded-lg px-2.5 py-1.5">
    КОМПОНЕНТ: {comps}
  </span>
</div>

{/* поле */}
<div className="bg-slate-950 rounded-xl p-3 borde
  <div className="absolute inset-0 bg-[radial-gra
  <svg viewBox="0 0 500 370" className="w-full h-

```

```

<text x={470} y={24} textAnchor="end" fill=
  {mode === "dfs" ? "стек DFS" : "parent[]"}
</text>
{mode === "dfs"
  ? frontier.slice(-8).map((v, i) => (
    <g key={i}>
      <rect x={440} y={32 + i * 20} width=
      <text x={453} y={44 + i * 20} textA
    </g>
  ))
  : (parentNow ?? []).slice(0, 10).map((pv,
    const color = COLORS[(setOf(parentNow
    return (
      <g key={i}>
        <rect x={432} y={32 + i * 15} wid
        <text x={436} y={41.5 + i * 15} f
      </g>
    );
  ))}
</g>
</svg>
</div>

```

```

{/* подпись кадра */}
<div className="mt-3 bg-slate-900/70 border borde
  <p className="text-sm font-bold text-white mb-1
    {runtime && vars && <Terminal className="w-3.
    {title}
  </p>
  <p className="text-xs text-slate-400 leading-re
</div>

```

```

{/* живые переменные из компилятора */}
<div className="mt-3 flex flex-wrap items-center
  <span className="text-[10px] uppercase tracking
  {vars ? (
    ["v", "comps", "c", "i", "e", "comp", "parent
    vars[name] === undefined ? null : (

```

```

    { id: 'E', label: 'E', x: 150, y: 200 },
    { id: 'F', label: 'F', x: 250, y: 200 },
    { id: 'G', label: 'G', x: 350, y: 200 },
  ];

const edges: Edge[] = [
  { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
  { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
  { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];

const adj: Record<string, string[]> = {
  'A': ['B', 'C'],
  'B': ['A', 'D', 'E'],
  'C': ['A', 'F', 'G'],
  'D': ['B'],
  'E': ['B'],
  'F': ['C'],
  'G': ['C'],
};

type Action = {
  type: 'visit' | 'process' | 'backtrack';
  node: string;
  desc: string;
  queueOrStack?: string[];
  visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
  const steps: Action[] = [];
  const vis = new Set<string>();
  const stack: string[] = []; // for visualization, j

  function dfs(v: string) {
    vis.add(v);
    stack.push(v);
    steps.push({ type: 'visit', node: v, desc: `Зависит от`

```

```

        q.push(u);
        steps.push({ type: 'visit', node: u, desc: `04
        tack: [...q], visitedNodes: [...Array.from(
        }
    }
}

steps.push({ type: 'backtrack', node: '', desc: `04
    om(vis)] });

return steps;
}

export function GraphTraversalViz() {
  const [mode, setMode] = useState<"dfs" | "bfs">("df
  const [autoPlay, setAutoPlay] = useState(false);
  const [stepIdx, setStepIdx] = useState(0);

  const steps = useMemo(() => mode === 'dfs' ? genera
  useVizStepSync(stepIdx, setStepIdx, steps.length -
    const v = vizString(vars.v);
    const structure = vizArray(vars.stack) ?? vizArra
    const visited = vizArray(vars.visited);
    if (v === null && !structure && !visited) return
    const candidates = steps
      .map(({step, index}) => ({ step, index }))
      .filter(({ step }) =>
        (v === null || step.node === v) &&
        (!structure || step.queueOrStack?.length ===
        (!visited || step.visitedNodes?.length === vi
      );
    return candidates.at(-1)?.index ?? null;
  });

  useEffect(() => {
    setStepIdx(0);
    setAutoPlay(false);
  });
}

```

```

    { /* SVG Graph View */}
    <div className="bg-slate-900 border border-
ive min-h-[300px]">
      <svg className="w-full h-full max-w-
      { /* Edges */}
      {edges.map((e, i) => {
        const u = nodes.find(n => n
        const v = nodes.find(n => n
        // Check if edge is part of
        const edgeTraversed = isVisi

        return (
          <line key={i} x1={u.x}
            className={`stroke-
500' : 'stroke-emerald-500') : 'stroke-slate
          });
        }}}

      { /* Nodes */}
      {nodes.map(n => {
        const visited = isVisited(n
        const current = isCurrent(n

        let fillColor = "fill-slate
        let strokeColor = "stroke-s
        let textColor = "fill-slate
        let radius = 18;

        if (visited) {
          fillColor = mode === 'd
          strokeColor = mode ===
          textColor = "fill-white
        }

        if (current && step.type !==
          strokeColor = "stroke-a
          textColor = "fill-amber
          radius = 22;
      })}
    </div>
  )}
}

```

```

        `}>
        Узел: {item}
        {idx === (mode === 'dfs'
          <span className="ml-1
        )}
      </div>
    )}
    {!!step.queueOrStack || step.que
      <div className="text-slate-5
    )}
  </div>
</div>
</div>

<div className="flex flex-col sm:flex-row g
  {/* Controls */}
  <div className="flex bg-slate-900 borde
    <button onClick={() => {setAutoPlay
      ="⏮️ Назад" title="⏮️ Назад" className="p
    0 disabled:opacity-30 disabled:backgroun
    <Undo strokeWidth={3} size={16}
    </button>
    <button onClick={() => setAutoPlay(
      justify-center min-w-[80px] ${mode === 'df
    >
      {autoPlay ? <><Pause size={16}
    </button>
    <button onClick={() => {setAutoPlay
      = steps.length - 1} aria-label="⏮️ Вперед"
    00 rounded-lg disabled:opacity-30 disabled:b
    <SkipForward strokeWidth={3} si
    </button>
    <button onClick={() => {setAutoPlay
      ame="p-2 text-slate-500 hover:text-rose-400
    <RefreshCw strokeWidth={3} size
    </button>
  </div>

```



```

    const parent = Math.floor((idx - 1) / 2);
    if (a[parent].priority < a[idx].priority) {
        [a[parent], a[idx]] = [a[idx], a[parent]];
        idx = parent;
    } else break;
}
return a;
}

```

```

function siftDown(arr: Patient[]): Patient[] {
    const a = arr.map(x => ({ ...x }));
    const n = a.length;
    let idx = 0;
    while (true) {
        let largest = idx;
        const l = 2 * idx + 1;
        const r = 2 * idx + 2;
        if (l < n && a[l].priority > a[largest].priority) largest = l;
        if (r < n && a[r].priority > a[largest].priority) largest = r;
        if (largest !== idx) {
            [a[largest], a[idx]] = [a[idx], a[largest]];
            idx = largest;
        } else break;
    }
    return a;
}

```

```

function heapInsert(arr: Patient[], item: Patient): Patient[] {
    return siftUp([...arr, { ...item }]);
}

```

// Position in tree layout

```

function nodePos(index: number): { x: number; y: number } {
    const level = Math.floor(Math.log2(index + 1));
    const posInLevel = index - (Math.pow(2, level) - 1);
    const count = Math.pow(2, level);
    const x = ((posInLevel + 0.5) / count) * 100;
    const y = 10 + level * 24;
}

```

```

        const tuple = vizArray(value);
        const priority = vizNumber(tuple?.[0]) ?? vizNum
        return {
            id: `python-${index}`,
            name: tuple ? String(tuple[1] ?? value) : Str
            emoji: " ",
            symptom: `индекс ${index}`,
            priority,
            animState: "idle",
        };
    })
    : heap;
const [customName, setCustomName] = useState('');
const [customPriority, setCustomPriority] = useState(
const [log, setLog] = useState<string>('');
const [shakeEmpty, setShake] = useState(false);
const [busy, setBusy] = useState(false);
const [healingPatient, setHealing] = useState<Patient>();

const addLog = (msg: string) => setLog(prev => [msg,

// --- INSERT: Новый пациент поступает в приемный пок
const insertPatient = useCallback((name: string, prio
    if (busy || heap.length >= 15) {
        addLog('⚠ Все палаты переполнены! Дождитесь выпис
        setShake(true);
        setTimeout(() => setShake(false), 400);
        return;
    }
    setBusy(true);

const preset = PATIENT_PRESETS[Math.floor(Math.rand
const finalName = name || preset.name.split(' ')[0]
const finalEmoji = preset.emoji;
const finalSymptom = preset.symptom;

const newPatient: Patient = {
    id: `p${uid++}`,

```

```

    setShake(true);
    addLog(`⚠ Нет пациентов для экстренной госпитализации`);
    setTimeout(() => setShake(false), 400);
    return;
  }
  setBusy(true);
  const topPatient = heap[0];
  setHealing(topPatient);

  addLog(`СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

  // Step 1: Root lights up as winner
  setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animState: 'winning' } : x));

  setTimeout(() => {
    // Step 2: Root exits the tree, tree reorganizes
    setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animState: 'exiting' } : x));

    setTimeout(() => {
      // Run binary heap extraction
      setHeap(prev => {
        if (prev.length === 0) return [];
        const a = prev.map(x => ({ ...x }));
        a[0] = { ...a[a.length - 1] };
        a.pop();
        return siftDown(a).map(x => ({ ...x, animState: 'sifting' }));
      });

      // Patient gets healed in the bed
      setTimeout(() => {
        setHealing(prev => prev ? { ...prev, animState: 'healed' } : null);
        addLog(`Успех! Пациент ${topPatient.name} перенес операцию`);
        setTimeout(() => {
          setHealing(null);
          setBusy(false);
        }, 600);
      }, 800);
    });
  }, 800);

```

```
});
```

```
return (
```

```
  <div className="flex flex-col gap-6">
```

```
    {/* МНЕМОНИКА / ПРАВИЛО */}
```

```
    <div className="bg-emerald-950/40 border border-emerald-500">
```

```
      <div className="text-3xl"> </div>
```

```
      <div>
```

```
        <h3 className="font-black text-emerald-400 text-center">
          Priority Queue</h3>
```

```
        <p className="text-xs text-slate-300 mt-1 leading-4">
```

```
          В реанимации не важно, кто пришел раньше, а кто в
          остояние</b> (максимальный приоритет).
```

```
          Куча (Heap) автоматически перестраивает папку,
          оказывался на самом верху дерева (в корне).
```

```
        </p>
```

```
      </div>
```

```
    </div>
```

```
    {/* УПРАВЛЕНИЕ */}
```

```
    <div className="flex flex-col lg:flex-row items-stretch">
```

```
      {/* Добавить случайного */}
```

```
      <button
```

```
        onClick={handleRandomInsert}
```

```
        disabled={busy || heap.length >= 15}
```

```
        className="flex-1 min-w-[150px] bg-gradient-to-r from-emerald-500 to-emerald-400
```

```
        opacity-40 disabled:cursor-not-allowed text-emerald-50 font-medium
```

```
        ve:scale-95 transition-all cursor-pointer focus:outline-none">
```

```
      >
```

```
        <span> Привезти по Скорой (INSERT)</span>
```

```
      </button>
```

```
      {/* Добавить кастомного */}
```

```
      <form onSubmit={handleCustomInsert} className="flex flex-col">
```

```
        <input
```

```
          type="text"
```

```
          required
```

```

    <button
      onClick={reset}
      className="px-5 py-3.5 rounded-2xl bg-slate-800
        r-pointer border border-slate-700"
    >
      Сброс
    </button>
  </div>

```

```

  { /* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */ }
  <div className="grid grid-cols-1 lg:grid-cols-12" >

```

```

    { /* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */ }
    <div className={`lg:col-span-8 bg-slate-900 rounded-2xl
      -[380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`} >
      <div className="absolute top-4 left-4 text-[14px] font-medium" >
        Сортировка по тяжести (Двоичная куча / Max-Heap)
      </div>

```

```

    <div className="relative w-full h-full min-h-[400px]" >
      <svg className="absolute inset-0 w-full h-full" >
        {edges}
      </svg>

```

```

    {visualHeap.map((patient, idx) => {
      const pos = nodePos(idx);
      const isRoot = idx === 0;
      const isCompilerNode = liveI === idx;
      return (
        <div
          key={patient.id}
          className={`
            absolute flex flex-col items-center
            -translate-x-1/2 -translate-y-1/2
            ${patient.animState === 'in' ? 'animate-in' : ''}
            ${patient.animState === 'winner' ? 'animate-winner' : ''}
            ${patient.animState === 'out' ? 'animate-out' : ''}
            ${isCompilerNode ? 'ring-4 ring-amber-500' : ''}

```

```

<div className="flex flex-col items-center" style={{width: 1000px, height: 1000px}}>
  <div className="relative" style={{width: 1000px, height: 1000px}}>
    <span className="text-6xl block animate-pulse" style={{width: 1000px, height: 1000px}}>
      <span className="absolute -top-4 -right-4" style={{width: 1000px, height: 1000px}}>
        <span className="absolute -bottom-2 -left-2" style={{width: 1000px, height: 1000px}}>
          </div>
        <div>
          <h4 className="text-white font-black" style={{width: 1000px, height: 1000px}}>
            <span>Пациент: {healingPatient.name}</span>
          </h4>
          <p className="text-emerald-400 text-x" style={{width: 1000px, height: 1000px}}>
            Состояние: Стабилизация ({healingPa
          </p>
          <div className="flex justify-center g" style={{width: 1000px, height: 1000px}}>
            <span className="w-2 h-2 rounded-fu" style={{width: 1000px, height: 1000px}}>
              <Heart className="w-4 h-4 text-rose" style={{width: 1000px, height: 1000px}}>
                </div>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
  <div>
    <div className="text-center text-slate-60" style={{width: 1000px, height: 1000px}}>
      <span className="text-5xl mb-2 block" style={{width: 1000px, height: 1000px}}>
        <p className="text-xs font-bold font-mo" style={{width: 1000px, height: 1000px}}>
          <p className="text-[10px] mt-1">Ждём са
        </p>
      </div>
    </div>
  </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-:
  <div className="text-[10px] font-mono text-slat
    {log.map((entry, idx) => (
      <div

```

```
export function useLoop(steps: number, ms = 900) {
  const [step, setStep] = useState(0);
  const paused = useContext(DemoPauseContext);
  const period = Math.round(ms * SPEED);

  useEffect(() => {
    if (steps <= 1 || paused) return;
    const t = setInterval(() => setStep((s) => (s + 1) % steps), period);
    return () => clearInterval(t);
  }, [steps, period, paused]);

  return step;
}

export const C = {
  idle: "#334155",
  idleStroke: "#475569",
  text: "#e2e8f0",
  dim: "#94a3b8",
  active: "#6366f1",
  done: "#10b981",
  warn: "#f59e0b",
  bad: "#f43f5e",
  edge: "#475569",
};

export const Svg: React.FC<{ children: React.ReactNode; }> = ({
  children,
  paused = false,
}) => {
  return (
    <div className="w-full">
      <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
        {children}
      </svg>
      {caption && (
        <div className="text-[10px] text-slate-400 text-center">
          {caption}
        </div>
      )}
    </div>
  );
}
```

```

        strokeWidth={width}
        strokeDasharray={dashed ? "4 3" : undefined}
        style={{ transition: MOVE }}
      />
    );

```

ФАЙЛ 48/111: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
  const step = useLoop(3, 1200);
  // i -> j напрямую 9; через k=1 получаем 3+4=7
  const pos: Record<string, [number, number]> = { i: [4, 3], j: [3, 4], k: [1, 1] };
  const viaOpen = step >= 1;
  const relaxed = step >= 2;
  return (
    <Svg
      caption={
        step === 0
          ? "k ещё запрещена: путь i → j = 9"
          : step === 1
            ? "Разрешаем пересадку через k: 3 + 4 = 7"
            : "7 < 9 → D[i][j] = 7"
        }
    >
      <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad : C.good} />
      <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done : C.good} />
      <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done : C.good} />

      <text x={130} y={112} textAnchor="middle" fontSize=

```



```

    )))

    {step >= 1 &&
      sorted.map(({v, i}) => (
        <g key={v}>
          <rect x={30 + i * 52} y={62} width={46} height={16}>
          <text x={53 + i * 52} y={77} textAnchor="middle">
            {v}
          </text>
        </g>
      )))

    {step >= 2 &&
      raw.map(({v, i}) => (
        <g key={`c${i}`}>
          <rect x={startX + i * (boxW + 4)} y={98} width={boxW} height={16}>
          <text x={startX + i * (boxW + 4) + boxW / 2} y={113}>
            {sorted.indexOf(v)}
          </text>
        </g>
      )))
  </Svg>
);
};

```

/* ----- Splay: три случая поворота ----- */

```
type Pt = [number, number];
```

```

const SplayFrames: React.FC<{
  frames: { pos: Record<string, Pt>; edges: [string, string][];
  captions: string[];
  ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
  const step = useLoop(frames.length, ms);
  const f = frames[step];
  const color = (id: string) =>
    id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "c" ? "#f1c40f" : "#2ecc71";

```

Универсальное пособие • полный исходный код

```
-----  
/** Zig-Zig – x и p с одной стороны, крутим сначала вер  
export const ZigZigDemo: React.FC = () => {  
  <SplayFrames  
    captions=[["Линия g → p → x («бамбук»)", "Поворот В  
    frames=[  
      { pos: { g: [176, 24], p: [126, 70], x: [76, 112]  
      { pos: { p: [130, 26], x: [78, 78], g: [186, 78]  
      { pos: { x: [102, 24], p: [152, 70], g: [202, 112]  
    ]}  
  />  
};
```

```
/** Zig-Zag – x и p с разных сторон, крутим сначала ниж  
export const ZigZagDemo: React.FC = () => {  
  <SplayFrames  
    captions=[["Зигзаг: g влево, x вправо", "Поворот НИ  
    frames=[  
      { pos: { g: [176, 24], p: [110, 70], x: [152, 112]  
      { pos: { g: [176, 24], x: [110, 70], p: [64, 112]  
      { pos: { x: [130, 30], p: [80, 100], g: [180, 100]  
    ]}  
  />  
};
```

ФАЙЛ 49/111: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
-----  
import React from "react";  
import { C, Edge, Node, Svg, useLoop } from "../demoKit"  
  
const GRAPH_POS: Record<string, [number, number]> = {  
  A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],  
};  
const GRAPH_EDGES: [string, string][] = [  
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
```

```

    });
};

export const ToposortDemo: React.FC = () => {
  const nodes = [
    { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
    { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
  ];
  const edges: [string, string][] = [
    ["труссы", "штаны"], ["носки", "боты"]
  ];
  const order = ["труссы", "носки", "штаны", "боты"];
  const step = useLoop(order.length + 1, 800);
  const placed = order.slice(0, step);
  const pos = Object.fromEntries(nodes.map((n) => [n.id, n.x]));
  return (
    <Svg caption={`Порядок: ${placed.join(" → ") || "..."} `}
      {edges.map(([u, v], i) => (
        <Edge key={i} a={pos[u]} b={pos[v]} color={placed[i]} />
      ))}
      {nodes.map((n) => {
        const idx = placed.indexOf(n.id);
        return (
          <g key={n.id}>
            <rect x={n.x - 34} y={n.y - 12} width={68} height={24} />
            <text x={n.x} y={n.y} textAnchor="middle" dx={
              idx >= 0 && <circle cx={n.x + 30} cy={n.y - 12} r={10} />
              {idx >= 0 && (
                <text x={n.x + 30} y={n.y - 12} textAnchor="middle">
                  {placed[idx]}
                </text>
              )}
            </g>
          );
        });
      }}
      <text x={224} y={62} textAnchor="middle" fontSize=16>
        Порядок: {placed.join(" → ") || "..."}
      </text>
      <text x={224} y={74} textAnchor="middle" fontSize=16>
        {placed[placed.length - 1]}
      </text>
    </Svg>
  );
};

```

```

    });
};

export const ArticulationDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
  };
  const edges: [string, string][] = [["A", "B"], ["A", "C"], ["A", "D"], ["B", "C"], ["B", "D"], ["C", "D"]];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : "Граф с вершиной C"}>
      {edges.map(([u, v], i) => (cut && (u === "C" || v === "C") ? null :
        {Object.entries(pos).map(([k, [x, y]]) =>
          cut && k === "C" ? (
            <circle key={k} cx={x} cy={y} r={12} fill="none" />
          ) : (
            <Node key={k} x={x} y={y} label={k} fill={k} />
          )
        )
      )}
    )}
    </Svg>
  );
};

```

```

export const MstDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 100], E: [190, 100]
  };
  const edges = [
    { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
    { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
  ];
  const chosen = ["A-D", "A-B", "B-C", "D-E"];
  const step = useLoop(chosen.length + 1, 800);
  const taken = new Set(chosen.slice(0, step));
  return (
    <Svg caption={`Жадно берём самые лёгкие рёбра без циклов`}>
      {edges.map(([u, v], i) => (taken.has(`${u}-${v}`) ? null :
        {Object.entries(pos).map(([k, [x, y]]) =>
          taken.has(k) ? (
            <circle key={k} cx={x} cy={y} r={12} fill="none" />
          ) : (
            <Node key={k} x={x} y={y} label={k} fill={k} />
          )
        )
      )}
    )}
    </Svg>
  );
};

```

```
import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

export const HeapDemo: React.FC = () => {
  const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [4, 8, 9, 12, 2], [4, 2, 9, 12, 8]];
  const step = useLoop(states.length, 1000);
  const heap = states[step];
  const pos: [number, number][] = [[130, 24], [80, 68], [130, 112], [80, 156]];
  const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
  return (
    <Svg caption="Новый элемент всплывает наверх, пока не окажется на вершине">
      <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b={pos[2]} />
      <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b={pos[2]} />
      {heap.map((v, i) => (
        <Node key={i} x={pos[i][0]} y={pos[i][1]} label={v} />
      ))}
    </Svg>
  );
};

export const StackDemo: React.FC = () => {
  const states = [["A"], ["A", "B"], ["A", "B", "C"], ["A", "B", "C", "D"], ["A", "B", "C", "D", "E"]];
  const step = useLoop(states.length, 800);
  const items = states[step];
  const popping = step >= 3;
  return (
    <Svg caption={popping ? "pop: забираем ВЕРХНИЮЮ часть стека" : "push: добавляем элемент в стек"}>
      <rect x={90} y={22} width={80} height={96} rx={6} />
      {items.map((v, i) => (
        <g key={v} style={{ transition: "all .3s" }}>
          <rect x={94} y={100 - i * 26} width={72} height={26} />
          <text x={130} y={111 - i * 26} textAnchor="middle">{v}</text>
        </g>
      ))}
    </Svg>
  );
};
```

```

    <Svg caption="find = «кто твой староста», union = 0"
      {p.map((par, i) =>
        par !== i ? {
          <path key={i} d={`M ${pos[i][0]} ${pos[i][1]}
            fill="none" stroke={rootColor[root(i)]} stroke-width={1}
          } : null
        ) : null
      )}
      {pos.map(([x, y], i) => <Node key={i} x={x} y={y} />)}
    </Svg>
  );
};

```

```

const TRIE_NODES = [
  { id: 0, x: 130, y: 22, ch: "•", parent: null as number },
  { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
  { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
  { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
  { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
  { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];

```

```

export const TrieDemo: React.FC = () => {
  const step = useLoop(TRIE_NODES.length + 1, 700);
  return (
    <Svg caption="Путь от корня по буквам = слово; общий путь = префикс"
      {TRIE_NODES.filter((n) => n.parent !== null).map((n) => {
        const p = TRIE_NODES[n.parent as number];
        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]} />
      })}
      {TRIE_NODES.map((n) => <Node key={n.id} x={n.x} y={n.y} />)}
    </Svg>
  );
};

```

```

export const TrieBfsDemo: React.FC = () => {
  const levels = [[0], [1, 2], [3, 4, 5]];
  const step = useLoop(levels.length + 1, 950);
  const doneIds = new Set(levels.slice(0, step).flat());

```

```
};
```

```
export const SegmentTreeDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const nodes = [
    { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
    { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
    { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
    { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
    { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
    { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
    { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
  ];
  return (
    <Svg caption="Запрос собирается из  $O(\log n)$  готовых  

      {nodes.map((n, i) => (
        <g key={i}>
          <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={20} fill={n.lvl === step ? C.active : C.idle} />
          <text x={n.x} y={n.y} textAnchor="middle" dom
            />
        </g>
      ))}
    </Svg>
  );
};
```

```
export const PrefixSumDemo: React.FC = () => {
  const a = [3, 1, 4, 1, 5, 9];
  const pref = a.reduce<number[]>((acc, v) => [...acc, v]);
  const step = useLoop(4, 1200);
  const l = 1, r = 3;
  const show = step >= 1;
  return (
    <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] - P[${l}]` : "о префиксные суммы P"}>
      <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
      {a.map((v, i) => (
        <g key={i}>
```



```
};
```

```
export const AmortizedDemo: React.FC = () => {
  const costs = [1, 1, 1, 8, 1, 1, 1, 1];
  const step = useLoop(costs.length + 1, 550);
  const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
  const avg = step ? (total / step).toFixed(2) : "-";
  return (
    <Svg caption={`Сделано ${step} операций, суммарно ${total} операций`}
      {costs.map((c, i) => (
        <rect key={i} x={14 + i * 30} y={104 - c * 10} width={30} height={10}
          fill={i < step ? (c > 2 ? C.bad : C.done) : C.done}
        ></rect>
      ))}
      <line x1={8} y1={106} x2={252} y2={106} stroke={C.done} strokeDash={[]}></line>
      <text x={130} y={20} textAnchor="middle" fontSize={14}>Шаг: {step}</text>
    </Svg>
  );
};
```

```
export const NpDemo: React.FC = () => {
  const step = useLoop(2, 1400);
  return (
    <Svg caption={step === 0 ? "Найти решение – долго (N^2)" : "Решение найдено"}
      <ellipse cx={130} cy={64} rx={112} ry={48} fill={C.done} stroke={C.done} strokeDash={[]}></ellipse>
      <ellipse cx={92} cy={64} rx={52} ry={32} fill="#333" stroke="#333" strokeDash={[]}></ellipse>
      <text x={92} y={64} textAnchor="middle" dominantBaseline="middle" fontStyle="italic">N</text>
      <text x={206} y={34} textAnchor="middle" fontStyle="italic">P</text>
      <circle cx={196} cy={84} r={13} fill={step === 1 ? C.done : C.wait} stroke={C.wait} strokeDash={[]}></circle>
      <text x={196} y={84} textAnchor="middle" dominantBaseline="middle" fontStyle="italic">1</text>
      <text x={130} y={124} textAnchor="middle" fontStyle="italic">Решение найдено</text>
    </Svg>
  );
};
```

```
export const PrefixFunctionDemo: React.FC = () => {
  const s = "abacaba";
  const pi = [0, 0, 1, 0, 1, 2, 3];
  return (
    <Svg caption="Префикс-функция"
      <rect key="s" x={14} y={104} width={140} height={10} fill={C.done} stroke={C.done} strokeDash={[]}></rect>
      <rect key="pi" x={14} y={114} width={140} height={10} fill={C.done} stroke={C.done} strokeDash={[]}></rect>
      <text x={130} y={104} textAnchor="middle" fontStyle="italic">s</text>
      <text x={130} y={114} textAnchor="middle" fontStyle="italic">pi</text>
      <text x={130} y={124} textAnchor="middle" fontStyle="italic">abacaba</text>
      <text x={130} y={134} textAnchor="middle" fontStyle="italic">0010123</text>
    </Svg>
  );
};
```

```

        {filled && {
          <text x={57 + c * 38} y={36 + r * 32} textAnchor="middle"
            fontSize={10} fontWeight="700" fill="black">{c}</text>
        }}
      </g>
    );
  })
})
<text x={130} y={122} textAnchor="middle" fontSize={14}>
  каждая клетка считается один раз и переиспользуется
</text>
</Svg>
);
};

```

ФАЙЛ 51/111: `src/components/hints/MiniDemo.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
  AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
  SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
} from "../demosStruct";
import {
  ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
  ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
  | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
  | "suffix-link" | "toposort" | "relax" | "prefix-sum"
  | "segment-tree" | "bridge" | "articulation" | "mst"
  | "prefix-function" | "dp" | "floyd" | "components" |

```

```

const [paused, setPaused] = useState(false);
const Cmp = REGISTRY[kind];
if (!Cmp) return null;
return (
  <DemoPauseContext.Provider value={paused}>
    <div
      className="bg-slate-950/70 rounded-lg border bo
      onMouseEnter={() => setPaused(true)}
      onMouseLeave={() => setPaused(false)}
    >
      <Cmp />
    </div>
  </DemoPauseContext.Provider>
);
};

```

ФАЙЛ 52/111: src/components/hints/SimulatorModal.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../../data/vizStepBu
import { VizLiveStatus } from "../VizLiveStatus";

```

```

interface Props {
  vizId: string | null;
  onClose: () => void;
}

```

```

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
  useEffect(() => {
    if (!vizId) return;

```

```
        </VizChapterContext.Provider>
      </div>
    </div>
  </div>,
  document.body
);
};
```

ФАЙЛ 53/111: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useEffect, useLayoutEffect, useRef, use
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";
```

```
export interface HintAnchor {
  termId: string;
  rect: DOMRect;
}
```

```
interface Props {
  anchor: HintAnchor | null;
  onClose: () => void;
  onOpenSimulator: (vizId: string) => void;
  onCardEnter: () => void;
  onCardLeave: () => void;
}
```

```
const CARD_W = 320;
const GAP = 10;
```

```
export const TermPopover: React.FC<Props> = ({ anchor,
```

```

const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
  <div
    ref={cardRef}
    onMouseEnter={onCardEnter}
    onMouseLeave={onCardLeave}
    className="fixed z-[100] term-popover"
    style={{
      top: pos ? pos.top : -9999,
      left: pos ? pos.left : -9999,
      width: Math.min(CARD_W, typeof window !== "unde
      visibility: pos ? "visible" : "hidden",
    }}
    role="dialog"
  >
    <div className="bg-slate-900 border border-indigo
      <div className="flex items-start gap-2 px-3 pt-
        <h4 className="text-sm font-bold text-indigo-
          <button
            onClick={onClose}
            className="text-slate-500 hover:text-white
            aria-label="Заккрыть подсказку"
          >
            <X className="w-4 h-4" />
          </button>
        </div>

      <div className="p-3 space-y-2.5 max-h-[60vh] ov
        <p className="text-[13px] leading-relaxed tex

          {entry.demo && <MiniDemo kind={entry.demo} />

          {entry.formal && (
            <p className="text-[11.5px] leading-relaxed
              {entry.formal}
            </p>

```

```

    const edge = `${vizString(vars.u) ?? ""}${vizString(vars.v) ?? ""}`;
    return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
  });

const nodes = [
  { id: 'S', x: 200, y: 150, label: 'S' },
  { id: 'A', x: 200, y: 50, label: 'A' },
  { id: 'B', x: 320, y: 220, label: 'B' },
  { id: 'C', x: 80, y: 220, label: 'C' },
];

const edges = [
  { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
  { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
  { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
  { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0, labelDx: 0, labelDy: 0 },
  { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0, labelDx: 0, labelDy: 0 },
  { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },
];

const isNodeVisible = (id: string) => {
  if (id === 'S' && (step === 0 || step === 7)) return true;
  return false;
};

const getPotential = (id: string) => {
  if (step >= 2 && step < 7) {
    if (id === 'S') return 0;
    if (id === 'A') return 0;
    if (id === 'B') return -2;
    if (id === 'C') return -1;
  }
  return null;
};

const getEdgeColorClass = (e: any) => {
  if (e.u === 'S') {
    if (step === 0 || step === 7) return "opacity: 0.5;";
  }
  return "";
};

```

```

    };

    const getEdgeContent = (e: any) => {
        if (e.u === 'S') return e.weight;

        if (step < 4) return e.weight;

        if (e.id === 'AB') {
            if (step === 4) return `${e.weight} + (${0}`
            return e.newWeight;
        }
        if (e.id === 'BC') {
            if (step < 5) return e.weight;
            if (step === 5) return `${e.weight} + (${-2}`
            return e.newWeight;
        }
        if (e.id === 'CA') {
            if (step < 6) return e.weight;
            if (step === 6) return `${e.weight} + (${-1}`
            return e.newWeight;
        }
        return e.weight;
    };

    const isEdgeTextHighlighted = (e: any) => {
        if (e.id === 'AB' && step === 4) return true;
        if (e.id === 'BC' && step === 5) return true;
        if (e.id === 'CA' && step === 6) return true;
        return false;
    };

    const isEdgeTextEmerald = (e: any) => {
        if (e.id === 'AB' && step > 4) return true;
        if (e.id === 'BC' && step > 5) return true;
        if (e.id === 'CA' && step > 6) return true;
        return false;
    };

```

```

    );
    case 5: return (
        <div>
            <b>Шаг 3.2: Пересчет ребра B → C.</b>
            Старый вес: 1. <br/>
            Вычисляем:  $1 + (-2) - (-1) =$  <span class="text-emerald-300">
        </div>
    );
    case 6: return (
        <div>
            <b>Шаг 3.3: Пересчет ребра C → A.</b>
            Старый вес: 2. <br/>
            Вычисляем:  $2 + (-1) - 0 =$  <span class="text-emerald-300">
        </div>
    );
    case 7: return (
        <div className="text-emerald-300">
            <b>Готово!</b> Фиктивную вершину S мы
            Все новые веса ребер в графе стали
            При этом старые кратчайшие пути оста
        </div>
    );
    default: return "";
}
};

return (
    <div className="bg-slate-800 p-4 rounded-xl border">
        <h4 className="text-sm md:text-base font-bold">
            <span className="w-2.5 h-2.5 bg-amber-400 rounded">
                Визуализация: Перевзвешивание алгоритмом
            </h4>

        <div className="flex flex-col md:flex-row gap-4">
            <div className="bg-[#0f172a] flex-1 min-h-100 p-4">
                <svg width="400" height="300" class="

```



```

        return (
          <g key={i}>
            <line
              x1={u.x} y1={u.y}
              className={`stroke-${u.id}`}
              markerEnd={getMarkerEnd(u.id)}
            />

            <text
              x={midX} y={midY}
              textAnchor="middle"
              dominantBaseline="bottom"
              className={`text-${u.id}`}
              style={{textHeight: 1.2, textEmergence: 1}}
              isS ? 'fill' : 'stroke'
            />
            {getEdgeContent(u.id)}
          </text>
        </g>
      )
    }
  }
}

{/* Nodes */}
{nodes.map(n => {
  if (!isNodeVisible(n.id)) return null
  const pot = getPotential(n.id)

  return (
    <g key={n.id} className={`node-${n.id}`}>
      <circle
        cx={n.x} cy={n.y}
        className={`stroke-${n.id}`}
        style={{strokeWidth: 2, fill: n.id === 'S' ? 'fill-[#2e8b57]' : 'fill-slate-500'}}
      />
      <text x={n.x} y={n.y}

```

```

        <Undo strokeWidth={3} size=
      </button>
      <button onClick={() => setStep(
        className={`flex-1 font-bol
          ${step === MAX_STEP
            ? 'bg-emerald-600/5
            : 'bg-indigo-600 ho
          {step === MAX_STEP ? "Готов
          {step !== MAX_STEP && <Skip
        </button>
      </div>

      {/* Progress Dots */}
      <div className="flex justify-center
        {Array.from({length: MAX_STEP +
          <div key={i} className={`w-
125' : i < step ? 'bg-indigo-900' : 'bg-sla
        }}}
      </div>
    </div>
  </div>
</div>
);
}

```

ФАЙЛ 55/111: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "...
import {
  Play,
  Pause,
  RotateCcw,
  Layers,

```

```

    sccMap: Record<number, number>; // node -> sccId
    currentSccId: number;
    activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
  const [steps, setSteps] = useState<AlgoStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
  const v = vizNumber(vars.v);
  const orderLength = vizArray(vars.order)?.length;
  if (v === null && orderLength === undefined) return null;
  const candidates = steps
    .map((step, index) => ({ step, index }))
    .filter(({ step }) => (v === null || step.currentNode === v));
  return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
  // Generate step-by-step Kosaraju steps
  const generatedSteps: AlgoStep[] = [];
  const visited = new Set<number>();
  const order: number[] = [];
  const sccMap: Record<number, number> = {};

  const recordStep = (
    phase: AlgoStep["phase"],
    desc: string,
    currentNode: number | null,
    transposed: boolean,
    currentSccId: number,
  ) => {
    generatedSteps.push({
      phase,
      desc,
      visited: new Set(visited),
      currentNode,
    });
  };

```

```

    order.push(u);
    recordStep(
      "dfs1",
      `DFS 1: вершина ${u} полностью обработана. Запи-
      u,
      false,
      -1,
    );
  };

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
  if (!visited.has(i)) {
    dfs1(i);
  }
}

const topoOrder = [...order].reverse();
recordStep(
  "transpose",
  `Первый проход завершен! Стек выхода в топологиче-
  ебра.` ,
  null,
  true,
  -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
  visited.add(u);

```

```

    }

    recordStep(
      "finished",
      `Алгоритм успешно завершен! Найдено ${sccId} компонента,
      null,
      true,
      sccId,
    );

    setSteps(generatedSteps);
    setCurrentStepIdx(0);
  }, []));

const step = steps[currentStepIdx] || {
  phase: "init",
  desc: "Загрузка...",
  visited: new Set(),
  currentNode: null,
  order: [],
  transposed: false,
  sccMap: {},
  currentSccId: -1,
  activeEdges: initialEdges,
};

useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlaying && currentStepIdx < steps.length - 1) {
    timer = setInterval(() => {
      setCurrentStepIdx((prev) => prev + 1);
    }, 1500);
  } else {
    setIsPlaying(false);
  }
  return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

```

```

        className="flex items-center justify-center
        ors"
    >
        <RotateCcw className="w-4 h-4" />
    </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
    { /* Playfield SVG */}
    <div className="lg:col-span-8 bg-[#0B1120] relative" >
        <svg className="w-full h-full max-w-[650px]" >
            { /* Markers for directed arrows */}
            <defs>
                <marker
                    id="arrow"
                    viewBox="0 0 10 10"
                    refX="22"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#6" />
                </marker>
                <marker
                    id="arrow-active"
                    viewBox="0 0 10 10"
                    refX="22"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#1" />
                </marker>
                <marker
                    id="arrow-transposed"

```

```

        markerId = "arrow-active";
    }
}

// Adjust slightly for bidirectional link
const isBidi =
    (edge.u === 3 && edge.v === 4) ||
    (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
    <line
        key={`edge-${idx}`}
        x1={x1}
        y1={y1 + dy}
        x2={x2}
        y2={y2 + dy}
        stroke={stroke}
        strokeWidth="2.5"
        markerEnd={`url(#${markerId})`}
        className="transition-all duration-300"
    />
);
}}}

{/* Nodes */}
{initialNodes.map((node) => {
    const isCurrent = step.currentNode === node.id;
    const isVisited =
        step.visited.has(node.id) ||
        (step.phase === "dfs1" && step.currentNode === node.id);
    const sccId = step.sccMap[node.id];

    let classes = "fill-slate-800 stroke-slate-800";
    if (sccId) {
        classes = getSccColor(sccId);
    } else if (isCurrent) {
        classes = "fill-amber-600 stroke-amber-600";
    }
    return (
        <div
            key={node.id}
            className={classes}
            style={{
                width: node.w,
                height: node.h,
                position: 'relative',
            }}
        >
            {node.label}
        </div>
    );
})}
```

```

        </g>
      );
    }}}
  </svg>

  {step.transposed && (
    <div className="absolute top-4 left-4 bg-in
    1 rounded">
      Граф транспонирован (Ребра обратные)
    </div>
  )}
</div>

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
00 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-4
  Порядок обхода и стек
</h4>

  <div className="bg-slate-900 border border-sl
    <p className="text-xs text-white leading-re
      {step.desc}
    </p>
  </div>

  <div className="flex-1 space-y-4 overflow-y-a
    {/* Top-Sort output / Order stack represent
  <div>
    <span className="text-[10px] text-slate-5
      СТЕК ВЫХОДА (DFS 1 ORDER):
    </span>
    <div className="flex flex-wrap gap-1 bg-s
      {step.order.length === 0 ? (
        <span className="text-slate-600 text-
      ) : (
        [...step.order].reverse().map((val, i
        <div

```



```

        <button
          disabled={currentStepIdx === 0}
          onClick={() => setCurrentStepIdx((prev)
          className="bg-slate-900 border border-s
        >
        Назад
      </button>
      <button
        disabled={currentStepIdx >= steps.length
        onClick={() => setCurrentStepIdx((prev)
        className="bg-slate-900 border border-s
      >
      Вперед
    </button>
  </div>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 56/111: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React, { useCallback, useContext, useEffect, use
import { Play, RotateCcw, SkipForward, HelpCircle, Chec
  from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime,
  from '../data/vizStepBus';

```

```

interface Vertex {
  id: number;
  label: string;
  x: number;

```

```
-----
{ id: '1-4', u: 1, v: 4, weight: 6, status: 'pending'
{ id: '4-5', u: 4, v: 5, weight: 7, status: 'pending'
{ id: '3-6', u: 3, v: 6, weight: 8, status: 'pending'
{ id: '2-5', u: 2, v: 5, weight: 9, status: 'pending'
{ id: '6-7', u: 6, v: 7, weight: 10, status: 'pending'
{ id: '4-7', u: 4, v: 7, weight: 11, status: 'pending'
{ id: '7-8', u: 7, v: 8, weight: 12, status: 'pending'
{ id: '5-8', u: 5, v: 8, weight: 13, status: 'pending'
];
```

```
export const KruskalSimulator: React.FC = () => {
  const [activeAlgo, setActiveAlgo] = useState<'kruskal'
  const [vertices, setVertices] = useState<Vertex[]>(in
  const [edges, setEdges] = useState<Edge[]>(initialEdg
  const [stepIndex, setStepIndex] = useState<number>(0)
  const [heapQueue, setHeapQueue] = useState<string[]>(
  const [activeCheatTab, setActiveCheatTab] = useState<

  const [logs, setLogs] = useState<StepLog[]>([
    {
      title: 'Введение в раскраску (Краскал)',
      description: '«Краскал – это раскраска?». Предста
        овы начинать!',
      type: 'info',
    },
  ]));
```

```
// --- KRUSKAL STEPS ---
```

```
const kruskalSteps = [
  // Step 1: Inspect A-B (2)
  () => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
      title: 'Шаг 1: Берем самое дешевое ребро A-B (в
      description: 'Оно соединяет две бесцветные верши
      type: 'info'
    }, ...prev]));
  },
```

```
// Step 6: Include B-C
() => {
  setVertices(prev => prev.map(v => v.id === 2 ? {
  setEdges(prev => prev.map(e => e.id === '1-2' ? {
  setLogs(prev => [{
    title: 'Успех: Ребро B-C в осто́ве',
    description: 'Вершина C перекрашена в красный ц
    type: 'success'
  }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
  setEdges(prev => prev.map(e => e.id === '0-3' ? {
  setLogs(prev => [{
    title: 'Шаг 4: Берем ребро A-D (вес 5)',
    description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
      ю – мы перекрашиваем всё в один цвет!"',
    type: 'warning'
  }, ...prev]));
},
// Step 8: Include A-D (Merge to purple)
() => {
  setVertices(prev => prev.map(v => v.color === 'bl
  setEdges(prev => prev.map(e => e.status === 'incl
  setLogs(prev => [{
    title: 'Слияние кланов: Ребро A-D добавлено',
    description: 'Мы объединили красный и синий кла
    type: 'success'
  }, ...prev]));
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {
  setLogs(prev => [{
    title: 'Шаг 5: Берем ребро B-E (вес 6)',
    description: 'НО: это ребро соединяет вершины B
    type: 'warning'
  }, ...prev]));
}
```

```
// Step 14: Include D-G
() => {
  setVertices(prev => prev.map(v => v.id === 6 ? {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
  setLogs(prev => [{
    title: 'Успех: Ребро D-G в осто́ве',
    description: 'Вершина G окрашена в фиолетовый.',
    type: 'success'
  }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '2-5' ? {
  setLogs(prev => [{
    title: 'Шаг 8: Берем ребро C-F (вес 9)',
    description: 'Оно соединяет вершины C и F, кото
    type: 'warning'
  }, ...prev]));
},
// Step 16: Reject C-F
() => {
  setEdges(prev => prev.map(e => e.id === '2-5' ? {
  setLogs(prev => [{
    title: 'Отказ: Цикл обнаружен!',
    description: 'Выбрасываем ребро C-F.',
    type: 'error'
  }, ...prev]));
},
// Step 17: Inspect G-H (10)
() => {
  setEdges(prev => prev.map(e => e.id === '6-7' ? {
  setLogs(prev => [{
    title: 'Шаг 9: Берем ребро G-H (вес 10)',
    description: 'Соединяет фиолетовую G и бесцветн
    type: 'info'
  }, ...prev]));
},
// Step 18: Include G-H
```

```

    setVertices(prev => prev.map(v => v.id === 8 ? {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: '    Успех: MST построено Краскалом!',
        description: 'Все 9 вершин окрашены в единый фи
            + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
        type: 'success'
    }, ...prev]));
    },
    ];

// --- PRIM STEPS ---
const primSteps = [
    // Step 1: Start at E
    () => {
        setVertices(prev => prev.map(v => v.id === 4 ? {
        setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E
        setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5
        setLogs(prev => [{
            title: 'Шаг 1: Плесень зарождается в Токио (Вер
            description: 'Мы выбрали вершину E как стартовую
                ритетную очередь (Heap): D-E(3), B-E(6), E-F(
            type: 'info'
        }, ...prev]));
    },
    // Step 2: Pop D-E (3)
    () => {
        setEdges(prev => prev.map(e => e.id === '3-4' ? {
        setLogs(prev => [{
            title: 'Шаг 2: Куча выдает самое дешевое ребро
            description: 'Плесень тянется по ребру D-E к вер
            type: 'info'
        }, ...prev]));
    },
    // Step 3: Include D-E, add D's edges to heap
    () => {
        setVertices(prev => prev.map(v => v.id === 3 ? {
        setEdges(prev => prev.map(e => e.id === '3-4' ? {

```

```
// Step 7: Include A-B, add B's edges
() => {
    setVertices(prev => prev.map(v => v.id === 1 ? {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
        eap' } : e));
    setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)']);
    setLogs(prev => [{
        title: 'Успех: Вершина B захвачена плесенью',
        description: 'В кучу добавлено B-C(4). Ребро B-E(6) добавлено в кучу.',
        type: 'success'
    }, ...prev]));
},
// Step 8: Pop B-C (4)
() => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
        description: 'Плесень тянется к вершине C.',
        type: 'info'
    }, ...prev]));
},
// Step 9: Include B-C, add C's edges
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
        eap' } : e));
    setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (9)']);
    setLogs(prev => [{
        title: 'Успех: Вершина C захвачена',
        description: 'Ребро C-F(9) добавлено в кучу.',
        type: 'success'
    }, ...prev]));
},
// Step 10: Pop B-E (6) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setLogs(prev => [{
        title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
```

```

        setLogs(prev => [{
            title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
            description: 'Плесень расширяется вниз к G.',
            type: 'info'
        }, ...prev]));
    },
    // Step 15: Include D-G, add G's edges
    () => {
        setVertices(prev => prev.map(v => v.id === 6 ? {
        setEdges(prev => prev.map(e => e.id === '3-6' ? {
            eap' } : e));
        setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
        setLogs(prev => [{
            title: 'Успех: Вершина G захвачена',
            description: 'В кучу добавлено G-H(10).',
            type: 'success'
        }, ...prev]));
    },
    // Step 16: Pop C-F (9) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '2-5' ? {
        setLogs(prev => [{
            title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
            description: 'Вершины C и F уже заражены плесенью',
            type: 'warning'
        }, ...prev]));
    },
    // Step 17: Reject C-F
    () => {
        setEdges(prev => prev.map(e => e.id === '2-5' ? {
        setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
        setLogs(prev => [{
            title: 'Отказ: Игнорируем ребро C-F',
            description: 'Выбрасываем из кучи.',
            type: 'error'
        }, ...prev]));
    },
    // Step 18: Pop G-H (10)

```

```

    },
    // Step 22: Pop H-I (12)
    () => {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setLogs(prev => [{
            title: 'Шаг 12: Куча выдает H-I (вес 12)',
            description: 'Плесень тянется к последней чистой',
            type: 'info'
        }, ...prev]));
    },
    // Step 23: Include H-I
    () => {
        setVertices(prev => prev.map(v => v.id === 8 ? {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setHeapQueue(['F-I (13 - цикл)']));
        setLogs(prev => [{
            title: '      Успех: Плесень захватила весь граф',
            description: 'Все 9 вершин поглощены. Итоговый ч
                иной – мы разрастались из одной точки с помо
            type: 'success'
        }, ...prev]));
    },
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
    // Step 1: Five-Year Plan 1 (Inspecting outgoing)
    () => {
        // Highlight the cheapest outgoing edges for EACH
        // Node 0(A) -> A-B(2)
        // Node 1(B) -> A-B(2)
        // Node 2(C) -> B-C(4)
        // Node 3(D) -> D-E(3)
        // Node 4(E) -> D-E(3)
        // Node 5(F) -> E-F(7)
        // Node 6(G) -> D-G(8)
        // Node 7(H) -> G-H(10)
        // Node 8(I) -> H-I(12)
    }
];

```



```

// Outgoing roads between {A, B, C} and {D, E, F},
// A-D (5), B-E (6), C-F (9).
// Cheapest is A-D (5).
setEdges(prev => prev.map(e => e.id === '0-3' ? {
setLogs(prev => [{
  title: 'Вторая пятилетка: Колхозы выбирают мост',
  description: 'Теперь Красный и Синий колхозы од
    -D с весом 5. Остальные мосты B-E(6) и C-F(9)
  type: 'warning'
}], ...prev));
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
  setVertices(prev => prev.map(v => ({ ...v, color:
  setEdges(prev => prev.map(e =>
    e.status === 'included' || e.id === '0-3' ? { .
  ));
  setLogs(prev => [{
    title: 'Объединение завершено: Один гигантский
    description: 'Оба колхоза объединились по мосту
    type: 'success'
  }, ...prev));
},
// Step 5: Check remaining edges & mark as rejected
() => {
  setEdges(prev => prev.map(e => e.status === 'pend
  setLogs(prev => [{
    title: ' Успех: MST построен Борувкой за 2 шаг
    description: 'Все лишние ребра (B-E, E-H, C-F,
      вес: 51. Благодаря параллельности, мы потрати
    type: 'success'
  }, ...prev));
}
];

const currentSteps =
  activeAlgo === 'kruskal' ? kruskalSteps :
  activeAlgo === 'prim' ? primSteps :

```

```

    setHeapQueue([]);
    setLogs(introLog(algo));
};

/* ----- связь с Python-компилятором -----
const chapterId = useContext(VizChapterContext);
const runtime = useVizRuntime();
const vars = runtime?.variables;

// активная вкладка = подрежим страницы: код в панели
useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, activeAlgo);
}, [chapterId, activeAlgo]);

/** Перемотка на кадр n: сброс и повтор первых n шагов
const goToStep = useCallback((n: number) => {
  const target = Math.max(0, Math.min(currentSteps.length, n));
  setVertices(initialVertices);
  setEdges(initialEdges);
  setHeapQueue([]);
  setLogs(introLog(activeAlgo));
  for (let i = 0; i < target; i++) currentSteps[i]();
  setStepIndex(target);
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [activeAlgo, currentSteps]);

/** Номер кадра из переменных кода: i / step / число
const selectByVariables = useCallback<VizStepSelector>((v) => {
  const i = vizNumber(v.i) ?? vizNumber(v.step);
  if (i !== null) return i + 1;
  const mst = vizArray(v.mst) ?? vizArray(v.answer) ?? vizArray(v.steps);
  if (mst) return mst.length;
  const taken = vizArray(v.taken) ?? vizArray(v.used);
  if (taken) return Math.max(0, taken.length - 1);
  return null;
}, []);

useVizStepSync(stepIndex, goToStep, currentSteps.length);

```

```

const PALETTE = ['red', 'blue', 'purple', 'mold'];
/** Цвет по корне компоненты: одинаковый корень → один
const colorOfComp = (root: string | undefined): string {
  if (!root) return 'none';
  let h = 0;
  for (let i = 0; i < root.length; i++) h = (h * 31 +
  return PALETTE[h % PALETTE.length];
};

/** Всё, что код пользователя рассказал про граф. */
const live = useМемо({}) => {
  if (!vars) return null;
  const parentRec = vizRecord(vars.parent) ?? vizRecord
  const parentArr = vizArray(vars.parent) ?? vizArray
  /** Метка множества вершины, как её записал код (бу
  const rawComp = new Map<number, string>();
  /** То же, но только если метка – вершина графа: по
  const parentIdx = new Map<number, number>();
  const feed = (id: number, val: unknown) => {
    const label = String(typeof val === 'string' ? val
    rawComp.set(id, label);
    const inner = typeof val === 'string' ? val : typ
    const pid = idOf(inner);
    if (pid !== null) parentIdx.set(id, pid);
  };
  if (parentRec) {
    Object.entries(parentRec).forEach(([k, val]) => {
      const id = idOf(k);
      if (id === null) return;
      feed(id, val);
    });
  } else if (parentArr) {
    parentArr.forEach((val, id) => {
      if (id > 8) return;
      feed(id, val);
    });
  }
}

```

```

// eslint-disable-next-line react-hooks/exhaustive-
}, [vars]));

```

```

/** Рёбра, которые рисуем: код пользователя важнее ка
const effEdges: Edge[] = useMemo(() => {
  if (!live) return edges;
  const key = (a: number, b: number) => `${Math.min(a
  const takenKey = new Set(live.takenEdges.map((e) =>
  const heapKey = new Set(live.heapEdges.map((e) => k
  const cheapKey = new Set(live.cheapestEdges.map((e)
  return edges.map((e) => {
    const k = key(e.u, e.v);
    const isCur = (live.curU === e.u && live.curV ===
    if (isCur) return { ...e, status: 'inspecting' as
    if (takenKey.has(k)) return { ...e, status: 'incl
    if (heapKey.has(k)) return { ...e, status: 'in_he
    if (cheapKey.has(k)) return { ...e, status: 'insp
    return { ...e, status: 'pending' as const };
  });
  // eslint-disable-next-line react-hooks/exhaustive-
}, [edges, live]);

```

```

const effVertices: Vertex[] = useMemo(() => {
  if (!live) return vertices;
  return vertices.map((v) => {
    const root = live.compOfVertex.get(v.id);
    if (root !== undefined) return { ...v, color: col
    if (live.takenVertices.includes(v.id)) return { .
    if (live.curU === v.id || live.curV === v.id) ret
    return { ...v, color: 'none' };
  });
  // eslint-disable-next-line react-hooks/exhaustive-
}, [vertices, live]);

```

```

const getColorClass = (color: string, id: number) =>
  switch (color) {
    case 'red': return 'bg-red-500 border-red-300 tex
    case 'blue': return 'bg-blue-500 border-blue-300

```

```

    <p className="text-slate-400 text-sm">
      Наблюдай за работой Краскала (Раскраска),
    </p>
  </div>

  { /* Algorithm Tabs */ }
  <div className="flex flex-wrap items-center gap-2">
    <div className="flex items-center bg-slate-100 p-2">
      <button
        onClick={() => handleReset('kruskal')}
        className={
          "flex items-center gap-1.5 px-3 py-2"
        +
          (activeAlgo === 'kruskal'
            ? 'bg-indigo-600 text-white shadow-md'
            : 'text-slate-400 hover:text-slate-600')
        >
        <GitGraph className="w-3.5 h-3.5" />
        <span>Краскал (Билет 18)</span>
      </button>
      <button
        onClick={() => handleReset('prim')}
        className={
          "flex items-center gap-1.5 px-3 py-2"
        +
          (activeAlgo === 'prim'
            ? 'bg-emerald-600 text-white shadow-md'
            : 'text-slate-400 hover:text-slate-600')
        >
        <Layers className="w-3.5 h-3.5" />
        <span>Прима (Билет 19)</span>
      </button>
      <button
        onClick={() => handleReset('boruvka')}
        className={
          "flex items-center gap-1.5 px-3 py-2"

```

```

    </div>
</div>

/* Что компилятор сообщил графу прямо сейчас */
<div className="mb-6 flex flex-wrap items-center" >
  <span className="flex items-center gap-1.5 text-sm" >
    <Terminal className="w-3.5 h-3.5" /> из компилятора
  </span>
  {live ? (
    <div>
      {live.curU !== null && live.curV !== null && (
        <span className="text-[11px] font-mono" >
          ребро {String.fromCharCode(65 + live.curU)} - {String.fromCharCode(65 + live.curV)}
        </span>
      )} : ''}
    </div>
  )}
  {live.takenEdges.length > 0 && (
    <span className="text-[11px] font-mono" >
      в осто́ве: {live.takenEdges.map((e) => e.join(' ')).join(', ')}
    </span>
  )}
  {live.compOfVertex.size > 0 && (
    <span className="text-[11px] font-mono" >
      компонент: {new Set(Array.from(live.compOfVertex)).join(', ')}
    </span>
  )}
  {live.takenVertices.length > 0 && (
    <span className="text-[11px] font-mono" >
      дерево выросло: {live.takenVertices.map((v) => String.fromCharCode(65 + v)).join(', ')}
    </span>
  )}
  {live.heapEdges.length > 0 && (
    <span className="text-[11px] font-mono" >
      в куче: {live.heapEdges.length}
    </span>
  )}
  {live.cheapestEdges.length > 0 && (
    <span className="text-[11px] font-mono" >
      в очереди: {live.cheapestEdges.length}
    </span>
  )}

```

```

        </div>
        <div className="flex items-center gap-2">
          <div className="w-3 h-3 rounded-full bg-slate-300">
            </div>
          </div>
        </>
      ) : activeAlgo === 'prim' ? (
        <>
          <div className="flex items-center gap-2">
            <div className="w-3 h-1 bg-sky-600">
              </div>
            <div className="flex items-center gap-2">
              <div className="w-3 h-3 rounded-full bg-slate-300">
                </div>
              </div>
            </div>
          </>
        ) : (
          <>
            <div className="flex items-center gap-2">
              <div className="w-3 h-3 rounded-full bg-slate-300">
                </div>
              <div className="flex items-center gap-2">
                <div className="w-3 h-3 rounded-full bg-slate-300">
                  </div>
                <div className="flex items-center gap-2">
                  <div className="w-3 h-3 rounded-full bg-slate-300">
                    </div>
                  </div>
                </div>
              </div>
            </div>
          </>
        )
      )
    </div>

    <div className="absolute top-4 right-4 bg-slate-300 p-2">
      ⌊ {stepIndex} / {currentSteps.length} ⌋
    </div>

    {/* SVG Edges */}
    <svg className="absolute inset-0 w-full h-full" style={{
      aspectRatio:"none"
    }}>
      {effEdges.map(edge => {

```

```

    { /* HTML Overlay for Vertices */ }
    <div className="absolute inset-0 w-full h-full" style={
      {effVertices.map(vertex => {
        <div
          key={vertex.id}
          style={{ top: vertex.y + "%", left: vertex.x + "%",
            className={"absolute -translate-x-1/2 -translate-y-1/2"},
            font-weight: "bold", text-align: "center", border: "2px solid black", transition: "all 0.3s ease-in-out"}
          id={vertex.id}
        >
          <span>{vertex.label}</span>
          {vertex.id === 4 && activeAlgo === 'prim' ? (
            <span className="text-[8px] block" style={{ display: "block", text-align: "center" }}>
              MST построено с помощью {activeAlgo}
            </span>
          ) : null}
        </div>
      )}
    >
    </div>

    {stepIndex >= currentSteps.length && (
      <div className="absolute inset-x-6 bottom-6 shadow-xl z-20" style={{
        padding: "10px", border: "1px solid black", background: "white"
      }}>
        <p className={"font-weight: bold; text-align: center; font-size: 1.2em; margin: 0"}>
          Шаг {stepIndex + 1} из {currentSteps.length}
        </p>
        <p className="text-slate-300 text-xs mt-2" style={{ margin: 0 }}>
          Общий вес минимального остова: {totalWeight}. Всего вершин: {vertices.length}.
        </p>
      </div>
    )}
  </div>

  { /* Logs / Heap */ }
  <div className="bg-slate-950 rounded-2xl border border-slate-900 p-4" style={{
    marginTop: 10px, background: "black", color: "white"
  }}>
    <div className="p-4 bg-slate-900 border-b border-slate-800" style={{ border: "1px solid black", background: "black", color: "white" }}>
      <h4 className="font-weight: bold; text-slate-200 text-sm margin: 0">Журнал выполнения
    </div>
    <div style="font-family: monospace; font-size: 0.9em; padding: 5px 0 0 20px; color: white">
      Ход выполнения
    </div>
  </div>

```



```

        <div className="flex items-center gap-2">
          {log.type === 'success' && <CheckCircle className="text-emerald-500" />}
          {log.type === 'warning' && <HelpCircle className="text-amber-500" />}
          {log.type === 'error' && <XCircle className="text-red-500" />}
          {log.type === 'info' && <Play className="text-blue-500" />}
          <h5 className="font-bold text-sm leading-tight">{log.message}</h5>
        </div>
        <p className="text-xs text-slate-300">{log.timestamp}</p>
      </div>
    )})
  </div>
</div>
</div>

```

```

/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-slate-700 p-4">
  <div className="flex flex-col sm:flex-row items-center gap-2">
    <div className="flex items-center gap-2.5">
      <BookOpen className="w-5 h-5 text-indigo-400" />
      <h4 className="text-xl font-bold text-white">Cheat Sheet</h4>
    </div>
    <div className="flex bg-slate-950 p-1 rounded">
      <button
        onClick={() => setActiveCheatTab('kruskal')}
        className={
          "px-3 py-1.5 rounded text-xs font-medium" +
          (activeCheatTab === 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300")
        }
      >
        Краскал (Билет 18)
      </button>
      <button
        onClick={() => setActiveCheatTab('prim')}
        className={
          "px-3 py-1.5 rounded text-xs font-medium" +
          (activeCheatTab === 'prim' ? "bg-emerald-600 text-white" : "bg-slate-800 text-slate-300")
        }
      >
        Прима (Билет 19)
      </button>
    </div>
  </div>

```



```

        </pre>
      </div>
    </div>
  </div>
})

{activeCheatTab === 'boruvka' && {
  <div className="grid grid-cols-1 lg:grid-cols-2" >
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-rose-500" />
        </p>
        <p className="text-2xl font-black text-white">Время
        <p className="text-slate-400 text-xs mt-1">
          p>
        </div>
        <div className="bg-slate-950 p-4 rounded-3xl">
          <p className="text-slate-400 text-xs font-medium">
            <Award className="w-3.5 h-3.5 text-rose-500" />
          </p>
          <p className="text-2xl font-black text-white">Награда
          <p className="text-slate-400 text-xs mt-1">
        </div>
        <div className="bg-slate-950 p-4 rounded-3xl">
          <p className="text-slate-400 text-xs font-medium">
            ении:</p>
            <p className="text-xs text-slate-300 leading-relaxed">
              , организовав массовое распараллеленное слияние
            </p>
          </div>
        </div>
      <div className="lg:col-span-2">
        <div className="bg-slate-950 p-4 rounded-3xl">
          <p className="text-slate-400 text-xs font-medium">
            <Code className="w-3.5 h-3.5 text-rose-500" />
          </p>
          <pre className="text-xs text-emerald-400 font-mono">
            80 flex-1">

```

```
/**
 * «Лёгкая» текстовая версия пособия.
 *
 * Открывается по адресу `?lite=1` (весь список тем) или
 * (одна тема). Здесь НЕТ навигации, сайдбара, интерактивных
 * панели компилятора – только сам текст тем, чтобы:
 *   1) любую страницу можно было распечатать/сохранить
 *   2) внешний парсер (ридер, «версия для слабовидящих»,
 *      LLM-индексатор) получал чистый текст без служебных
 *      элементов
 *
 * Все спойлеры <details> раскрыты, визуализация описания
 * показывает, какие у неё вкладки и какие переменные
 */
```

```
const LIGHT_CSS = `
:root { color-scheme: light; }
body { background: #ffffff !important; color: #111827; }
.lite-wrap { max-width: 46rem; margin: 0 auto; padding:
  1.6; }
.lite-wrap h1 { font-size: 1.75rem; font-weight: 700; }
.lite-wrap h2 { font-size: 1.35rem; font-weight: 700;
  color: #111827; }
.lite-wrap h3 { font-size: 1.1rem; font-weight: 700; }
.lite-wrap h4 { font-size: 1rem; font-weight: 700; margin:
  0 0 1.6 0; }
.lite-wrap p, .lite-wrap li { color: #1f2937; font-size:
  1.1; }
.lite-wrap a { color: #1d4ed8; }
.lite-meta { color: #4b5563; font-size: .85rem; margin:
  0 0 1.6 0; }
.lite-topics { font-size: .8rem; color: #374151; margin:
  0 0 1.6 0; }
.lite-viz { margin: 1.25rem 0; padding: .8rem 1rem; border:
  1px solid #f3f4f6; }
.lite-viz h4 { margin: 0 0 .35rem; font-size: .95rem; }
.lite-viz ul { margin: .35rem 0 0 1.1rem; padding: 0; }
.lite-viz code, .lite-wrap code { font-family: ui-monospace;
  padding: 0 .25rem; }
.lite-wrap pre { background: #f3f4f6; border: 1px solid
  #d1d5db; padding: 1.45; }
.lite-wrap table { border-collapse: collapse; width:
  100%; }
```

```

    return one.length ? one : chapters;
  }, [topicId]));

return (
  <div className="lite-wrap">
    <style dangerouslySetInnerHTML={{ __html: LIGHT_C
    <h1>Универсальное пособие: алгоритмы и структуры
    <p className="lite-meta">
      Билеты 1–24 • текстовая версия (без навигации,
      {topicId ? `одна тема: ${list[0]?.title ?? topic
      <a href="?">полная интерактивная версия</a>
    </p>

    {!topicId && (
      <nav className="lite-toc">
        <h3>Содержание</h3>
        <ol>
          {list.map((c) => (
            <li key={c.id}>
              <a href={`?lite=1&topic=${encodeURIComponent
                {chapterTopics[c.id]?.length ? ` – биле
              ""}
            </li>
          )}}
        </ol>
      </nav>
    )}

    {list.map((chapter) => {
      const viz = getViz(chapter.id);
      const topics = chapterTopics[chapter.id] ?? [];
      const syncKeys = Object.keys(PAGE_SYNC).filter(
      return (
        <article key={chapter.id} id={chapter.id}>
          <h2>{chapter.title}</h2>
          <p className="lite-topics">
            {topics.length > 0 && <>Билет{topics.leng

```

В интерактивной версии эта демонстрация переменных перерисовывают её напрямую

```

    </p>
  </div>
})

{
  quizzes[chapter.id] ?? []
}

<div className="lite-quiz">
  <h3>Блиц: вопросы по теме</h3>
  <ol>
    {
      quizzes[chapter.id] ?? []
    }.map((q, i) => {
      <li key={i}>
        <p>
          <b>{q.question}</b>
        </p>
        <ul>
          {
            q.options.map((option, j) => {
              <li key={j}>
                {j === q.correctIndex ? <b>Правильно</b> : <b>Неправильно</b>}
              </li>
            })
          }
        </ul>
        <p className="lite-meta">{q.explanation}</p>
      </li>
    })
  </ol>
</div>
})
</article>
);
}}}
</div>
);
};

export default LiteGuide;

```

```

*
* Схема демонстрации нарисована на фиксированном наборе
* S...F у Форда–Беллмана). Если в коде пользователя верши
* {0...6, v1...v9}, картинка физически не может их показат
* просто пропадали, и казалось, что визуализация живёт
* выводятся рядом, а подсказка объясняет, как связать
*/
export function LiveDistTable({ dist, prev, done, demoIds }) {
  if (!dist) return null;

  const keys = Object.keys(dist);
  if (keys.length === 0) return null;

  const demoSet = new Set(demoIds);
  const unknown = keys.filter((key) => !demoSet.has(key));
  // Все вершины совпали со схемой – таблица не нужна,
  if (unknown.length === 0) return null;

  const doneSet = new Set((done ?? []).map((value) => S...F));
  const shown = keys.slice(0, 64);

  return (
    <div className="mb-6 rounded-xl border border-amber-500 p-4">
      <p className="text-[11px] leading-relaxed text-amber-500">
        <b>{title}</b> В вашем коде вершины{" "}
        <span className="font-mono">
          {unknown.slice(0, 8).join(", ")}
          {unknown.length > 8 ? " ..." : ""}
        </span>
        , а на схеме демо-графа нарисованы{" "}
        <span className="font-mono">{demoIds.join(", ")}
        показаны отдельной таблицей. Чтобы они управлялись
        (кнопка-«глаз» в панели Python подставляет его
      </p>
      <div className="mt-2.5 flex flex-wrap gap-1.5">
        {shown.map((key) => {
          const parent = prev ? fmt(prev[key]) : null;
          const fixed = doneSet.has(key);

```

```
const cards = [
  {
    img: dijkstraImg,
    alt: 'Декстер Дуглас и Фриказоид (Дейкстра)',
    title: '× Фриказоид и Декстер Дуглас',
    desc: 'Декстер смотрит в CRT-монитор; синий электрический ток',
    highlight: 'неотрицательными',
    highlightColor: 'text-emerald-400',
    rest: 'весами ( $w \geq 0$ ). Подсунь отрицательное ребро',
    complexity: ' $O((V+E) \log V)$ ',
    border: 'border-blue-500/30 hover:border-blue-400/60',
    titleColor: 'text-blue-400',
    badge: 'text-blue-400/70 bg-blue-950/40',
  },
  {
    img: bellmanImg,
    alt: 'Бэтмен на машине Форда (Форд-Беллман)',
    title: 'Бэтмен на машине Форда',
    desc: 'Патрулирует n-1 ночей в кабине машины Генри',
    highlight: 'отрицательные циклы',
    highlightColor: 'text-rose-400',
    rest: '(бесконечную кормушку Джокера).',
    complexity: ' $O(V \cdot E)$ ',
    border: 'border-rose-500/30 hover:border-rose-400/60',
    titleColor: 'text-rose-400',
    badge: 'text-rose-400/70 bg-rose-950/40',
  },
  {
    img: floydImg,
    alt: 'Fortnite Все против Всех (Флойд-Уоршелл)',
    title: 'Все против Всех / Fortnite (Флойд)',
    desc: 'Матрица  $N \times N$  + три цикла (',
    highlight: 'k, i, j',
    highlightColor: 'text-emerald-400',
    rest: '). Все против всех – ищет кратчайшие пути от',
    complexity: ' $O(V^3)$ ',
    border: 'border-emerald-500/30 hover:border-emerald-400/60',
  },
]
```



```

    selectedChapterId: string;
    setSelectedChapterId: (id: string) => void;
    /** Заголовок текущей темы – показываем в кнопке, что
currentTitle: string;
index: number;
total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedCh
  const [open, setOpen] = useState(false);

  useEffect(() => {
    document.body.style.overflow = open ? "hidden" : ""
    return () => {
      document.body.style.overflow = "";
    };
  }, [open]);

  useEffect(() => {
    const onKey = (e: KeyboardEvent) => e.key === "Escap
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown",
  }, []);

  return (
    <>
      <div className="lg:hidden sticky top-[57px] z-40"
        <button
          onClick={() => setOpen(true)}
          className="w-full flex items-center gap-2.5 p
          aria-label="Открыть содержание"
        >
          <span className="p-1.5 rounded-lg bg-indigo-600"
            <List className="w-4 h-4" />

```

ФАЙЛ 61/111: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState } from 'react';
import { Sparkles, FileDown, FileText, Code2, Loader2, } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  /** Открыта ли боковая панель компилятора. */
  compilerOpen: boolean;
  onToggleCompiler: () => void;
}
```

```
/**
 * Шапка приложения.
 *
 * Кнопка «Скачать PDF» всегда на самом видном месте во
 */
```

```
export const Navbar: React.FC<NavbarProps> = ({ compilerOpen, onToggleCompiler }) => {
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };
};
```

```
  return (
    <header className="bg-slate-900 border-b border-slate-800">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
```

```

<div id="download-menu" className="flex items
  <a
    id="download-guide-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/
    download="guide_lite.pdf"
    target="_blank"
    rel="noreferrer"
    className="flex-1 sm:flex-none flex items
    bg-indigo-600 hover:bg-indigo-500 text-white
    n-200 whitespace-nowrap"
    title="Скачать PDF пособия: компактный код"
  >
    <FileDown className="w-4 h-4 text-indigo-600" />
  </a>

<details className="relative shrink-0">
  <summary
    className="flex items-center justify-center
    bg-slate-700/60 border border-slate-600 cu
    title="Ещё форматы: PDF с кодом, TXT и .
  >
    <span>Ещё</span>
  </summary>
  <div className="absolute right-0 top-full
  5 shadow-xl shadow-black/50 min-w-[170px]">
    <a
      id="download-pdf-btn"
      href={` ${import.meta.env.BASE_URL}exp
      download="full_code_all_pages.pdf"
      target="_blank"
      rel="noreferrer"
      className="flex items-center gap-2 px
      rald-600/20 border border-emerald-500/30 tr
      title="Полный PDF-дамп исходного кода"
    >
      <FileDown className="w-3.5 h-3.5" />
    </a>

```

```

-----
import React, { useCallback, useContext, useEffect, use
import { ChevronLeft, ChevronRight, Play, Pause, Rotate
import { PlanarityDemo } from "./PlanarityDemo";
import { VizChapterContext, emitVizDemo, useVizRuntime,

/**
 * Билет 7 – «Графы. Планарные. Покраска».
 *
 * Четыре вкладки:
 * 1. «Планарность» – K5 и K3,3 с подсвеченными пересече
 * 2. «Жадная покраска» –  $\chi \leq \Delta + 1$ : вершины красятся
 *    достаётся наименьший цвет, не занятый соседями;
 * 3. «Двудольность» – проверка 2-раскрашиваемости BFS
 *    нечётный цикл = двумя красками не обойтись;
 * 4. «Покраска рёбер» –  $\chi' \leq \Delta + 1$  (Визинг), а для дв
 *     $\chi' = \Delta$  (теорема Кёнига).
 *
 * Два графа на выбор: «основной» (8 вершин,  $\Delta = 3$ , ест
 * и «двудольный» (6 вершин,  $\Delta = 3$ ,  $\chi = 2$ ,  $\chi' = 3$ ).
 *
 * Синхронизация с Python-компилятором (PAGE_SYNC["plan
 * color/col – массив цветов вершин (красит кружки нап
 * ecolor/ec – массив цветов рёбер, v – текущая вершина
 * u – сосед, chi/k – число цветов, delta/D – максима
 * i/step – номер кадра.
 */

type Tab = "planar" | "greedy" | "bipartite" | "edges";
type GraphKey = "main" | "bip";

interface Graph {
  key: GraphKey;
  label: string;
  n: number;
  edges: [number, number][];
  adj: number[][];
  pos: { x: number; y: number }[];
  names: string[];

```

```

    title: string;
    note: string;
    color: number[];           // 0 = не покрашена
    ecolor?: number[];        // 0 = не покрашено
    v: number | null;
    u: number | null;
    e: number | null;
    used?: number[];
    bad?: [number, number] | null;
}

/** Жадная покраска вершин:  $\chi \leq \Delta + 1$ . */
function greedyFrames(g: Graph): Frame[] {
    const frames: Frame[] = [];
    const color = new Array(g.n).fill(0);
    const delta = Math.max(...g.adj.map((a) => a.length))
    frames.push({
        title: "Старт: ни одна вершина не покрашена",
        note: `color = [0]·n. Порядок обхода – просто по номерам.
        Конечно уложится в  $\Delta + 1 = \{\text{delta} + 1\}$  цветов: у вершин
        color: [...color], v: null, u: null, e: null,
    });
    for (let v = 0; v < g.n; v++) {
        const used = Array.from(new Set(g.adj[v].filter((x) => x !== v)));
        let c = 1;
        while (used.includes(c)) c += 1;
        color[v] = c;
        const deg = g.adj[v].length;
        frames.push({
            title: `Вершина ${g.names[v]}: берём цвет ${c}`,
            note: used.length
                ? `Соседи уже заняли цвета ${used.join(", ")}.
                Всегда хватит ( $\chi \leq \Delta + 1$ ).`
                : `У вершины ${g.names[v]} ещё нет покрашенных соседей.`,
            color: [...color], v, u: null, e: null, used,
        });
    }
    const chi = Math.max(...color);

```

```

        title: `${g.names[v]} → ${g.names[u]}: цвет
        note: `Сосед не покрашен: color[${g.names[u]
        x}).join(", ") || "пуста"}]`,
        color: [...color], v, u, e: null,
    });
} else if (color[u] === color[v]) {
    conflict = [v, u];
    frames.push({
        title: `Конфликт: ${g.names[v]} и ${g.names
        note: `Рёбро (${g.names[v]}, ${g.names[u]})
        покрасить:  $\chi \geq 3$ .`,
        color: [...color], v, u, e: null, bad: conf
    });
}
}
}
}
frames.push({
    title: conflict ? "Граф НЕ двудольный" : "Граф двуд
    note: conflict
        ? `BFS упёрся в нечётный цикл (${g.names[conflict
        краски — их и даёт жадный алгоритм на вкладке «
        : "Все вершины разбиты на две доли, рёбер внутри
        праведлива теорема Кёнига:  $\chi' = \Delta$  (рёберная пок
        color: [...color], v: null, u: null, e: null, bad:
    });
return frames;
}

```

/** Жадная покраска рёбер: $\Delta \leq \chi' \leq \Delta + 1$ (Визинг), для

```

function edgeFrames(g: Graph): Frame[] {
    const frames: Frame[] = [];
    const ecolr = new Array(g.edges.length).fill(0);
    const inc: Map<number, number>[] = Array.from({ length
    const delta = Math.max(...g.adj.map((a) => a.length))
    frames.push({
        title: "Старт: рёбра не покрашены",
        note: `Правильная рёберная покраска: у каждой верши

```

```
-----
  { id: "edges", label: "4 · Покраска рёбер (Визинг, Кё  
];
```

```
export const PlanarColoringViz: React.FC = () => {
  const chapterId = useContext(VizChapterContext);
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const [tab, setTab] = useState<Tab>("greedy");
  const [graphKey, setGraphKey] = useState<GraphKey>("m
  const [step, setStep] = useState(0);
  const [playing, setPlaying] = useState(false);
```

```
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, tab);
  }, [chapterId, tab]);
```

```
  const graph = useMemo(() => makeGraph(graphKey), [gra
  const frames = useMemo(() => {
    if (tab === "greedy") return greedyFrames(graph);
    if (tab === "bipartite") return bipartiteFrames(gra
    if (tab === "edges") return edgeFrames(graph);
    return [] as Frame[];
  }, [tab, graph]);
  const maxStep = Math.max(0, frames.length - 1);
```

```
  useEffect(() => {
    setStep(0);
    setPlaying(false);
  }, [tab, graphKey]);
```

```
  useEffect(() => {
    if (!playing) return;
    const t = window.setInterval(() => {
      setStep((s) => {
        if (s >= maxStep) {
          setPlaying(false);
          return s;
        }
      })
    }, 1000);
  }, [maxStep, playing]);
```

```

        setPlaying(false);
        setStep((s) => Math.max(0, Math.min(maxStep, s + d)
    }, [maxStep]));

    if (tab === "planar") {
        return (
            <div className="bg-slate-800/90 p-5 sm:p-6 rounded-
                <Header />
                <Tabs tab={tab} setTab={setTab} />
                <PlanarityDemo />
                <p className="text-[11px] text-slate-500 mt-2">
                    Формула Эйлера  $V - E + F = 2$  и оценка  $E \leq 3V$ 
                </p>
            </div>
        );
    }

    return (
        <div className="bg-slate-800/90 p-5 sm:p-6 rounded-
            <Header />
            <Tabs tab={tab} setTab={setTab} />

            <div className="flex flex-wrap items-center gap-2
                <span className="text-[10px] uppercase tracking
                    {[["main", "bip"] as GraphKey[]].map((k) => (
                        <button
                            key={k}
                            onClick={() => setGraphKey(k)}
                            className={"px-3 py-1.5 rounded-lg text-[11px]
                                "bg-slate-900/60 text-slate-400 hover:text-
                            >
                                {makeGraph(k).label}
                            </button>
                        )))
                </div>

            <div className="flex flex-wrap items-center gap-2
                <button onClick={() => { setStep(0); setPlaying

```



```

        />
        {tab === "edges" && ec > 0 && {
            <text x={{graph.pos[a].x + graph.pos[b].x} y={{graph.pos[a].y + graph.pos[b].y} / 2} fill={colorOf(ec)} fontSize="10" fontWeight="bold">E</text>
        }}
    </g>
  );
}}}
{graph.pos.map((p, id) => {
  const c = nodeColor(id);
  const isCur = id === curV;
  const isNbr = id === curU;
  return (
    <g key={id}>
      <circle cx={p.x} cy={p.y} r={isCur ? 10 : 5} fill={isCur ? "#38bdf8" : c > 0 ? colorOf(c) : "#475569"} stroke="black" stroke-width={isCur ? 2 : 1}></circle>
      <text x={p.x} y={p.y + 4} textAnchor="center" font-size="10" font-weight="bold">{names[id]}</text>
      {c > 0 && tab !== "edges" && <text x={p.x} y={p.y + 10} font-size="10" font-weight="normal">{c}</text>}
      {tab === "edges" && <text x={p.x} y={p.y + 10} font-size="10" font-weight="normal">{id}</text>}
    </g>
  );
})}
</svg>
</div>

```

```

<div className="lg:col-span-2 space-y-3">
  <div className="bg-slate-900/70 border border-slate-800 p-4">
    <p className="text-sm font-bold text-white">Graph</p>
    <p className="text-xs text-slate-400 leading-relaxed">
      {tab === "edges" ? graph.edges.map((_, i) => {
        <span key={i} className="px-1.5 py-0.5">{i}</span>
      }) : graph.nodes.map((_, i) => {
        <span key={i} className="px-1.5 py-0.5">{i}</span>
      })
    }
  </div>
  <div className="bg-slate-950 border border-slate-800 p-4">
    <p className="text-[10px] uppercase tracking-wider">Edges</p>
    <div className="flex flex-wrap gap-1.5 font-mono text-sm">
      {tab === "edges" ? graph.edges.map((_, i) => {
        <span key={i} className="px-1.5 py-0.5">{i}</span>
      }) : graph.nodes.map((_, i) => {
        <span key={i} className="px-1.5 py-0.5">{i}</span>
      })
    }
  </div>

```

```

        </div>
    );
};

const Header: React.FC = () => {
    <div className="flex flex-wrap items-center justify-b
    <div className="flex items-center gap-3">
        <div className="p-2.5 bg-fuchsia-600 text-white r
            <Palette className="w-5 h-5" />
        </div>
        <div>
            <h3 className="text-lg sm:text-xl font-bold tex
            <p className="text-xs text-fuchsia-300">K5/K3,3
        </div>
    </div>
</div>
);

const Tabs: React.FC<{ tab: Tab; setTab: (t: Tab) => vo
    <div className="flex flex-wrap gap-2 mb-4">
        {TABS.map((t) => {
            <button
                key={t.id}
                onClick={() => setTab(t.id)}
                className={"flex items-center gap-1.5 px-3 py-2
                    -600 text-white" : "bg-slate-900/60 text-slat
            >
                {t.icon} {t.label}
            </button>
        })}
    </div>
);

export default PlanarColoringViz;

```

```

    }
    const crosses: number[] = [];
    for(let i=0;i<edges.length;i++) {
      for(let j=i+1;j<edges.length;j++) {
        if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
          if (!crosses.includes(i)) crosses.push(i);
          if (!crosses.includes(j)) crosses.push(j);
        }
      }
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
      const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
      const xing = crosses.includes(i);
      lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
        stroke={xing ? "#f43f5e" : "#4f46e5"} />);
    }
    const labels = ["A","B","C","D","E"];
    for(const p of pts) {
      circles.push(<g key={`c${p.id}`}>
        <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" />
        <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
      </g>);
    }
    return <>{lines}{circles}</>;
  })())
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20 z-20">
  V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
  <div className="absolute top-2 left-2 bg-slate-900 p-2 text-white">
    00/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
    <svg className="w-full h-full absolute inset-0">

```

```

        <circle cx={p.x} cy={p.y} r={8} fill=
        <text x={p.x} y={p.y+3} textAnchor="m
    </g>);
    }
    return <>{lines}{circles}</>;
  }}())}
</svg>
<div className="absolute bottom-2 left-2 right
  z-20">
  V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>
</div>

<div className="bg-amber-950/40 border border-amb
  ▲ Запомни: K5 и K3,3 – два кита непланарности.
  гомеоморфный K5 или K3,3 – он непланарен. Теорем
</div>
</div>
);
}

```

ФАЙЛ 64/111: src/components/PythonCompiler.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useCallback, useEffect, useMemo, useRef, useState } from 'react'
import {
  ArrowRightLeft,
  Check,
  CheckCircle2,
  ChevronDown,
  ChevronLeft,
  ChevronRight,
  ChevronUp,
  CircleStop,

```

```

chapterId: string;
chapterTitle: string;
/** Размер панели (px) управляется родителем и сохран
width?: number;
height?: number | null;
onWidthChange?: (w: number) => void;
onHeightChange?: (h: number) => void;
/** Перейти к странице пособия (посмотреть визуализац
/** Кнопка «открыть страницу в пособии» – не нужна, е
onOpenGuide?: () => void;
/** Скрыть панель. */
onClose: () => void;
}

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void;
  setStderr: (options: { batched: (message: string) => void;
  setStdin: (options: { stdin: () => string | number | null;
});

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideModule>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

/**
 * Подмена интерпретатора для проверок: DOM-харнес (jsdom)
 * из node_modules) и CI-скриншоты кладут сюда готовый
 * на CDN. В браузере у пользователя поле не определено
 */
type PythonRuntimeProvider = PyodideRuntime | (() => Promise<PyodideRuntime>);

declare global {
  interface Window {
    __algv0PythonRuntime?: PythonRuntimeProvider;
  }
}

```

Универсальное пособие • полный исходный код

```
-----  
/* скелет, шаг-в-шаг с подсветкой визуализации (см. data-visualizer.py)  
/* -----
```

```
/* -----  
/* Шпаргалка Python: команды сортировки, циклы, структуры данных  
/* «Вставить» кладёт сниппет в редактор под курсор.  
/* -----
```

```
interface Snippet {  
  label: string;  
  tip: string;  
  code: string;  
}
```

```
const SNIPPETS: { group: string; items: Snippet[] }[] =  
  [  
    {  
      group: "Сортировка",  
      items: [  
        {  
          label: "arr.sort() / sorted()",  
          tip: "In-place сортировка по возрастанию и новая массив  
          code: 'arr = [5, 2, 8, 1]\narr.sort()\n        ) # копия по убыванию\n',  
        },  
        {  
          label: "sorted(key=...)",  
          tip: "Сортировка по ключу: рёбра по весу, пары по длине  
          code: 'edges = [(3, "A", "B"), (1, "B", "C"), (2, "C", "A")]\nedges\n',  
        },  
      ],  
    },  
    {  
      group: "Циклы",  
      items: [  
        {  
          label: "for i in range(n)",  
          tip: "Вложенные циклы по i и j – как обход матрицы",  
          code: 'for i in range(n):\n  for j in range(n):\n    # ...'\n',  
        },  
      ],  
    },  
  ]
```

```

        },
        {
            label: "Множество visited",
            tip: "Посещённые вершины: in/add за O(1).",
            code: 'visited = set()\nvisited.add("A")\nprint',
        },
    ],
},
{
    group: "Вывод",
    items: [
        {
            label: "f-строка",
            tip: "Подставить значения переменных шага в строку",
            code: 'i, j, k = 1, 2, 0\nprint(f"dist[{i}][{j}] = {k}")',
        },
    ],
},
];

```

/**

* Переживает сккрытие панели: код пользователя не теряется
*

* Хранилище — ПО СТРАНИЦАМ. Раньше слот был один, и код
* восстанавливался на новой странице: редактор показывал
* и визуализация жили значениями чужого кода.

*/

```
const savedEditors = new Map<string, { code: string; base: string>>();
```

```
export function PythonCompiler({ chapterId, chapterTitle, chapterTitleProps }) {
```

```
    /** Активная вкладка демонстрации страницы (у sparse-массива)
```

```
    const [demoId, setDemoId] = useState<string | null>(null);
```

```
    const sync = useMemo(() => getPageSync(chapterId, demoId), [chapterId, demoId]);
```

```
    const syncVarBases = useMemo(() =>
```

```
        () =>
```

```
        [...new Set(
```

```
            sync.variables.flatMap((v) =>
```

```

* компонент, но `runPythonAsync` к тому моменту може
* этого флага доехавший до конца прогон публиковал с
* после закрытия: панель исчезла, а визуализация про
* кода, которого на экране больше нет.
*/
const aliveRef = useRef(true);
useEffect(() => {
  aliveRef.current = true;
  return () => {
    aliveRef.current = false;
  };
}, []);

// Код хранится по страницам, поэтому при открытии па
// бывает: рассинхрон возможен только после смены вкл
const [desyncedFrom, setDesyncedFrom] = useState<string>
const prevTplRef = useRef({ base: initTemplate, full:

/** Активная трасса. input нужен, чтобы изменение std
const [debug, setDebug] = useState<{ result: DebugRes
/** Esc оставляет неизменённый текст в редакторе, пок
const [editHold, setEditHold] = useState<string | null>
/** Автопроход включается пользователем; первый шаг п
const [playing, setPlaying] = useState(false);
const [playbackSpeed, setPlaybackSpeed] = useState<num
  if (typeof window === "undefined") return 1;
  const saved = Number(window.localStorage.getItem(PL
  return PLAYBACK_SPEEDS.includes(saved as (typeof PL
));
const listRef = useRef<HTMLDivElement>(null);

useEffect(() => {
  window.localStorage.setItem(PLAYBACK_SPEED_KEY, Str
}, [playbackSpeed]);

useEffect(() => {
  savedEditors.set(chapterId, { code, base: initTempl
}, [code, initTemplate, chapterId]);

```



```

    try {
      await navigator.clipboard.writeText(code);
      setCopied(true);
      setTimeout(() => setCopied(false), 1600);
    } catch {
      setRuntimeMessage("Не получилось скопировать – бу
    }
  }, [code]));

```

/* — Пошаговый отладчик

/** Связь больше не зависит от эталонных строк: совпа
 const hasViz = sync.variables.length > 0;

/**

* Снимок ПОСЛЕ выделенной строки. Контейнеры берём и
 * (там уже виден append/st[i][j]=...), но индексы цикл
 * события: после st[i][j]= Python уже мог увеличить
 */

```

const snapshotAt = useCallback((result: DebugResult,
  const step = result.steps[idx];
  const next = result.steps[idx + 1];
  const afterLocals = next?.locals ?? result.finalLoca
  const afterValues = next?.values ?? result.finalVal
  const beforeValues = step?.values ?? {};
  const line = step?.line ? source.split("\n")[step.l
  // Нас интересуют только простые скалярные цели (`i
  // `i +=`). Контейнеры и так всегда берутся из afte
  // внутри print(f"i={i}") не будет ошибочно принят
  const assignment = line.match(
    /^\\s*([A-Za-z_][A-Za-z0-9_]*(?:\\s*,\\s*[A-Za-z_][A
  );
  const assigned = new Set(assignment?.[1].split(",")

```

```

const variables = { ...afterValues };
const locals = { ...afterLocals };
for (const [name, value] of Object.entries(beforeVa
  const scalar = value === null || typeof value !==

```

```

        setDebug((d) => {
            if (!d || d.result.steps.length === 0) return d
            const idx = Math.max(0, Math.min(d.result.steps
            return { ...d, idx };
        });
    },
    []
);

const stepBy = useCallback(
    (delta: number) => {
        setDebug((d) => {
            if (!d || d.result.steps.length === 0) return d
            const idx = Math.max(0, Math.min(d.result.steps
            return { ...d, idx };
        });
    },
    []
);

const stepTo = useCallback((idx: number) => goDebug(i

const stopDebug = useCallback(() => {
    setPlaying(false);
    setEditHold(code);
    setDebug(null);
    clearVizState(chapterId);
}, [chapterId, code]);

// Базовый интервал 650 мс; множитель скорости хранитс
useEffect(() => {
    if (!playing || !debug) return;
    if (debug.idx >= debug.result.steps.length - 1) {
        setPlaying(false);
        return;
    }
    const t = window.setTimeout(() => stepBy(1), 650 /
    return () => window.clearTimeout(t);

```

```
runtime.setStdin({ stdin: () => (inputLines.length
```

```
const raw = await runtime.runPythonAsync(buildDebu
// Панель могли закрыть, пока Python считал: выход
if (!aliveRef.current) return;
const result = JSON.parse(String(raw)) as DebugRe
const printed = clipOutput([...stdout, ...stderr]
```

```
if (result.steps.length === 0) {
  // Шагов нет (только комментарии, синтаксическая
  // заменяется пустым листингом, а визуализация
  // показывает собственный встроенный сценарий (
  setDebug({ result, idx: 0, src: source, input,
  clearVizState(chapterId);
  setOutputOpen(true);
  setOutput(
    result.error
    ? `Ошибка:\n${result.error}`
    : printed || "В коде нет исполняемых строк
  });
  setRuntimeMessage(
    result.error
    ? "Код не выполнен — редактор остался отк
    : "Нет шагов трассы: связь с визуализацией
  );
} else {
  setDebug({ result, idx: 0, src: source, input,
  setPlaying(autoPlay && result.steps.length > 1)
  // Программа что-то напечатала — значит, вывод
  // результат своего же print() панель не имеет
  if (printed.trim() !== "") setOutputOpen(true);
  setOutput(
    (printed || "Программа ничего не вывела — тра
    (result.error ? `\\n\\nОшибка (выполнение пре
  );
  if (result.error) setOutputOpen(true);
  setRuntimeMessage(
    result.truncated
```

```

    }
    if (e.key === "ArrowLeft") {
      e.stopPropagation();
      e.preventDefault();
      setPlaying(false);
      stepBy(-1);
    }
    if (e.key === " ") {
      e.stopPropagation();
      e.preventDefault();
      setPlaying((v) => !v);
    }
    if (e.key === "Escape") stopDebug();
  };
  // capture-фаза: чтобы глобальный обработчик навига
  window.addEventListener("keydown", onKey, true);
  return () => window.removeEventListener("keydown",
}, [debug, stepBy, stopDebug]);

// держим текущую строку листинга в видимой области
/**
 * Панель показывает листинг трассы, только если шаги
 * Пустой код или синтаксическая ошибка больше не пря
 * раньше на их месте возникал пустой листинг без воз
 * «исчезала» и кнопка, и само поле ввода.
 */
const showTrace = !!debug && debug.result.steps.length
/** Трасса построена по старому тексту: пользователь
const stale = !!debug && debug.src !== code;
/** Листинг трассы можно свернуть, но редактор от это
const [traceOpen, setTraceOpen] = useState(true);
const curDebug = debug ? { step: debug.result.steps[d
const curDebugLine = curDebug?.step?.line;
useEffect(() => {
  if (curDebugLine === undefined || !listRef.current)
    listRef.current
      .querySelector(`[data-line="${curDebugLine}"]`)
      ?.scrollIntoView({ block: "nearest" });

```

```

        requestAnimationFrame(() => {
            ta.focus();
            ta.selectionStart = ta.selectionEnd = start + p;
        });
        return next;
    });
}, []));

return (
    <aside
        aria-label="Python-компилятор"
        className="fixed z-40 inset-x-0 bottom-0 h-[58dvh]
            -[520px] 2xl:w-[600px] flex flex-col bg-slate-900"
        style={typeof window !== "undefined" ? {
            ...(width && window.innerWidth >= 1024 ? { width: "100%" },
            ...(height ? { height: Math.min(height, window.innerHeight) },
            {} : undefined)
        } : undefined}
    >
        /* Верхний край меняет высоту на телефоне и десктопе */
        {onHeightChange && (
            <div
                role="separator"
                aria-orientation="horizontal"
                aria-label="Потяните, чтобы изменить высоту терминала"
                title="Потяните, чтобы изменить высоту терминала"
                className="absolute -top-1.5 left-0 right-0 h-10"
                onPointerDown={(event) => {
                    const el = event.currentTarget;
                    el.setPointerCapture(event.pointerId);
                    const startY = event.clientY;
                    const startH = el.parentElement?.getBoundingClientRect().height;
                    const onMove = (moveEvent: PointerEvent) => {
                        const maxHeight = window.innerHeight - (window.innerHeight > 260 ? 260 : 0);
                        const minHeight = Math.min(260, maxHeight);
                        const next = Math.round(startH + startY - moveEvent.clientY);
                        onHeightChange(Math.max(minHeight, Math.min(maxHeight, next)));
                    };
                    const onUp = () => {

```

```

    >
      <div className="absolute left-1/2 top-1/2 -tr
        digo-500 transition-colors" />
    </div>
  })
  { /* — Заголовок панели
<div className="shrink-0 flex flex-wrap items-cen
  <span className={`w-2 h-2 rounded-full shrink-0
  <Terminal className="w-4 h-4 text-emerald-400 s
  <span className="text-[13px] font-bold text-whi
  <span className="hidden sm:inline text-[10px] t

  <div className="ml-auto flex flex-wrap items-cen
    <Tooltip
      side="bottom"
      content={running ? "Python выполняется..." :
        "д и запустить автопроход"}
    >
      <button
        type="button"
        onClick={runOrTogglePlayback}
        disabled={running}
        className={`inline-flex items-center gap-
        0 ${
          playing
            ? "bg-amber-500/15 text-amber-300 hov
            : "bg-emerald-500/15 text-emerald-300
          }`}
        aria-label={running ? "Python выполняется
      >
        {running ? <Loader2 className="h-4 w-4 an
        " />}
        <span>{playbackSpeed}x</span>
      </button>
    </Tooltip>
    <Tooltip side="bottom" content="Скопировать к
      <button
        type="button"

```

```

type="button"
onClick={() => {
  const next = !refOn;
  setRefOn(next);
  // «Убрать эталон» – значит убрать код
  // выполняется, поэтому демонстрация бо
  // значений и честно показывает встроенн
  // странице скажет об этом явно).
  setCode(next ? template : "");
  setDesyncedFrom(null);
  setPlaying(false);
  setEditHold(null);
  setDebug(null);
  clearVizState(chapterId);
  setTraceRequest((n) => n + 1);
}}
className={`p-1.5 rounded-lg transition-c
  refOn
    ? "bg-indigo-500/30 text-white ring-1
    : "text-indigo-300 hover:text-white h
  }`
aria-label={refOn ? "Убрать эталонный код
ктор"}
aria-pressed={refOn}
>
  {refOn ? <Eye className="w-4 h-4" /> : <E
</button>
</Tooltip>
<label
  className="ml-1 inline-flex items-center gap
  der border-slate-700 hover:border-emerald-5
  title="Скорость автоматического проигрывани
>
  {running ? <Loader2 className="w-3.5 h-3.5
  ld-400" />}
  <span className="hidden xl:inline">{running
  <select
    aria-label="Скорость автопрохода"
```

```

        ? "Связь идёт по именам ниже: реально
а не сравнивается."
        : "На выбранной странице нет интеракт
    }
>
    <span
      tabIndex={0}
      className={`cursor-help inline-flex item
        hasViz ? "text-emerald-400" : "text-s
      }`}
    >
      <Link2 className="w-3 h-3" />
      {hasViz ? "синхронизировано" : "нет виз
    </span>
  </Tooltip>
</div>
</div>

{stripTab === "vars" ? {
  <div className="px-3 pb-2.5 space-y-2 max-h-3
    <div className="flex items-center gap-2 min
      {onOpenGuide ? {
        <Tooltip side="bottom" content="Открыть
          <button
            type="button"
            onClick={onOpenGuide}
            className="inline-flex items-center
e-700 text-slate-300 hover:text-white hover
          >
            <span className="truncate">{chapter
          </button>
        </Tooltip>
      } : {
        <span className="inline-flex items-cent
text-slate-300 max-w-full">
          <span className="truncate">{chapterTi
        </span>
      }}
  }}

```



```

        >
        {v.name}
      </code>
    </Tooltip>
  )))
  {sync.stepNote && {
    <Tooltip side="bottom" content={`War
      <span
        tabIndex={0}
        className="cursor-help inline-flex
text-[10px] text-slate-400 hover:text-white
      >
        <ArrowRightLeft className="w-3 h-3
        <span className="truncate max-w-5
      </span>
    </Tooltip>
  }}
</div>
))
</div>
) : {
  <div className="px-3 pb-2.5 max-h-36 overflow
    {SNIPPETS.map(({gItem}) => {
      <div key={gItem.group}>
        <div className="text-[9px] font-bold up
        <div className="flex flex-wrap gap-1.5"
          {gItem.items.map(({s}) => {
            <Tooltip key={s.label} side="bottom
              <button
                type="button"
                onClick={() => insertSnippet(s.
                className="inline-flex items-ce
              -[10.5px] text-indigo-300 hover:text-white
                title=""
              >
                <Plus className="w-3 h-3 text-s
                  {s.label}
                </button>

```

```

        setCode(event.target.value);
    }}
    onKeyDown={(event) => {
        if ((event.ctrlKey || event.metaKey) && event.defaultPrevented) {
            event.preventDefault();
            void debugRun();
        }
        event.stopPropagation();
    }}
    spellCheck={false}
    aria-label="Код Python"
    className="flex-[2] min-h-[110px] w-full resize-none focus:ring-2 focus:ring-inset focus:ring-blue-500"
    placeholder="Напишите Python-код... (пустой редактор)"
/>

```

```

{showTrace && debug && (
    <div className={`flex flex-col border-t border-l border-r border-b`}
        {/* панель управления шагами */}
        <div className="shrink-0 flex flex-wrap items-center gap-2"
            <Tooltip content={playing ? `Пауза автопрогона` : `Включить автопрогон`}
                роиграть трассу заново со скоростью ${playbackRate} раз`}
                <button
                    type="button"
                    onClick={runOrTogglePlayback}
                    className={`mx-0.5 inline-flex items-center justify-center gap-2 rounded-md border border-transparent bg-white px-2.5 py-1.5 text-sm font-medium transition-colors`}
                    aria-label={playing ? `Пауза автопрогона` : `Включить автопрогон`}
                >
                    {playing ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
                    <span className="text-[9px] font-bold leading-tight">{playing ? "Пауза" : "Включить"}</span>
                </button>
            </Tooltip>
            <Tooltip content="В начало трассы">
                <button type="button" aria-label="В начало трассы"
                    disabled={!curDebug?.step} className="p-1 transition-colors">

```

```

        ЛИМИТ
      </span>
    </Tooltip>
  })
  <Tooltip content={hasViz ? "Визуализация :
st... Эталонный текст не обязателен – пишите
нет имён, связанных с демонстрацией; Python
    <span
      tabIndex={0}
      className={`cursor-help inline-flex i
"border-emerald-500/40 bg-emerald-500/10 t
    >
      {hasViz ? <Link2 className="w-3 h-3"
      {hasViz ? "имена → демо" : "свободный
    </span>
  </Tooltip>
  <Tooltip content={traceOpen ? "Свернуть л
и переменные текущего шага."}>
    <button
      type="button"
      aria-label={traceOpen ? "Свернуть лис
      aria-expanded={traceOpen}
      onClick={() => setTraceOpen((v) => !v
      className="ml-auto inline-flex shrink
ext-[10px] font-bold text-slate-300 transit
    >
      {traceOpen ? <ChevronDown className="
    ЛИСТИНГ
  </button>
</Tooltip>
  <Tooltip content="Остановить прогон (Esc)
на показывает встроенное демо. Автозапуск н
    <button
      type="button"
      aria-label="Остановить прогон трассы"
      onClick={stopDebug}
      className="inline-flex shrink-0 items
ext-[10px] font-bold text-rose-300 transit

```

```

<div className="flex items-center gap-1.5"
  <Variable className="w-3 h-3 text-indigo-500" value={value} />
  Переменные на этом шаге
  <span className="normal-case tracking-normal" />
</div>
{debug.result.error && (
  <pre className="mt-1 whitespace-pre-wrap" />
)}
{debugOrdered.length === 0 && !debug.result.error && (
  <div className="mt-1 text-[11px] text-slate-500" />
)}
</div>
)}
<div className="mt-1 flex flex-wrap gap-1"
  {debugOrdered.map(([name, value]) => {
    const changed = debugChanged[name];
    const synced = syncVarSet.has(name);
    return (
      <span
        key={name}
        className={`inline-flex items-baseline gap-1.5
          ${changed ? "border-amber-500/50 bg-amber-500/10" : ""}
          ${synced ? "border-emerald-500/40 bg-emerald-500/10" : ""}
          ${!changed & !synced ? "border-slate-700 bg-slate-700/10" : ""}
        `}
      >
        <span className={changed ? "text-amber-500" : ""}>{value}</span>
        <span className="text-slate-500">{name}</span>
        <span className={changed ? "text-amber-500" : ""}>{syncVarSet.get(name)}</span>
      </span>
    );
  })}
</div>
</div>
</>

```

```
        <span className="inline-flex items-center gap-2">
          {runtimeReady && <CheckCircle2 className="w-4 h-4 text-green-500"/>}
          <span className="text-slate-600">{outputOpen ? 'Running' : 'Ready'}</span>
        </span>
      </button>
      {outputOpen && (
        <pre
          data-testid="compiler-output"
          className="flex-1 min-h-[70px] overflow-auto text-slate-500 font-mono text-sm padding-5 text-slate-300"
        >
          {output}
        </pre>
      )}
    <div className="px-3 py-1.5 border-t border-slate-200 border-l border-r border-b">
      </div>
    </aside>
  );
}
```

ФАЙЛ 65/111: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray, vizString } from '../viz-runtime';
```

```
interface Customer {
  id: string;
  name: string;
  emoji: string;
  color: string;
  bubbleText: string;
  animState: 'in' | 'idle' | 'eating' | 'out';
}
```

```

        animState: "idle",
      )))
    : queue;

const [servedHistory, setServedHistory] = useState<st
const [isServing, setIsServing]         = useState(fa
const [shakeEmpty, setShake]             = useState(fa
const [log, setLog]                      = useState<st

const addLog = (msg: string) => setLog(prev => [msg,

// ENQUEUE: Встать в конец очереди (хвост / Tail)
const enqueue = useCallback(() => {
  if (isServing) return;
  if (queue.length >= 6) {
    addLog('⚠ Хвост очереди уже на улице, повару тяже
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }

  const preset = PRESETS[counter % PRESETS.length];
  counter++;

  const newGuest: Customer = {
    id: Date.now().toString(),
    ...preset,
    animState: 'in',
  };

  setQueue(prev => [...prev, newGuest]);
  addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoj

  setTimeout(() => {
    setQueue(prev => prev.map(c => c.id === newGuest.
  }, 450);
}, [queue, isServing]);

```

```

        апишу ИИ за порцию борща!', animState: 'in' },
        { id: 'r3', name: 'Кот Борис', emoji: ' ', color: 'red',
          без сметаны, плиз.', animState: 'in' },
      ]));
      setServedHistory([]);
      setIsServing(false);
      setLog([' Столовая сброшена. Снова голодная толпа!']);
      setTimeout(() => {
        setQueue(prev => prev.map(c => ({ ...c, animState: 'in' })), 500);
      });

      return (
        <div className="flex flex-col gap-6">
          {/* МНЕМОНИКА / ПРАВИЛО */}
          <div className="bg-indigo-950/40 border border-indigo-400">
            <div className="text-3xl"> </div>
            <div>
              <h3 className="font-black text-indigo-400 text-2xl">ПРАВИЛО</h3>
              <p className="text-xs text-slate-300 mt-1 leading-tight">
                В очереди за супом всё честно. Кто первый встал, тот и ест.
                Новые гости встают строго <b>в конец</b> очереди.
                Здесь нет обхода и блатов — только строгий порядок.
              </p>
            </div>
          </div>

          {/* УПРАВЛЕНИЕ */}
          <div className="flex items-center gap-3 flex-wrap">
            <button
              onClick={enqueue}
              disabled={!isServing}
              className="flex-1 min-w-[150px] bg-gradient-to-r from-indigo-500 to-indigo-400 opacity-40 disabled:cursor-not-allowed text-white font-medium rounded-lg shadow-lg active:scale-95 transition-all cursor-pointer">
              <span> Встать в очередь (ENQUEUE)</span>
            </button>

```



```

        }}
      </div>

    </div>
  </div>

  {/* Пол кухни */}
  <div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
<div className="md:col-span-3 bg-slate-900 round
  <div className="absolute top-4 left-4 text-[1
    Сытые гости (Архив FIFO)
  </div>

  <div className="absolute top-4 right-4 flex i
    order-emerald-800/80 text-[9px] font-mono">
    <Sparkles className="w-3.5 h-3.5 animate-pu
    <span>СЫТЫ </span>
  </div>

  <div className="flex-1 flex flex-col justify-
    {servedHistory.length === 0 ? (
      <div className="flex-1 flex flex-col item
        <span className="text-5xl mb-2"> </span>
        <p className="text-xs font-bold font-mo
        <p className="text-[10px] mt-1">Здесь 6
      </div>
    ) : (
      <div className="flex flex-col gap-1.5 w-fr
        {servedHistory.map((item, idx) => (
          <div
            key={idx}
            className="w-full py-1.5 px-3 round
          0 flex items-center gap-2 text-[11px] text-
            >
            <span>{item}</span>

```

```

interface TrieNode {
  id: number;
  char: string;
  parent: number;
  children: { [key: string]: number };
  fail: number;
  term_link: number;
  is_terminal: boolean;
  words: string[];
  depth: number;
  x?: number;
  y?: number;
}

export const SalmonAutomatonWidget: React.FC = () => {
  const [words, setWords] = useState<string[]>(['CAR',
  const [newWord, setNewWord] = useState('');
  const [nodes, setNodes] = useState<{ [key: number]: T
  const [simulationStep, setSimulationStep] = useState<
  const [bfsQueue, setBfsQueue] = useState<number[]>([
  const [currentNode, setCurrentNode] = useState<number
  const [speechText, setSpeechText] = useState<string>('
    'Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и
      буль!')
  });
  const [isTalking, setIsTalking] = useState<boolean>(f
  const [isBlinking, setIsBlinking] = useState<boolean>

  /* — связь с Python-компилятором (PAGE_SYNC["aho-con
    words/patterns – набор образцов: бор ПЕРЕСТРАИВАЕТ
    state – текущая вершина автомата, step – шаг BFS,
  const chapterId = useContext(VizChapterContext);
  const runtime = useVizRuntime();
  const vars = runtime?.variables;

  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, "aho");

```

```

        if (q.length) setBfsQueue(q);
    }
    // eslint-disable-next-line react-hooks/exhaustive-
}, [vars]));

const compilerLinked = Boolean(liveWordsSig || liveSt

// Регулярное моргание рыбы
useEffect(() => {
    const blinkInterval = setInterval(() => {
        setIsBlinking(true);
        setTimeout(() => setIsBlinking(false), 200);
    }, 4000);
    return () => clearInterval(blinkInterval);
}, []);

// Эффект разговора при смене текста
useEffect(() => {
    setIsTalking(true);
    const timer = setTimeout(() => setIsTalking(false),
    return () => clearTimeout(timer);
}, [speechText]);

// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
    const newNodes: { [key: number]: TrieNode } = {
        0: { id: 0, char: 'ROOT', parent: 0, children: {}
    };
    let count = 0;

    wordList.forEach((word) => {
        let curr = 0;
        for (let i = 0; i < word.length; i++) {
            const c = word[i].toUpperCase();
            if (newNodes[curr].children[c] === undefined) {
                count++;
                newNodes[curr].children[c] = count;
                newNodes[count] = {

```

```

        newNodes[u].x = leftX === -1 ? 40 + currentX * 2 : leftX;
        if (leftX === -1) currentX++;
    }
};

layoutDFS(0, 0);

setNodes(newNodes);
setSimulationStep(0);
setBfsQueue([]);
setCurrentNode(null);
setSpeechText(
    `Отлично! Я построил базовое префиксное дерево (БПД)
    S, чтобы расставить fail-ссылки!`
);
};

useEffect(() => {
    buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
    e.preventDefault();
    const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
    if (!cleanWord) return;
    if (words.includes(cleanWord)) {
        setSpeechText(`Слово «${cleanWord}» уже есть в слове S`);
        return;
    }
    const updated = [...words, cleanWord];
    setWords(updated);
    setNewWord('');
    buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
    if (words.length <= 1) {
        setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем построить БПД');
        return;
    }
    const updated = words.filter(word => word !== wordToRemove);
    setWords(updated);
    buildInitialTrie(updated);
};

```

```

    const r = bfsQueue[0];
    const newQueue = bfsQueue.slice(1);
    const updatedNodes = { ...nodes };
    const rNode = updatedNodes[r];
    const childrenIds = Object.values(rNode.children);

    let logs = `Рассматриваем вершину #${r} (${rNode.char})`;

    for (const [char, u] of Object.entries(rNode.children)) {
        newQueue.push(u);
        let v = rNode.fail;
        while (v !== 0 && updatedNodes[v].children[char] === undefined) {
            v = updatedNodes[v].fail;
        }
        const failTarget = updatedNodes[v].children[char];
        updatedNodes[u].fail = failTarget;

        // Рассчитываем терминальную ссылку (term_link)
        if (updatedNodes[failTarget].is_terminal) {
            updatedNodes[u].term_link = failTarget;
        } else {
            updatedNodes[u].term_link = updatedNodes[failTarget].term_link;
        }

        logs += `Для #${u} ('${char}') fail = #${failTarget}`;
    }

    if (childrenIds.length === 0) {
        logs += 'У этой вершины нет детей, просто переходим к следующему';
    }

    setNodes(updatedNodes);
    setBfsQueue(newQueue);
    setCurrentNode(r);
    setSpeechText(logs);
};

return {

```

```

>
  { /* Тело лосося */ }
  <ellipse cx="100" cy="100" rx="70" ry="45" fill="#e67e22" />
  <path d="M 40 85 Q 100 55 160 85 Q 100 115 40 85 Z" fill="#e67e22" />

  { /* Хвост */ }
  <path
    d="M 35 100 L 5 70 L 15 100 L 5 130 Z"
    fill="#be123c"
    className="origin-[35px_100px] animate-rotate-90deg"
  />

  { /* Плавники */ }
  <path d="M 90 55 L 70 30 L 110 55 Z" fill="#f87171" />
  <path d="M 90 145 L 70 170 L 110 145 Z" fill="#f87171" />
  <path
    d="M 120 115 L 100 135 L 130 130 Z"
    fill="#f87171"
    className="origin-[120px_115px] animate-rotate-90deg"
  />

  { /* Глаз (с морганием) */ }
  <circle cx="145" cy="85" r="12" fill="#fff" />
  {isBlinking ? (
    <line x1="133" y1="85" x2="157" y2="85" />
  ) : (
    <circle cx="148" cy="85" r="6" fill="#fff" />
    <circle cx="151" cy="82" r="2" fill="#fff" />
  )}

  { /* Рот (анимация разговора) */ }
  {isTalking ? (
    <path d="M 165 105 Q 155 120 165 125 Q 165 110 155 115 168 118 Z" />
  ) : (
    <path d="M 165 110 Q 155 115 168 118 Z" />
  )}

```



```

        stroke={isCurrent ? '#60a5fa' : 'rgb(128, 128, 128)'}
        strokeWidth="2"
        markerEnd="url(#arrowhead)"
      />
      <text
        x={({parent.x + node.x) / 2 - 10}
        y={({parent.y + node.y) / 2}
        fill="#94a3b8"
        fontSize="12"
        fontWeight="bold"
      >
        {node.char}
      </text>
    </g>
  );
}
}

/* Draw Fail Links */
simulationStep > 0 && Object.values(nodes).length > 1 &&
  if (node.id === 0 || node.fail === 0) return;
  // Wait, all nodes get fail link. Let's wait until
  step >= 1 and computed it
  // To avoid too many lines, let's draw only one
  const target = nodes[node.fail];
  if (!target || !node.x || !node.y || !target.x || !target.y) return;

  // Simple bezier curve for fail link
  const isCurrent = currentNode === node;

  return (
    <path
      key={`fail-${node.id}`}
      d={`M ${node.x} ${node.y - 10} Q ${node.x + 16} ${node.y - 16} ${target.x} ${target.y - 10}`}
      fill="none"
      stroke={isCurrent ? '#f43f5e' : 'rgb(128, 128, 128)'}
      strokeWidth={isCurrent ? 2 : 1.5}
    />
  );
}
}

```



```

        x={node.x}
        y={node.y + 4}
        fill="white"
        fontSize={isRoot ? "10" : "12"}
        fontWeight="bold"
        textAnchor="middle"
      >
        {isRoot ? 'R' : node.char}
    </text>
    {!isRoot && {
      <text
        x={node.x + 12}
        y={node.y - 12}
        fill="#94a3b8"
        fontSize="9"
      >
        {node.id}
      </text>
    }}
  </g>
);
}}
</svg>
);
}}()
</div>

```

```

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
  <div className="bg-slate-900/50 p-5 rounded-xl">
    <div>
      <h5 className="text-white font-bold mb-3">
        <span> </span> Словарь (Паттерны)
      </h5>
      <div className="flex flex-wrap gap-2 mb-4">
        {words.map((w) => {
          <span
            key={w}

```

```

<h5 className="text-white font-bold mb-3 flex
  <span className="flex items-center gap-1.5">
    <span> </span> Таблица переходов и fail-
  </span>
  <span className="text-xs text-slate-400 font
    Всего вершин: {Object.keys(nodes).length}
  </span>
</h5>

```

```

<div className="max-h-[220px] overflow-y-auto
  <table className="w-full text-left border-c
    <thead>
      <tr className="border-b border-slate-700
        <th className="py-2 px-3">ID Вершины</th>
        <th className="py-2 px-3">Символ</th>
        <th className="py-2 px-3">Родитель</th>
        <th className="py-2 px-3">fail-ссылка</th>
        <th className="py-2 px-3">term_link</th>
        <th className="py-2 px-3">Дети</th>
        <th className="py-2 px-3">Слова (dict
      </tr>
    </thead>
    <tbody className="divide-y divide-slate-800
      {Object.values(nodes).map((node) => {
        const isCurrent = currentNode === node.id
        const isInQueue = bfsQueue.includes(node.id)

        return (
          <tr
            key={node.id}
            className={`transition-colors ${
              isCurrent
                ? 'bg-blue-900/40 font-bold text-white'
                : isInQueue
                ? 'bg-slate-800/80'
                : 'hover:bg-slate-800/40'
            }`}
          >

```

```

                #{node.term_link}
            </span>
        )}
    </td>
    <td className="py-2.5 px-3 text-slate-600">
        {Object.keys(node.children).length > 0 ? (
            {Object.entries(node.children)
                .map([c, id] => `${c}→#id=${id}`)
                .join(', ')}
        ) : ''}
    </td>
    <td className="py-2.5 px-3">
        {node.words.length === 0 ? (
            <span className="text-slate-600">
                {node.words.join(', ')}
            </span>
        ) : (
            <span className="bg-emerald-900 text-white">
                {node.words.join(', ')}
            </span>
        )}
    </td>
</tr>
</tbody>
</table>
</div>
</div>
</div>

```

```

/* что сейчас сообщил компилятор */
<div className="mt-4 flex flex-wrap items-center">
    <span className="text-[10px] uppercase tracking-wider">
        <Terminal className="w-3.5 h-3.5" /> из компилятора
    </span>
    {compilerLinked ? (
        <div>
            {liveWordsSig && <span className="text-[11px] font-mono">
                words = {words.join(", ")}</span>
            }
        </div>
    ) : ''}
</div>

```

```
-----
    arrayHighlight?: [number, number];           // [L, R]
    result?: number;                             // partial
}
```

// ===== PURE LOGIC: build tree =====

```
function buildTreeFull(arr: number[]): Record<number, number> {
    const n = arr.length;
    const tree: Record<number, number> = {};
    function go(v: number, l: number, r: number) {
        if (l === r) { tree[v] = arr[l]; return; }
        const m = (l + r) >> 1;
        go(2 * v, l, m);
        go(2 * v + 1, m + 1, r);
        tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
    }
    go(1, 0, n - 1);
    return tree;
}
```

// ===== STEP GENERATORS =====

// 1. Build steps (recursive top-down)

```
function genBuildSteps(arr: number[]): Step[] {
    const n = arr.length;
    const steps: Step[] = [];
    let id = 0;
    const tree: Record<number, number> = {};

    const CODE = [
        "build(v, l, r) {",           // 1
        "    if (l === r) {",         // 2
        "        tree[v] = A[l]; return", // 3
        "    }",                       // 4
        "    mid = (l + r) >> 1",      // 5
        "    build(2v, l, mid)",        // 6
        "    build(2v+1, mid+1, r)",    // 7
        "    tree[v] = tree[2v]+tree[2v+1]", // 8
        "}",                           // 9
    ];
```

```

        steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
            codeLine: 2, treeState: tree, highlightNodes: [v]
        });
        return 0;
    }
    if (ql <= l && r <= qr) {
        steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
            codeLine: 4, treeState: tree, highlightNodes: [v]
        });
        return tree[v];
    }
    const m = (l + r) >> 1;
    steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
        codeLine: 6, treeState: tree, highlightNodes: [v]
    });
    const left = go(2 * v, l, m, ql, qr);
    const right = go(2 * v + 1, m + 1, r, ql, qr);
    const res = left + right;
    steps.push({ id: id++, desc: `Возврат в узел ${v}:`,
        codeLine: 8, treeState: tree, highlightNodes: [v],
        rangeChildren: [2 * v, 2 * v + 1], arrayHighlight: [2 * v, 2 * v + 1]
    });
    return res;
}
const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}]`,
    codeLine: 10, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: ans
});
return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree)

```

function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
    const n = arr.length;
    // Build iterative tree: leaves at [n..2n-1]
    const tree: Record<number, number> = {};
    for (let i = 0; i < n; i++) tree[n + i] = arr[i];
    for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]);

    const steps: Step[] = [];
    let id = 0;
    let l = qL + n;
    let r = qR + n + 1; // half-open [l, r)
    let res = 0;

```

```

return { v, l, r, val: treeState[v] ?? null, left: build(
    v + 1, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```

const BUILD_CODE = [
    "build(v, l, r) {",
    "    if (l === r) { tree[v] = A[l]; return; }",
    "    // -----",
    "    // -----",
    "    mid = (l + r) >> 1;",
    "    build(2*v, l, mid);",
    "    build(2*v+1, mid+1, r);",
    "    tree[v] = tree[2*v] + tree[2*v+1];",
    "}"
]

```

```

];
const QUERY_TD_CODE = [
    "query(v, l, r, ql, qr) {",
    "    if (ql > r || qr < l) return 0;",
    "    // -----",
    "    if (ql <= l && r <= qr) return tree[v];",
    "    // -----",
    "    mid = (l + r) >> 1;",
    "    // спускаемся в обоих детей",
    "    return query(left) + query(right);",
    "}"
]

```

```

];
const QUERY_BU_CODE = [
    "queryBU(ql, qr) {",
    "    res = 0;",
    "    l = ql + n; r = qr + n + 1;",
    "    // -----",
    "    while (l < r) {",
    "        if (l & 1) res += tree[l++];",
    "        // -----",
    "        if (r & 1) res += tree[--r];",
    "        l >>= 1; r >>= 1;",
    "    }",
    "    return res; }"
]

```

```

if (!p) return null;
const { step, idx, setIdx, playing, setPlaying, speed
const n = arr.length;

const fullTree = useMemo(() => buildTreeFull(arr), [a
const vTree = useMemo(() => buildVisualTree(step.tree

const renderNode = useCallback((nd: VNode): React.Rea
  const isCompilerActive = liveV === nd.v;
  const isHigh = step.highlightNodes.includes(nd.v) |
  const isChild = step.mergeChildren?.includes(nd.v);

  // Live compiler value or step value or pre-calcula
  let displayVal: number | string = nd.val !== null ?
  if (liveTree && liveTree[nd.v] !== undefined && typ
    displayVal = liveTree[nd.v] as number;
  }

  let cls = 'bg-slate-800 border-slate-700 text-slate
  if (isHigh) cls = `${clr.ring} text-white ring-2 sc
  else if (isChild) cls = `${clr.childRing} text-ambe
  else cls = `${clr.filled} text-slate-200`;

  return (
    <div key={nd.v} className="flex flex-col items-ce
      <div className={`flex flex-col items-center jus
        ] ${cls}`}>
        <div className="flex items-center gap-1 text-
          <span>v{nd.v}</span>
          {isCompilerActive && <span className="bg-am
        </div>
        <span className="text-[9px] font-bold text-am
        <span className="text-xs font-black font-mono
          {nd.l === nd.r ? `A[${nd.l}] = ${displayVal
        </span>
      </div>
    {(nd.left || nd.right) && (

```

```

        <span className="text-[12px] font-extralight">
      </div>
    );
  }}}
</div>
</div>

{/* Tree */}
<div className="flex-1 p-2 overflow-x-auto bg-slate-900">
  <div className="scale-[0.85] origin-top min-w-max">
    <pre>
      <pre className="px-3 py-2 text-[9px] md:text-[10px] font-mono">
        {code.map((line, i) => {
          const ln = i + 1;
          const active = step.codeLine === ln;
          return (
            <div key={ln} className={`px-2 py-0.5 rounded-2 border-indigo-500 text-indigo-200' : color === 'emerald' ? 'text-emerald-300' : 'text-emerald-400'}`>
              <span className="inline-block w-4 text-slate-400">{ln}</span>
            </div>
          );
        })}
      </pre>

      {/* Description + result */}
      <div className="px-3 py-2.5 bg-slate-900/60 border-slate-700">
        <span className={`mt-0.5 inline-block w-2 h-2 rounded-2 border-indigo-500 text-indigo-200' : color === 'emerald' ? 'text-emerald-300' : 'text-emerald-400'}`>{active}</span>
        <span className="leading-relaxed">{step.desc}</span>
        {step.result !== undefined && (
          <span className={`ml-auto shrink-0 font-mono text-[9px] md:text-[10px] font-mono' : color === 'emerald' ? 'text-emerald-300' : 'text-emerald-400'}`>
            {step.result}
          </span>
        )}
      </div>
    </pre>
  </div>
</div>

```



```

        <span className="text-xs text-slate-400">
        <input
          type="number"
          min={0}
          max={arr.length - 1}
          value={qL}
          onChange={e => setQL(Math.min(Number(e.target.value),
            className="w-14 bg-slate-900 border border-indigo-500
            focus:border-indigo-500"
          />
        </div>
        <div className="flex items-center gap-1">
        <span className="text-xs text-slate-400">
        <input
          type="number"
          min={0}
          max={arr.length - 1}
          value={qR}
          onChange={e => setQR(Math.min(Number(e.target.value),
            className="w-14 bg-slate-900 border border-indigo-500
            focus:border-indigo-500"
          />
        </div>
        </div>
        <div className="text-[10px] text-slate-500">
        Запрос:  $\text{sum}(A[\{qL\}..\{qR\}]) = \{arr.slice(qL, qR+1).reduce((sum, val) => sum + val, 0))\}$ 
        </div>
        </div>
        </div>
        </div>
        </div>

        {/* ---- TABS ---- */}
        <div className="flex flex-wrap items-center gap-1">
        <button onClick={() => setView('build')} className="text-sm font-medium
          -colors ${view === 'build' ? 'border-indigo-500 text-slate-200'}`}>
        <Hammer className="w-3.5 h-3.5" /> Построить
        </button>

```



```

        <p className="text-sm text-slate-300 leading-4">
          <strong className="text-white">Итеративный способ проверки корректности. Но если нужны <strong className="text-white">рекурсивный (сверху-вниз)</strong> способ.
        </p>
      </div>
    </div>
  )}
</div>
);
}

```

ФАЙЛ 68/111: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } from "lucide-react";
import { chapters, SECTIONS, sectionOf } from "../data/chapters";
import { VIZ_REGISTRY } from "../vizRegistry";

interface SidebarProps {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Мобильный режим: список живёт в выдвижной панели */
  onNavigate?: () => void;
  /** Узкий режим: номер билета и название в одну строку */
  compact?: boolean;
}

export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId, onNavigate, compact }) => {
  const [query, setQuery] = useState("");

  /**
   * Группы – это разделы пособия (SECTIONS), а не случайные темы.
   * связанные темы стоят рядом, порядок разделов не важен.
   */

```

```

        >
        <X className="w-3.5 h-3.5" />
      </button>
    })
  </div>
})
{!compact && (
  <span className="text-[10px] text-slate-500 i
    Порядок чтения: билеты 1 → 24
  </span>
)}
</div>

<div className="flex-1 overflow-y-auto px-3 pb-4
  {groups.length === 0 && <p className="text-sm t

{groups.map(({ section, items }) => (
  <div key={section.id}>
    <div className="flex items-baseline gap-1.5
      <span className="text-[10px] font-bold up
        {section.from > 0 && (
          <span className="text-[10px] text-slate
        )}
      </div>
    <div className="space-y-1">
      {items.map((chapter) => {
        const isSelected = chapter.id === selec
        const hasViz = Boolean(VIZ_REGISTRY[cha
        // «18–20. Графы. Основное дерево...» → з
        const num = chapter.title.match(/^(\d+{
        const label = num ? chapter.title.slice
        return (
          <button
            key={chapter.id}
            onClick={() => {
              setSelectedChapterId(chapter.id);
              onNavigate?.();
              window.scrollTo({ top: 0, behavio

```

ФАЙЛ 69/111: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar
```

```
interface Plate {
  id: string;
  name: string;
  icon: string;
}
```

```
interface Person {
  id: string;
  name: string;
  icon: string;
}
```

```
interface Hypothesis {
  id: string;
  text: string;
  probability: number;
}
```

```
export const Simulator: React.FC = () => {
  const [activeTab, setActiveTab] = useState<'stack' |

  // Stack state
  const [stack, setStack] = useState<Plate[]>([
    { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
    { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
    { id: '3', name: 'Блюдец от торта', icon: ' ' },
  ]));
  const [popMessage, setPopMessage] = useState<string |
```

```

const popPlate = () => {
  if (stack.length === 0) {
    setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
    return;
  }
  const topPlate = stack[stack.length - 1];
  setStack(prev => prev.slice(0, prev.length - 1));
  setPopMessage(`Вымыта верхняя тарелка (LIFO): ${topPlate.name}`);
};

const resetStack = () => {
  setStack([
    { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
    { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
    { id: '3', name: 'Блюдец от торта', icon: '🍰' },
  ]);
  setPopMessage("Стол тарелок восстановлен.");
};

// Queue handlers
const addPerson = () => {
  const presets = [
    { name: 'Любитель борща', icon: '🥘' },
    { name: 'Курьер доставки', icon: '📦' },
    { name: 'Усталый админ', icon: '🧑' },
    { name: 'Турист с рюкзаком', icon: '🎒' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPerson = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon,
  };
  setQueue(prev => [...prev, newPerson]);
  setDequeueMessage(`Встал в конец очереди: ${newPerson.name}`);
};

const servePerson = () => {

```

```

    let maxIdx = 0;
    for (let i = 1; i < heap.length; i++) {
      if (heap[i].probability > heap[maxIdx].probability) {
        maxIdx = i;
      }
    }
    const best = heap[maxIdx];
    setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
    setHeapMessage(`  Выбран самый "тяжелый" кандидат:
  `);
};

```

```

const resetHeap = () => {
  setHeap([
    { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
    { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
    { id: '3', text: 'Кандидат: "Привет, я испанский язык"' },
  ]);
  setHeapMessage("  Гипотезы восстановлены.");
};

```

```

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability);

```

```

return (
  <div className="bg-slate-900/90 border border-slate-800 p-3">
    <div className="border-b border-slate-800 pb-6 mb-6">
      <h2 className="text-2xl font-bold text-transparent">
        Интерактивный симулятор структур данных
      </h2>
      <p className="text-slate-400 text-sm mt-2">
        Проверь работу принципов LIFO, FIFO и Приоритетной очереди
      </p>
    </div>

    { /* Tabs */ }
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      <button

```



```

    <button
      onClick={() => setActiveTab('heap')}
      className={`flex items-center justify-between
        activeTab === 'heap'
          ? 'bg-emerald-600/20 border-emerald-500 te
          : 'bg-slate-800/60 border-slate-700/60 te
      `}
    >
    <div className="flex items-center gap-3">
      <Award className={`w-5 h-5 ${activeTab ===
      <div className="text-left">
        <div className="font-bold text-sm text-wh
        <div className="text-xs opacity-80">Приор
      </div>
    </div>
    <span className="text-xs bg-emerald-500/20 te
      Beam Search
    </span>
  </button>
</div>

{/* STACK TAB */}
{activeTab === 'stack' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Симуляция стопки тарелок</span>
          <span className="text-xs font-mono bg-b
        </h3>
        <p className="text-slate-400 text-xs mt-1
          Новые тарелки ставятся наверх. Мыть мож
        </p>
      </div>
    <div className="flex flex-wrap items-center
      <button

```

```

        <p className="text-base font-bold">Стоп
        <p className="text-xs mt-1">Добавьте гр
    </div>
  ) : (
    <div className="w-full max-w-sm flex flex
      <div className="absolute top-0 text-xs
full flex items-center gap-1">
      <span>ВЕРШИНА СТЕКА (TOP)</span>
      <ArrowDown className="w-3.5 h-3.5 ani
    </div>

    {/* Reversed map so top of stack is vis
    {stack.slice().reverse().map((plate, ind
    const isTop = index === 0;
    return (
      <div
        key={plate.id}
        className={`w-full flex items-cen
          isTop
            ? 'bg-gradient-to-r from-blue
le-102'
            : 'bg-slate-900/90 border-sla
        }`}
      >
        <div className="flex items-center
          <span className="text-2xl">{pla
          <span className={`font-bold tex
            {plate.name}
          </span>
        </div>
        <span className={`text-xs font-mo
          isTop ? 'bg-blue-500 text-white
        }`}>
          {isTop ? 'Готово к помывке' : `
        </span>
      </div>
    );
  }}}

```

```

        </button>
        <button
          onClick={resetQueue}
          title="Сбросить"
          className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
        >
          <RotateCcw className="w-5 h-5" />
        </button>
      </div>
    </div>

    {dequeueMessage && (
      <div className="bg-indigo-950/60 border border-
p-3 animate-fade-in">
        <span className="inline-block w-2 h-2 round
        {dequeueMessage}
      </div>
    )}

    {/* Visualization */}
    <div className="bg-slate-950/60 p-6 rounded-2
overflow-x-auto">
      {queue.length === 0 ? (
        <div className="text-center py-12 text-sla
          <div className="text-5xl mb-3"> </div>
          <p className="text-base font-bold">Очер
          <p className="text-xs mt-1">Добавьте ро
        </div>
      ) : (
        <div className="w-full flex items-center
          <div className="flex flex-col items-cen
300">
          <span className="text-3xl"> </span>
          <span className="text-xs font-bold font
        </div>

        <div className="text-indigo-500 flex it

```

```

        </div>
      </div>
    })
    <div className="mt-6 text-xs text-slate-500">
      ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
    </div>
  </div>
</div>
})

{/* HEAP TAB */}
{activeTab === 'heap' && (
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Приоритетная очередь / Куча</sp
          <span className="text-xs font-mono bg-e
x-Heap</span>
        </h3>
        <p className="text-slate-400 text-xs mt-1
          Здесь не важно, кто пришел первым. Важн
        </p>
      </div>
      <div className="flex items-center gap-3 w-f
        <button
          onClick={popHeap}
          className="flex-1 md:flex-none flex ite
py-2.5 rounded-xl text-sm font-bold shadow-
        >
          Взять кандидата с макс. вероятностью
        </button>
        <button
          onClick={resetHeap}
          title="Сбросить"
          className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"

```



```
                {percent}%
            </span>
        </div>
    </div>
    );
    }}}
</div>
})
<div className="mt-6 text-xs text-slate-500"
    НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
</div>
</div>
</div>
})
</div>
);
};
```

ФАЙЛ 70/111: src/components/SplayPlayground.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useContext, useEffect, useState } from 'react';
import { VizChapterContext, emitVizDemo } from '../data';
import { SplayRotationsViz } from './SplayRotationsViz';
import SplayWalkViz from './SplayWalkViz';
import { SplayRotationSandbox } from './SplayRotationSandbox';
```

```
/**
```

```
 * Страница splay – три вкладки по нарастанию сложности
 * 1. «Механика» – покадровый разбор Zig / Zig-Zig / Zig-Zag
 *    (прицел → поворот, призраки старых позиций, обход)
 * 2. «Большое дерево» – те же случаи внутри настоящего
 *    поворот – кадры «прицел → отстёгиваем среднее по
 * 3. «Собери сам» – песочница: подними узел кликами, и
```

```
 */
```

```

        {t.label}
      </button>
    )))
  </div>

  {tab === "cases" && <SplayRotationsViz />}
  {tab === "big" && <SplayWalkViz />}
  {tab === "play" && <SplayRotationSandbox />}

  {tab === "play" && {
    <p className="text-xs text-slate-500 mt-3">
      Порядок поворотов в песочнице сверяется с пра
    </p>
  }}
</div>
);
};

export default SplayPlayground;

```

ФАЙЛ 71/111: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useCallback, useMemo, useState } from "react";
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, RotateCw } from "lucide-react";

/**
 * Песочница поворотов.
 *
 * Здесь ничего не подсказывается: дано настоящее дерево,
 * поднять отмеченный узел в корень. Клик по узлу = оди
 * Порядок поворотов выбираешь сам; правильный ли он по
 * только после того, как узел окажется наверху (или по
 */

```

```

    target: 2,
    build: () => ({ key: 4, left: { key: 2, left: leaf(
},
{
  name: "Смешанный",
  hintCase: "Zig-Zag, затем Zig-Zig",
  target: 5,
  build: () => {
    const n5 = leaf(5);
    const n6: TNode = { key: 6, left: n5, right: leaf(
    const n4: TNode = { key: 4, left: leaf(3), right:
    return { key: 8, left: n4, right: leaf(9) };
  },
},
];

/* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number): TNode[] => {
  const path: TNode[] = [];
  let cur = root;
  while (cur) {
    path.push(cur);
    if (key === cur.key) return path;
    cur = key < cur.key ? cur.left : cur.right;
  }
  return [];
};

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
  const path = findPath(root, key);
  if (path.length < 2) return root;

  const x = path[path.length - 1];
  const p = path[path.length - 2];
  const gg = path.length >= 3 ? path[path.length - 3] :

```



```
function layout(root: TNode | null): { nodes: Placed[];
  const nodes: Placed[] = [];
  const edges: [number, number][] = [];
  let order = 0;

  const walk = (n: TNode | null, depth: number) => {
    if (!n) return;
    walk(n.left, depth + 1);
    nodes.push({ key: n.key, x: order++, y: depth, depth: depth });
    if (n.left) edges.push([n.key, n.left.key]);
    if (n.right) edges.push([n.key, n.right.key]);
    walk(n.right, depth + 1);
  };
  walk(root, 0);

  const cols = Math.max(1, order);
  const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth)));
  const colW = 46;
  const rowH = 62;
  return {
    nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH + rowH / 2 })),
    edges,
    width: cols * colW,
    height: rows * rowH + 20,
  };
}
```

/* ----- КОМПОНЕНТ ----- */

```
export const SplayRotationSandbox: React.FC = () => {
  const [presetIdx, setPresetIdx] = useState(0);
  const preset = PRESETS[presetIdx];

  const [tree, setTree] = useState<TNode>(() => preset.root);
  const [history, setHistory] = useState<TNode[]>([]);
  const [moves, setMoves] = useState<{ node: number; count: number }>({});
  const [showAnswer, setShowAnswer] = useState(false);
```

```
const minimal = useMemo(() => findPath(preset.build())
```

```
return (
```

```
  <div className="w-full bg-slate-950 rounded-2xl border
```

```
    <div className="flex items-start justify-between
```

```
      <div>
```

```
        <h4 className="text-sm sm:text-base font-bold
```

```
        <p className="text-[12px] text-slate-400 lead
```

```
          Клик по узлу = один поворот с его родителем
```

```
          <span className="font-mono font-bold text-in
```

```
          поворотов выбираешь сам, разбор покажем в к
```

```
        </p>
```

```
      </div>
```

```
      <button
```

```
        onClick={() => reset((presetIdx + 1) % PRESET
```

```
        className="flex items-center gap-1.5 px-3 py-1
```

```
        ext-xs font-bold transition-colors shrink-0
```

```
      >
```

```
        <Dices className="w-3.5 h-3.5" /> Другое дере
```

```
      </button>
```

```
    </div>
```

```
  <div className="flex gap-2 mb-3 overflow-x-auto n
```

```
    {PRESETS.map((p, i) => {
```

```
      <button
```

```
        key={p.name}
```

```
        onClick={() => reset(i)}
```

```
        className={`px-3 py-1.5 rounded-lg text-[11
```

```
          i === presetIdx ? "bg-indigo-600 text-whi
```

```
        }`}
```

```
      >
```

```
        {p.name}
```

```
      </button>
```

```
    })}
```

```
  </div>
```

```
  <div className="bg-slate-900 rounded-xl border bo
```

```
    <svg
```

```

        <circle r={22} fill="none" stroke="#6366f1"
        >>
        <circle
          r={16}
          fill={isTarget ? "#6366f1" : path.has
            stroke={isTarget ? "#a5b4fc" : "#475568"}
            strokeWidth={2}
          />
        <text textAnchor="middle" dominantBaseline="middle"
          {n.key}
        </text>
      </g>
    );
  }
</svg>
</div>

<div className="flex flex-wrap items-center gap-2" >
  <button
    onClick={undo}
    disabled={history.length === 0}
    className="flex items-center gap-1.5 px-3 py-1.5 text-white disabled:opacity-40 text-xs font-bold"
  >
    <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
  </button>
  <button
    onClick={() => reset()}
    className="flex items-center gap-1.5 px-3 py-1.5 text-white font-bold transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" /> ЗАНОВО
  </button>
  <button
    onClick={() => setShowAnswer((s) => !s)}
    className={`flex items-center gap-1.5 px-3 py-1.5 text-white
      ${showAnswer
        ? "bg-amber-600/20 border-amber-500 text-amber-500"
        : "bg-slate-800/20 border-slate-700 text-slate-300"
      }
    >
      <RotateCcw className="w-3.5 h-3.5" />
    </button>
  </div>

```

```

        <li key={i}>
          ход {i + 1}: крутили <span className={m.expected}>
        </li>
      )
    })
  </ul>
})
{wrong === 0 && moves.length > minimal && (
  <div className="opacity-80">Порядок верный,
  </div>
)}
</div>
);
};

```

ФАЙЛ 72/111: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useEffect, useMemo, useRef, useState } from 'react';
import { Gauge, Pause, Play, RotateCcw, Terminal } from 'react-d3';
import { useVizRuntime, vizNumber, vizString } from 'viz-runtime';

```

```
/**
```

```
* Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag;
```

```
*
```

```
* Что сделано ради понятности:
```

```
* * у узлов настоящие числа (20 / 40 / 60), а не абстрактные;
```

```
* * сразу видно, что дерево остаётся деревом поиска;
```

```
* * под деревом печатается обход слева направо: он ОДНОЗНАЧЕН;
```

```
* * это и есть доказательство, что поворот ничего не меняет;
```

```
* * кадр «прицел» ничего не двигает, только подсвечивает;
```

```
* * на кадре «результат» остаются призраки – пунктирные узлы;
```

```
* * местах и стрелки «откуда → куда»;
```

```

/* ----- Zig -----
const ZIG_BEFORE = {
  pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
  edges: [{"p", "x"}, {"p", "C"}, {"x", "A"}, {"x", "B"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
  key: "zig",
  label: "Zig",
  when: "родитель x — уже корень, деда нет",
  rotations: "1 поворот",
  keys: { x: 20, p: 40 },
  inorder: ["A", "20", "B", "40", "C"],
  ranges: "A < 20 · B между 20 и 40 · C > 40",
  frames: [
    {
      ...ZIG_BEFORE,
      kind: "start",
      title: "Было: 20 висит под корнем 40",
      text: "Нам нужен ключ 20, а он на глубине 1. Деду
      ms: RESULT_MS,
    },
    {
      ...ZIG_BEFORE,
      kind: "aim",
      title: "Прицел: крутим ребро 40 — 20",
      text: "Ничего пока не двигается. Смотри на подсве
      focus: [{"p", "x"}],
      ms: AIM_MS,
    },
    {
      pos: { x: [160, 50], A: [80, 130], p: [245, 130],
      edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
      kind: "result",
      title: "Стало: 20 — корень",
      text: "40 стало правым сыном двадцатки. Поддерев
        ева от 40 — это одно и то же место.",

```

```

        text: "Главная хитрость этого случая: первым крути
        focus: ["g", "p"],
        ms: AIM_MS,
    },
    {
        ...ZZ_S1,
        kind: "result",
        title: "Поворот 1: 40 поднялось над 60",
        text: "40 встало в корень, 60 опустилось к нему по
            убине 1, а не 2.",
        moved: ["p", "g", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZZ_S1,
        kind: "aim",
        title: "Прицел 2: ребро 40 – 20",
        text: "А теперь обычный одиночный поворот – тот же
        focus: ["p", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [100, 45], A: [45, 118], p: [188, 118],
        edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
        kind: "result",
        title: "Стало: 20 в корне, ветка x подтянулась",
        text: "Сравни с первым кадром: А было на глубине
            ась. Splay не «уменьшает высоту вообще», он под
            сумме запросов (амортизация).",
        moved: ["x", "p", "A", "B"],
        ms: RESULT_MS,
    },
    ],
};

```

/* ----- Zig-Zag -----

```

const ZG_S0 = {
    pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:

```

```

        moved: ["x", "p", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZG_S1,
        kind: "aim",
        title: "Прицел 2: ребро 60 – 40",
        text: "Остался одиночный поворот вокруг деда – и",
        focus: ["g", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [180, 50], p: [90, 130], g: [275, 130],
        edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
        kind: "result",
        title: "Стало: 20 и 60 – два сына сорока",
        text: "Дерево стало идеально симметричным: все че
            очку». ",
        moved: ["x", "p", "g", "C"],
        ms: RESULT_MS,
    },
    ],
};

```

```
const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];
```

```

const COLOR: Record<string, { fill: string; stroke: str
    x: { fill: "#6366f1", stroke: "#a5b4fc" },
    p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
    g: { fill: "#f59e0b", stroke: "#fcd34d" },
    sub: { fill: "#1e293b", stroke: "#475569" },
};

```

```

const ROLE_RU: Partial<Record<NodeId, string>> = {
    x: "x – ищем его",
    p: "p – родитель",
    g: "g – дед",
};

```

```

        />
      })
    </g>
  });
})))

{edges.map(([a, b], i) => {
  const pa = pos[a];
  const pb = pos[b];
  if (!pa || !pb) return null;
  const hot = Boolean(focus && focus.includes(a));
  return (
    <line
      key={`${a}-${b}-${i}`}
      x1={pa[0]}
      y1={pa[1]}
      x2={pb[0]}
      y2={pb[1]}
      stroke={hot ? "#fbbf24" : "#475569"}
      strokeWidth={hot ? 5 : 2}
      opacity={focus && !hot ? 0.25 : 1}
      style={{ transition: "all 1200ms cubic-bezier(0.4, 0, 0.2, 1)" }}
    />
  );
})))

(({Object.keys(pos) as NodeId[]).map((id) => {
  const p = pos[id];
  if (!p) return null;
  const sub = isSubtree(id);
  const c = COLOR[sub ? "sub" : id];
  const r = sub ? 15 : 20;
  const hot = Boolean(focus && focus.includes(id));
  return (
    <g
      key={id}
      style={{
        transform: `translate(${p[0]}px, ${p[1]}px)`,

```



```

/* — связь с Python-компилятором: case переключает в
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveCase = vars ? vizString(vars.case) ?? vizSt
const liveStep = vars ? vizNumber(vars.step) ?? vizNum
const liveX = vars ? vizNumber(vars.xk) ?? vizNumber(
const liveP = vars ? vizNumber(vars.pk) ?? vizNumber(
const liveG = vars ? vizNumber(vars.gk) ?? vizNumber(
const lastSig = useRef<string | null>(null);

useEffect(() => {
  if (!liveCase && liveStep === null) return;
  const norm = (liveCase ?? "").trim().toLowerCase();
  const idx = CASE_BY_NAME[norm] ?? caseIdx;
  const sig = `${idx}:${liveStep ?? ""}`;
  if (sig === lastSig.current) return;
  const first = lastSig.current === null;
  lastSig.current = sig;
  setPlaying(false);
  if (first || idx !== caseIdx) {
    setCaseIdx(idx);    // новый случай — начинаем его
    setFrame(0);
  } else {
    // тот же случай, следующий шаг кода — пролистывае
    setFrame((f) => Math.min(CASES[idx].frames.length
  }
  // eslint-disable-next-line react-hooks/exhaustive-
}, [liveCase, liveStep]);

const active = CASES[caseIdx];
const total = active.frames.length;
const idx = Math.min(frame, total - 1);
const current = active.frames[idx];
const prevPos = active.frames[Math.max(idx - 1, 0)].p
const duration = useMemo(() => Math.round(current.ms
const last = idx === total - 1;

useEffect(() => {

```

```

        ? "bg-slate-700 text-slate-300"
        : "bg-indigo-500/20 text-indigo-300"
      }` }
    >
      {current.kind === "aim" ? "прицел" : current.title}
    </span>
    <span className="text-sm font-bold text-white">
      Шаг {idx + 1} из {total}. {current.title}
    </span>
  </div>
  <p className="text-[13px] text-slate-400 leading-5">
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800">
  <Tree frame={current} prev={prevPos} keys={activeKeys}>

    {/* обход слева направо — одинаковый на всех кадрах */}
    <div className="mt-2 pt-2 border-t border-slate-800">
      <div className="flex items-center gap-1.5 flex-wrap">
        <span className="text-slate-500 mr-1">Обход
        {active.inorder.map((t, i) => {
          <span
            key={i}
            className={`px-1.5 py-0.5 rounded font-mono text-[10px]
              /\d/.test(t) ? "bg-emerald-500/15 text-white" : "bg-slate-800 text-slate-400"
            }` }
          >
            {t}
          </span>
        ))}
        <span className="text-emerald-400/80 ml-1">
      </div>
      <p className="text-[11px] text-slate-500 mt-1">
    </div>
  </div>

  {/* индикатор автопрокрутки */}
  <div className="h-1 w-full bg-slate-800 rounded-f

```

```

        {playing ? <Pause className="w-3.5 h-3.5" />
        {playing ? "Стоп" : "Авто"}}
    </button>
    <button
      onClick={() => setSlow((s) => !s)}
      title="Автопрокрутка ещё медленнее"
      className={`flex items-center gap-1.5 px-3 py-3
        slow
          ? "bg-emerald-600/20 border-emerald-500 text-emerald-500"
          : "bg-slate-800 border-slate-700 text-slate-200"
        }`}
    >
      <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1" : "1"}
    </button>
    <button
      onClick={() => {
        setFrame(0);
        setPlaying(false);
      }}
      className="flex items-center gap-1.5 px-3 py-3 text-xs font-bold transition-colors"
    >
      <RotateCcw className="w-3.5 h-3.5" /> Сброс
    </button>

    <div className="flex items-center gap-3 ml-auto"
      {([ "x", "p", "g" ] as const).map((id) =>
        active.keys[id] === undefined ? null : (
          <span key={id} className="flex items-center justify-center"
            <span className="w-3 h-3 rounded-full" style={styles[id]} />
          </span>
        )
      )}
    </div>
  </div>

  {/* что сейчас сообщил компилятор */}
  <div className="mt-3 pt-3 border-t border-slate-800" />

```

```
import React, { useEffect, useMemo, useRef, useState }

/**
 * Splay на БОЛЬШОМ дереве – по урокам treap-прохода.
 *
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный поворот.
 *   aim – прицел на тройку g–p–x, назван случай;
 *   cut – отстёгиваем среднее поддереву β (пунктир, поворот);
 *   rot – поворот: зелёным подсвечены рёбра, которых не было.
 * Рядом с логом всегда висит врезка «Механика одного поворота».
 * На каждом кадре – живая автопроверка in-order (сортировка).
 *
 * Чистые функции (splaySteps, rotateUp, bstInsert, cloneWithParents).
 */

export interface SN {
  key: number;
  left: SN | null;
  right: SN | null;
  parent: SN | null;
}

export function cloneWithParents(t: SN | null): SN | null {
  if (!t) return null;
  const n: SN = { key: t.key, left: null, right: null, parent: null };
  const l = t.left ? cloneWithParents(t.left) : null;
  const r = t.right ? cloneWithParents(t.right) : null;
  if (l) {
    n.left = l;
    l.parent = n;
  }
  if (r) {
    n.right = r;
    r.parent = n;
  }
}
```

```

    p.parent = child;
  }
  child.parent = g;
  if (!g) return;
  if (g.left === p) g.left = child;
  else g.right = child;
}

/** Рёбра в виде "p→c" — для diff между кадрами. */
export function collectEdges(t: SN | null): string[] {
  const es: string[] = [];
  const walk = (n: SN | null) => {
    if (!n) return;
    if (n.left) es.push(`${n.key}→${n.left.key}`);
    if (n.right) es.push(`${n.key}→${n.right.key}`);
    walk(n.left);
    walk(n.right);
  };
  walk(t);
  return es.sort();
}

const diffEdges = (before: string[], after: string[]):
  const rm = before.filter((e) => !after.includes(e));
  const ad = after.filter((e) => !before.includes(e));
  const parts: string[] = [];
  for (const a of ad) {
    const [p, c] = a.split("→");
    if (rm.includes(`${c}→${p}`)) parts.push(`ребро ${p}`);
    else parts.push(`поддерево ${c} переехало к ${p}`);
  }
  return parts.join("; ");
};

export type SplayStepKind = "search" | "aim" | "cut" |
export interface SplayStep {
  kind: SplayStepKind;
  root: SN | null;

```

```

steps.push({
  kind: "search",
  root: cloneWithParents(root)!,
  pathKeys: [...pathKeys],
  note: isInsert
    ? `Вставили ${key} обычным BST-спуском. Теперь по
    : found
      ? `Спуск по ключу ${key}: путь ${pathKeys.join(
      : `Ключа ${key} нет. Спуск упёрся в ${last.key}
  rotCount: 0,
});

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
  while (root && root.parent) root = root.parent;
};
/** Один поворот = три кадра: aim → cut → rot. */
const emitRotation = (child: SN, label: string, aimEdge: string) => {
  steps.push({
    kind: "aim",
    root: cloneWithParents(root)!,
    aimEdge,
    caseLabel: label,
    note: aimNote,
    rotCount,
  });
  const p = child.parent!;
  const mid = p.left === child ? child.right : child.left;
  steps.push({
    kind: "cut",
    root: cloneWithParents(root)!,
    cutEdge: [child.key, mid ? mid.key : child.key],
    cutLabel: mid ? `β = поддерево ${mid.key}` : "β не найдено",
    cutDest: mid ? (p.left === child ? "справа у p" : "слева у p") : null,
    caseLabel: label,
    note: mid
  });

```

```

        [g.key, p.key],
        `Zig-Zig (${side}: линия ${g.key}—${p.key}—${target.key},
        ${g.key}, а ${target.key} пока не трогаем.`),
    );
    steps.push({
        kind: "aim",
        root: cloneWithParents(root)!,
        aimEdge: [target.parent.key, target.key],
        caseLabel: "Zig-Zig",
        note: `Теперь обычный зиг: ребро ${target.parent.key}—${target.key},
        rotCount,
    });
    emitRotation(
        target,
        "Zig-Zig",
        [target.parent.key, target.key],
        `Доводим: ${target.key} над ${target.parent.key}`,
    );
} else {
    const side = `${g.key} → ${pIsLeft ? "влево" : "вправо"}`;
    emitRotation(
        target,
        "Zig-Zag",
        [p.key, target.key],
        `Zig-Zag (змейка: ${side}). Порядок ОБРАТНЫЙ: ${p.key}—${target.key}`,
    );
    steps.push({
        kind: "aim",
        root: cloneWithParents(root)!,
        aimEdge: [target.parent.key, target.key],
        caseLabel: "Zig-Zag",
        note: `Второе ребро: ${target.parent.key}—${target.key}`,
        rotCount,
    });
    emitRotation(
        target,
        "Zig-Zag",
        [target.parent.key, target.key],

```

```

{ line: 1, text: "while x has parent:" },
{ line: 2, text: "    p = parent(x); g = parent(p)" },
{ line: 3, text: "    if g is None:                                # Z" },
{ line: 4, text: "        rotate(x over p)" },
{ line: 5, text: "    elif p,g,x на одной линии:                            # " },
{ line: 6, text: "        rotate(p over g)                                # B" },
{ line: 7, text: "        rotate(x over p)" },
{ line: 8, text: "    else:                                                  # Z" },
{ line: 9, text: "        rotate(x over p)                                # H" },
{ line: 10, text: "        rotate(x over g)" },
];

```

/* ----- врезка «механика одного по

```

const MechanismInset: React.FC<{ step: SplayStep | null
const isCut = step?.kind === "cut";
const isRot = step?.kind === "rot";
return (
  <div className="bg-slate-950/80 rounded-lg border b
    <h4 className="text-xs font-bold text-slate-300 m
    <p className="text-[10px] text-slate-500 mb-2">
      Как вправление вывиха: одна кость ходит вокруг
    </p>
    <svg viewBox="0 0 340 190" className="w-full">
      <g transform="translate(10,6)">
        <text y="10" fill="#94a3b8" fontSize="11" fon
        <line x1="55" y1="46" x2="30" y2="86" stroke=
        <line x1="55" y1="46" x2="80" y2="86" stroke=
        <line x1="30" y1="86" x2="12" y2="126" stroke=
        <line
          x1="30" y1="86" x2="48" y2="126"
          stroke={isCut ? "#f59e0b" : "#64748b"}
          strokeWidth={isCut ? 3 : 2}
          strokeDasharray={isCut ? "5,3" : "none"}
        </>
        <circle cx="55" cy="46" r="15" fill="#1e293b"
        <text x="55" y="50" textAnchor="middle" fill=
        <circle cx="30" cy="86" r="15" fill="#78350f"

```



```

        stroke={isRot ? "#10b981" : "#64748b"}
        strokeWidth={isRot ? 2.5 : 1.5}
      />
      <text x="63" y="130" textAnchor="middle" fill="black">
      <circle cx="95" cy="126" r="13" fill="#0f172a" stroke="black" />
      <text x="95" y="130" textAnchor="middle" fill="black">
      <text x="112" y="92" fill={isRot ? "#10b981" : "#64748b"}
        {isRot ? "↓ β теперь при р" : "β при р"}
      </text>
      <text x="30" y="160" fill="#64748b" fontSize=14>
    </g>
  </svg>
  <p className="text-[10px] text-slate-500 leading-5">
    Обход слева-направо α x β р γ не меняется никог
    числа поворотов. Единственный, кто меняет хозяи
  </p>
</div>
);
};

/* ----- рендер -----

const VIEW_W = 760;
const SLOT_W = 88;
const LEVEL_H = 76;

const layout = (root: SN | null) => {
  const pos = new Map<number, { px: number; py: number }>();
  let slot = 0;
  const walk = (n: SN | null, d: number) => {
    if (!n) return;
    walk(n.left, d + 1);
    pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d * LEVEL_H });
    slot += 1;
    walk(n.right, d + 1);
  };
  walk(root, 0);
  return pos;
};

```

```

const es: { a: number; b: number }[] = [];
const walk = (n: SN | null) => {
  if (!n) return;
  if (n.left) es.push({ a: n.key, b: n.left.key });
  if (n.right) es.push({ a: n.key, b: n.right.key });
  walk(n.left);
  walk(n.right);
};
walk(viewRoot);
return es;
}, [viewRoot, treeVers, idx]));

const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.px));
const autoH = Math.max(300, ...nodes.map((n) => n.py));

const dBefore = useMemo(() => (steps ? depthOf(steps[steps.length - 1].root) : 0));
const dNow = steps ? depthOf(viewRoot, steps[steps.length - 1].root) : 0;
const hBefore = useMemo(() => (steps ? heightOf(steps[steps.length - 1].root) : 0));
const hNow = heightOf(viewRoot);

// переехавшие рёбра: разница с предыдущим кадром
const prevEdgeSet = useMemo(() => {
  if (!steps || !step || idx <= 0) return null;
  return new Set(collectEdges(steps[idx - 1].root));
}, [steps, step, idx]);
const movedCount = useMemo(() => {
  if (!prevEdgeSet) return 0;
  return edges.filter((e) => !prevEdgeSet.has(`${e.a}${e.b}`)).length;
}, [prevEdgeSet, edges]);

const ino0 = useMemo(() => (steps ? inorderKeys(steps[steps.length - 1].root) : ''));
const inoNow = inorderKeys(viewRoot).join(",");
const checkOk = !steps || ino0 === inoNow;

const doInsert = () => {
  const v = parseInt(input, 10);
  if (Number.isNaN(v)) return;
  const withNew = bstInsert(baseRef.current, v);

```

```

        <p className="text-sm text-indigo-300">Клик
    </div>
</div>
<div className="flex flex-wrap items-center gap-2">
    <input
        value={input}
        onChange={({e}) => setInput(e.target.value)}
        onKeyDown={({e}) => {
            if (e.key === "Enter") {
                const v = parseInt(input, 10);
                if (!Number.isNaN(v)) {
                    const has = inorderKeys(baseRef.current);
                    if (has) runKey(v);
                    else doInsert();
                    setInput("");
                }
            }
        }}
        placeholder="ключ + Enter"
        className="w-28 bg-slate-950 border border-slate-500 border-indigo-500"
    />
    <button
        onClick={() => {
            baseRef.current = buildInitial();
            setSteps(null);
            setIdx(0);
            setPlaying(false);
            setTreeVers((v) => v + 1);
        }}
        className="px-3 py-2 rounded-lg text-xs font-medium"
    >
        Исходное
    </button>
    <button
        onClick={() => {
            const shuffled = [...INITIAL_KEYS].sort(() => Math.random() - 0.5);
            let r: SN | null = null;

```

```

00/60">
    {step.caseLabel}
  </span>
})
</div>
})

{/* Дерево */}
<div className="bg-indigo-950/20 rounded-xl p-4 b
  {!steps && {
    <div className="absolute top-3 left-3 bg-indi
      ont-bold">
        Кликни по любому узлу – splay покадрово
      </div>
    })
    <svg viewBox={`0 0 ${autoW} ${autoH}`} className
      {edges.map((e) => {
        const a = pos.get(e.a);
        const b = pos.get(e.b);
        if (!a || !b) return null;
        const isAim = step?.aimEdge && step.aimEdge
        const isCutEdge =
          step?.kind === "cut" && step.cutEdge && s
          .cutEdge[1];
        const isMoved = !isAim && !isCutEdge && pre
        return (
          <g key={`e-${e.a}-${e.b}`}>
            <line
              x1={a.px} y1={a.py} x2={b.px} y2={b.py}
              stroke={isAim ? "#f59e0b" : isCutEdge
              strokeWidth={isAim ? 4 : isCutEdge ?
              strokeDasharray={isAim ? "6,4" : isCu
              className="transition-all duration-500
            />
            {isCutEdge && (
              <text x={({a.px + b.px} / 2 + 14} y={
                nospace" className="transition-all duration
                  {step?.cutLabel} → {step?.cutDest

```

```

    <p className="font-semibold text-amber-400"
      {step?.kind === "aim"
        ? " Прицел:"
        : step?.kind === "cut"
          ? "× Отстёгиваем среднее поддерево:"
          : step?.kind === "rot"
            ? " Поворот:"
            : step?.kind === "search"
              ? " Спуск:"
              : "Статус:"}
    </p>
    <p className="min-h-[40px] flex items-center"
      {step ? step.note : "Кликни по узлу или в
      ке справа – механика одного поворота."}
    </p>
    {steps && step && step.kind !== "search" &&
      <p className={`text-xs mt-2 ${checkOk ? "
        {checkOk ? " " : " "} проверка кадра: o
        {inorderKeys(viewRoot).length}
        {movedCount > 0 ? ` · переехало рёбер: `
      </p>
    }}
  </div>
  <div className="w-full lg:w-[300px] shrink-0">
    <MechanismInset step={step} />
  </div>
</div>
</div>

```

```

{/* Счётчики + код */}
<div className="flex flex-col lg:flex-row gap-6 m
  <div className="w-full lg:w-1/2 bg-rose-950/10
    <h4 className="text-sm font-bold text-rose-300"
      {steps ? (
        <table className="w-full text-sm">
          <tbody className="font-mono">
            <tr className="border-b border-slate-800"
              <td className="py-2 text-slate-400">r

```

```
        </div>
      </div>
    );
  };
};

export default SplayWalkViz;
```

ФАЙЛ 74/111: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSteps';
```

```
interface Plate {
  id: string;
  name: string;
  emoji: string;
  color: string;
  isDirty: boolean;
  animState: 'in' | 'idle' | 'washing' | 'clean';
}
```

```
const PLATE_PRESETS = [
  { name: 'Тарелка из-под спагетти', emoji: '🍝', color: 'from-pasta' },
  { name: 'Тарелка из-под пиццы', emoji: '🍕', color: 'from-pizza' },
  { name: 'Блюдец от торта', emoji: '🍰', color: 'from-cake' },
  { name: 'Тарелка от тако', emoji: '🌮', color: 'from-taco' },
  { name: 'Тарелка из-под суши', emoji: '🍣', color: 'from-sushi' },
  { name: 'Сковорода от глазуньи', emoji: '🍳', color: 'from-pancake' },
];
```

```
let counter = 0;
```

```
export const StackViz: React.FC = () => {
```

```

    }

    const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
    counter++;

    const newPlate: Plate = {
      id: Date.now().toString(),
      ...preset,
      isDirty: true,
      animState: 'in',
    };

    setDirtyStack(prev => [...prev, newPlate]);
    addLog(`  PUSH: Положили ${newPlate.emoji} сверху с ${newPlate.id}`);

    setTimeout(() => {
      setDirtyStack(prev => prev.map(p => p.id !== newPlate.id ? p : newPlate));
    }, 500);
  }, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length === 0) {
    setShake(true);
    addLog('⚠ POP: Нечего мыть! Все тарелки чистые.');
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsWashing(true);
  const topPlate = dirtyStack[dirtyStack.length - 1];

  addLog(`  POP: Начинаем мыть верхнюю тарелку с ${topPlate.emoji} (${topPlate.id})`);

  // Step 1: Start washing animation (sponge and soap)
  setDirtyStack(prev => prev.map(p => p.id !== topPlate.id ? p : topPlate));
  setShowSoap(true);

```

```

<div className="bg-blue-950/40 border border-blue-950"
  <div className="text-3xl"> </div>
  <div>
    <h3 className="font-black text-blue-400 text-2xl">
      Ты складываешь грязные тарелки друг на друга.
      А когда приходит время мыть, ты берёшь именно эту.
      Попробуй вытащить нижнюю – всё рухнет!
    </h3>
  </div>
</div>

```

```

{/* УПРАВЛЕНИЕ */}
<div className="flex items-center gap-3 flex-wrap"
  <button
    onClick={pushPlate}
    disabled={isWashing}
    className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
      opacity-40 disabled:cursor-not-allowed text-white font-medium rounded-2xl
      transition-all cursor-pointer flex items-center justify-center"
    >
    <span> Положить грязную (PUSH)</span>
  </button>

  <button
    onClick={popPlate}
    disabled={isWashing || dirtyStack.length === 0}
    className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
      opacity-40 disabled:cursor-not-allowed text-white font-medium rounded-2xl
      transition-all cursor-pointer flex items-center justify-center"
    >
    <span> Помыть верхнюю (POP)</span>
  </button>

  <button
    onClick={reset}
    className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-medium
      transition-all cursor-pointer border border-slate-700"
  >
    <span> Сбросить (RESET)</span>
  </button>
</div>

```



```

        <p className="text-white font-black text-1">
        <p className="text-slate-500 text-xs mt-1">
    </div>
) : (
    <div className={`flex-1 flex flex-col justify-between`>

        {/* Указатель TOP */}
        <div
            className="absolute right-8 flex items-center"
            style={{ bottom: `${24 + topIndex * 44}px` }}
        >
            <span className="text-xs font-mono text-slate-500">
er gap-1">
                <ArrowUp className="w-3.5 h-3.5" />
                <span>ВЕРШИНА (TOP)</span>
            </span>
            <span className="text-blue-400 animate-pulse">
            </div>

        {/* Тарелки (отрисовываем в обратном порядке) */}
        {displayStack.slice().reverse().map((plate, idx) => {
            const idx = displayStack.length - 1 - idx;
            const isTop = idx === topIndex;
            return (
                <div
                    key={plate.id}
                    className={`
                        w-full max-w-[280px] h-10 rounded-2xl
                        shadow-md select-none transition-all duration-300
                        ${plate.color}
                        ${plate.animState === 'in' ? 'animate-in' : ''}
                        ${plate.animState === 'washing' ? 'animate-washing' : ''}
                        ${isTop ? 'ring-4 ring-blue-500/50' : ''}
                    `}
                >
                    <span className="text-lg">{plate.name}</span>
                    <span className="text-slate-800 text-sm">
                        {isTop && (

```

```

        className="w-full h-8 rounded-full flex items-center justify-center gap-2 opacity-50"
      >
        <span className="text-sm"> {plateName}
        <span className="text-[10px] text-end"> {plateSize}
      </div>
    )))
  </div>
})
</div>

{/* Столешница сушилки */}
<div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800" />
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4" >
  <div className="text-[10px] font-mono text-slate-400" >
    {log.map((entry, idx) => (
      <div
        key={idx}
        className={`text-xs font-mono flex items-center justify-between`
      >
        {idx === 0 ? <span className="text-blue-500">
          <span>{entry}</span>
        </div>
      )))
    </div>
  </div>
);
};

```

```

        if (j === pattern.length) {
            steps.push({ i, j, pi: [...pi], found: true
        }
    }
    return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
    if (!pattern) pattern = " ";
    const s = pattern + "#" + text;
    const n = s.length;
    const z = new Array(n).fill(0);
    const steps: any[] = [];

    let l = 0, r = 0;
    steps.push({ i: 0, l, r, z: [...z], desc: `Начало.` });

    for (let i = 1; i < n; i++) {
        steps.push({ i, l, r, z: [...z], desc: `Шаг i=$
            i

        if (i <= r) {
            steps.push({ i, l, r, z: [...z], desc: `Вни
                !` });
            z[i] = Math.min(r - i + 1, z[i - l]);
            steps.push({ i, l, r, z: [...z], desc: `Пре
                }. Копипаста сработала.` });
        }

        steps.push({ i, l, r, zTarget: z[i], zSource: i
        while (i + z[i] < n && s[z[i]] === s[i + z[i]])
            steps.push({ i, l, r, zTarget: z[i], zSource
                падают! Окно растёт.` });
            z[i]++;
        }

        if (i + z[i] < n) {
            steps.push({ i, l, r, zTarget: z[i], zSource
                [i]]}' разные. Стоп.` });
        }
    }
}

```

```

const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
  setStepIdx(0);
  setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
  if (autoPlay) {
    timer.current = setInterval(() => {
      setStepIdx(prev => {
        if (prev >= steps.length - 1) { set
        return prev + 1;
      });
    }, 1200);
  }
  return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const baseStep = steps[stepIdx] || steps[0];
const vars = runtime?.variables;
const liveValues = vizArray(mode === "kmp" ? vars?.l
const hasLiveValues = !!vars && Object.prototype.ha
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const liveL = vizNumber(vars?.l);
const liveR = vizNumber(vars?.r);
const compilerLinked = liveI !== null || hasLiveVal
const step = compilerLinked
  ? {

```

```
</div>
```

```
<div className="bg-slate-950 rounded-xl border-2 border-slate-900 px] flex items-center justify-start">
  <div className="flex gap-1.5 px-4 h-full">
    {s.split("").map((char, index) => {
      const isPattern = index < pattern.length;
      const isSeparator = index === pattern.length;

      let borderColor = "border-slate-900";
      let bgColor = "bg-slate-900";
      let textColor = isSeparator ? "text-slate-50" : "text-slate-900";

      // KMP Logic
      if (mode === "kmp") {
        if (step.highlight?.include(char)) {
          borderColor = "border-emerald-400";
          bgColor = step.match === char ? "bg-emerald-400" : "bg-slate-900/40";
        }
      }

      // Z Logic
      if (mode === "z") {
        if (index >= step.l && index <= step.r) {
          bgColor = "bg-emerald-900/40";
          borderColor = "border-emerald-400";
        }
        if (step.zTarget === index) {
          borderColor = "border-emerald-400";
          bgColor = step.match === char ? "bg-emerald-400" : "bg-slate-900/40";
        }
      }

      return (
        <div key={index} className={textColor} style={{
          /* L/R markers for Z */

```

```

        </div>
    }}

    {/* Z fingers */}
    {mode == "z" && step.i
        <div className="abs
e z-20">
            <span className
        </div>
    }}
    {mode == "z" && step.z
        <div className="abs
            <span className
        </div>
    }}
    {mode == "z" && step.z
        <div className="abs
            <span className
        </div>
    }}

    {/* Found flash effect
    {step.found && mode ==
        <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
    }}
    {step.found && mode ==
        <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
    }}
    </div>
    )
    }}}
    </div>
</div>

<div className="flex flex-col sm:flex-row g
    {/* Controls */}

```

Мы пишем **Шаблон + # + Текст**.

Когда значени {mode === 'kmp' ? "пр
о было найдено!

Пройди шаги вручную, посмотри, как
ля пропуска работы (в Z).

</p>

</div>

```
/* Code Sneak Peek */
```

```
<div className="bg-slate-900 border border-
```

```
  <div className="bg-slate-800 px-4 py-2 b
```

```
late-400 uppercase">
```

```
  <span>Код C++ ({mode === 'kmp' ? 'КМП
```

```
  </div>
```

```
  <pre className="p-4 text-xs font-mono te
```

```
{mode === 'kmp' ? `vector<int> pi(s.length());
```

```
for (int i = 1; i < s.length(); i++) {
```

```
  int j = pi[i-1];
```

```
  while (j > 0 && s[i] != s[j]) j = pi[j-1];
```

```
  if (s[i] == s[j]) j++;
```

```
  pi[i] = j;
```

```
}` : `int l = 0, r = 0;
```

```
for (int i = 1; i < n; i++) {
```

```
  if (i <= r) z[i] = min(r - i + 1, z[i - l]);
```

```
  while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
```

```
  if (i + z[i] - 1 > r) {
```

```
    l = i;
```

```
    r = i + z[i] - 1;
```

```
  }
```

```
}`}
```

```
  </pre>
```

```
  </div>
```

```
</div>
```

```
);
```

```
}
```

```
        l duration-150 ${
          side === "top" ? "bottom-full mb-2" : "top-full"
        } ${open ? "visible translate-y-0 opacity-100" : ""}
      >
      {content}
    <span
      className={`absolute left-1/2 h-2 w-2 -translate-x-50%
        side === "top" ? "-bottom-[5px] border-b border-gray-400" : ""
      }`
    />
  </span>
</span>
);
};
```

ФАЙЛ 77/111: `src/components/TopologicalSortViz.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "../viz";
import {
  Play,
  Pause,
  RotateCcw,
  Clock,
} from "lucide-react";
```

```
interface TNode {
  id: number;
  x: number;
  y: number;
  label: string;
  name: string;
}
```



```

        .map((step, index) => ({ step, index }))
        .filter(({ step }) => (v === null || step.currentNode !== null))
        ;
    return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
    const listSteps: TStep[] = [];
    const visited = new Set<number>();
    const fullyProcessed = new Set<number>();
    const topoList: number[] = [];
    const dfsStack: number[] = [];

    const recordStep = (desc: string, currentNode: number) => {
        listSteps.push({
            desc,
            visited: new Set(visited),
            fullyProcessed: new Set(fullyProcessed),
            currentNode,
            topoList: [...topoList],
            dfsStack: [...dfsStack],
        });
    };

    recordStep(
        "Начало топологической сортировки. Используем классический DFS",
        null,
    );

    // Adj list
    const adj: number[][] = Array.from({ length: 5 }, () => []);
    dagEdges.forEach((e) => adj[e.u].push(e.v));

    const dfs = (u: number) => {
        visited.add(u);
        dfsStack.push(u);
        recordStep(
            `DFS: посещаем узел ${u}, добавляем в стек`,
            u,
        );
        for (const v of adj[u]) {
            if (!visited.has(v)) {
                dfs(v);
            }
        }
        recordStep(
            `DFS: завершили обработку узла ${u}, удаляем из стека`,
            u,
        );
        topoList.push(u);
    };

    dfs(0);
    return topoList;
}, []);

```

```

        "Топологическая сортировка завершена! Полученный
        null,
    );
    setSteps(listSteps);
    setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {
  desc: "Заныск...",
  visited: new Set(),
  fullyProcessed: new Set(),
  currentNode: null,
  topoList: [],
  dfsStack: [],
};

useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlaying && currentStepIdx < steps.length - 1)
    timer = setInterval(() => {
      setCurrentStepIdx((prev) => prev + 1);
    }, 1600);
  } else {
    setIsPlaying(false);
  }
  return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIdx(0);
};

const getTopoListString = () => {
  return [...step.topoList]
    .reverse()
    .map((id) => dagNodes[id].name)

```

```

    { /* Playfield SVG */}
    <div className="lg:col-span-8 bg-[#0B1120] rela
      <svg className="w-full h-full max-w-[650px]"
        <defs>
          <marker
            id="dag-arrow"
            viewBox="0 0 10 10"
            refX="24"
            refY="5"
            markerWidth="6"
            markerHeight="6"
            orient="auto-start-reverse"
          >
            <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
          </marker>
          <marker
            id="dag-arrow-active"
            viewBox="0 0 10 10"
            refX="24"
            refY="5"
            markerWidth="6"
            markerHeight="6"
            orient="auto-start-reverse"
          >
            <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
          </marker>
        </defs>

    { /* Edges */}
    {dagEdges.map(({edge, idx}) => {
      const u = dagNodes.find((n) => n.id === e
      const v = dagNodes.find((n) => n.id === e

      const isActive =
        (step.currentNode === edge.u || step.cu
        step.visited.has(edge.v));

      return (

```

```

return (
  <g key={node.id}>
    <rect
      x={node.x - 55}
      y={node.y - 22}
      width="110"
      height="44"
      rx="8"
      fill={fill}
      stroke={stroke}
      strokeWidth="2"
      className="transition-all duration-1
    />
    <text
      x={node.x}
      y={node.y - 4}
      textAnchor="middle"
      fill="white"
      fontSize="11px"
      fontWeight="bold"
      className="pointer-events-none"
    >
      ({node.label}) {node.name.split(" "
    </text>
    <text
      x={node.x}
      y={node.y + 12}
      textAnchor="middle"
      fill="#94a3b8"
      fontSize="9px"
      className="pointer-events-none"
    >
      {node.name.split(" ").slice(1).join
    </text>
    </g>
  );
})}

```

```

    </span>
    <div className="flex items-center gap-2">
      <span className="w-3.5 h-3.5 rounded bg-slate-400">
        <span className="text-slate-400 text-[12px] font-medium">
          Серые (зашли)
        </span>
      </span>
    </div>
    <div className="flex items-center gap-2">
      <span className="w-3.5 h-3.5 rounded bg-slate-400">
        <span className="text-slate-400 text-[12px] font-medium">
          Черные (готово)
        </span>
      </span>
    </div>
  </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">
  <span>
    Шаг {currentStepIdx + 1} из {steps.length}
  </span>
  <div className="flex gap-1">
    <button
      disabled={currentStepIdx === 0}
      onClick={() => setCurrentStepIdx((prev) => prev - 1)}
      className="bg-slate-900 border border-slate-200 text-white"
    >
      Назад
    </button>
    <button
      disabled={currentStepIdx >= steps.length - 1}
      onClick={() => setCurrentStepIdx((prev) => prev + 1)}
      className="bg-slate-900 border border-slate-200 text-white"
    >
      Вперед
    </button>
  </div>
</div>

```

```
-----  
const pairId = (key: number, grade: number): NodeId =>  
  
interface TNode {  
  id: NodeId;  
  key: number;  
  grade: number;  
  left: TNode | null;  
  right: TNode | null;  
}  
  
export const POINTS: Pair[] = [  
  { key: 45, grade: 8 },  
  { key: 3, grade: 6 },  
  { key: 6, grade: 5 },  
  { key: 2, grade: 4 },  
  { key: 9, grade: 3 },  
  { key: 7, grade: 2 },  
  { key: 12, grade: 0 },  
  { key: 1, grade: -4 },  
];  
  
/** Порядок ввода (как в черновике) – намеренно НЕ отсортирован  
export const INPUT_ORDER: Pair[] = [  
  { key: 3, grade: 5 },  
  { key: 45, grade: 8 },  
  { key: 3, grade: 6 },  
  { key: 1, grade: -4 },  
  { key: 7, grade: 2 },  
  { key: 6, grade: 5 },  
  { key: 2, grade: 4 },  
  { key: 9, grade: 3 },  
  { key: 12, grade: 0 },  
];  
// (3,5) дубль ключа/приоритета – уберём: ввод без повт  
export const INPUT_CLEAN: Pair[] = INPUT_ORDER.filter((p)  
  
const sortedByY = (ps: Pair[]): Pair[] => [...ps].sort((a, b) => a.grade - b.grade);
```

```

-----
    return { l: res.l, r: res.r, trace };
}

export interface MergeResult {
    root: TNode | null;
    trace: { aId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null; note: string }[];
}

export function mergeTree(a: TNode | null, b: TNode | null): MergeResult {
    const trace: MergeResult["trace"] = [];
    const go = (a: TNode | null, b: TNode | null): TNode | null => {
        if (!a || !b) {
            trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosenId: null, note: `Соединение двух узлов целиком.` });
            return a ?? b;
        }
        if (a.grade > b.grade) {
            trace.push({ aId: a.id, bId: b.id, chosenId: a.id, note: `Выбор узла a, так как его правое поддеревое с (${b.key}, ${b.grade}) не содержит узла с ключом ${a.key}.` });
            a.right = go(a.right, b);
            return a;
        }
        trace.push({ aId: a.id, bId: b.id, chosenId: b.id, note: `Выбор узла b, так как его левое поддеревое с (${a.key}, ${a.grade}) не содержит узла с ключом ${b.key}.` });
        b.left = go(a, b.left);
        return b;
    };
    return { root: go(a, b), trace };
}

export function eraseNode(root: TNode | null, key: number): MergeResult {
    const trace: { aId: NodeId | null; note: string }[] = [];
    const go = (t: TNode | null): TNode | null => {
        if (!t) return null;
        if (key < t.key) {
            trace.push({ aId: t.id, note: `Удаление узла с ключом ${key} из левого поддерева.` });
            t.left = go(t.left);
            return t;
        }
        if (key > t.key) {
            trace.push({ aId: t.id, note: `Удаление узла с ключом ${key} из правого поддерева.` });
            t.right = go(t.right);
            return t;
        }
        // key === t.key
        trace.push({ aId: t.id, note: `Удаление узла с ключом ${key}.` });
        if (!t.left) return t.right;
        if (!t.right) return t.left;
        // both left and right exist
        let left = t.left;
        let right = t.right;
        while (right.left) right = right.left;
        right.left = left;
        return right;
    };
    return { root: go(root), trace };
}

```



```

    </svg>
    {subtitle && <p className="text-xs text-slate-400
  </div>
  });
};

const pairParts = (id: NodeId): [number, number] => {
  const [a, b] = id.split(":").map(Number);
  return [a, b];
};

/* ----- плоскость (клетчатая бумага) -----

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H

interface PlaneProps {
  points: Pair[];
  drawnEdges: { a: Pair; b: Pair }[];
  pending: Pair | null;
  interactive?: boolean;
  selected?: Pair | null;
  cursor?: { x: number; y: number } | null;
  badEdge?: { a: Pair; b: Pair } | null;
  onPointDown?: (p: Pair) => void;
  onPointUp?: (p: Pair) => void;
  onMove?: (e: React.PointerEvent<SVGSVGElement>) => vo
}

const Plane: React.FC<PlaneProps> = ({ points, drawnEdges
  }) => {
  const byPair = (p: Pair) => ({ x: PX(p.key), y: PY(p.
  return (
    <svg
      viewBox={`0 0 ${PL_W} ${PL_H}`}
      className={`w-full ${interactive ? "touch-none se
      onPointerMove={onMove}

```

```

const done = drawnEdges.some((e) => {e.a.key ===
const isSel = selected && selected.key === p.key
return (
  <g
    key={`pp-${p.key}:${p.grade}`}
    className="transition-all duration-500"
    onPointerDown={interactive && onPointDown ?
    onPointerUp={interactive && onPointUp ? () :
    style={interactive ? { cursor: "pointer" }
  >
    <circle cx={PX(p.key)} cy={PY(p.grade)} r={
    stroke={isSel ? "#a5b4fc" : done ? "#38bdf8"
    <text x={PX(p.key)} y={PY(p.grade) + 3.5} f
    ">{p.key}</text>
  </g>
  );
}})

{pending && (
  <g className="transition-all duration-500">
    <circle cx={PX(pending.key)} cy={PY(pending.g
    >
    <text x={PX(pending.key)} y={PY(pending.grade
    ly="monospace">{pending.key}</text>
  </g>
  )}
</svg>
);
};

/* ===== ИГРА: соединение мышкой =====

export type GEdge = { a: Pair; b: Pair }; // a – родите

export const CANON_GAME: GEdge[] = (() => {
  const es: GEdge[] = [];
  const walk = (n: TNode | null) => {

```

```

-----
    if (left.length > 1 || right.length > 1) return false
    return ok(left[0], lo, n.key) && ok(right[0], n.key, hi)
  };
  return roots.every((r) => ok(r, -Infinity, Infinity))
}

```

/** Полный вердикт по текущим линиям. */

```

export function gameVerdict(edges: GEdge[], all: Pair[]) {
  const heap = edges.every((e) => e.a.grade > e.b.grade)
  const oneParent = new Set(edges.map((e) => gpid(e.b)))
  const acyclic = !edges.some((e) => reaches(edges, e.a, e.b))
  const bst = forestIsBst(edges, all);
  const roots = all.filter((p) => !edges.some((e) => gpid(e.a) === p.key))
  const complete = edges.length === all.length - 1 && heap
  return { heap, oneParent, acyclic, bst, complete, win: complete }
}

```

/** Попытка хода: почему нельзя – или ОК. */

```

export function tryConnect(edges: GEdge[], a: Pair, b: Pair) {
  if (sameP(a, b)) return { ok: false, reason: "Это одна точка" };
  if (a.grade === b.grade)
    return { ok: false, reason: "«!» равны – «кто выше?»" };
  const parent = a.grade > b.grade ? a : b;
  const child = a.grade > b.grade ? b : a;
  if (parentOf(edges, gpid(child)))
    return { ok: false, reason: `У точки (${child.key}) уже есть родительская линия.` };
  if (reaches(edges, parent, child))
    return { ok: false, reason: "Цикл: линия замкнёт саму себя" };
  const next = [...edges, { a: parent, b: child }];
  if (!forestIsBst(next, POINTS))
    return { ok: false, reason: "Линия вниз по у допустима только если не образует цикла" };
  return { ok: true };
}

```

/* ===== РЕЖИМ BUILD ===== */

```

type BuildPhase = "sort" | "connect" | "game";

```

```

-----
export function canConnect(
  edges: GameEdge[],
  x: Pair,
  y: Pair,
): { ok: boolean; edge?: GameEdge; reason?: string } {
  if (x.key === y.key && x.grade === y.grade) return { ok: false, reason: "такая линия уже проведена" };
  for (const e of edges) {
    if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) || (pid(e.a) === pid(y) && pid(e.b) === pid(x))) {
      return { ok: false, reason: "такая линия уже проведена" };
    }
  }
  if (x.grade === y.grade) return { ok: false, reason: "линии с одинаковым уровнем не соединяются" };
  const [parent, child] = x.grade > y.grade ? [x, y] : [y, x];
  if (edges.some((e) => pid(e.b) === pid(child))) {
    return { ok: false, reason: `у точки (${child.key}) уже есть родитель` };
  }
  // цикл: родитель уже сидит в компоненте ребёнка (ребёнок уже в компоненте родителя)
  const childComp = new Set<string>([pid(child)]);
  let grew = true;
  while (grew) {
    grew = false;
    for (const e of edges) {
      if (childComp.has(pid(e.b)) && !childComp.has(pid(e.a))) {
        childComp.add(pid(e.a));
        grew = true;
      }
    }
  }
  if (childComp.has(pid(parent))) return { ok: false, reason: "линия в treap уже есть" };
  return {
    ok: true,
    edge: { a: parent, b: child },
    reason:
      x.grade > y.grade ? undefined : "линия в treap была бы запрещена, но мы её не добавляем",
  };
}

export function checkGameState(
  points: Pair[],

```

```

        seen.add(pid(p));
        const kids = children.get(pid(p)) ?? [];
        if (kids.length > 2) {
            bstOk = false;
            return;
        }
        const sorted = [...kids].sort((u, v) => u.key - v.key);
        let left: Pair | null = null;
        let right: Pair | null = null;
        if (sorted.length === 2) {
            if (sorted[0].key === sorted[1].key || sorted[0].key > sorted[1].key) {
                bstOk = false; // двое детей с равным x или с
                return;
            }
            left = sorted[0];
            right = sorted[1];
        } else if (sorted.length === 1) {
            // одиночный ребёнок: сторона важна – определяем
            if (sorted[0].key <= p.key) left = sorted[0];
            else right = sorted[0];
        }
        if (left) walk(left);
        inorder.push(p.key);
        if (right) walk(right);
    };
    walk(roots[0]);
    if (seen.size !== n) bstOk = false;
    for (let i = 1; i < inorder.length; i++)
        if (inorder[i] < inorder[i - 1]) bstOk = false;
    }
    return { heapOk, singleParent, connected, countOk, bstOk };
}

```

```

/** Дерево из проведённых линий – для показа результата
export function edgesToTree(edges: GameEdge[]): TNode | null {
    const parentOf = new Map<string, GameEdge>();
    for (const e of edges) parentOf.set(pid(e.b), e);
    const roots = edges.filter((e) => !parentOf.has(pid(e.b)));
    if (roots.length > 1) return null;
    if (roots.length === 0) return null;
    const root = roots[0];
    const tree = new TNode(root.b, root.k);
    const seen = new Set<string>();
    const inorder = [];
    const walk = (p: TNode) => {
        if (seen.has(pid(p))) return;
        seen.add(pid(p));
        const kids = children.get(pid(p)) ?? [];
        if (kids.length > 2) return;
        const sorted = [...kids].sort((u, v) => u.key - v.key);
        let left: TNode | null = null;
        let right: TNode | null = null;
        if (sorted.length === 2) {
            if (sorted[0].key === sorted[1].key || sorted[0].key > sorted[1].key) return;
            left = sorted[0];
            right = sorted[1];
        } else if (sorted.length === 1) {
            // одиночный ребёнок: сторона важна – определяем
            if (sorted[0].key <= p.key) left = sorted[0];
            else right = sorted[0];
        }
        if (left) walk(left);
        inorder.push(p.key);
        if (right) walk(right);
    };
    walk(root);
    if (seen.size !== edges.length) return null;
    for (let i = 1; i < inorder.length; i++)
        if (inorder[i] < inorder[i - 1]) return null;
    return tree;
}

```

```

    setConnIdx(10000);          // все рёбра проведены – ср
    setPlaying(false);
    // eslint-disable-next-line react-hooks/exhaustive-
  }, [live.pairsText]);
  const [phase, setPhase] = useState<BuildPhase>("sort");
  const [algo, setAlgo] = useState<SortAlgo>("none");
  const [order, setOrder] = useState<ConnectOrder>("y-d
  const [sortIdx, setSortIdx] = useState(0);
  const [connIdx, setConnIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [sortCost, setSortCost] = useState<string | nul

  const parsed = useMemo(() => parsePairs(raw), [raw]);
  const userPairs = parsed.errors.length ? [] : parsed.p

  const n = userPairs.length;

  const userTree = useMemo(() => (userPairs.length >= 1
  const canonEdges = useMemo(() => {
    const es: { a: Pair; b: Pair }[] = [];
    const walk = (node: TNode | null) => {
      if (!node) return;
      if (node.left) { es.push({ a: { key: node.key, gr
        left}); }
      if (node.right) { es.push({ a: { key: node.key, g
        de.right}); }
    };
    walk(userTree);
    return es;
  }, [userTree]);
  const leftRightEdges = useMemo(() => [...canonEdges].

  useEffect(() => {
    if (!playing || phase === "game") return;
    const t = setTimeout(() => {
      if (phase === "sort") {
        if (algo === "none") { setPlaying(false); return
        if (sortIdx < 3) setSortIdx(sortIdx + 1);
      }
    }, 1000);
  }, [playing, phase, algo, sortIdx]);

```

```

const startGame = () => {
  resetGame();
  setPhase("game");
  setPlaying(false);
};

const verdict = useMemo(() => checkGameState(userPair)
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree

const attemptConnect = (x: Pair, y: Pair) => {
  const r = canConnect(gameEdges, x, y);
  if (r.ok && r.edge) {
    setGameEdges((prev) => [...prev, r.edge!]);
    setGameMsg(r.reason ?? null);
    setBadEdge(null);
  } else {
    setGameMsg(r.reason ?? "так нельзя");
    setBadEdge({ a: x, b: y });
    window.setTimeout(() => setBadEdge(null), 900);
  }
};

const pointDown = (p: Pair) => {
  if (phase !== "game") return;
  if (!sel) {
    setSel(p);
    setGameMsg(`Выбрана (${p.key}; ${p.grade}) — теперь
  } else if (sel.key === p.key && sel.grade === p.grade) {
    setSel(null);
    setGameMsg(null);
  } else {
    attemptConnect(sel, p);
    setSel(null);
  }
};

const pointUp = (p: Pair) => {
  if (phase !== "game" || !sel) return;
  if (sel.key === p.key && sel.grade === p.grade) return;

```

```

placeholder="3 5, 45 8, 1 -4 ..."
/>
{parsed.errors.length > 0 && (
  <div className="mt-2 text-xs text-rose-300 bg-
    {parsed.errors.map((e, i) => <div key={i}>{e}
  </div>
)}
{parsed.errors.length === 0 && (parsed.equalX |
  <div className="mt-2 text-xs text-amber-300 bg-
    {parsed.equalY && <div>✂ Есть <b>равные y</b>
    кто <b>раньше</b> в списке</b> (сортировка ста
    {parsed.equalX && <div>✂ Есть <b>равные x</b>
    de.key → left).</div>}
  </div>
)}
</div>

{/* Координатная плоскость – НАВЕРХУ */}
<div className="bg-emerald-950/20 rounded-xl p-4"
  <div className="flex flex-wrap items-center jus
    <h4 className="text-base font-bold text-emera
    <div className="flex items-center gap-2">
      {phase === "sort" ? (
        <div>
          <span className="text-xs text-slate-400"
            {[ "quicksort", "counting" ] as const }.m
          <button
            key={a}
            onClick={() => { setAlgo(a); setSort
            Math.round(n * Math.log2(n)) : 0} сравнений
            disabled={n < 2}
            className={`px-3 py-1.5 rounded-lg
            600 text-white" : "bg-slate-700 text-slate-
          >
            {a === "quicksort" ? "quicksort (сп
            </button>
          )}}
        </div>
      )}
    </div>
  )}

```



```

        onPointUp={pointUp}
        onMove={planeMove}
    />
    <div className="mt-2 text-xs text-slate-300 bg-
      {phase === "sort" ? (
        algo === "none" ? (
          <>Ввод приходит <b>как попало</b> ({n} па
          ть</b>.</>
        ) : sortIdx < 3 ? (
          algo === "quicksort" ? (
            <>⚡ quicksort: счётчик сравнений: <b cl
            {comparisons}</b> ( $n \cdot \log_2 n$ ).</>
          ) : (
            <> counting: точки в корзины по значени
            , потому что у – маленькие целые.</>
          )
        ) : (
          <>Отсортировано ({sortCost ?? "способ не
          <b>Игру    </b></>
        )
      ) : phase === "game" ? (
        gameWon ? (
          <> Готово! {hintUsed ? "Решение подсказан
          ево: линии вниз по у, обход по х отсортиров
        ) : gameMsg ? (
          <>△ {gameMsg}</>
        ) : (
          <>Клики точку-родителя (выше по «!»), за
        )
      ) : connDone ? (
        <> Проведено {connEdges.length} линий в по
        </b>: точки прибиты, дерево одно. А теперь
      ) : (
        <>Линий проведено: {connIdx} из {connEdgesA
      )}
    </div>
  </div>

```

```

        ществует." />
      </div>
    })

    {/* Дерево */}
    {order === "y-desc" && userTree && (
      <div className="bg-slate-900/40 rounded-xl p-4" >
        <h4 className="text-sm font-bold text-slate-300" >
          <TreeView root={userTree} subtitle="Слева-направо" />
        </div>
      </div>
    )}
  </div>
);
};

/* ===== РЕЖИМ SPLIT ===== */

const SPLIT_KEYS = [2, 3, 6, 7, 9, 12, 45];

const SplitMode: React.FC = () => {
  const [x0, setX0] = useState(7);
  const [stepIdx, setStepIdx] = useState(0);
  const live = useTreapLive();
  const fullTree = useMemo(() => buildTree(POINTS), [POINTS]);
  const split = useMemo(() => splitTree(buildTree(POINTS), x0), [fullTree, x0]);
  const steps = split.trace;
  const idx = Math.min(stepIdx, steps.length - 1);

  // mid из кода выбирает разрез, step листает кадры split
  useEffect(() => {
    if (live.mid === null || !SPLIT_KEYS.includes(live.mid)) {
      setX0(live.mid);
      setStepIdx(0);
      // eslint-disable-next-line react-hooks/exhaustive-deps
    }, [live.mid]);
  }, [live.mid]);
  useEffect(() => {
    if (live.step === null) return;
    setStepIdx(Math.max(0, Math.min(steps.length - 1, stepIdx)));
  }, [live.step, stepIdx]);
};

```

```

        {steps[idx]?.note}
      </div>

      {done && (
        <div className="grid grid-cols-1 md:grid-cols-2"
          <div className="bg-sky-950/20 rounded-xl p-3"
            <h5 className="text-sm font-bold text-sky-300"
              <TreeView root={lTree} width={480} height={400} />
            </div>
            <div className="bg-rose-950/20 rounded-xl p-3"
              <h5 className="text-sm font-bold text-rose-300"
                <TreeView root={rTree} width={480} height={400} />
              </div>
            </div>
          </div>
        )}

      <div className="text-xs text-slate-400 bg-slate-900 p-3"
        <b>Как это работает на самом деле:</b> идём с левым деревом), отдаётся <b>вместе с веткой</b> – д
      </div>
    </div>
  );
};

/* ===== РЕЖИМ MERGE ===== */

const MergeMode: React.FC = () => {
  const [stepIdx, setStepIdx] = useState(0);
  const live = useTreapLive();
  const { l, r } = useMemo(() => splitTree(buildTree(P0, P1)))
  const merged = useMemo(() => mergeTree(cloneTree(l), cloneTree(r)))
  const steps = merged.trace;

  // step из кода листает кадры слияния
  useEffect(() => {
    if (live.step === null) return;
    setStepIdx(Math.max(0, Math.min(steps.length - 1, live.step)));
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [live]);
};

```

```

        <TreeView root={merged.root} subtitle="merge("
      </div>
    ))

    <div className="text-xs text-slate-400 bg-slate-900"
      <b>Грамотно про «слияние»:</b> у корней сравни
      в (то, что сохраняет порядок x) отправляется ,
    </div>
  </div>
);
};

/* ===== РЕЖИМ ERASE ===== */

const EraseMode: React.FC = () => {
  const [key, setKey] = useState(9);
  const [stepIdx, setStepIdx] = useState(0);
  const live = useTreapLive();
  const fullTree = useMemo(() => buildTree(POINTS), [])
  const res = useMemo(() => eraseNode(buildTree(POINTS)
  const steps = res.trace;

  // key из кода выбирает удаляемую точку, step листает
  useEffect(() => {
    if (live.key === null || !POINTS.some((p) => p.key === key))
      setKey(live.key);
    setStepIdx(0);
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [live.key]);
  useEffect(() => {
    if (live.step === null) return;
    setStepIdx(Math.max(0, Math.min(steps.length - 1, live.step)));
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [live.step, steps.length]);
  const idx = Math.min(stepIdx, steps.length - 1);
  const done = idx >= steps.length - 1;

  const pathIds = useMemo(() => {

```

```

        </div>
      )}

      <div className="text-xs text-slate-300 bg-slate-900" data-bbox="172 172 1000 238">
        {steps[idx]?.note}
      </div>

      <div className="text-xs text-slate-400 bg-slate-900" data-bbox="172 268 1000 334">
        <b>Ответ на «убрать можно только самый верхний»  

        ягивает merge двух детей. Удаляется <b>любая</b>
      </div>
    </div>
  );
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {
  const [pick, setPick] = useState<"heap" | "treap">("h
  // Полное дерево из 7 узлов (куча): индексы 1..7
  const heapArr = [0, 98, 45, 90, 32, 21, 76, 12];
  const full = useMemo(() => buildTree(POINTS), []);
  // Честная слотовая разметка treap: индексы как в куче
  const slots = useMemo(() => {
    const arr: (string | null)[] = Array(16).fill(null);
    const walk = (n: TNode | null, i: number) => {
      if (!n || i >= 16) return;
      arr[i] = n.id;
      walk(n.left, 2 * i);
      walk(n.right, 2 * i + 1);
    };
    walk(full, 1);
    return arr;
  }, [full]);
  const nodeCount = slots.filter(Boolean).length;

  return (
    <div className="space-y-4">

```

```

        });
      }
    }
  </div>
  <p className="text-xs text-rose-300 mt-2">Смо
    то левая ветка ушла на 4 уровня и заняла да
    казателями (left/right), а не арифметикой.<
  </div>
  })
}

<div className="text-xs text-slate-400 bg-slate-9
  <b>Грабля «дети в массиве по  $2k/2k+1$ »:</b> фор
  ивым: слои дырявые, и «адрес» ребёнка посчита
</div>
</div>
});
};

/* ===== РЕЖИМ SEARCH (поиск ≠ сортировка) =====

const SearchMode: React.FC = () => {
  // Подобрано так, что бинарный поиск ГАРАНТИРОВАННО пр
  // mid (индекс 3) = 9 > 7 → уходит влево, а цель лежи
  const shuffled: Pair[] = [
    { key: 12, grade: 0 },
    { key: 3, grade: 6 },
    { key: 45, grade: 8 },
    { key: 9, grade: 3 },
    { key: 6, grade: 5 },
    { key: 2, grade: 4 },
    { key: 7, grade: 2 },
    { key: 1, grade: -4 },
  ];
  const target = 7;
  const sortedByKey = [...shuffled].sort((a, b) => a.key
  const [stage, setStage] = useState<0 | 1 | 2>(0); //
  const [bidx, setBidx] = useState(0);

  // шаги бинарного поиска на отсортированном массиве д

```

```

    {(stage === 2 ? sortedByKey : shuffled).map((p,
      const highlight = stage === 1 && i === wrongM
      const inBs = stage === 2 && (() => { const s
      const isMid = stage === 2 && (() => { const s
      const isFound = stage === 2 && bidx >= bsStep
      return (
        <div key={` ${p.key}:${p.grade}-${i}`} class=
        nsition-all ${highlight ? "border-rose-500 b
        r-amber-500 bg-amber-950/30" : inBs ? "bord
        <div className="text-white font-bold">{p.
        <div className="text-slate-400 text-[10px
        </div>
      );
    })}
  </div>

  <div className="text-xs text-slate-300 bg-slate-9
    {stage === 0 && <>Массив пар <b>не отсортирован
    а беспорядок.</>}
    {stage === 1 && <> Бинарный поиск посмотрел на
    ю половину... <b>в которой лежала цель</b>. На
    реставил</b>: массив как был в беспорядке, та
    {stage === 2 && bidx < bsSteps.length - 1 && <>
    {stage === 2 && bidx >= bsSteps.length - 1 && <
    сравнения. Но чтобы поиск вообще заработал,
    сравнения на <b>всех</b>. «Отсортирую бинарни
  </div>

  <div className="text-xs text-slate-400 bg-slate-9
    <b>Есть ли разница, чем сортировать?</b> Срав
    няя граница, каждое сравнение даёт 1 бит. Но
    ля маленьких целых у (вкладка «Собрать», кноп
    задать порядок соединения.
  </div>
</div>
);
};

```

```

<div className="flex flex-wrap items-center gap-2"
  <button onClick={() => { setEdges(edges.slice(0
    rounded-lg text-xs font-bold bg-slate-700 tex
  <button onClick={() => { setEdges([]); setSel(n
    3 py-1.5 rounded-lg text-xs font-bold bg-slat
  <button onClick={() => { setEdges(CANON_GAME.map
    ); }} className="px-3 py-1.5 rounded-lg text-:
  <span className="text-xs text-slate-400">линий:
</div>

```

```

<div className="bg-emerald-950/20 rounded-xl p-4"
  <svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} class=
    <defs>
      <pattern id="gm-grid" width="14" height="14"
        <path d="M 14 0 L 0 0 0 14" fill="none" s
      </pattern>
    </defs>
    <rect x="34" y="16" width={PL_W - 46} height=
    <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={P
    <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y
    {edges.map((e, i) => {
      <line
        key={`ge-${i}`}
        x1={PX(e.a.key)} y1={PY(e.a.grade)}
        x2={PX(e.b.key)} y2={PY(e.b.grade)}
        stroke={i === edges.length - 1 ? "#34d399"
        strokeWidth={i === edges.length - 1 ? 3.5
        className="transition-all duration-300"
      />
    })}
  )}

```

```

{POINTS.map((p) => {
  const isSel = sel && sameP(sel, p);
  const isHover = hover && sameP(hover, p) &&
  return {
    <g key={`gp-${p.key}:${p.grade}`} onClick=
    l)} className="cursor-pointer">

```



```

-----
/**
 * Живые переменные Python-кода страницы treap:
 * points / pairs – набор пар (x; y): по нему ПЕРЕСТРАИВАЕМ
 * mid             – разрез для вкладки Split;
 * key             – удаляемая точка для вкладки Erase;
 * step / i        – номер шага (листаем кадры Split / Erase);
 * height, n       – высота дерева и число узлов (показываем);
 */
const useTreapLive = () => {
  const runtime = useVizRuntime();
  const vars = runtime?.variables;

  const pairsText = useMemo(() => {
    if (!vars) return null;
    const rawPts = vizArray(vars.points) ?? vizArray(vars.pairs);
    if (!rawPts || rawPts.length === 0) return null;
    const parts: string[] = [];
    rawPts.forEach((it) => {
      if (Array.isArray(it) && it.length >= 2) {
        const x = vizNumber(it[0]);
        const y = vizNumber(it[1]);
        if (x !== null && y !== null) parts.push(`${x} ${y}`);
      } else if (typeof it === "string") {
        const m = it.match(/-?\d+(?:\.\d+)?/g);
        if (m && m.length >= 2) parts.push(`${m[0]} ${m[1]}`);
      }
    });
    return parts.length ? parts.join(", ") : null;
  }, [vars]);

  const mid = vars ? vizNumber(vars.mid) : null;
  const key = vars ? vizNumber(vars.key) : null;
  const step = vars ? vizNumber(vars.step) ?? vizNumber(0) : null;
  const height = vars ? vizNumber(vars.height) ?? vizNumber(0) : null;
  const n = vars ? vizNumber(vars.n) : null;
  return { vars, pairsText, mid, key, step, height, n };
};

```

```

<div className="flex flex-wrap gap-2 mb-5">
  {TABS.map((t) => {
    <button
      key={t.id}
      onClick={() => setTab(t.id)}
      className={`px-4 py-2 rounded-lg text-xs font-medium border
        900/60 text-slate-400 hover:text-white border
      >
        {t.label}
      </button>
    )})}
</div>

```

```

{/* что сейчас сообщил компилятор */}
<div className="mb-4 flex flex-wrap items-center">
  <span className="text-[10px] uppercase tracking-wider">
    <Terminal className="w-3.5 h-3.5" /> из компилятора
  </span>
  {live.vars ? (
    <>
      {live.pairsText && <span className="text-[10px] font-medium">
        d px-2 py-1">points = {live.pairsText}</span>
      {live.n !== null && <span className="text-[10px] font-medium">
        py-1">n = {live.n}</span>
      {live.height !== null && <span className="text-[10px] font-medium">
        px-2 py-1">height = {live.height}</span>
      {live.mid !== null && <span className="text-[10px] font-medium">
        2 py-1">mid = {live.mid}</span>
      {live.key !== null && <span className="text-[10px] font-medium">
        x-2 py-1">key = {live.key}</span>
      {live.step !== null && <span className="text-[10px] font-medium">
        x-2 py-1">step = {live.step}</span>
      </>
    ) : (
      <span className="text-[11px] text-slate-500">
        открой панель Python: переменные <code class="font-mono text-emerald-300">mid</code>, <code class="font-mono text-emerald-300">for
        step</code> управляют этой демонстрацией
      </span>
    )
  )}

```

```

-----
* размерности, — выглядело так, будто компилятор исполн
* картинка живёт отдельной жизнью. Теперь источник данн
* «данные из компилятора (i, j, dist 5x5)», «компилятор
* пока нет» (отладчик стоит до первой строки) или «встр
*/
export function VizLiveStatus({ demoTitle, className =
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const names = vars ? Object.keys(vars).filter((name) :
  const linked = names.length > 0;

  const matrixSource = pickLiveVar(vars, MATRIX_NAMES);
  const matrix = liveMatrix(matrixSource?.value);

  const detail = matrix
    ? `${matrixSource?.name} ${matrix.sizeLabel}`
    : names.length
      ? names.slice(0, 6).join(", ") + (names.length >
      : undefined;

  return (
    <VizSourceBadge
      linked={linked}
      awaiting={!linked && !!runtime}
      line={runtime?.line}
      detail={detail}
      demoTitle={demoTitle}
      className={className}
    />
  );
}

```

```
/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
  "stack-dfs": {
    title: "Стек и обход в глубину",
    hint: "Кладите и снимайте тарелки – это ровно то, ч
    Component: StackViz,
  },
  "queue-bfs": {
    title: "Очередь и обход в ширину",
    hint: "Очередь слева, живой BFS по графу справа.",
    Component: () => (
      <div className="space-y-10">
        <QueueViz />
        <BfsViz />
      </div>
    ),
  },
  "heap-beam-search": {
    title: "Куча (priority queue)",
    hint: "Добавляйте гипотезы – куча сама держит лучшую
    Component: HeapViz,
  },
  "segment-trees": {
    title: "Дерево отрезков",
    hint: "Запросы и обновления на отрезке за  $O(\log n)$ .
    Component: SegmentTreeVisualizer,
  },
  "sparse-table": {
    title: "Разреженная таблица (1D и 2D)",
    hint: "Наведите курсор на любую ячейку – подсветитс
    Component: SparseTableViz,
  },
  treap: {
```

```

"bellman-ford": {
  title: "Форд–Беллман",
  Component: BellmanFordViz,
},
floyd: {
  title: "Флойд–Уоршелл",
  Component: FloydViz,
},
"johnson-algo": { title: "Алгоритм Джонсона", Component: JohnsonViz },
mst: {
  title: "Остовное дерево: Краскал, Прима, Борувка",
  hint: "Три алгоритма (билеты 18–20) на одном графе. Переменные parent/mst/taken/u/v/w перекрашивают граф.",
  Component: KruskalSimulator,
},
"graph-components": {
  title: "Компоненты связности: обход и DSU",
  hint: "Две вкладки: покраска обходом (каждый новый компонент из вашего кода красят вершины.",
  Component: GraphComponentsViz,
},
"complexity-classes": {
  title: "Классы сложности и сведения",
  hint: "Иерархия  $P \subseteq NP \subseteq PSPACE \subseteq EXP$ , сведение  $A \leq B$ ",
  Component: ComplexityViz,
},
"string-kmp": { title: "Префикс-функция (КМП)", Component: KmpViz },
"string-z-func": { title: "Z-функция", Component: ZFuncViz },
"aho-corasick": {
  title: "Ахо–Корасик",
  hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
  Component: () => (
    <div className="space-y-10">
      <WaterfallAnimationWidget />
      <SalmonAutomatonWidget />
    </div>
  ),
},

```

```

detail?: string;
/** Как называется встроенная демонстрация (для подписи) */
demoTitle?: string;
/** Размеры демо-сетки и данных пользователя не совпадают */
mismatch?: string | null;
className?: string;
}

/**
 * Явный индикатор источника данных визуализации.
 *
 * Без него демонстрация молча проигрывала собственный код
 * когда редактор Python пуст или в нём другие имена/переменные,
 * так, будто компилятор «исполняет какой-то свой код»,
 * «из компилятора» или «встроенное демо».
 */
export function VizSourceBadge({ linked, awaiting = false }) {
  const tip = linked
    ? `Значения берутся из выполненного Python-кода${demoTitle ? demoTitle : ''}`
    : awaiting
      ? `Панель Python выполнила код и стоит на строке ${currentLine}.  
Нажмите «шаг вперёд» (→) или «в конец трассы» — тогда визуализация их не увидит: список имён этой темы есть в коде.  
: "Сейчас показана встроенная демонстрация компонента. Когда вы закончите, визуализация переключится на ваши переменные."`
      : ''

  return (
    <div className={`flex flex-wrap items-center gap-1.5`} >
      <Tooltip side="bottom" content={tip}>
        <span
          tabIndex={0}
          className={`cursor-help inline-flex items-center gap-1.5`}
          linked
            ? "border-emerald-500/50 bg-emerald-500/10"
            : awaiting
              ? "border-sky-500/50 bg-sky-500/10 text-slate-800"
              : "border-slate-600 bg-slate-800/80 text-slate-800"
          >
        </span>
      </div>
    </div>
  )
}

```



```

        if (clean && clean !== textToScan) {
            setTextToScan(clean);
            setCurrentIndex(0);
            setCurrentNode(0);
            setFoundMatches([]);
            setIsSearchDone(false);
            setJumpPath(null);
            setActiveMode("search");
        }
    }
    if (liveState !== null && liveState >= 0 && WATERFA
    if (liveI !== null && liveI >= 0) setCurrentIndex(M
        ToScan).length));
    if (liveMatchesSig) {
        const parsed = (JSON.parse(liveMatchesSig) as unk
            .map((it) => {
                if (Array.isArray(it) && it.length >= 2) {
                    const idx = vizNumber(it[0]);
                    const word = vizString(it[1]);
                    if (idx !== null && word) return { index: i
                }
                if (typeof it === "string") {
                    const m = it.match(/(\d+)\D+([A-Za-zA-Яa-я]
                    if (m) return { index: Number(m[1]), word:
                }
                return null;
            })
            .filter((x): x is { index: number; word: string
        if (parsed.length) setFoundMatches(parsed);
    }
    // eslint-disable-next-line react-hooks/exhaustive-
}, [vars]));

const compilerLinked = Boolean(liveText || (liveState

// === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
const [textToScan, setTextToScan] = useState<string>(
const [currentIndex, setCurrentIndex] = useState<numb

```



```

let curr = currentNode;
let logMsg = `[Символ '${char}'] `;

let jumpType: 'normal' | 'fail' | null = null;
void jumpType;
let originalCurr = curr;

if (TRIE_TRANSITIONS[curr][char] !== undefined) {
  const nextNode = TRIE_TRANSITIONS[curr][char];
  logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label}). `;
  setJumpPath({ from: curr, to: nextNode, type: 'normal' });
  curr = nextNode;
  setActiveSearchCodeLine(3);
} else {
  setActiveSearchCodeLine(2);
  let jumps = 0;
  while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
    curr = WATERFALL_NODES[curr].fail;
    jumps++;
  }
  const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
  if (originalCurr === 0) {
    logMsg += `В корне нет перехода по '${char}'. Логично.`;
    setJumpPath(null);
  } else {
    logMsg += `Поток по '${char}' у вершины #${originalCurr} выходит  

    рыжок по fail-ссылке в #${curr} (${WATERFALL_NODES[curr].label}). `;
    setJumpPath({ from: originalCurr, to: nextNode, type: 'fail' });
  }
  curr = nextNode;
}

const currentMatches: { index: number; word: string } = [];
const matchedWordsForLog: string[] = [];

let temp = curr;

```

```

title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
desc: 'Мы обрабатываем первую вершину из очереди -
      просто не существует. fail[H] автоматически нап
codeLine: 5,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'H',
checkingBranchesFrom: null,
},
{
  targetNodeId: 7,
  title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
  desc: 'Обрабатываем дочернюю вершину #7 (S) на De
    ,
  codeLine: 5,
  scoutPos: 0,
  inspectedParentFail: 0,
  checkTargetChar: 'S',
  checkingBranchesFrom: null,
},
{
  targetNodeId: 2,
  title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
  desc: 'Переходим на Depth 2 к вершине #2 (HE). Ро
    а-разведчик плавает в ROOT и ищет ветку по "E".
  codeLine: 3,
  scoutPos: 0,
  inspectedParentFail: 0,
  checkTargetChar: 'E',
  checkingBranchesFrom: 0,
},
{
  targetNodeId: 5,
  title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
  desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
    H" и "S" не подходят. Ветки нет, fail[HI] = ROOT
  codeLine: 3,
  scoutPos: 0,

```

```

    {
      targetNodeId: 9,
      title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С...
      desc: '    МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
        ба плывет в #1 (H) и ищет ветку "E". У вершины :
      codeLine: 4,
      scoutPos: 1,
      inspectedParentFail: 1,
      checkTargetChar: 'E',
      checkingBranchesFrom: 1,
    },
    {
      targetNodeId: 4,
      title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
      desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро...
        #7 (S). fail[HERS] = #7 (S). Автомат полностью :
      codeLine: 4,
      scoutPos: 0,
      inspectedParentFail: 0,
      checkTargetChar: 'S',
      checkingBranchesFrom: 0,
    },
  ],
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTra

```

```

return (
  <div className="bg-slate-800/95 rounded-2xl p-6 border-1px border-slate-700">
    {/* Шапка и переключатель режимов */}
    <div className="flex flex-wrap items-center justify-between">
      <div className="flex items-center space-x-3">
        <span className="text-3xl"> </span>
        <div>
          <h4 className="text-2xl font-bold text-white">
            Анимация водопада: Полное обучение автомата
          </h4>

```

```

<div className="bg-slate-900/90 p-5 rounded-x
  <div>
    <h5 className="text-indigo-300 font-bold
      <span> </span> Полный обход в ширину (В
    </h5>
    <p className="text-xs text-slate-300 mb-3
      Показаны все шаги очереди BFS (начиная с
      арг 8 (SHE → HE)</strong>, где наглядно видно
    </p>
    <div className="space-y-1.5 mb-6 overflow
      {TRAINING_STEPS_INFO.map((step, idx) =>
        <button
          key={idx}
          onClick={() => setTrainStep(idx)}
          className={`w-full text-left px-3 p-3
            ${
              idx === trainStep
                ? 'bg-indigo-600 text-white font-medium'
                : 'bg-slate-800 text-slate-400'
            }`}
        >
          <span className="line-clamp-1">{step.name}</span>
          {idx === trainStep && <span className="font-medium">
            </button>
          </div>
        </div>

```

```

<div className="flex gap-2">
  <button
    onClick={() => setTrainStep((prev) => Math.max(0, prev - 1))}
    disabled={trainStep === 0}
    className="flex-1 bg-slate-700 hover:bg-slate-600 transition-all"
  >
    Назад
  </button>
  <button

```

```

        <span>3. while (v !== root && trie[v]
        {currentTrainInfo.codeLine === 3 && <
т, возврат в корень</span>}
    </div>
    <div className={`p-1.5 rounded flex item
r-l-4 border-emerald-500 text-emerald-300 f
    <span>4. if (trie[v][char] !== undefin
/span>
    {currentTrainInfo.codeLine === 4 && <
ан IF!</span>}
    </div>
    <div className={`p-1.5 rounded flex item
-4 border-blue-500 text-blue-300 font-bold'
    <span>5. else fail[u] = root; // Bero
    {currentTrainInfo.codeLine === 5 && <
root</span>}
    </div>
  </div>
</div>

<div>
  <h5 className="text-slate-200 font-bold m
    <Lightbulb className="w-4 h-4 text-ambe
  </h5>
  <div className="bg-slate-800 p-4 rounded-
w-inner min-h-[72px] flex items-center">
    {currentTrainInfo.desc}
  </div>
</div>
</div>
</div>
) : (
  /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
  <div className="grid grid-cols-1 lg:grid-cols-3
    <div className="bg-slate-900/60 p-5 rounded-x
    <div>
      <h5 className="text-white font-bold mb-2
      <span>» </span> Текст для сканирования

```

```

        <button
          onClick={stepSearchSimulation}
          disabled={isSearchDone || textToScan.length === 0}
          className={`w-full py-2.5 rounded-lg font-medium transition-colors
            ${isSearchDone || textToScan.length === 0
              ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
              : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
          `}
        >
          <span>{currentIndex === 0 ? 'Начать план' : 'Продолжить'}</span>
          <ArrowRight className="w-4 h-4" />
        </button>
      </div>
    </div>

    <div className="lg:col-span-2 bg-slate-900/60 p-4 rounded-2xl">
      <div>
        <h5 className="text-white font-bold mb-3">
          <span>✂</span> Текущее действие в коде
        </h5>
        <div className="bg-slate-950 p-4 rounded-2xl">
          <div className={`p-1.5 rounded flex items-center justify-between
            border-blue-500 text-blue-300 font-bold`} :
            <span>1. curr = state; char = text[i]
            {activeSearchCodeLine === 1 && <span>Инициализация</span>}
          </div>
          <div className={`p-1.5 rounded flex items-center justify-between
            border-rose-500 text-rose-300 font-bold`} :
            <span>2. while (curr !== root && !trie[curr].get(char)) {
              {activeSearchCodeLine === 2 && <span>Выход по fail!</span>}
            }
          </div>
          <div className={`p-1.5 rounded flex items-center justify-between
            border-blue-400 text-blue-200 font-bold`} :
            <span>3. curr = trie[curr].get(char)

```

```

<h5 className="text-white font-bold text-base" style={{width: 100%;}}>
  <span> </span> Ветвящийся синий водопад (Борис)
</h5>
<div className="flex flex-wrap gap-4 text-xs" style={{width: 100%;}}>
  <span className="flex items-center gap-1 text-white" style={{width: 50%;}}>
    <span className="w-3 h-0.5 bg-blue-500 inline-block" style={{width: 100%;}}>
    </span>
    <span className="flex items-center gap-1 text-white" style={{width: 50%;}}>
      <span className="w-3 h-0.5 border-t border-blue-500 inline-block" style={{width: 100%;}}>
      </span>
    </span>
  </div>
</div>

{/* Само SVG дерево водопада */}
<div className="w-full overflow-x-auto custom-scrollbar" style={{width: 100%;}}>
  <svg viewBox="0 0 800 520" className="w-full h-full" style={{width: 100%; height: 100%;}}>

    {/* Горизонтальные линии и подписи уровней */}
    {[
      { depth: 0, y: 60, label: 'Depth 0 (Корень)' },
      { depth: 1, y: 160, label: 'Depth 1 (H, S)' },
      { depth: 2, y: 260, label: 'Depth 2 (HE, L)' },
      { depth: 3, y: 360, label: 'Depth 3 (HER, L)' },
      { depth: 4, y: 460, label: 'Depth 4 (HERS, L)' },
    ]}.map((level) => {
      <g key={`depth-line-${level.depth}`}>
        <line
          x1="20"
          y1={level.y}
          x2="780"
          y2={level.y}
          stroke="#334155"
          strokeWidth="1"
          strokeDasharray="4,4"
          opacity="0.4"
        />
        <rect x="25" y={level.y - 12} width="14" height="24"

```

```

        fill={isTrainingExamined ? (isMatchBranch ? 'red' : 'green') : 'blue'}
        fontSize="12"
        fontWeight="bold"
        textAnchor="middle"
      >
        {char}
      </text>

      {/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В ПУНКТИРНЫЕ ВОДОПАДЫ */}
      {isTrainingExamined && (
        <g transform={`translate(${(fromNode.x - toNode.x) / 2})`}>
          <rect x="-15" y="-12" width="30" height="24"
            stroke="black" strokeWidth="2" />
          <text x="0" y="5" fontSize="14" fontStyle="italic">
            {isMatchBranch ? ' ' : 'X'}
          </text>
        </g>
      )}
    </g>
  </g>
);
});
}}}

```

```

{/* Рендер fail-ссылки (пунктирные водопады) */}
Object.values(WATERFALL_NODES).map((node) => {
  if (node.id === 0) return null;
  const targetNode = WATERFALL_NODES[node.failToId];

```

```

  // В режиме обучения показываем fail-ссылку
  // Найдем индекс шага, на котором обработали этот шаг
  const stepIdxForNode = TRAINING_STEPS_INFO[node.id].stepIdx;
  if (activeMode === 'training' && trainStep === stepIdxForNode) {

```

```

    let isHighlighted = activeMode === 'search' || activeMode === 'fail';
    let isJustEstablished = activeMode === 'training' && !isHighlighted;
    // Если не обработали этот шаг, то не показываем fail-ссылку

```



```

        <circle cx={node.x} cy={node.y} r="
    }}
    {isTargetChild && (
        <circle cx={node.x} cy={node.y} r="
    }}

    {/* Основной круг вершины */}
    <circle
        cx={node.x}
        cy={node.y}
        r={isCurrent || isTargetChild ? '24'
        fill={isCurrent ? '#2563eb' : isTar
        stroke={isCurrent ? '#ffffff' : isT
        strokeWidth={isCurrent || isTargetC
        className="transition-all duration-
    />

    {/* Текст внутри вершины */}
    <text
        x={node.x}
        y={node.y + 5}
        fill={isCurrent || isTargetChild ?
        fontSize={isCurrent || isTargetChil
        fontWeight="bold"
        textAnchor="middle"
    >
        {node.label}
    </text>

    {/* Метка искомой вершины в режиме об
    {isTargetChild && (
        <g transform={`translate(${node.x},
            <rect x="-35" y="-12" width="70"
            <text x="0" y="2" fontSize="10" f
                ИЩЕМ FAIL
            </text>
        </g>
    )}

```



```
import React, { useContext, useEffect, useMemo, useState } from 'react';
import {
  emitVizDemo,
  liveMatrix,
  pickLiveVar,
  useVizRuntime,
  vizArray,
  vizNumber,
  VizChapterContext,
} from '../../../data/vizStepBus';

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка – наведение на число:
 * * в 1D наводим на P[i] – подсвечивается кусок массива
 * * в 1D наводим на a[i] – видно, в какие префиксы он входит
 * * в 2D наводим на ячейку – подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения
 */

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];

const A2 = [
  [1, 2, 3, 4],
  [5, 6, 7, 8],
  [9, 1, 2, 3],
  [4, 5, 6, 7],
];

const cellBase = `flex items-center justify-center rounded-md font-mono text-sm
w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center`;

export const PrefixSumViz: React.FC = () => {
```

```
);
```

```
/** Что сейчас под курсором: префикс P[i] или элемент
const [hover, setHover] = useState<{ kind: "pref" | "a" }>({ kind: "pref" });
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 });
/** Снимок i/P из автоматически выполненного Python-кода
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveP = vizArray(vars?.P);
const hasLiveP = !!vars && Object.prototype.hasOwnProperty.call(vars, "P");
const compilerLinked = liveI !== null || hasLiveP;
// i/P управляют ячейками напрямую; индекс Python-строки
const shownDriven = compilerLinked
  ? liveI !== null
    ? Math.max(0, Math.min(A1.length, Math.trunc(liveI)))
    : Math.max(0, Math.min(A1.length, (liveP?.length || 0)))
  : null;
```

```
const covered =
  hover?.kind === "pref"
    ? { from: 0, to: hover.i - 1 }
    : hover?.kind === "a"
      ? { from: hover.i, to: hover.i }
      : null;
```

```
const sum = pref[range.r + 1] - pref[range.l];
```

```
return (
  <div className="space-y-5">
    <div className="overflow-x-auto pb-1">
      <div className="inline-block min-w-full">
        /** индексы */
        <div className="flex gap-1.5 mb-1 pl-[52px]">
          {A1.map((_, i) => (
            <div key={i} className={`_${cellBase} !h-5`}>
              {i}
            </div>
          ))}
        </div>
      </div>
    </div>
  </div>
);
```

```

const isL = i === range.l;
const isR = i === range.r + 1;
const liveValue = liveP?.[i];
const blank = hasLiveP && (liveValue === null);
const masked = !hasLiveP && shownDriven !== null;
const isDrivenCur = shownDriven !== null;
const shownValue = hasLiveP ? liveValue : null;
return (
  <div
    key={i}
    onMouseEnter={() => setHover({ kind: 'enter' })}
    onMouseLeave={() => setHover(null)}
    className={` ${cellBase} cursor-help ${
      isDrivenCur
        ? "bg-emerald-600 text-white ring-2 ring-emerald-500"
        : blank
        ? "bg-slate-950 text-slate-600"
        : masked
        ? "bg-slate-800/50 text-slate-700"
        : active
        ? "bg-emerald-500 text-white"
        : isR
        ? "bg-emerald-700 text-white"
        : isL
        ? "bg-rose-700 text-white"
        : "bg-slate-900 text-slate-500"
      }` }
    title={blank || masked ? `P[${i}] - n` : ''}
  >
    {blank || masked ? "." : String(shownValue)}
  </div>
);
}}}
</div>
</div>
</div>

{/* Пояснение под курсором */}

```

```

        y <b className="text-indigo-400">a</b> – ку,
      </p>
    })
  </div>

  { /* Запрос */
    <div className="bg-slate-900 border border-indigo
      <div className="text-xs text-slate-400 mb-2">
        Запрос суммы на отрезке (кликайте по массиву
      </div>
      <p className="font-mono text-sm sm:text-base te
        sum(a[{range.l}..{range.r}]) = P[{range.r + 1
        <span className="text-emerald-400">{pref[range
        <span className="text-rose-400">{pref[range.l
        <span className="text-indigo-300 font-bold">{
      </p>
      <p className="text-[11px] text-slate-500 mt-1">
        Один вычитание вместо цикла: запрос за  $O(1)$  п
      </p>
    </div>
  </div>
  );
};

```

```

/* ----- 2D -----
const Prefix2D: React.FC = () => {
  const n = A2.length;
  const m = A2[0].length;
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const liveI = vizNumber(vars?.i);
  const liveJ = vizNumber(vars?.j);

  const P = useMemo(() => {
    const p = Array.from({ length: n + 1 }, () => Array
    for (let i = 1; i <= n; i++) {
      for (let j = 1; j <= m; j++) {
        p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i]

```

```

const sizeMismatch =
  hasLiveS && liveS && (liveS.rows !== P.length || liveS.cols !== P[0].length)
  ? `Демо построено на матрице ${n}x${m} (сетка преобразована в массив, который имеет размер ${liveS.sizeLabel}). Ниже нарисована сетка
    : null;

useEffect(() => {
  const r1 = vizNumber(vars?.r1);
  const c1 = vizNumber(vars?.c1);
  const r2 = vizNumber(vars?.r2);
  const c2 = vizNumber(vars?.c2);
  if (r1 === null || c1 === null || r2 === null || c2 === null) return;
  setRect({
    r1: Math.max(0, Math.min(srcRows - 1, Math.trunc(r1))),
    c1: Math.max(0, Math.min(srcCols - 1, Math.trunc(c1))),
    r2: Math.max(0, Math.min(srcRows - 1, Math.trunc(r2))),
    c2: Math.max(0, Math.min(srcCols - 1, Math.trunc(c2))),
  });
}, [vars, srcRows, srcCols]);

// Границы запроса держим внутри реальной сетки (у пользователя может быть не та сетка)
const r1 = Math.max(0, Math.min(rect.r1, prefRows - 1));
const c1 = Math.max(0, Math.min(rect.c1, prefCols - 1));
const r2 = Math.max(r1, Math.min(rect.r2, prefRows - 1));
const c2 = Math.max(c1, Math.min(rect.c2, prefCols - 1));

const D = cellAt(r1, c1);
const B = cellAt(r1, c2 + 1);
const Cc = cellAt(r2 + 1, c1);
const Aa = cellAt(r2 + 1, c2 + 1);
const total = Aa !== null && B !== null && Cc !== null ? 3 : 0;
const fmt = (v: number | null) => (v === null ? "-" : v);

const cornerOf = (i: number, j: number) => {
  if (i === r2 + 1 && j === c2 + 1) return "A";
  if (i === r1 && j === c2 + 1) return "B";
  if (i === r2 + 1 && j === c1) return "C";
}

```

```

        </div>
      );
    }
  }
  </div>
}
</div>
</div>

{/* Матрица префиксных сумм */}
<div className="overflow-x-auto">
  <div className="text-xs text-slate-400 mb-2">
    Префиксные суммы {liveS ? `${livePrefSource`
  </div>
  <div className="inline-block">
    {grid.map((row, i) => {
      <div key={i} className="flex gap-1.5 mb-1">
        {row.map((v, j) => {
          const blank = v === null || v === und
          const shownValue = v;
          const corner = cornerOf(i, j);
          const isHover = hover?.i === i && hov
          const isCompilerCell = liveI === i &&
          const cornerColor =
            corner === "A"
              ? "bg-emerald-600 text-white"
              : corner === "D"
                ? "bg-sky-600 text-white"
                : corner
                  ? "bg-rose-600 text-white"
                  : "bg-slate-900 border border
        return {
          <div
            key={j}
            onMouseEnter={() => setHover({ i,
            onMouseLeave={() => setHover(null
            className={`${cellBase} cursor-he
              isCompilerCell
                ? "bg-emerald-600 text-white

```



```

    <p className="font-mono text-sm sm:text-base text-slate-500">
      сумма = <span className="text-emerald-400">A</span>
      <span className="text-rose-400">C</span> + <span
        className="text-indigo-300 font-bold">{r1}</span>
    </p>
    <p className="font-mono text-[11px] text-slate-500">
      прямоугольник ({r1}, {c1}) ... ({r2}, {c2}) · c
    </p>
    <p className="text-[11px] text-slate-500 mt-1">
      Вычли два «хвоста» и вернули дважды вычитенный
    </p>
  </div>
</div>
);
};

```

ФАЙЛ 84/111: src/components/visualizers/SparseTableViz.
 КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 899 | РАЗМЕР: 1000

```

import React, { useContext, useEffect, useRef, useState } from "react";
import {
  emitVizDemo,
  liveMatrix,
  useVizRuntime,
  vizArray,
  vizNumber,
  vizRecord,
  VizChapterContext,
} from "../../data/vizStepBus";
import { motion, AnimatePresence } from "motion/react";
import {
  Play,
  RotateCcw,
  ChevronRight,
} from "lucide-react";

```

```

for (let r = 0; r < 8; r++) {
  for (let c = 0; c < 8; c++) {
    ST2D[r][c][0][0] = val2D[r][c];
  }
}

for (let kx = 0; kx <= 3; kx++) {
  for (let ky = 0; ky <= 3; ky++) {
    if (kx === 0 && ky === 0) continue;
    for (let r = 0; r + (1 << kx) <= 8; r++) {
      for (let c = 0; c + (1 << ky) <= 8; c++) {
        if (kx > 0) {
          ST2D[r][c][kx][ky] = Math.min(
            ST2D[r][c][kx-1][ky],
            ST2D[r + (1 << (kx-1))][c][kx-1][ky]
          );
        } else {
          ST2D[r][c][kx][ky] = Math.min(
            ST2D[r][c][kx][ky-1],
            ST2D[r][c + (1 << (ky-1))][kx][ky-1]
          );
        }
      }
    }
  }
}

export const SparseTableViz: React.FC = () => {
  // Эта страница посвящена именно двумерной таблице: 1D
  // вкладкой, но больше не подменяет 2D-визуализацию и
  const [tab, setTab] = useState<"1d" | "2d_build" | "2d_viz">("1d");
  const chapterId = useContext(VizChapterContext);

  // Каждая вкладка = своё демо с собственным скелетом:
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, tab === "2d_build" ? "2d_build" : "2d_viz", [chapterId, tab]);
  }, [chapterId, tab]);
}

```

```

const VizID: React.FC = () => {
  const [step, setStep] = useState(0);
  const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
  const runtime = useVizRuntime();
  const runtimeVars = runtime?.variables;
  const liveI = vizNumber(runtimeVars?.i);
  const liveJ = vizNumber(runtimeVars?.j);
  const liveSource = vizArray(runtimeVars?.a);
  const liveRows = vizArray(runtimeVars?.st);
  const hasLiveSource = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'a');
  const hasLiveTable = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'st');
  const compilerLinked = liveI !== null || liveJ !== null;
  const shownSource = hasLiveSource
    ? liveSource?.length
      ? liveSource
      : Array.from({ length: N }, () => null)
    : arrID;
  const liveTarget =
    liveI !== null && liveJ !== null && Number.isInteger(liveI) && Number.isInteger(liveJ)
      ? { i: liveI, j: liveJ }
      : null;

  /** Размерность таблицы пользователя: сетка демо 8x4, */
  const liveTable = liveMatrix(runtimeVars?.st);
  const dimMismatch = liveTable
    ? liveTable.rows !== N || liveTable.cols !== LOG
      ? `Демо-сетка построена для массива из ${N} элементов. Таблица ниже рисует ячейки демо-сетки: значения элементов панели переменных компилятора.`
      : null
    : hasLiveSource && liveSource && liveSource.length
      ? `Демо-массив из ${N} элементов, а в вашем коде используется демо-размера ${N}x${LOG}.`
      : null;

  // Total steps: we animate computing each element for
  const computeSteps: { i: number; j: number }[] = [];

```

```

        className="p-2 bg-indigo-600 hover:bg-indigo-700"
        disabled={step === maxStep}
      >
        <ChevronRight size={20} />
      </button>
      <button
        onClick={() => setStep(0)}
        className="p-2 bg-slate-800 hover:bg-slate-700"
      >
        <RotateCcw size={20} />
      </button>
    </div>
  </div>

  <div className="bg-slate-900 rounded-xl border border-slate-800" >
    <div className="text-[11px] uppercase tracking-wider" >
      <div className="flex gap-1 sm:gap-1.5 overflow-hidden" >
        {shownSource.map((v, idx) => {
          const inCover = !!covered && idx >= covered
          const display = v === undefined || v === null
            ? "."
            : typeof v === "object"
              ? JSON.stringify(v)
              : String(v);
          return (
            <div key={idx} className="flex flex-col items-center justify-center" >
              <div
                className={`flex items-center justify-center w-[44px] h-[clamp(26px,8vw,44px)] text-[clamp(9px,1.2em)] ${
                  inCover ? "bg-indigo-500 text-white" : "text-slate-400"
                }`}
              >
                {display}
              </div>
              <span className="text-[9px] text-slate-400" >
                {v}
              </span>
            </div>
          );
        })}
      </div>
    </div>
  </div>

```

```

    2^1 (L=2)
  </div>
  <div className="text-center font-bold text-rap">
    2^2 (L=4)
  </div>
  <div className="text-center font-bold text-rap">
    2^3 (L=8)
  </div>

  {/* Matrix */}
  {Array.from({ length: N }).map((_, i) => {
    <React.Fragment key={i}>
      <div className="text-slate-500 font-mon
        {i}
      </div>
      {Array.from({ length: LOG }).map((_, j) => {
        const isValid = i + (1 << j) <= N;
        const liveRow = vizArray(liveRows?.[i]);
        const liveValue = liveRow?.[j];
        // Уровень j=0 – это исходный a[i], о
        // очищаемого рабочего массива и не д
        const sourceValue = j === 0 ? liveSou
        const runtimeValue = liveValue ?? sou
        const runtimeHasValue = runtimeValue
        const manualVisible =
          isValid &&
          (j === 0 || computeSteps.findIndex(
        const isVisible = compilerLinked ? run

      let isActive = !!liveTarget && liveTa
      let isSrc1 = false;
      let isSrc2 = false;

      if (compilerLinked && liveTarget && l
        isSrc1 = i === liveTarget.i && j ===
        isSrc2 = i === liveTarget.i + (1 <<

```

```

        ? "bg-sky-500 text-white"
        : isActive
        ? "bg-indigo-500 text-white"
        : isSrc1
        ? "bg-emerald-500 text-white"
        : isSrc2
        ? "bg-amber-500 text-white"
        : isVisible
        ? "bg-slate-800 text-white"
        : compilerLinked
        ? "bg-slate-950 text-white"
        : "bg-transparent"
    }`}
    title={compilerLinked ? `st[${state}]` : ""}
  >
    {displayValue}
  </motion.div>
)}

```

```

{/* Arrows visualization */}
{(isSrc1 || isSrc2) && (
  <motion.svg
    initial={{ opacity: 0 }}
    animate={{ opacity: 1 }}
    className="absolute pointer-events-none"
    style={{
      width: 100,
      height: isSrc2 ? 40 + (1 << 2) : 40,
      right: -50,
      top: 24,
      overflow: "visible",
    }}
  >
    <path
      d={`M 0 0 C 30 0, 30 ${isSrc1 ? 40 : 40 + (1 << 2)} 0`}
      fill="none"
      stroke={isSrc1 ? "#10b981" : "#f59e00"}
      strokeWidth="3"
    />
  </motion.svg>
)}

```

```

        </span>
        ) &rarr; min(
        <span className="text-emerald-400">{st1D[c.i + half][c.j - 1]}
        <span className="text-amber-400">
            {st1D[c.i + half][c.j - 1]}
        </span>
        ) ={" "}
        <span className="text-indigo-400 font-b"
            {st1D[c.i][c.j]}
        </span>
    </p>
    );
  })()
) : {
  <p className="text-slate-500 italic">
    Нажмите далее, чтобы увидеть, как блоки сво
  </p>
  })
</div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
  const [k, setK] = useState(1);
  const [hover, setHover] = useState<{ r: number; c: number}>({ r: 0, c: 0 });
  const [locked, setLocked] = useState<{ r: number; c: number}>({ r: 0, c: 0 });
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const liveK = vizNumber(vars?.k);
  /**
   * Шаг, закреплённый кликом по вкладке. Без него live
   * перетирал выбор пользователя и вкладки «Шаг 1/2/3»
   * клик есть, а матрица не меняется. Закрепление живё
   */
  const [pinnedK, setPinnedK] = useState<number | null>(liveK);
  const prevLiveK = useRef<number | null>(liveK);
  useEffect(() => {

```

```

return (
  <div className="flex flex-col xl:flex-row gap-6">
    <div className="flex-1 min-w-0 bg-slate-900 p-3 s
      <h3 className="text-xl font-bold text-indigo-400">
      <p className="text-sm text-slate-400 mb-6 text-
        Для наглядности показываем только <b classNam
        <br/>
        <span className="text-rose-300 animate-pulse m
          иц!</span>
      </p>
      {dimMismatch2D ? (
        <p className="mb-4 max-w-[520px] rounded-lg b
          {dimMismatch2D}
        </p>
      ) : null}

    <div className="flex bg-slate-950 p-1 rounded-l
      {[0, 1, 2, 3].map(step => (
        <button
          key={step}
          onClick={() => { setK(step); setPinnedK(s
          className={`px-4 py-2 rounded-md text-sm
            'text-slate-400 hover:text-white hover:bg-
          >
            {step === 0 ? "Оригинал (1x1)" : `Шаг ${s
          </button>
        )}}
      </div>
      {pinnedK !== null && compilerLinked && (
        <p className="-mt-6 mb-6 text-[11px] text-sla
          Шаг закреплён вручную: когда <span classNam
          демонстрация вернётся к шагу кода.
        </p>
      )}

    <div className="flex flex-col items-center gap-
      scrollbar">

```



```

const isActive = !!activeCell &&
ll.c + L) && ((r - activeCell.r) % (1 << st

    if (isActive) {
        if (isCurr) {
            // Значения в ячейке ещё не
            // контур «сюда смотрит код
            // залитая пустая клетка вы
            highlightClass = isBlank
                ? `bg-indigo-500/10 text-
== c ? 'ring-[3px] ring-white ring-offset-2
                : `bg-indigo-500 text-whi
&& locked.c === c ? 'ring-[3px] ring-white
                zIndex = 'z-10';
        } else {
            const prevL = 1 << (shownK
            const isTop = r < activeCel
            const isLeft = c < activeCe

                if (isTop && isLeft) {
                    highlightClass = 'bg-ros
md:scale-[1.08]';
                } else if (isTop && !isLeft
                    highlightClass = 'bg-blur
] md:scale-[1.08]';
                } else if (!isTop && isLeft
                    highlightClass = 'bg-eme
9,0.3)] md:scale-[1.08]';
                } else {
                    highlightClass = 'bg-amb
3)] md:scale-[1.08]';
                }
                zIndex = 'z-10';
            }
        }

    return (
        <div

```



```

        <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
        <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
        </div>
        <div className="flex iter
        <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{activ
        <span className="text
[activeCell.r + prevL][activeCell.c + prevL
        </div>
        </>
    )
  }}())}
</div>

    <div className="pt-3 mt-3 border-t
    ) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][shownK
    </div>
    <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
    </div>
    ) : {
      <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
        Наведите курсор на матрицу {L}x{L
        </div>
      )}
    </div>
  )}
</div>
</div>
</div>
);
};

```

```
const Viz2DQuery: React.FC = () => {
```

```

<div className="flex-1 min-w-0 bg-slate-900
  <h3 className="text-xl font-bold text-ro
  <p className="text-slate-400 text-[13px]
ерх, r2, c2 - правый низ).</p>
  {queryMismatch ? (
    <p className="mb-3 rounded-lg border b
      {queryMismatch}
    </p>
  ) : null}

  <div className="flex flex-col items-centr
    <div className="grid grid-cols-[repea
ative shadow-inner w-max max-w-full">
      {Array.from({length: 64}).map((_,
        const r = Math.floor(idx / 8);
        const c = idx % 8;
        const isInQuery = r >= r1 && r
          return (
            <div key={idx} className={`
y-center rounded text-[clamp(9px,2.4vw,12px
0 border-rose-500/50 border scale-105 z-10'
              {val2D[r][c]}
            </div>
          )
        })))

    <div
      className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
      style={{
        top: `${(r1 / 8) * 100}%`,
        left: `${(c1 / 8) * 100}%`,
        marginTop: '8px',
        marginLeft: '8px',
        height: 'calc(' + (H/8*100) +
        width: 'calc(' + (W/8*100) +
      }}

```

```

        </div>
    </div>

    <div className="flex-1 bg-slate-900 p-6 rounded-2xl" >
        <h3 className="text-xl font-bold text-emerald-400" >
            <p className="text-sm font-sans text-slate-300" >
                Мы знаем, что максимальная степень дв
            rounded text-white">{Lx}</span>. Аналогично
            te">{Ly}</span>.
        </p>
        <p className="text-sm font-sans text-slate-300" >
            Чтобы покрыть весь прямоугольник, мы
            Name="font-mono bg-[#0a0f1e] px-1 rounded b
            роса. (Цветные прямоугольники слева).
        </p>

        <div className="bg-[#0a0f1e] rounded-xl p-4" >
            text-slate-300 shadow-inner overflow-x-auto" >
                <div className="absolute right-3 top-3" >
                    <span className="text-slate-500">// 1.
                    <span className="text-purple-400">int<
                    ext-amber-300">1</span>); <span className="
                    <span className="text-purple-400">int<
                    ext-amber-300">1</span>); <span className="
                    <span className="text-purple-400">int<
                    "text-slate-500">// Выс: {Lx}</span><br/>
                    <span className="text-purple-400">int<
                    "text-slate-500">// Шир: {Ly}</span><br/><b
                    <span className="text-slate-500">// 2.
                    <span className="text-indigo-400">int<
                    &nbsp;&nbsp;&nbsp;<span className="text-slate-300" >
                    В запроса)</span><br/>
                    &nbsp;&nbsp;&nbsp;<span className="text-rose-900" >
                    r1</span>][<span className="text-white bg-r
                    &nbsp;&nbsp;&nbsp;<span className="text-slate-300" >
                    во, чтобы правый край уперся в c2)</span><b

```

```

ue-300 border-2"; badge = "TR"; }
    else if (isBL) { color = "bg-
-emerald-300 border-2"; badge = "BL"; }
    else if (isBR) { color = "bg-
mber-300 border-2"; badge = "BR"; }

```

```

    return (
      <div key={idx} className={
tion-all duration-300 relative shrink-0 ${c
        {txt}
        {badge} && <div classNam
r-slate-700 text-white px-1 font-bold round
      </div>

```

```

    )
  }}}}
</div>
</div>
</div>

```

```

<div className="mt-auto bg-slate-950 fon
)]">

```

```

  <div className="text-center font-bold
wrap bg-[#0a0fle] py-3 px-2 rounded-lg bord
    <span className="text-slate-400 fon
      <span className="text-rose-400">
      <span className="text-blue-400 mi
n>

```

```

      <span className="text-emerald-40
span>

```

```

      <span className="text-amber-400 r
      &nbsp;<span className="text-slate-4
      <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
    </div>

```

```

  </div>

```

```

</div>

```

```

</div>

```

```

        if (wtfContainer) wtfContainer.classList.remove('wtf');

        // Remove the URL hash to avoid scrolling
        history.pushState("", document.title, window.location.pathname);
        window.scrollTo(0, 0);
    } else {
        document.body.classList.remove('wtf-mode');
        if (aside) aside.style.display = '';
        if (headerMain) headerMain.style.display = '';
        if (questionsContainer) questionsContainer.classList.remove('hidden');
        if (wtfContainer) wtfContainer.classList.remove('wtf');
    }
}

function setTheme(theme) {
    const body = document.body;
    body.classList.remove('theme-dark', 'theme-light', 'theme-notebook');
    body.classList.add('theme-' + theme);

    // Just some visual feedback on buttons
    document.querySelectorAll('[id^="theme-"]')
        .forEach(btn => {
            btn.classList.remove('outline', 'outline-none');
        });

    if (theme === 'dark') document.getElementById('btn-theme-dark').classList.add('text-white transition-colors');
    if (theme === 'light') document.getElementById('btn-theme-light').classList.add('text-white transition-colors');
    if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors');

    document.getElementById('btn-theme-dark').classList.add('bg-gray-900 text-white transition-colors');
    document.getElementById('btn-theme-light').classList.add('bg-gray-100 text-white transition-colors');
    document.getElementById('btn-theme-notebook').classList.add('bg-gray-800 text-white transition-colors');
}

```

```

    if (drawerOverlay) {
      drawerOverlay.addEventListener("click",
    }

    // Close on link click
    document.querySelectorAll("#mobile-drawer a
      link.addEventListener("click", () => {
        if (mobileDrawer && !mobileDrawer.c
          toggleDrawer();
        }
      });
    });

    const observerOptions = {
      root: null,
      rootMargin: '-10% 0px -70% 0px',
      threshold: 0
    };

    const observer = new IntersectionObserver((
      entries.forEach(entry => {
        if (entry.isIntersecting && !document
          // Highlight active TOC link lo
          const id = entry.target.id;
          document.querySelectorAll('aside
            if (link.getAttribute('href
              link.classList.add('bg-
            } else {
              link.classList.remove('l
            }
          });
        }
      });
    }, observerOptions);

    document.querySelectorAll('section[id^="que
      observer.observe(section);
    });

```



```
        font-family: 'Arial Black', Impact, sans-serif;
    }
    .uno-card::before, .uno-card::after {
        content: attr(data-val); position: absolute;
        text-shadow: 1px 1px 0 #000, -1px -1px 0 #000;
    }
    .uno-card::before { top: 2px; left: 4px; }
    .uno-card::after { bottom: 2px; right: 4px; transform: rotate(180deg); }
    .uno-card.mini { width: 45px !important; height: 45px !important; }
    .uno-card.mini::before, .uno-card.mini::after {
        font-size: 10px;
    }

    /* Global formula scroll settings */
    .formula-scroll {
        overflow-x: auto;
        overflow-y: hidden;
        -webkit-overflow-scrolling: touch;
        scrollbar-width: none; /* Firefox */
        -ms-overflow-style: none; /* IE and Edge */
    }
    .formula-scroll::-webkit-scrollbar {
        display: none;
    }

    mjx-container {
        max-width: 100% !important;
    }
    mjx-container[display="true"] {
        overflow-x: auto;
        overflow-y: hidden;
        max-width: 100%;
        scrollbar-width: none;
        -ms-overflow-style: none;
        -webkit-overflow-scrolling: touch;
    }
    mjx-container[display="true"]::-webkit-scrollbar {
        display: none;
    }
```

```

king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-center shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 font-

/* THEME OVERRIDES */
body.theme-light {
    background-color: #f8fafc !important;
    color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light
body.theme-light .bg-slate-800 { background-color:
body.theme-light .bg-slate-700, body.theme-light
    important; }
body.theme-light .text-slate-200, body.theme-light
body.theme-light .text-slate-400 { color: #47556b; }
body.theme-light .border-slate-600, body.theme-light
body.theme-light .border-slate-500 { border-color:
body.theme-light #top-controls { background-color:

body.theme-notebook {
    background-color: #fcf8eb !important; /* light beige */
    color: #3b2818 !important; /* brown ink */
    background-image: repeating-linear-gradient(45deg,
    background-attachment: local !important;
    font-family: "Comic Sans MS", "Caveat", "Serif";
}
body.theme-notebook .bg-slate-900, body.theme-notebook
    .bg-slate-700 {
    background-color: rgba(255, 255, 255, 0.7);
    border-color: #cbd5e1 !important;
    box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook
    .text-slate-400 { color: #47556b; }
body.theme-notebook .text-slate-400 { color: #5b738d; }
body.theme-notebook .border-slate-600, body.theme-notebook
    .border-slate-500 { border-color: #94a3b8 !important; }

```

```

        </div>
    </div>
    <div class="flex items-center gap-3">
        <span class="text-sm font-bold uppercase tr
        <div class="bg-slate-800 p-1 rounded-lg fle
            <button onclick="setTheme('dark')" id="
        transition-colors">Темная</button>
            <button onclick="setTheme('light')" id=
        ext-white hover:bg-slate-700 transition-col
            <button onclick="setTheme('notebook')"
        over:text-white hover:bg-slate-700 transiti
        </div>
    </div>
</div>

<!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
<svg width="0" height="0" class="absolute">
    <defs>
        <marker id="arrow-yellow" markerWidth="6" m
            <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d"
        </marker>
        <marker id="arrow-orange" markerWidth="6" m
            <path d="M0,0 L6,3 L0,6 Z" fill="#f9731d"
        </marker>
        <marker id="arrow-pink" markerWidth="6" mar
            <path d="M0,0 L6,3 L0,6 Z" fill="#f472b2"
        </marker>
        <marker id="arrow-cyan" markerWidth="6" mar
            <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4bf"
        </marker>
        <marker id="arrow-indigo" markerWidth="6" m
            <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8"
        </marker>
        <marker id="arrow-lime" markerWidth="6" mar
            <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d"
        </marker>
        <marker id="arrow-green" markerWidth="6" ma
            <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80"

```

```

        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="0" y1="2" x2="40" y2="2" stro
        <text x="20" y="14" fill="#475569" font

    </g>
  </defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
  0/50 z-50 hover:bg-blue-500 transition-transform
    <svg xmlns="http://www.w3.org/2000/svg" class="
      <path stroke-linecap="round" stroke-linejoin
    </svg>
  </button>

<!-- Затемнение фона на мобилках при открытом меню
<div id="drawer-overlay" class="lg:hidden fixed inset-0
  background-color: #000000; opacity: 0.5;

<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-

  <!-- Левое меню (Оглавление) -->
  <aside id="mobile-drawer" class="fixed inset-y-0 left-0
    nslate-x-full transition-transform duration-300
    w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
    <div class="h-full flex flex-col p-6 overflow-y-auto
      lg:rounded-2xl lg:border lg:border-slate-700
      <div class="flex justify-between items-center
        <h3 class="font-black text-transparent bg-clip-text
        ider text-sm">
          Содержание
        </h3>
        <button id="close-drawer-btn" class="
          <svg xmlns="http://www.w3.org/2000/svg" class="
            <path stroke-linecap="round" stroke-linejoin="round"
          </svg>
        </button>
      </div>
    </div>
  </div>

```

```
<div class="pl-6 space-y-2 text
  <a href="#section-3-1" class="b
  <a href="#section-3-2" class="b
  <a href="#section-3-3" class="b
  <a href="#section-3-4" class="b
</div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-4" class:
er-amber-500 pl-3 font-bold text-amber-300">
    4. Степени и корни
  </a>
  </summary>
  <div class="pl-6 space-y-2 text
    <a href="#q4-def" class="block l
    <a href="#q4-moivre" class="blo
    <a href="#q4-root" class="block
    <a href="#q4-count" class="bloc
  </div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-5" class:
er-fuchsia-500 pl-3 font-bold text-fuchsia-:
    5. Группа корней из единицы
  </a>
  </summary>
  <div class="pl-6 space-y-2 text
    <a href="#q5-def" class="block l
    <a href="#q5-group" class="bloc
    <a href="#q5-prim" class="block
    <a href="#q5-crit" class="block
    <a href="#q5-cyclic" class="blo
  </div>
</details>
```

er-violet-500 pl-3 font-bold text-violet-300">8. Сравнимость и Поля В

</summary>

<div class="pl-6 space-y-2 text

<a href="#q8-1" class="bloc

<a href="#q8-2" class="bloc

<a href="#q8-3" class="bloc

<a href="#q8-4" class="bloc

<a href="#q8-5" class="bloc

<a href="#q8-6" class="bloc

<a href="#q8-7" class="bloc

</div>

</details>

<details class="group">

<summary class="list-none curso

<a href="#question-9" class

er-green-500 pl-3 font-bold text-green-400">

9. Кольцо многочленов

</summary>

<div class="pl-6 space-y-2 text

<a href="#q9-1" class="bloc

<a href="#q9-2" class="bloc

<a href="#q9-3" class="bloc

<a href="#q9-4" class="bloc

</div>

</details>

<details class="group">

<summary class="list-none curso

<a href="#question-10" clas

der-rose-500 pl-3 font-bold text-rose-400">

10. Делимость и Схема Г

</summary>

<div class="pl-6 space-y-2 text


```

        </a>
        <a href="#question-20" class="block
-500 pl-3 text-slate-400">
            <span class="font-bold text-ros
        </a>
        <a href="#question-21" class="block
go-500 pl-3 text-slate-400">
            <span class="font-bold text-ind
        </a>
        <a href="#question-22" class="block
sia-500 pl-3 text-slate-400">
            <span class="font-bold text-fuc
        </a>

        <a href="#proof-algorithms" class="
-red-500 pl-3 mt-4 text-red-500 font-bold m
        ▲ Ликбез: Алгоритмы выживания
        </a>

</nav>
<script>
    document.querySelectorAll('details.
        detail.addEventListener('toggle
            if (detail.open) {
                document.querySelectorA
                    if (other !== detail
                        other.removeAtt
                    }
            }
        });
    });
});

// Scroll spy logic
document.addEventListener('DOMConten
    const sections = document.query
    = "q5-", div[id^="q6-"], div[id^="q7-"], di
    v[id^="q13-"], div[id^="q14-"], div[id^="q15-"]

```



```

        }
      });
    }, {
      root: null,
      rootMargin: '-10% 0px -70% 0px',
      threshold: [0, 0.5]
    });

    sections.forEach(section => {
      observer.observe(section);
    });
  });
});

```

```

function fallbackModal(contentRaw, observer) {
  const modal = document.createElement('div');
  modal.style.position = 'fixed';
  modal.style.top = '0';
  modal.style.left = '0';
  modal.style.width = '100vw';
  modal.style.height = '100vh';
  modal.style.backgroundColor = 'black';
  modal.style.zIndex = '99999';
  modal.style.display = 'flex';
  modal.style.flexDirection = 'column-reverse';
  modal.style.alignItems = 'center';
  modal.style.justifyContent = 'center';

  const box = document.createElement('div');
  box.style.width = '80%';
  box.style.height = '80%';
  box.style.backgroundColor = '#1a1a1a';
  box.style.padding = '20px';
  box.style.borderRadius = '10px';
  box.style.display = 'flex';
  box.style.flexDirection = 'column-reverse';
  box.style.color = '#fff';

  const header = document.createElement('div');

```

```

        copyBtn.innerText = 'Скопировать';
    }
    setTimeout(() => copyBtn.innerText = 'Скопировано', 1000);
};

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Закрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = '#333';
closeBtn.style.color = '#fff';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5px';
closeBtn.style.cursor = 'pointer';
closeBtn.onclick = () => modal.close();

box.appendChild(header);
box.appendChild(subHeader);
if (title !== 'PDF') {
    box.appendChild(textarea);
    btnContainer.appendChild(copyBtn);
}
btnContainer.appendChild(closeBtn);
box.appendChild(btnContainer);
modal.appendChild(box);
document.body.appendChild(modal);
}

```

```

function downloadMD() {
    let clone = document.querySelector('#md-content');

    // Smart markdown parsing based on DOM
    // Smart markdown parsing based on DOM
    clone.querySelectorAll('h2').forEach(h2 => {
    clone.querySelectorAll('h3').forEach(h3 => {
    clone.querySelectorAll('.analog').forEach(analog => {
    clone.querySelectorAll('li').forEach(li => {
    clone.querySelectorAll('svg').forEach(svg => {

```

```

        style.id = 'theme-style';

        if (theme === 'dark') {
            body.classList.add('bg-slate');
            document.getElementById('theme-style').innerHTML = `
                body.bg-slate section h1 { color: white; }
                body.bg-slate main { color: white; }
                body.bg-slate main p { color: white; }
                /* Universal text color */
                body.bg-slate .text-slate { color: white; }
                body.bg-slate .text-slate-2 { color: white; }
                body.bg-slate .text-slate-3 { color: white; }
                body.bg-slate [class*="text-slate"] { color: white; }

                /* Specific backgrounds */
                body.bg-slate aside > div { background-color: #1a202c; }
                body.bg-slate [class*="bg-slate"] { background-color: #1a202c; }

                body.bg-slate [class*="bg-slate-2"] { background-color: #2d3748; }
                body.bg-slate [class*="bg-slate-3"] { background-color: #4a5568; }
                body.bg-slate .analogy-label { background-color: #4a5568; }
                body.bg-slate .img-placeholder { background-color: #4a5568; }
                body.bg-slate .info-block { background-color: #4a5568; }

                body.bg-slate .analogy-label { background-color: #4a5568; }
                body.bg-slate .img-captions { background-color: #4a5568; }

                /* Tweak card blocks */
                body.bg-slate .card-red { background-color: #f1271d; }

                body.bg-slate aside { background-color: #1a202c; }
            `;
        } else if (theme === 'light') {
            body.classList.add('bg-white');
            document.getElementById('theme-style').innerHTML = `
                body.bg-white section h1 { color: #1a202c; }
                body.bg-white main { color: #1a202c; }
                body.bg-white main p { color: #1a202c; }
                /* Universal text color */
                body.bg-white .text-slate { color: #1a202c; }
                body.bg-white .text-slate-2 { color: #1a202c; }
                body.bg-white .text-slate-3 { color: #1a202c; }
                body.bg-white [class*="text-slate"] { color: #1a202c; }

                /* Specific backgrounds */
                body.bg-white aside > div { background-color: #e2e3e5; }
                body.bg-white [class*="bg-white"] { background-color: #e2e3e5; }

                body.bg-white [class*="bg-white-2"] { background-color: #c6c8ca; }
                body.bg-white [class*="bg-white-3"] { background-color: #a6a8aa; }
                body.bg-white .analogy-label { background-color: #a6a8aa; }
                body.bg-white .img-placeholder { background-color: #a6a8aa; }
                body.bg-white .info-block { background-color: #a6a8aa; }

                body.bg-white .analogy-label { background-color: #a6a8aa; }
                body.bg-white .img-captions { background-color: #a6a8aa; }

                /* Tweak card blocks */
                body.bg-white .card-red { background-color: #f1271d; }

                body.bg-white aside { background-color: #e2e3e5; }
            `;
        }
    }
}

```

```

        font-family: monospace;
    }
    body.bg-black svg path,
        stroke: #22c55e !important;
        fill: transparent !important;
    }
    `;
}

document.head.appendChild(style)
}
</script>

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 border-1
    <!-- Кнопки Конвертации / Экспорта
    <div class="flex flex-col gap-2">
        <div class="text-xs text-slate-500 font-
        <button onclick="downloadWord()"
ont-bold flex items-center justify-center gap-2
            <span> </span> Word
        </button>
        <a href="algebra.pdf" target="_blank"
-sm font-bold flex items-center justify-center gap-2
            <span> </span> PDF
        </a>
        <button onclick="downloadMD()"
-bold flex items-center justify-center gap-2
            <span> </span> Markdown
        </button>
        <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center gap-2
            <span> </span> HTML
        </button>
    </div>

```

```

<li><a href="#question-4">4. Стр
<li><a href="#question-5">5. Гр
<li><a href="#question-6">6. Де
<li><a href="#question-7">7. Ал
<li><a href="#question-8">8. Кл
<li><a href="#question-9">9. Ко
<li><a href="#question-10">10.
<li><a href="#question-11">11.
<li><a href="#question-12">12.
<li><a href="#question-13">13.
<li><a href="#question-14">14.
<li><a href="#question-15">15.
<li><a href="#question-17">17.
<li><a href="#question-18">18.
<li><a href="#question-19">19.
<li><a href="#question-20">20.
<li><a href="#question-21">21.
<li><a href="#question-22">22.
    </ul>
  </div>
</header>

<!-- ===== WTF CONTAINER =====>
<div id="wtf-container" class="hidden">
  <h2 class="text-3xl font-black text-cen

    <div class="overflow-x-auto w-full pb-4">
      <div class="grid grid-cols-3 gap-4">
        <!-- Column headers -->
        <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
        <div class="bg-orange-900/40 bo
ольцо полиномов</div>
        <div class="bg-cyan-900/40 bord
сные числа</div>

        <!-- Row 1: База -->
        <a href="#question-1" onclick="setM

```

```

        <a href="#question-12" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange
        <div class="font-bold text-slate
    </a>
    <!-- Empty 3rd row cell -->
    <div class="hidden md:block"></div>

    <!-- Row 4: Факторизация / Структур
    <a href="#question-8" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
    </a>
    <a href="#question-11" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange
        <div class="font-bold text-slate
    </a>
    <a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-5
        <div class="font-bold text-slate
    </a>

    <!-- Row 5: Корни / Уравнения -->
    <div class="hidden md:block"></div>
    <a href="#question-13" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange
        <div class="font-bold text-slate
    </a>
    <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-5
        <div class="font-bold text-slate
    </a>

```

```

<a href="#question-20" onclick="setl
-l-4 border-rose-500 hover:bg-slate-700 tra
<div class="text-xs text-rose-5
<div class="font-bold text-slate
</a>
<a href="#question-21" onclick="setl
-l-4 border-indigo-500 hover:bg-slate-700 t
<div class="text-xs text-indigo
<div class="font-bold text-slate
</a>
<a href="#question-22" onclick="setl
-l-4 border-fuchsia-500 hover:bg-slate-700
<div class="text-xs text-fuchsia
<div class="font-bold text-slate
</a>

<!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ
<div class="col-span-1 md:col-span-3
border-b border-red-500/30 pb-2">Кластер 4:

<a href="#proof-algorithms" onclick="setl
:col-span-3 bg-red-900/20 p-4 border-l-4 bo
<div class="text-xs text-red-500
<div class="font-bold text-slate
</a>
</div>
</div>

<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="m
<div class="flex items-center mb-6">
<div class="bg-red-500/20 p-3 r
<svg class="w-8 h-8 text-red-500"
rg/2000/svg">
<path stroke-linecap="r
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
</svg>
</div>

```

```

<!-- Единственность -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
  <h3 class="text-xl text-eme
  <p class="text-sm text-slate-400" style="color: #a0aec0;">
  <ul class="list-disc pl-5 sp
    <li><span class="text-w
, r_2$).</li>
    <li><span class="text-w
    <li><span class="text-w
счет ограничений (размер остатка) докажи, ч
  </ul>
</div>

<!-- Индукция -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
  <h3 class="text-xl text-yel
  <p class="text-sm text-slate-400" style="color: #a0aec0;">
  <ul class="list-disc pl-5 sp
    <li><span class="text-w
валось.</li>
    <li><span class="text-w
Просто перепиши формулу с буквой $k$.</li>
    <li><span class="text-w
усок $k$, замени его на жареного карася из
  </ul>
</div>

<!-- Двойная делимость -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
  <h3 class="text-xl text-ora
  <p class="text-sm text-slate-400" style="color: #a0aec0;">
  <ul class="list-disc pl-5 sp
    <li><span class="text-w
вверх по уравнениям).</li>
    <li><span class="text-w
рху вниз, показывая, что $\delta \vdots r_i
    <li><span class="text-w

```


</div>

<h2 class="text-2xl font-bold b
(Правила экстрасенса)</h2>

<p class="text-sm text-slate-400">
тношения), а ты их не помнишь. Юзай базовые
нейность) для делимости.</p>

<ul class="space-y-4 pb-10">

<li class="bg-slate-800/50 p-4">

<div>

<h4 class="text-cyan-400">

<p class="text-sm t">

<code class="text-x">

</div>

<li class="bg-slate-800/50 p-4">

<div>

<h4 class="text-cyan-400">

<p class="text-sm t">

х двух, потом последних.

<code class="text-x">

</div>

<li class="bg-slate-800/50 p-4">

<div>

<h4 class="text-cyan-400">

<p class="text-sm t">

есть сложение — есть ноль. Если есть умноже

<code class="text-x">

</div>

ФАЙЛ 88/111: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 341 | РАЗМЕР: 15.5 KB

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";
import { annotateTicketLinks } from "../ticketLinks";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили защиты внутрь, инт
 * его можно открыть с флешки, отправить в мессенджер и
 * Живые React-визуализации в статику не переносятся —
 * подставляется заметка со ссылкой на онлайн-версию, а
 * которые в приложении всплывают по наведению, собираю
 * в конце документа.
 */

/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
  const parts: string[] = [];
  for (const sheet of Array.from(document.styleSheets))
    try {
      const rules = (sheet as CSSStyleSheet).cssRules;
      parts.push(Array.from(rules).map((r) => r.cssText));
    } catch {
      const href = (sheet as CSSStyleSheet).href;
      if (!href) continue;
      try {
        const res = await fetch(href);
        if (res.ok) parts.push(await res.text());
      } catch {
        /* чужой домен — пропускаем */
      }
    }
  }
}
```

```
-----
const escapeHtml = (s: string) =>
  s.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(

/**
 * Размечает термины и превращает их в якорные ссылки на
 * Возвращает готовый HTML главы и список найденных терминов
 */
function prepareBody(html: string, opts: { selfId?: string;
  body: string;
  termIds: string[];
} {
  const host = document.createElement("div");
  host.innerHTML = html;

  try {
    annotateTerms(host);
  } catch {
    /* если браузер не осилил регулярку – выгружаем текст
    */

  /*
   * Упоминания «билет 15» становятся якорями на главу 15
   * только для книги, где все главы на месте: в файле book.html
   * вёл бы в никуда, а битые ссылки – ровно то, от чего мы
   */
  if (opts.ticketAnchors) {
    try {
      annotateTicketLinks(host, { selfId: opts.selfId });
      host.querySelectorAll<HTMLAnchorElement>("a[data-goto]")
        .forEach(a => {
          a.setAttribute("href", `#ch-${a.dataset.goto}`);
          a.removeAttribute("data-goto");
        });
    } catch {
      /* ссылки – улучшение, а не обязательная часть файла
      */
    }
  }

  const ids: string[] = [];
```

```
</section>`;
}
```

```
function wrap(opts: { title: string; subtitle: string;
  const stamp = new Date().toLocaleString("ru-RU");
  return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, init
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
<div class="export-shell">
  <header class="export-head">
    <div style="font-size:.7rem;text-transform:uppercase
    <h1 style="color:#fff;font-size:1.6rem;font-weight:
    <div style="font-size:.72rem;color:#475569;margin-t
  </header>

  <div class="export-note">
    Это автономная копия: стили внутри файла, интернет
    Интерактивные тренажёры и анимации в статичный HTML
    они живут в онлайн-версии: <a href="${escapeHtml(op
  </div>

  <article class="prose prose-invert max-w-none">
${opts.body}
  </article>

${opts.gloss}

  <footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
```



```

const css = await collectCss();
const allTerms: string[] = [];
const bodies: string[] = [];

const toc = chapters
  .map((c) => `<li style="margin:.2rem 0"><a href="#c
  .join("\n");

for (const c of chapters) {
  const { body: rawBody, termIds } = prepareBody(c.co
  // картинки глав встраиваются сразу: книга одна, ас
  // снимки демонстраций подтягиваются из /export/viz
  const body = (await vizFigures(c.id)) + (await inli
  termIds.forEach((t) => {
    if (!allTerms.includes(t)) allTerms.push(t);
  });
  bodies.push(`<div id="ch-${escapeHtml(c.id)}" style=
<p style="text-align:right;font-size:.72rem;margin:0 0
  }

const body = `<nav id="toc" style="background:#0f172a
  rem">
  <h2 style="color:#fff;font-size:1.05rem;font-weight:8
  <ol style="margin:0;padding-left:1.1rem;font-size:.85
</nav>
`${bodies.join("\n")}`;

const html = wrap({
  title: "Универсальное пособие по алгоритмам",
  subtitle: `Все темы, ${chapters.length} шт.`,
  css,
  body,
  gloss: glossarySection(allTerms, true),
  origin: window.location.origin,
});
return html;
}

```

Универсальное пособие • полный исходный код

```
-----  
/** id страницы по номеру билета (undefined, если такого нет)  
export const chapterForTicket = (ticket: number): string | undefined
```

```
/** «билет», «билеты», «билетов», «билете», «билетам»...  
const TICKET_RE = /билет(?:ы|ов|ам|ах|ом|у|а|е)?\s*(\d{1,2})?/
```

```
/**  
 * Константы TreeWalker числами: глобальный `NodeFilter`  
 * во всех окружениях (в jsdom его нет) – с числами разницы  
 */
```

```
const SHOW_TEXT = 4;  
const FILTER_ACCEPT = 1;  
const FILTER_REJECT = 2;
```

```
/** Внутри чего ссылки не ставим: код, уже существующие  
const SKIP_SELECTOR = "code, pre, script, style, a, button, form, input, select, textarea, div, span, p, h1, h2, h3, h4, h5, h6, h7, h8, h9, h10, h11, h12, h13, h14, h15, h16, h17, h18, h19, h20, h21, h22, h23, h24, h25, h26, h27, h28, h29, h30, h31, h32, h33, h34, h35, h36, h37, h38, h39, h40, h41, h42, h43, h44, h45, h46, h47, h48, h49, h50, h51, h52, h53, h54, h55, h56, h57, h58, h59, h60, h61, h62, h63, h64, h65, h66, h67, h68, h69, h70, h71, h72, h73, h74, h75, h76, h77, h78, h79, h80, h81, h82, h83, h84, h85, h86, h87, h88, h89, h90, h91, h92, h93, h94, h95, h96, h97, h98, h99, h100, h101, h102, h103, h104, h105, h106, h107, h108, h109, h110, h111, h112, h113, h114, h115, h116, h117, h118, h119, h120, h121, h122, h123, h124, h125, h126, h127, h128, h129, h130, h131, h132, h133, h134, h135, h136, h137, h138, h139, h140, h141, h142, h143, h144, h145, h146, h147, h148, h149, h150, h151, h152, h153, h154, h155, h156, h157, h158, h159, h160, h161, h162, h163, h164, h165, h166, h167, h168, h169, h170, h171, h172, h173, h174, h175, h176, h177, h178, h179, h180, h181, h182, h183, h184, h185, h186, h187, h188, h189, h190, h191, h192, h193, h194, h195, h196, h197, h198, h199, h200, h201, h202, h203, h204, h205, h206, h207, h208, h209, h210, h211, h212, h213, h214, h215, h216, h217, h218, h219, h220, h221, h222, h223, h224, h225, h226, h227, h228, h229, h230, h231, h232, h233, h234, h235, h236, h237, h238, h239, h240, h241, h242, h243, h244, h245, h246, h247, h248, h249, h250, h251, h252, h253, h254, h255, h256, h257, h258, h259, h260, h261, h262, h263, h264, h265, h266, h267, h268, h269, h270, h271, h272, h273, h274, h275, h276, h277, h278, h279, h280, h281, h282, h283, h284, h285, h286, h287, h288, h289, h290, h291, h292, h293, h294, h295, h296, h297, h298, h299, h300, h301, h302, h303, h304, h305, h306, h307, h308, h309, h310, h311, h312, h313, h314, h315, h316, h317, h318, h319, h320, h321, h322, h323, h324, h325, h326, h327, h328, h329, h330, h331, h332, h333, h334, h335, h336, h337, h338, h339, h340, h341, h342, h343, h344, h345, h346, h347, h348, h349, h350, h351, h352, h353, h354, h355, h356, h357, h358, h359, h360, h361, h362, h363, h364, h365, h366, h367, h368, h369, h370, h371, h372, h373, h374, h375, h376, h377, h378, h379, h380, h381, h382, h383, h384, h385, h386, h387, h388, h389, h390, h391, h392, h393, h394, h395, h396, h397, h398, h399, h400, h401, h402, h403, h404, h405, h406, h407, h408, h409, h410, h411, h412, h413, h414, h415, h416, h417, h418, h419, h420, h421, h422, h423, h424, h425, h426, h427, h428, h429, h430, h431, h432, h433, h434, h435, h436, h437, h438, h439, h440, h441, h442, h443, h444, h445, h446, h447, h448, h449, h450, h451, h452, h453, h454, h455, h456, h457, h458, h459, h460, h461, h462, h463, h464, h465, h466, h467, h468, h469, h470, h471, h472, h473, h474, h475, h476, h477, h478, h479, h480, h481, h482, h483, h484, h485, h486, h487, h488, h489, h490, h491, h492, h493, h494, h495, h496, h497, h498, h499, h500, h501, h502, h503, h504, h505, h506, h507, h508, h509, h510, h511, h512, h513, h514, h515, h516, h517, h518, h519, h520, h521, h522, h523, h524, h525, h526, h527, h528, h529, h530, h531, h532, h533, h534, h535, h536, h537, h538, h539, h540, h541, h542, h543, h544, h545, h546, h547, h548, h549, h550, h551, h552, h553, h554, h555, h556, h557, h558, h559, h560, h561, h562, h563, h564, h565, h566, h567, h568, h569, h570, h571, h572, h573, h574, h575, h576, h577, h578, h579, h580, h581, h582, h583, h584, h585, h586, h587, h588, h589, h590, h591, h592, h593, h594, h595, h596, h597, h598, h599, h600, h601, h602, h603, h604, h605, h606, h607, h608, h609, h610, h611, h612, h613, h614, h615, h616, h617, h618, h619, h620, h621, h622, h623, h624, h625, h626, h627, h628, h629, h630, h631, h632, h633, h634, h635, h636, h637, h638, h639, h640, h641, h642, h643, h644, h645, h646, h647, h648, h649, h650, h651, h652, h653, h654, h655, h656, h657, h658, h659, h660, h661, h662, h663, h664, h665, h666, h667, h668, h669, h670, h671, h672, h673, h674, h675, h676, h677, h678, h679, h680, h681, h682, h683, h684, h685, h686, h687, h688, h689, h690, h691, h692, h693, h694, h695, h696, h697, h698, h699, h700, h701, h702, h703, h704, h705, h706, h707, h708, h709, h710, h711, h712, h713, h714, h715, h716, h717, h718, h719, h720, h721, h722, h723, h724, h725, h726, h727, h728, h729, h730, h731, h732, h733, h734, h735, h736, h737, h738, h739, h740, h741, h742, h743, h744, h745, h746, h747, h748, h749, h750, h751, h752, h753, h754, h755, h756, h757, h758, h759, h760, h761, h762, h763, h764, h765, h766, h767, h768, h769, h770, h771, h772, h773, h774, h775, h776, h777, h778, h779, h780, h781, h782, h783, h784, h785, h786, h787, h788, h789, h790, h791, h792, h793, h794, h795, h796, h797, h798, h799, h800, h801, h802, h803, h804, h805, h806, h807, h808, h809, h810, h811, h812, h813, h814, h815, h816, h817, h818, h819, h820, h821, h822, h823, h824, h825, h826, h827, h828, h829, h830, h831, h832, h833, h834, h835, h836, h837, h838, h839, h840, h841, h842, h843, h844, h845, h846, h847, h848, h849, h850, h851, h852, h853, h854, h855, h856, h857, h858, h859, h860, h861, h862, h863, h864, h865, h866, h867, h868, h869, h870, h871, h872, h873, h874, h875, h876, h877, h878, h879, h880, h881, h882, h883, h884, h885, h886, h887, h888, h889, h890, h891, h892, h893, h894, h895, h896, h897, h898, h899, h900, h901, h902, h903, h904, h905, h906, h907, h908, h909, h910, h911, h912, h913, h914, h915, h916, h917, h918, h919, h920, h921, h922, h923, h924, h925, h926, h927, h928, h929, h930, h931, h932, h933, h934, h935, h936, h937, h938, h939, h940, h941, h942, h943, h944, h945, h946, h947, h948, h949, h950, h951, h952, h953, h954, h955, h956, h957, h958, h959, h960, h961, h962, h963, h964, h965, h966, h967, h968, h969, h970, h971, h972, h973, h974, h975, h976, h977, h978, h979, h980, h981, h982, h983, h984, h985, h986, h987, h988, h989, h990, h991, h992, h993, h994, h995, h996, h997, h998, h999, h1000, h1001, h1002, h1003, h1004, h1005, h1006, h1007, h1008, h1009, h1010, h1011, h1012, h1013, h1014, h1015, h1016, h1017, h1018, h1019, h1020, h1021, h1022, h1023, h1024, h1025, h1026, h1027, h1028, h1029, h1030, h1031, h1032, h1033, h1034, h1035, h1036, h1037, h1038, h1039, h1040, h1041, h1042, h1043, h1044, h1045, h1046, h1047, h1048, h1049, h1050, h1051, h1052, h1053, h1054, h1055, h1056, h1057, h1058, h1059, h1060, h1061, h1062, h1063, h1064, h1065, h1066, h1067, h1068, h1069, h1070, h1071, h1072, h1073, h1074, h1075, h1076, h1077, h1078, h1079, h1080, h1081, h1082, h1083, h1084, h1085, h1086, h1087, h1088, h1089, h1090, h1091, h1092, h1093, h1094, h1095, h1096, h1097, h1098, h1099, h1100, h1101, h1102, h1103, h1104, h1105, h1106, h1107, h1108, h1109, h1110, h1111, h1112, h1113, h1114, h1115, h1116, h1117, h1118, h1119, h1120, h1121, h1122, h1123, h1124, h1125, h1126, h1127, h1128, h1129, h1130, h1131, h1132, h1133, h1134, h1135, h1136, h1137, h1138, h1139, h1140, h1141, h1142, h1143, h1144, h1145, h1146, h1147, h1148, h1149, h1150, h1151, h1152, h1153, h1154, h1155, h1156, h1157, h1158, h1159, h1160, h1161, h1162, h1163, h1164, h1165, h1166, h1167, h1168, h1169, h1170, h1171, h1172, h1173, h1174, h1175, h1176, h1177, h1178, h1179, h1180, h1181, h1182, h1183, h1184, h1185, h1186, h1187, h1188, h1189, h1190, h1191, h1192, h1193, h1194, h1195, h1196, h1197, h1198, h1199, h1200, h1201, h1202, h1203, h1204, h1205, h1206, h1207, h1208, h1209, h1210, h1211, h1212, h1213, h1214, h1215, h1216, h1217, h1218, h1219, h1220, h1221, h1222, h1223, h1224, h1225, h1226, h1227, h1228, h1229, h1230, h1231, h1232, h1233, h1234, h1235, h1236, h1237, h1238, h1239, h1240, h1241, h1242, h1243, h1244, h1245, h1246, h1247, h1248, h1249, h1250, h1251, h1252, h1253, h1254, h1255, h1256, h1257, h1258, h1259, h1260, h1261, h1262, h1263, h1264, h1265, h1266, h1267, h1268, h1269, h1270, h1271, h1272, h1273, h1274, h1275, h1276, h1277, h1278, h1279, h1280, h1281, h1282, h1283, h1284, h1285, h1286, h1287, h1288, h1289, h1290, h1291, h1292, h1293, h1294, h1295, h1296, h1297, h1298, h1299, h1300, h1301, h1302, h1303, h1304, h1305, h1306, h1307, h1308, h1309, h1310, h1311, h1312, h1313, h1314, h1315, h1316, h1317, h1318, h1319, h1320, h1321, h1322, h1323, h1324, h1325, h1326, h1327, h1328, h1329, h1330, h1331, h1332, h1333, h1334, h1335, h1336, h1337, h1338, h1339, h1340, h1341, h1342, h1343, h1344, h1345, h1346, h1347, h1348, h1349, h1350, h1351, h1352, h1353, h1354, h1355, h1356, h1357, h1358, h1359, h1360, h1361, h1362, h1363, h1364, h1365, h1366, h1367, h1368, h1369, h1370, h1371, h1372, h1373, h1374, h1375, h1376, h1377, h1378, h1379, h1380, h1381, h1382, h1383, h1384, h1385, h1386, h1387, h1388, h1389, h1390, h1391, h1392, h1393, h1394, h1395, h1396, h1397, h1398, h1399, h1400, h1401, h1402, h1403, h1404, h1405, h1406, h1407, h1408, h1409, h1410, h1411, h1412, h1413, h1414, h1415, h1416, h1417, h1418, h1419, h1420, h1421, h1422, h1423, h1424, h1425, h1426, h1427, h1428, h1429, h1430, h1431, h1432, h1433, h1434, h1435, h1436, h1437, h1438, h1439, h1440, h1441, h1442, h1443, h1444, h1445, h1446, h1447, h1448, h1449, h1450, h1451, h1452, h1453, h1454, h1455, h1456, h1457, h1458, h1459, h1460, h1461, h1462, h1463, h1464, h1465, h1466, h1467, h1468, h1469, h1470, h1471, h1472, h1473, h1474, h1475, h1476, h1477, h1478, h1479, h1480, h1481, h1482, h1483, h1484, h1485, h1486, h1487, h1488, h1489, h1490, h1491, h1492, h1493, h1494, h1495, h1496, h1497, h1498, h1499, h1500, h1501, h1502, h1503, h1504, h1505, h1506, h1507, h1508, h1509, h1510, h1511, h1512, h1513, h1514, h1515, h1516, h1517, h1518, h1519, h1520, h1521, h1522, h1523, h1524, h1525, h1526, h1527, h1528, h1529, h1530, h1531, h1532, h1533, h1534, h1535, h1536, h1537, h1538, h1539, h1540, h1541, h1542, h1543, h1544, h1545, h1546, h1547, h1548, h1549, h1550, h1551, h1552, h1553, h1554, h1555, h1556, h1557, h1558, h1559, h1560, h1561, h1562, h1563, h1564, h1565, h1566, h1567, h1568, h1569, h1570, h1571, h1572, h1573, h1574, h1575, h1576, h1577, h1578, h1579, h1580, h1581, h1582, h1583, h1584, h1585, h1586, h1587, h1588, h1589, h1590, h1591, h1592, h1593, h1594, h1595, h1596, h1597, h1598, h1599, h1600, h1601, h1602, h1603, h1604, h1605, h1606, h1607, h1608, h1609, h1610, h1611, h1612, h1613, h1614, h1615, h1616, h1617, h1618, h1619, h1620, h1621, h1622, h1623, h1624, h1625, h1626, h1627, h1628, h1629, h1630, h1631, h1632, h1633, h1634, h1635, h1636, h1637, h1638, h1639, h1640, h1641, h1642, h1643, h1644, h1645, h1646, h1647, h1648, h1649, h1650, h1651, h1652, h1653, h1654, h1655, h1656, h1657, h1658, h1659, h1660, h1661, h1662, h1663, h1664, h1665, h1666, h1667, h1668, h1669, h1670, h1671, h1672, h1673, h1674, h1675, h1676, h1677, h1678, h1679, h1680, h1681, h1682, h1683, h1684, h1685, h1686, h1687, h1688, h1689, h1690, h1691, h1692, h1693, h1694, h1695, h1696, h1697, h1698, h1699, h1700, h1701, h1702, h1703, h1704, h1705, h1706, h1707, h1708, h1709, h1710, h1711, h1712, h1713, h1714, h1715, h1716, h1717, h1718, h1719, h1720, h1721, h1722, h1723, h1724, h1725, h1726, h1727, h1728, h1729, h1730, h1731, h1732, h1733, h1734, h1735, h1736, h1737, h1738, h1739, h1740, h1741, h1742, h1743, h1744, h1745, h1746, h1747, h1748, h1749, h1750, h1751, h1752, h1753, h1754, h1755, h1756, h1757, h1758, h1759, h1760, h1761, h1762, h1763, h1764, h1765, h1766, h1767, h1768, h1769, h1770, h1771, h1772, h1773, h1774, h1775, h1776, h1777, h1778, h1779, h1780, h1781, h1782, h1783, h1784, h1785, h1786, h1787, h1788, h1789, h1790, h1791, h1792, h1793, h1794, h1795, h1796, h1797, h1798, h1799, h1800, h1801, h1802, h1803, h1804, h1805, h1806, h1807, h1808, h1809, h1810, h1811, h1812, h1813, h1814, h1815, h1816, h1817, h1818, h1819, h1820, h1821, h1822, h1823, h1824, h1825, h1826, h1827, h1828, h1829, h1830, h1831, h1832, h1833, h1834, h1835, h1836, h1837, h1838, h1839, h1840, h1841, h1842, h1843, h1844, h1845, h1846, h1847, h1848, h1849, h1850, h1851, h1852, h1853, h1854, h1855, h1856, h1857, h1858, h1859, h1860, h1861, h1862, h1863, h1864, h1865, h1866, h1867, h1868, h1869, h1870, h1871, h1872, h1873, h1874, h1875, h1876, h1877, h1878, h1879, h1880, h1881, h1882, h1883, h1884, h1885, h1886, h1887, h1888, h1889, h1890, h1891, h1892, h1893, h1894, h1895, h1896, h1897, h1898, h1899, h1900, h1901, h1902, h1903, h1904, h1905, h1906, h1907, h1908, h1909, h1910, h1911, h1912, h1913, h1914, h1915, h1916, h1917, h1918, h1919, h1920, h1921, h1922, h1923, h1924, h1925, h1926, h1927, h1928, h1929, h1930, h1931, h1932, h1933, h1934, h1935, h1936, h1937, h1938, h1939, h1940, h1941, h1942, h1943, h1944, h1945, h1946, h1947, h1948, h1949, h1950, h1951, h1952, h1953, h1954, h1955, h1956, h1957, h1958, h1959, h1960, h1961, h1962, h1963, h1964, h1965, h1966, h1967, h1968, h1969, h1970, h1971, h1972, h1973, h1974, h1975, h1976, h1977, h1978, h1979, h1980, h1981, h1982, h1983, h1984, h1985, h1986, h1987, h1988, h1989, h1990, h1991, h1992, h1993, h1994, h1995, h1996, h1997, h1998, h1999, h2000, h2001, h2002, h2003, h2004, h2005, h2006, h2007, h2008, h2009, h2010, h2011, h2012, h2013, h2014, h2015, h2016, h2017, h2018, h2019, h2020, h2021, h2022, h2023, h2024, h2025, h2026, h2027, h2028, h2029, h2030, h2031, h2032, h2033, h2034, h2035, h2036, h2037, h2038, h2039, h2040, h2041, h2042, h2043, h2044, h2045, h2046, h2047, h2048, h2049, h2050, h2051, h2052, h2053, h2054, h2055, h2056, h2057, h2058, h2059, h2060, h2061, h2062, h2063, h2064, h2065, h2066, h2067, h2068, h2069, h2070, h2071, h2072, h2073, h2074, h2075, h2076, h2077, h2078, h2079, h2080, h2081, h2082, h2083, h2084, h2085, h2086, h2087, h2088, h2089, h2090, h2091, h2092, h2093, h2094, h2095, h2096, h2097, h2098, h2099, h2100, h2101, h2102, h2103, h2104, h2105, h2106, h2107, h2108, h2109, h2110, h2111, h2112, h2113, h2114, h2115, h2116, h2117, h2118, h2119, h2120, h2121, h2122, h2123, h2124, h2125, h2126, h2127, h2128, h2129, h2130, h2131, h2132, h2133, h2134, h2135, h2136, h2137, h2138, h2139, h2140, h2141, h2142, h2143, h2144, h2145, h2146, h2147, h2148, h2149, h2150, h2151, h2152, h2153, h2154, h2155, h2156, h2157, h2158, h2159, h2160, h2161, h2162, h2163, h2164, h2165, h2166, h2167, h2168, h2169, h2170, h2171, h2172, h2173, h2174, h2175, h2176, h2177, h2178, h2179, h2
```

```

): number {
  if (typeof document === "undefined") return 0;
  const walker = document.createTreeWalker(root, SHOW_TEXT,
    acceptNode(node) {
      const parent = (node as Text).parentElement;
      if (!parent) return FILTER_REJECT;
      if (parent.closest(SKIP_SELECTOR)) return FILTER_REJECT;
      const text = node.nodeValue ?? "";
      if (!/билет/i.test(text)) return FILTER_REJECT;
      return FILTER_ACCEPT;
    },
  ));

  const targets: Text[] = [];
  let current = walker.nextNode();
  while (current) {
    targets.push(current as Text);
    current = walker.nextNode();
  }

  let created = 0;
  for (const textNode of targets) {
    const text = textNode.nodeValue ?? "";
    TICKET_RE.lastIndex = 0;
    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;
    while ((m = TICKET_RE.exec(text)) !== null) {
      const from = Number(m[1]);
      const to = m[2] ? Number(m[2]) : from;
      const id = chapterForRange(from, to);
      if (!id || id === opts.selfId) continue;
      if (m.index > last) frag.appendChild(document.createTextNode(
        const a = document.createElement("a");
        a.className = "ticket-link";
        a.dataset.goto = id;
        a.setAttribute("href", hrefFor(id, opts.lite ?? false));
        a.textContent = m[0];
      ));
    }
  }
}

```



```

-----
*   node scripts/audit-coverage.mjs           # отчёт
*   node scripts/audit-coverage.mjs --md      # markd
*/
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
  entryPoints: [path.join(ROOT, "src/data/content.ts")],
  bundle: true,
  format: "esm",
  platform: "node",
  outfile: bundle,
  logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/registry.json"));
const registryBody = registry.slice(registry.indexOf("VIZUALIZATORY"), registry.length);
const vizIds = new Set([...registryBody.matchAll(/^\\s{2}[a-zA-Z0-9_]+/g)].map(m => m[0]));

const rows = chapters.map((c) => {
  const html = c.content ?? "";
  const analogies = (html.match(/Аналогия/gi) ?? []).length;
  return {
    id: c.id,
    title: c.title,
    num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
    analogies,
  };
});

```

```
-----  
    console.log(`Пропущенные номера билетов 1-24: ${missi  
    console.log(`Визуализаторы без главы: ${orphanViz.joi  
}
```

ФАЙЛ 91/111: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```
-----  
  
/**  
 * Проверка покрытия глав всплывающими подсказками:  
 * в каждой ли теме находятся термины из глоссария.  
 *  
 *   node scripts/audit-terms.mjs  
 */  
import { build } from "esbuild";  
import path from "node:path";  
  
const ROOT = process.cwd();  
await build({  
  entryPoints: ["src/data/content.ts", "src/data/glossa  
  bundle: true, format: "esm", platform: "node",  
  outdir: "tmp/audit-terms", outExtension: { ".js": ".m  
  external: ["react", "react-dom", "motion", "lucide-re  
});  
  
const { chapters } = await import(path.join(ROOT, "tmp/  
const { GLOSSARY_LABELS, GLOSSARY } = await import(path  
  
const esc = (s) => s.replace(/[.*+?^${}()|[\]\\"/>  
const re = new RegExp(  
  `(?<![\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e  
  "giu"  
);  
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label  
  
// те же лимиты, что и в src/hooks/useTermHints.ts
```

Универсальное пособие • полный исходный код

```
-----  
console.log("Не встретились:", GLOSSARY.filter((g) => !
```

```
-----  
ФАЙЛ 92/111: scripts/export/build-guide-pdf.mjs
```

```
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТР0К: 865 | РАЗМ
```

```
-----  
/**  
 * Компактный PDF пособия: public/export/guide_lite.pdf  
 *  
 * текст всех тем (без служебной разметки)  
 * + снимки визуализаций, если они отрендерены (tmp/viz/  
 * capture-viz-shots.mjs в CI через headless-браузер)  
 * + иллюстрации страниц (jpg из src/assets)  
 * + текстовое описание каждой демонстрации и переменны  
 * Python-компилятор (работает и без браузера – локал  
 * – БЕЗ кода панели компилятора и без навигации прилож  
 *  
 * Запуск: npm run export:guide  
 * Цель: 40–60 страниц (жёсткий предел – 100).  
 *  
 * Вёрстка целиком на собственном курсоре (cy): pdfkit  
 * сам, длинные таблицы/код/боксы режутся на фрагменты  
 */  
import fs from "node:fs";  
import path from "node:path";  
import { build } from "esbuild";  
import PDFDocument from "pdfkit";  
import { ROOT, OUT_DIR, ensureDir, humanSize } from "../  
import { htmlToBlocks } from "../html-to-blocks.mjs";  
  
const GUIDE_PDF_PATH = path.join(OUT_DIR, "guide_lite.p  
const SHOTS_DIR = path.join(ROOT, "tmp", "viz-shots");  
const ENTRY = path.join(ROOT, "tmp", "guide-entry.ts");
```

```

nodes: [
  { id: "S", label: "S", x: 40, y: 60, dist: "0", s
  { id: "A", label: "A", x: 130, y: 30, dist: "4",
  { id: "B", label: "B", x: 130, y: 90, dist: "5",
  { id: "C", label: "C", x: 220, y: 60, dist: "3",
  { id: "D", label: "D", x: 310, y: 30, dist: "6",
  { id: "E", label: "E", x: 310, y: 90, dist: "7",
  { id: "F", label: "F", x: 400, y: 60, dist: "8",
],
edges: [
  { u: "S", v: "A", w: 4, highlighted: true },
  { u: "S", v: "B", w: 5, highlighted: true },
  { u: "B", v: "C", w: -2, highlighted: true },
  { u: "A", v: "C", w: 1 },
  { u: "C", v: "D", w: 3, highlighted: true },
  { u: "C", v: "E", w: 4, highlighted: true },
  { u: "D", v: "F", w: 2 },
  { u: "E", v: "F", w: 1, highlighted: true },
  { u: "D", v: "A", w: -2 },
],
table: [
  ["Проход (итерация)", "Рёбра в работе", "Изменени
  ["Пара 1", "S→A, S→B", "dist[A]=4, dist[B]=5", "н
  ["Пара 2", "B→C (-2)", "dist[C]=3", "нормальный х
  ["Пара 3", "C→D, C→E", "dist[D]=6, dist[E]=7", "н
  ["Пара 4", "E→F (1)", "dist[F]=8", "нормальный хо
  ["Итерация V-1", "все рёбра сошлись", "без измене
  ["Итерация V (контроль)", "все рёбра", "без измен
],
},
floyd: {
  title: "Матрица кратчайших путей (Фloyd–Уоршелл)",
  table: [
    ["из \ в", "S (0)", "A (1)", "B (2)", "C (3)", "
    ["S (0)", "0", "4", "5", "3", "6"],
    ["A (1)", "∞", "0", "∞", "1", "4"],
    ["B (2)", "∞", "∞", "0", "-2", "1"],
    ["C (3)", "∞", "∞", "∞", "0", "3"],

```

```
const TOP = PAGE.margin;
const BOTTOM = PAGE.height - PAGE.margin - 16;

/* ----- шрифты -----

const CANDIDATES = {
  regular: [
    "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
    "/usr/share/fonts/dejavu/DejaVuSans.ttf",
    "/usr/local/share/fonts/DejaVuSans.ttf",
    "/System/Library/Fonts/Supplemental/Arial Unicode.ttf",
    "C:/Windows/Fonts/arial.ttf",
  ],
  bold: [
    "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
    "/usr/share/fonts/dejavu/DejaVuSans-Bold.ttf",
    "/usr/local/share/fonts/DejaVuSans-Bold.ttf",
    "C:/Windows/Fonts/arialbd.ttf",
  ],
  oblique: [
    "/usr/share/fonts/truetype/dejavu/DejaVuSans-Oblique.ttf",
    "/usr/share/fonts/dejavu/DejaVuSans-Oblique.ttf",
  ],
  mono: [
    "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
    "/usr/share/fonts/dejavu/DejaVuSansMono.ttf",
    "/usr/local/share/fonts/DejaVuSansMono.ttf",
    "C:/Windows/Fonts/consola.ttf",
  ],
};

const pick = (list) => list.find((f) => fs.existsSync(f));

/**
 * Убираем символы, которых нет в DejaVu (эмодзи и пиктограммы)
 * на их месте появляется «пустой» глиф. Стрелки, математические
 * геометрические фигуры DejaVu знает – их оставляем.
 */
```

```

        logLevel: "silent",
        loader: { ".ts": "ts", ".tsx": "tsx", ".jpg": "data:
            ss": "empty" },
    });
    return import("data:text/javascript;base64," + Buffer
}

```

/* ————— снимки визуализаций —————

```

function shotsFor(chapterId) {
    const dirs = [SHOTS_DIR, path.join(ROOT, "public", "e
    for (const dir of dirs) {
        if (fs.existsSync(dir)) {
            const files = fs
                .readdirSync(dir)
                .filter((f) => /\.(png|jpe?g)$/i.test(f))
                .filter((f) => f === `${chapterId}.png` || f.st
                .sort();
            if (files.length > 0) {
                return files.map((f) => ({
                    file: path.join(dir, f),
                    name: f.replace(/\.(png|jpe?g)$/i, "").split(
                }));
            }
        }
    }
    return [];
}

```

/* ————— отрисовка PDF —————

```

class Renderer {
    constructor(doc, fonts) {
        this.doc = doc;
        this.fonts = fonts;
        this.footerTitle = "";
        this.pages = 0;
        this.cy = TOP;
    }
}

```

```

firstPage(footerTitle) {
  if (footerTitle !== undefined) this.footerTitle = footerTitle;
  this.doc.addPage({ size: [PAGE.width, PAGE.height],
    this.pages += 1;
    this.cy = TOP;
  }

  /** Сколько места нужно, чтобы элемент не оторвался от
  ensure(h, extra = 0) {
    if (this.cy + h + extra > BOTTOM) this.newPage();
  }

  height(text, { font, size, width, lineGap = 0 }) {
    return this.doc.font(font).fontSize(size).heightOfText(text, {
  }

  /**
   * Режет текст на фрагменты не выше max пунктов: сначала по
   * предложений, слишком длинное предложение – по словам.
   *
   * Без этого pdfkit сам переносит остаток на новую строку, что
   * создаётся в обход рендерера – без нижнего колонтитула.
   * из-за чего в файле оказывалось на одну страницу больше.
   */
  splitToFit(text, opts, max) {
    const measure = (t) => this.height(t, opts);
    const source = String(text);
    if (measure(source) <= max) return [source];
    const chunks = [];
    let current = "";
    const flush = () => {
      if (current.trim()) chunks.push(current.trim());
      current = "";
    };
    for (const sentence of source.split(/(?<=[.!?…;:])\s/)) {
      const withSentence = current ? `${current} ${sentence}` : sentence;
      if (measure(withSentence) <= max) {
        current = withSentence;
      } else {
        flush();
        current = sentence;
      }
    }
    flush();
    return chunks;
  }
}

```

```

    const text = plain(rawText);
    if (!text) return;
    const h = this.height(text, { font: this.fonts.bold });
    this.ensure(h + 6, 20);
    this.cy += 6;
    this.doc.font(this.fonts.bold).fontSize(size).fillColor(color);
    this.cy = Math.max(this.cy, this.doc.y) + 3;
}

h3(rawText, { size = 11.2, color = "#334155" } = {}) {
    const text = plain(rawText);
    if (!text) return;
    const h = this.height(text, { font: this.fonts.bold });
    this.ensure(h + 4, 16);
    this.cy += 4;
    this.doc.font(this.fonts.bold).fontSize(size).fillColor(color);
    this.cy = Math.max(this.cy, this.doc.y) + 2;
}

/** Абзац: режим по высоте страницы сами – автоперенос
para(rawText, { size = 10.0, indent = 0, color = "#1f2f4f" } = {}) {
    const text = plain(rawText);
    if (!text) return;
    const font = mono ? this.fonts.mono : italic && this.fonts.sans;
    const width = CONTENT_W - indent;
    const opts = { font, size, width, lineGap };
    const chunks = this.splitToFit(text, opts, BOTTOM - 10);
    chunks.forEach((chunk, index) => {
        const last = index === chunks.length - 1;
        const h = this.height(chunk, opts);
        this.ensure(h + (last ? gapAfter : 0), 6);
        const y0 = this.cy;
        this.doc.font(font).fontSize(size).fillColor(color);
        this.cy = Math.max(y0 + h, this.doc.y) + (last ? 0 : 10);
        if (this.cy > BOTTOM) this.newPage();
    });
}

```



```

    while (i < lines.length) {
      const candidate = chunk.concat(lines[i]).join(" ");
      const ch = this.height(candidate, { font, size, weight });
      if (chunk.length && ch > maxH) break;
      chunk.push(lines[i]);
      h = ch;
      i += 1;
    }
    const body = chunk.join("\n");
    const boxH = h + 14;
    this.ensure(boxH);
    const y0 = this.cy;
    this.doc.roundedRect(PAGE.margin, y0, CONTENT_W, boxH, 6);
    this.doc.font(font).fontSize(size).fillColor("#1f77b4");
    this.cy = y0 + boxH + (i < lines.length ? 2 : 7);
    if (i < lines.length) this.newPage();
  }
}

vectorGraph(nodes = [], edges = [], { height = 130, title }) {
  this.ensure(height + 20, 10);
  const startY = this.cy;
  const boxW = CONTENT_W;

  this.doc
    .roundedRect(PAGE.margin, startY, boxW, height, 6)
    .fillAndStroke("#f8fafc", "#cbd5e1");

  if (title) {
    this.doc
      .font(this.fonts.bold)
      .fontSize(9.5)
      .fillColor("#1e293b")
      .text(title, PAGE.margin + 12, startY + 8);
  }

  // Рёбра графа
  for (const e of edges) {

```

```

    const fillColor = n.status === "done" ? "#dcfce7"
    const strokeColor = n.status === "done" ? "#16a34a"

    this.doc
      .circle(nx, ny, r)
      .lineWidth(1.5)
      .fillAndStroke(fillColor, strokeColor);

    this.doc
      .font(this.fonts.bold)
      .fontSize(8.5)
      .fillColor("#0f172a")
      .text(String(n.label ?? n.id), nx - r, ny - 4,

    if (n.dist !== undefined) {
      this.doc
        .font(this.fonts.mono)
        .fontSize(7)
        .fillColor("#059669")
        .text(`d=${n.dist}`, nx - 20, ny + r + 2, { w
    }
  }

  this.cy = startY + height + 8;
}

/** Таблица: строки режутся по страницам, заголовок п
table(rawRows, { size = 8.5 } = {}) {
  const rows = rawRows.map((r) => r.map((c) => plain(
  if (!rows.length) return;
  const cols = Math.max(...rows.map((r) => r.length))
  const widths = new Array(cols).fill(CONTENT_W / col
  const pad = 5;
  const header = rows[0];

  const rowHeight = (cells) => {
    let max = 0;

```

```
quote(rawText) {
  const text = plain(rawText);
  if (!text) return;
  const font = this.fonts.oblique ?? this.fonts.regular;
  const width = CONTENT_W - 22;
  const opts = { font, size: 9.5, width, lineGap: 1 };
  // Длинная цитата режется по страницам заранее: рис
  // текст уйдёт за BOTTOM и pdftkit добавит страницу
  const chunks = this.splitToFit(text, opts, BOTTOM - 10);
  chunks.forEach((chunk) => {
    const h = this.height(chunk, opts);
    this.ensure(h + 10);
    const y0 = this.cy;
    this.doc.rect(PAGE.margin, y0, 3, h + 4).fill("#a6a6a6");
    this.doc.font(font).fontSize(9.5).fillColor("#374151");
    this.doc.text(chunk, PAGE.margin, y0, { width, height: h });
    this.cy = y0 + h + 10;
    if (this.cy > BOTTOM) this.newPage();
  });
}
```

```
image(file, { maxWidth = CONTENT_W, maxHeight = 320, caption }) {
  if (!fs.existsSync(file)) return false;
  let img;
  try {
    img = this.doc.openImage(file);
  } catch {
    return false;
  }
  const capH = caption ? 14 : 0;
  const scale = Math.min(maxWidth / img.width, (maxHeight - capH) / img.height);
  const w = img.width * scale;
  const h = img.height * scale;
  this.ensure(h + capH + 10, 20);
  const y0 = this.cy;
  const x0 = PAGE.margin + (CONTENT_W - w) / 2;
  this.doc.image(file, x0, y0, { width: w, height: h });
  this.doc.rect(x0, y0, w, h).lineWidth(0.6).strokeColor("black").stroke();
}
```

```

        y += this.height(l, { font: this.fonts.regular,
    }
    this.cy = y0 + boxH + 7;
    if (rest.length) this.newPage();
  }
}
}
}

```

/* _____ сборка _____

```

async function main() {
  const fonts = {
    regular: pick(CANDIDATES.regular),
    bold: pick(CANDIDATES.bold),
    oblique: pick(CANDIDATES.oblique),
    mono: pick(CANDIDATES.mono),
  };
  if (!fonts.regular) throw new Error("Не найден TTF-шрифт");
  fonts.bold = fonts.bold ?? fonts.regular;
  fonts.mono = fonts.mono ?? fonts.regular;

  const { chapters, chapterTopics, PAGE_SYNC, VIZ_REGISTER } = require('algvd');
  ensureDir(OUT_DIR);

  const doc = new PDFDocument({
    size: [PAGE.width, PAGE.height],
    margin: PAGE.margin,
    autoFirstPage: false,
    info: {
      Title: "Универсальное пособие – алгоритмы и структуры данных",
      Author: "algvd • автоэкспорт",
      Subject: "Текст всех тем + визуализации, без кода",
      CreationDate: new Date(),
    },
  });
  doc.pipe(stream);
  const stream = fs.createWriteStream(GUIDE_PDF_PATH);
  doc.pipe(stream);
}

```

```

chapters.forEach((c, i) => {
  const topics = chapterTopics[c.id] ?? [];
  R.para(
    `${String(i + 1).padStart(2, "0")}.  ${c.title}${
      : ""}` ,
    { size: 9.3, gapAfter: 2 }
  );
});

/* — темы — */
let shotsTotal = 0;
for (const chapter of chapters) {
  const topics = chapterTopics[chapter.id] ?? [];
  const shortTitle = chapter.title.replace(/^\\d+([.-])
  R.newPage(shortTitle);

  if (topics.length) {
    R.para(`Билет${topics.length > 1 ? "ы" : ""} ${to
      : 2 }));
  }
  R.h1(chapter.title.replace(/^\\d+([.-])?\\s*/ , ""), {

  const chapterTitle = chapter.title.replace(/^\\d+([.-]
  for (const b of htmlToBlocks(chapter.content)) {
    if (b.type === "p" && /^Билет\\s/i.test(b.text)) c
    if (b.type === "heading" && b.level <= 2 && b.tex

    if (b.type === "heading") {
      if (b.level <= 2) R.h2(b.text);
      else if (b.level === 3) R.h3(b.text);
      else R.para(b.text, { size: 10.2, color: "#3341
    } else if (b.type === "summary") {
      R.para("► " + b.text, { size: 9.8, color: "#373
    } else if (b.type === "li") {
      R.bullet(b.text, { ordinal: b.ordinal, depth: b
    } else if (b.type === "code") {
      R.code(b.text);
    } else if (b.type === "table") {

```

```

        lines.push("• Референсный код открывается в инт
        if (lines.length) R.box(title, lines);
    }
}

/* блиц: те же вопросы, что в приложении, но сразу
const quiz = (quizzes && quizzes[chapter.id]) || []
if (quiz.length) {
    R.h2("Блиц: вопросы по теме");
    quiz.forEach((q, qi) => {
        R.para(`${qi + 1}. ${q.question}`, { size: 9.6,
        q.options.forEach((option, oi) => {
            R.bullet(oi === q.correctIndex ? `${option} —
        });
        R.para(q.explanation, { size: 8.8, color: "#4b5
    });
}
}

R.stampFooter();
doc.end();
await new Promise((resolve, reject) => {
    stream.on("finish", resolve);
    stream.on("error", reject);
});

const size = fs.statSync(GUIDE_PDF_PATH).size;
console.log("[guide] PDF пособия собран");
console.log(`        тем: ${chapters.length} • снимко
console.log(`        выход: ${path.relative(ROOT, GUI

// Самопроверка: страниц в файле должно быть ровно ст
// рендерер. Расхождение означает, что pdftk создал
// в файл без нижнего колонтитула и без нумерации.
const pdfLib = await import("pdf-lib");
const physical = (await pdfLib.PDFDocument.load(fs.re
console.log(`        страниц: ${R.pages} (рендер) / $
if (R.pages > 100) console.warn(`        ВНИМАНИЕ: ${R

```

```
    .filter((f) => f.endsWith(".png"))
    .sort();

if (pages.length === 0) throw new Error("Не найдено н

ensureDir(OUT_DIR);

const doc = new PDFDocument({
  size: [A4.width, A4.height],
  margin: 0,
  autoFirstPage: false,
  info: {
    Title: "Универсальное пособие – полный исходный к
    Author: "CI: rebuild-pdf",
    Subject: "Экспорт всего кода проекта (код -> txt
    CreationDate: new Date(),
  },
});

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
  doc.addPage({ size: [A4.width, A4.height], margin: 0
  doc.image(path.join(PAGES_DIR, file), 0, 0, { width:
}

doc.end();
await new Promise((resolve, reject) => {
  stream.on("finish", resolve);
  stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`      страниц: ${pages.length}`);
console.log(`      выход:   ${path.relative(ROOT, PDF_P
}
}
```

```

-----
  mst: ["Краскал", "Прим", "Борувка"],
  treap: ["Собрать", "Split", "Merge"],
  "splay-tree": ["Обход", "Повороты", "Прогулка"],
};

const slug = (s) =>
  s
    .toLowerCase()
    .replace(/[^a-zA-яё0-9]+/gi, "-")
    .replace(/^-+|-+$/g, "")
    .slice(0, 24);

async function vizIds() {
  const bundle = await build({
    stdin: {
      contents: `export { activeVizIds } from "../../src/";`,
      resolveDir: path.join(ROOT, "scripts", "export"),
      loader: "ts",
    },
    bundle: true,
    write: false,
    format: "esm",
    platform: "node",
    logLevel: "warning",
  });
  const mod = await import("data:text/javascript;base64:" +
    btoa(bundle.code));
  return mod.activeVizIds;
}

fs.mkdirSync(OUT_DIR, { recursive: true });

const ids = await vizIds();
console.log(`[export:shots] страниц со визуализациями: ${ids.length}`);

const browser = await puppeteer.launch({
  headless: true,
  args: ["--no-sandbox", "--disable-setuid-sandbox", "--no-dev-shm-usage"],
});

```



```
    } catch (error) {
      failed += 1;
      console.log(`[export:shots] FAIL ${id}: ${String(error)}`);
    }
  }

const manifest = fs
  .readdirSync(OUT_DIR)
  .filter((f) => /\.(png|jpe?g)$/i.test(f))
  .sort()
  .map((f) => ({ file: f, name: f.replace(/\.(png|jpe?g)/, ".txt") }));
fs.writeFileSync(path.join(OUT_DIR, "manifest.json"), JSON.stringify(manifest));

await browser.close();
console.log(`[export:shots] снимков: ${done}, пропусков: ${failed}`);
// Пропущенные страницы не повод ронять пайплайн: PDF д
// текстовое описание демонстрации.
process.exit(0);
```

ФАЙЛ 95/111: scripts/export/collect-code.mjs
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМЕР: 1.2 КБ

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текстовый файл
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть)
 * с фоллбэком на обход файловой системы, если git недоступен
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,
```

```

    "CI / автоматизация",
    "Прочее",
];

function listFiles() {
    let files;
    try {
        files = execFileSync("git", ["ls-files", "--cached"], {
            cwd: ROOT,
            encoding: "utf8",
        })
            .split("\n")
            .filter(Boolean);
    } catch {
        files = [];
        const walk = (dir) => {
            for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
                const abs = path.join(dir, e.name);
                const rel = path.relative(ROOT, abs).split(path.sep);
                if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
                if (e.isDirectory()) walk(abs);
                else files.push(rel);
            }
        };
        walk(ROOT);
    }

    return files
        .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d + path.sep)))
        .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
        .filter((rel) => CODE_EXT.has(path.extname(rel)))
        .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
        .sort((a, b) => {
            const ia = ORDER.indexOf(categoryOf(a));
            const ib = ORDER.indexOf(categoryOf(b));
            return ia !== ib ? ia - ib : a.localeCompare(b);
        });
}

```

```

push();
push(`Дата генерации:                ${stamp}`);
push(`Файлов исходного кода:         ${files.length}`);
push(`Суммарный объём кода:           ${humanSize(totalByteSize)}`);
push(`Глав/билетов в пособии:        ${chapters.length}`);
push();

```

```

push(HR);
push("                                ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
push(HR);
chapters.forEach((c, i) => {
  push(`  ${String(i + 1).padStart(3, " ")}. ${c.title}`);
});
push();

```

```

push(HR);
push("                                ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
push(HR);
let current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push();
    push(`  # ${cat}`);
  }
  push(`  ${String(i + 1).padStart(3, " ")}. ${rel}`);
});
push();
push();

```

```

current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push(HR);
    push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
  }
});

```

```
* Общие настройки и утилиты пайплайна экспорта.
*
* код -> full_code_all_pages.txt      (collect-code.mjs)
* txt  -> page-XXXX.png               (render-pages.mjs)
* png  -> full_code_all_pages.pdf     (build-pdf.mjs)
*/
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
export const PAGE = {
  /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100 */
  density: Number(process.env.EXPORT_DENSITY ?? 150),
  /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_GRAYSCALE=1 */
  monochrome: process.env.EXPORT_GRAYSCALE !== "1",
  size: "A4",
  font: "DejaVu-Sans-Mono",
  fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
  pointsize: 8,
  /** Максимум символов в строке (далее – жёсткий перебор) */
  cols: 138,
  /** Строк содержимого на странице (+2 служебные: шапка, подвал) */
  rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
```

```
-----
* таблицы, код, спойлеры <details>. Служебная разметка
* svg, кнопки) отбрасывается, содержимое спойлеров – не
*
* Никаких внешних зависимостей: собственный мини-парсер
* куски хранятся отдельными узлами #text, поэтому инлайн
* (<b>, <code>, <span>) не «уезжает» в конец абзаца.
*/
```

```
/** Содержимое этих тегов не нужно в текстовой версии.
const SKIP = new Set([
  "svg", "script", "style", "button", "iframe", "video",
  "path", "circle", "rect", "line", "polyline", "polygon",
  "defs", "marker", "tspan", "ellipse", "use", "clipPath",
]);
```

```
/** Пустые элементы (без закрывающего тега). */
const VOID = new Set(["br", "hr", "img", "input", "meta",
```

```
/** Блочные теги: их содержимое собирается в отдельный блок.
const BLOCK = new Set([
  "h1", "h2", "h3", "h4", "h5", "h6", "p", "li", "pre",
  "summary", "figcaption", "table", "tr", "td", "th", "thead",
  "article", "ul", "ol", "details", "figure", "header",
]);
```

```
const HEADING = new Set(["h1", "h2", "h3", "h4", "h5",
```

```
/**
* Разбор HTML в дерево: { tag, attrs, children }, текст
* Намеренно простой: страницы пособия генерируются нами
*/
```

```
export function parseHtml(html) {
  const root = { tag: "#root", attrs: {}, children: [] };
  const stack = [root];
  const re = /<!--[\s\S]*?-->|<\/([a-zA-Z0-9]+)\s*>|<([a-zA-Z0-9]+)\s*/;
  let m;
  while ((m = re.exec(html)) !== null) {
    const [full, closeTag, openTag, attrsStr, selfClose] = m;
```

```

    .replace(/&#39;/g, "'")
    .replace(/&nbsp;/g, "\u00a0")
    .replace(/&mdash;/g, "—")
    .replace(/&ndash;/g, "—")
    .replace(/&laquo;/g, "«")
    .replace(/&raquo;/g, "»")
    .replace(/&hellip;/g, "...")
    .replace(/&times;/g, "×")
    .replace(/&le;/g, "≤")
    .replace(/&ge;/g, "≥")
    .replace(/&ne;/g, "≠")
    .replace(/&infin;/g, "∞")
    .replace(/&rarr;/g, "→")
    .replace(/&larr;/g, "←")
    .replace(/&amp;/g, "&")
    .replace(/&#(\d+)/g, (_, d) => String.fromCharCode

```

```

const clean = (s) => decodeEntities(s).replace(/\s+/g,

```

```

const isEl = (n) => n.tag !== "#text";

```

```

const isBlockEl = (n) => isEl(n) && BLOCK.has(n.tag) &&

```

```

/** Есть ли у узла блочные потомки (тогда его текст «ра

```

```

const hasBlockChild = (node) => node.children.some(isBl

```

```

/** Текст инлайнового поддерева в порядке документа. */

```

```

function inlineText(node) {
  let out = "";
  for (const c of node.children) {
    if (c.tag === "#text") {
      out += c.text;
    } else if (SKIP.has(c.tag)) {
      continue;
    } else if (BLOCK.has(c.tag)) {
      continue;
    } else if (c.tag === "br") {
      out += "\n";
    } else {

```

```
* Обход дерева: собираем блоки в порядке документа.
* @returns {Array<{type:string,text?:string,rows?:string[]}>}
*/
export function htmlToBlocks(html) {
  const root = parseHtml(html);
  const out = [];

  const push = (block) => {
    const prev = out[out.length - 1];
    if (prev && prev.type === block.type && prev.text === block.text) {
      out.push(block);
    }
  };

  const emitListItem = (li, ordinal, depth) => {
    const text = collectText(li);
    if (text) push({ type: "li", text, ordinal: ordinal });
    // вложенные списки внутри li
    li.children.forEach((c) => {
      if (c.tag === "ul" || c.tag === "ol") emitList(c, depth + 1);
    });
  };

  const emitList = (list, depth) => {
    let i = 0;
    for (const li of list.children) {
      if (li.tag !== "li") continue;
      i += 1;
      emitListItem(li, list.tag === "ol" ? i : null, depth);
    }
  };

  const walk = (node, listDepth = 0) => {
    if (node.tag !== "#root" && SKIP.has(node.tag)) return;

    if (node.tag === "table") {
      const rows = tableRows(node);
      if (rows.length) push({ type: "table", rows });
      return;
    }
  };
}
```

```

    const text = inlineText(node);
    if (!text) return;
    if (HEADING.has(node.tag)) push({ type: "heading",
    else if (node.tag === "li") emitListItem(node, null
    else if (node.tag === "summary") push({ type: "summ
    else if (node.tag === "blockquote") push({ type: "q
    else if (node.tag === "figcaption") push({ type: "c
    else push({ type: "p", text });
  };

  root.children.forEach((c) => walk(c, 0));
  return out;
}

/** То же самое, но простым текстом (для .txt-дампа и в
export function htmlToPlainText(html) {
  const lines = [];
  for (const b of htmlToBlocks(html)) {
    if (b.type === "heading") lines.push("", "#".repeat
    else if (b.type === "summary") lines.push("", "▶ "
    else if (b.type === "li") lines.push("  ".repeat(b
    else if (b.type === "code") lines.push("", ...b.tex
    else if (b.type === "table") {
      lines.push("");
      b.rows.forEach((r) => lines.push("| " + r.join("
      lines.push("");
    } else if (b.type === "quote") lines.push("> " + b.
    else if (b.type === "caption") lines.push("  " + b
    else lines.push(b.text, "");
  }
  return lines.join("\n").replace(/\n{3,}/g, "\n\n").tr
}

```



```

const source = txt.replace(/\r\n/g, "\n").split("\n");
const pages = [];
for (let i = 0; i < source.length; i += PAGE.rows) {
  pages.push(source.slice(i, i + PAGE.rows));
}
return pages;
}

```

```

async function main() {
  if (!fs.existsSync(TXT_PATH)) {
    throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}
  }

```

```

const im = imageMagickCmd();
const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

```

```

fs.rmSync(PAGES_DIR, { recursive: true, force: true });
ensureDir(PAGES_DIR);

```

```

const total = pages.length;
const jobs = pages.map((lines, idx) => {
  const num = String(idx + 1).padStart(4, "0");
  const header = `Универсальное пособие • полный исходный код
    Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
  const body = [header, "-".repeat(PAGE.cols), ...lines];
  const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
  const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
  fs.writeFileSync(txtFile, body + "\n", "utf8");
  return { txtFile, pngFile, num };
});

```

```

const args = (job) => [
  "-density", String(PAGE.density),
  "-page", PAGE.size,
  "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "Helvetica",
  "-pointsize", String(PAGE.pointsize),
  `text:${job.txtFile}`,
];

```

```
-----  
    console.error("Ошибка рендера страниц:", err.message)  
    process.exit(1);  
});  
-----
```

ФАЙЛ 99/111: scripts/verify/browser-checks.tsx

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 1065 | РА

```
-----  
/**  
 * DOM-харнес: настоящее приложение в jsdom + настоящий  
 * Монтирует App, кликает кнопки, печатает в редакторе  
 * визуализациям матрицы чужих размерностей и ловит пад  
 */  
import { act } from "react";  
import { createRoot } from "react-dom/client";  
import type { Root } from "react-dom/client";  
import App from "../../src/App";  
import { chapters, chapterTopics, SECTIONS, sectionOf }  
import { quizzes } from "../../src/data/quizzes";  
import { VIZ_REGISTRY } from "../../src/components/vizR  
import { VizLiveStatus } from "../../src/components/Viz  
import { PAGE_SYNC, buildInitTemplate } from "../../src  
import { VizChapterContext, emitVizState, clearVizState  
import { buildDebugRunner, type DebugResult } from "../  
  
type Py = { runPythonAsync: (src: string) => Promise<un  
  
export interface TestResult {  
  group: string;  
  name: string;  
  ok: boolean;  
  ms: number;  
  detail: string;  
  consoleErrors: string[];  
}  
-----
```

```

}

function findButton(match: string | RegExp): HTMLButtonElement {
  const list = Array.from(document.querySelectorAll("button"));
  return list.find((b) => {
    const hay = [b.getAttribute("aria-label"), b.getAttribute("value")];
    return typeof match === "string" ? hay.includes(match) : match.test(hay);
  });
}

async function click(el: Element | undefined | null, label: string) {
  if (!el) throw new Error(`не найдена ${label}`);
  await act(async () => {
    el.dispatchEvent(new window.MouseEvent("click", { bubbles: true }));
    await sleep(20);
  });
}

/** Панель компилятора – все поиски внутри неё, чтобы не искать по всему документу */
function panel(): HTMLElement | null {
  return document.querySelector<HTMLElement>('aside[aria-label="Compiler Panel"]');
}

function panelText(): string {
  return (panel()?.textContent ?? "").replace(/\s+/g, " ");
}

function textarea(): HTMLTextAreaElement | undefined {
  return panel()?.querySelector<HTMLTextAreaElement>("text-area");
}

function typeInto(ta: HTMLTextAreaElement, value: string) {
  const setter = Object.getOwnPropertyDescriptor(window, "document").value;
  if (!setter) throw new Error("нет нативного setter'a для document");
  act(() => {
    setter.call(ta, value);
    ta.dispatchEvent(new window.Event("input", { bubbles: true }));
  });
}

```

```

async function mountApp(topic: string) {
  if (root) {
    await act(async () => {
      root?.unmount();
    });
    root = null;
  }
  if (host) {
    host.remove();
  }
  host = freshHost();
  window.history.replaceState({}, "", `/?topic=${encodeURIComponent(topic)}`);
  root = createRoot(host);
  await act(async () => {
    root!.render(<App />);
  });
  await settle(80);
  return host;
}

```

/** Смонтировать одну визуализацию так, как это делает

```

async function mountViz(vizId: string) {
  if (root) {
    await act(async () => {
      root?.unmount();
    });
    root = null;
  }
  if (host) host.remove();
  host = freshHost();
  const entry = VIZ_REGISTRY[vizId];
  if (!entry) throw new Error(`в реестре нет визуализации с id ${vizId}`);
  const Comp = entry.Component;
  root = createRoot(host);
  await act(async () => {
    root!.render(
      <VizChapterContext.Provider value={vizId}>
        <VizLiveStatus demoTitle={entry.title} />
      </VizChapterContext.Provider>
    );
  });
  await settle(80);
  return host;
}

```

```

    consoleErrors = [];
    const t0 = Date.now();
    let ok = true;
    let detail = "";
    try {
        const d = await fn();
        detail = d ?? "";
    } catch (error) {
        ok = false;
        detail = error instanceof Error ? `${error.message}` :
    }
    const ms = Date.now() - t0;
    const errs = consoleErrors.filter((e) => !/not wrapped/.test(e));
    if (errs.length > 0) {
        ok = false;
        detail = `${detail}\nconsole.error (${errs.length})
    }
    results.push({ group, name, ok, ms, detail, consoleError });
    const mark = ok ? "PASS" : "FAIL";
    realConsoleError === console.error;
    process.stdout.write(`${mark} [${group}] ${name}${ok ? "" : " failed"}\n`);
    if (!ok) process.stdout.write(detail.split("\n").map((line) => `  ${line}\n`));
    return ok;
}

/* -----
/*  сценарии
/*  -----

/** Фильтры прогона: HAR_CHAPTERS=segment-trees,floyd и
const onlyChapters = (process.env.HAR_CHAPTERS || "").split(",");
const skipGroups = new Set((process.env.HAR_SKIP || "").split(","));
/** HAR_SYNC=floyd,dijkstra – прогнать только эти записи
const onlySync = new Set((process.env.HAR_SYNC || "").split(","));

export async function runSuite({ py }: { py: Py }): Promise<void> {
    const chapterList = onlyChapters.length ? chapters.filter((c) => onlyChapters.includes(c)) : chapters;
    const want = (group: string) => !skipGroups.has(group) && (onlySync.size ? onlySync.has(group) : true);
    const results = [];
    for (const group of chapterList) {
        const result = await runTest(group);
        results.push(result);
    }
    console.log("Results:");
    for (const result of results) {
        console.log(result);
    }
}

```

```

const done = await waitFor(() => RUN_DONE.some((m) =>
  if (!done) throw new Error(`запуск не завершился`);
const msg = RUN_DONE.find((m) => bodyText().includes(m));
const sync = PAGE_SYNC[chapter.id];
if (!sync?.code) return `кода синхронизации нет`;
if (/Ошибка|Не удалось/.test(msg)) throw new Error(msg);
// после настоящего прогона визуализация обязана быть
const linked = await waitFor(() => /данные из коммента/);
if (!linked) return `${msg}; визуализация осталась прежней`;
return `${msg} → визуализация переключилась на данные`;
});
}

```

/* — 2. Флойд: матрицы ЛЮБОЙ размерности из кода полнотелого

```

const floydCases: { name: string; vars: Record<string, any> } = [
  { name: "5×5 (меньше демо-7×7)", vars: { n: 5, k: 1, i: 0, j: 0,
    [9, 6, 9, 9, 0] } }, expect: "5×5" },
  { name: "10×10 (больше демо – старый краш nodeNames)", vars: {
    from({ length: 10 }, (_, c) => (r === c ? 0 : (r + c) % 10)) },
    expect: "10×10" },
  { name: "3×3 с бесконечностями", vars: { n: 3, k: 0, i: 0, j: 0,
    [0, 1, 2] }, expect: "3×3" },
  { name: "не квадратная 4×6", vars: { n: 4, k: 1, i: 0, j: 0,
    [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 0, 20] } },
    expect: "Демо-схема графа" },
  { name: "словарь словарей d[0][1]", vars: { n: 4, k: 0, i: 0, j: 0,
    2: { 0: 9, 1: 9, 2: 0, 3: 1 }, 3: { 0: 3, 1: 9, 2: 0, 3: 0 } },
    expect: "словарь словарей" },
  { name: "ключи-пары \"(i, j)\"", vars: { n: 3, k: 0, i: 0, j: 0,
    "(1, 2)": 2, "(2, 0)": 1, "(2, 1)": 9, "(2, 2)": 0 },
    expect: "ключи-пары" },
  { name: "плоский список – это НЕ матрица", vars: { n: 0, k: 0, i: 0, j: 0,
    [0] }, expect: "плоский список" },
  { name: "80×80 (обрезка до 64)", vars: { n: 80, k: 0, i: 0, j: 0,
    (_, c) => (r === c ? 0 : 1 + ((r + c) % 50))) },
    expect: "80×80" },
  { name: "пустая матрица", vars: { n: 0, k: 0, i: 0, j: 0,
    [0] }, expect: "пустая матрица" },
  { name: "матрица под именем D и k/i/j вне диапазона", vars: {
    [10, 11, 12, 0] }, expect: "4×4" },
];

```

```

for (const c of floydCases) {
  await test("В. Флойд: размерности", c.name, async () => {
    await mountViz("floyd");
  });
}

```

```

        if (!c.expect.test(txt)) throw new Error(`ожидали
const defects = domDefects(host);
if (defects.length) throw new Error(`артефакты в
return /размерность не совпадает/.test(txt) ? "пр
    });
}

await test("С. матричные страницы", "2D-построение: o
await mountViz("sparse-table");
const A8 = Array.from({ length: 8 }, (_, i) => Array
await act(async () => {
    emitVizState("sparse-table", { line: 7, func: "<m
    });
    await sleep(40);
});
const grids = () => Array.from((host ?? document).q
let g0 = grids()[0];
if (!g0) throw new Error("нет ни одной матрицы блок
const firstCell = () => (g0.children[0]?.textContent
// «Оригинал (1x1)» = сама матрица A: значения видн
if (firstCell() !== "1") throw new Error(`оригинал
if ((g0.children[7]?.textContent ?? "").trim() !==
// шаги кликаются даже под live-k из компилятора
await click(findButton("Шаг 2 (4x4)"), "вкладка шаг
const txt = (host?.textContent ?? "").replace(/\s+/
if (!/Матрица блоков 4x4/.test(txt)) throw new Erro
g0 = grids()[0];
const cls = (g0.children[0] as HTMLElement)?.classN
// пустая активная клетка (код смотрит на (0,0), st
if (!/border-dashed/.test(cls) || !/indigo/.test(cl
    throw new Error(`пустая активная клетка нарисован
}
if (domDefects(host).length) throw new Error(`артеф
return "оригинал из A с первого шага, вкладка «Шаг
});

/* — 4. Дейкстра / Форд-Беллман: чужие имена вершин
for (const [viz, vars] of [

```

```

const seen = new Set<string>();
for (const s of result.steps) for (const n of Object.keys(result.finalValues ?? {}))
for (const n of Object.keys(result.finalValues ?? {}))
// в документации имя может быть видом «хк / х»,
// проверяем, что хотя бы один python-идентификатор
const declared = sync.variables.map({v} => v.name);
const ident = (t: string) => /^[A-Za-z_][A-Za-z0-9_]*$/;
const missing = declared
    .map({raw} => raw.split(/[/[\],\s+или\s+\/]/).map(s => s.trim()).filter(s => s.length))
    .filter({alts} => alts.some(ident))
    .filter({alts} => !alts.some(a => ident(a) && !seen.add(a)))
    .map({alts} => alts.join(" / "));
if (missing.length) throw new Error(`код не создан`);

```

```

if (!VIZ_REGISTRY[chapterId]) return `шагов ${result.steps.length}`;
await mountViz(chapterId);
const finalVars = result.finalValues ?? result.steps[result.steps.length - 1].finalValues;
await act(async () => {
    emitVizState(chapterId, { line: result.steps[result.steps.length - 1].line,
        ed: Object.keys(finalVars) });
    await sleep(40);
});
const txt = (host?.textContent ?? "").replace(/[/[\],\s+или\s+\/]/g, " ");
if (!/данные из компилятора/.test(txt)) throw new Error(`некорректный код`);
const defects = domDefects(host);
if (defects.length) throw new Error(`артефакты в коде`);
return `шагов ${result.steps.length}${result.steps[result.steps.length - 1].finalValues ? " переменных " : ""} `;

```

```

});

```

```

}

```

/* — 6. Поведение редактора: пустой код, ошибки, гла...

```

const ch = "dijkstra";

```

```

await test("F. редактор", "только комментарии: редактор", async () => {
    await mountApp(ch);
    await click(findButton("Боковая панель Python"), "п...

```



```

    if (domDefects(host).length) throw new Error(`артефакты: ${domDefects(host).length}
    return `вывод обрезан до ${out.length} знаков, хвост: ${out.length - 1}
  });

```

```

await test("F. редактор", "полная очистка: ничего не осталось", async (ch) => {
  await mountApp(ch);
  await click(findButton("Боковая панель Python"), "python");
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(ta, "");
  const done = await waitFor(() => bodyText().include("очистка"));
  if (!done) throw new Error(`на пустой редактор панель не исчезла`);
  if (!textarea()) throw new Error("редактор исчез после очистки");
  if (!/встроенное демо/.test(bodyText())) throw new Error("неверно настроено демо");
  if (allButtons().length < 5) throw new Error(`кнопок: ${allButtons().length}, ожидалось 5`);
  return `пустой код: кнопок ${allButtons().length}, ожидалось 5`;
});

```

```

await test("F. редактор", "глаз: эталон попадает в редактор", async (ch) => {
  await mountApp(ch);
  await click(findButton("Боковая панель Python"), "python");
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  const init = ta.value;
  const editorButtons = allButtons();
  const ref = PAGE_SYNC[ch]?.code ?? "";
  if (!ref) throw new Error("у страницы нет эталонного кода");
  const done0 = await waitFor(() => RUN_DONE.some((m) => m === "глаз"));
  if (!done0) throw new Error(`инициализация не выполнена`);

```

```

// — глаз ВКЛ: эталон лежит В РЕДАКТОРЕ, и редактор не пуст
await click(findButton("Вставить эталонный код"), "python");
const trace = await waitFor(() => !!findButton(/Остаток/));
if (!trace) throw new Error(`эталон не запустился. ${findButton(/Остаток/).length} кнопок`);
if (!textarea()) throw new Error("редактор исчез во время запуска");
if (/Ошибка:/.test(panelText())) throw new Error(`эталон не вставлен`);
const ta2 = textarea();
const firstRefLine = ref.split("\n").find((l) => l.startsWith("// —"));

```

```

    if (!back) throw new Error(`\u0438 не вернул инициализацию`);
    return "эталон вставляется в редактор, ведёт демонстрацию";
  });

```

```

await test("F. редактор", "код не перетекает между страницами");
await mountApp(ch);
await click(findButton("Боковая панель Python"), "показать панель");
const ta = textarea();
if (!ta) throw new Error("нет редактора");
typeInto(ta, "# MY-MARKER-14\nx = 41\nx += 1\nprint x\n");
await waitFor(() => RUN_DONE.some((m) => bodyText().includes(m)));
if (!bodyText().includes("42")) throw new Error("свернуть не удалось");
// переходим на другую страницу, не закрывая панель
await mountApp("floyd");
await click(findButton("Боковая панель Python"), "показать панель");
const ta2 = textarea();
if (!ta2) throw new Error("на новой странице нет редактора");
if (ta2.value.includes("MY-MARKER-14")) throw new Error("код не перетекает");
if (!ta2.value.includes("# «")) throw new Error("на новой странице нет редактора");
await mountApp(ch);
await click(findButton("Боковая панель Python"), "показать панель");
const ta3 = textarea();
if (!ta3?.value.includes("MY-MARKER-14")) throw new Error("код не перетекает");
return "у каждой страницы свой код, возврат сохраняется";
});

```

```

await test("F. редактор", "редактор живёт и во время прогона трассы");
await mountApp(ch);
await click(findButton("Боковая панель Python"), "показать панель");
const done = await waitFor(() => bodyText().includes("трасса собралась"));
if (!done) throw new Error(`трасса не собралась. Панель не появилась`);
if (!textarea()) throw new Error("во время прогона редактор не появился");
if (!findButton("/Остановить прогон трассы/")) throw new Error("кнопка не появилась");
// Листинг сворачивается (освобождая место редактору)
// и редактор остаётся на месте.
const codeLines = () => panel()?.querySelectorAll("pre");
if (codeLines() === 0) throw new Error("в раскрытой панели нет кода");
await click(findButton("Свернуть листинг трассы"), "свернуть");

```

```

const listing = await waitFor(() => !!findButton(/0
if (!listing) throw new Error("панель не показала тр
const namelessListing = namelessButtons(panel() ??
const listingBtns = allButtons().length;
if (namelessEditor.length) throw new Error(`в редак
    , ")}`);
if (namelessListing.length) throw new Error(`в листин
    (" , ")}`);
const required = [/Остановить прогон трассы/, /Сверн
    начало трассы/, /В конец трассы/, /автопроход/i]
const btns = allButtons().join(" | ");
const missing = required.filter((re) => !re.test(btns)
if (missing.length) throw new Error(`в листинге нет
return `редактор: ${editorBtns} кнопок, листинг: ${
});

```

```

await test("Н. доступность", "бейдж честно различает
    await mountApp(ch);
// 1) демо: компилятор ещё ничего не присылал
if (!/встроенное демо/.test(bodyText())) throw new Error
if (/переменных пока нет/.test(bodyText())) throw new Error
await click(findButton("Боковая панель Python"), "панель")
const listing = await waitFor(() => !!findButton(/0
if (!listing) throw new Error("панель не показала тр
// 2) код выполнен, отладчик на первой строке: переменных пока нет
const awaiting = await waitFor(() => /переменных пока нет/
if (!awaiting) throw new Error(`после запуска бейджа не
    `${bodyText().match(/встроенное демо|данные из компилятора/
// 3) дошли до конца трассы – картинка обязана жить
await click(findButton("В конец трассы"), "в конец трассы")
const linked = await waitFor(() => /данные из компилятора/
if (!linked) throw new Error("в конце трассы визуализация не
// 4) сняли связь (Esc + пустой код) – снова демо
await key("Escape");
await settle(80);
const ta = textarea();
if (!ta) throw new Error("редактор не вернулся по expected
typeInto(ta, "");

```

```

    if (!el) throw new Error(`в шапке нет кнопки «${w}»`);
    const href = el.getAttribute("href") ?? "";
    if (!href.endsWith(w.file)) throw new Error(`«${w}» не в файле`);
    const name = [el.getAttribute("aria-label"), el.getAttribute("title")].filter(Boolean).join(" ");
    if (name.trim().length < 5) throw new Error(`у кнопки нет названия`);
  }
  if (!menu.querySelector("#download-html-btn")) throw new Error("нет кнопки загрузки HTML");
  const py = document.querySelector<HTMLButtonElement>("#download-python");
  if (!py) throw new Error("нет кнопки панели Python");
  if (!py.getAttribute("aria-label")) throw new Error("у кнопки нет aria-label");
  if (!py.getAttribute("title")) throw new Error("у кнопки нет title");
  return "4 загрузки на месте, href ведут на файлы, кнопки работают";
});

```

```

await test("I. ссылки", "упоминание «билет 5» кликабельно", async () => {
  await mountApp("sparse-table");
  const links = Array.from(document.querySelectorAll("a"));
  if (links.length === 0) throw new Error("на странице нет ссылок");
  const toPrefix = links.find((a) => a.dataset.goto === "5");
  if (!toPrefix) throw new Error("нет ссылки на билет 5");
  if (!/билет 5/i.test(toPrefix.textContent ?? "")) throw new Error("ссылка не содержит «билет 5»");
  const href = toPrefix.getAttribute("href") ?? "";
  if (!href.includes("topic=prefix-sums-2d")) throw new Error("ссылка не содержит «topic=prefix-sums-2d»");
  await click(toPrefix, "ссылка на билет 5");
  await settle(120);
  const txt = bodyText();
  if (!/Двумерная матрица префиксных сумм/.test(txt))
    throw new Error(`клик по «билет 5» не открыл страницу`);
  return `ссылка на страницу: ${links.length}, переход работает`;
});

```

```

await test("I. ссылки", "содержание: разделы по порядку", async () => {
  await mountApp("bridges-code");
  const sidebar = document.querySelector("#sidebar-content");
  if (!sidebar) throw new Error("нет сайдбара с содержанием");
  const headers = Array.from(sidebar.querySelectorAll("h2, h3"));
  if (headers.length !== 4) throw new Error("содержание не соответствует");
  return "содержание соответствует";
});

```

```

await test("I. ссылки", "введение: базовая база, обзор");
await mountApp("intro");
const txt = bodyText();
for (const must of ["Дерево как массив", "Куча", "Обзор"])
  if (!txt.includes(must)) throw new Error(`во введении не найдено: ${must}`);
if (!txt.includes("2n") || !txt.includes("2n+1")) throw new Error("не найдено: 2n или 2n+1");
if (!txt.includes("≤ 1")) throw new Error("не найдено: ≤ 1");
if (/Обзор: Кратчайшие пути/.test(txt)) throw new Error("неверно: кратчайшие пути");
const goto = Array.from(document.querySelectorAll("a[href]"));
if (!goto.includes("dijkstra")) throw new Error("не найдено: дикий");
if (domDefects(document.querySelector("#chapter-viz")))
  throw new Error("артефакты в DOM введения");
return "введение = куча/очередь/бинпоиск/сортировка";
});

```

```

await test("I. блиц", "вопросы отвечают, объясняют и");
await mountApp("sparse-table");
const list = quizzes["sparse-table"] ?? [];
if (list.length === 0) throw new Error("у билета 2 вопроса");
const section = Array.from(document.querySelectorAll("div"));
if (!section) throw new Error("блиц не отрисован на странице");
const ask = (label: string | RegExp) =>
  Array.from(section.querySelectorAll("button")).find(
    b => b.getAttribute("aria-label").includes(label)
  );
const hay = [b.getAttribute("aria-label"), b.textContent];
return typeof label === "string" ? hay.includes(label) : false;
});
const shown = () => (section.textContent ?? "").includes(list[0].question.slice(0, 24));
if (!shown()) throw new Error(`первый вопрос не показан: ${list[0].question}`);
const wrong = list[0].options.findIndex((_, i) => i !== 0);
await click(ask(list[0].options[wrong].slice(0, 20)));
await settle(80);
if (!/Неверно/.test(shown())) throw new Error("на странице не показан ответ");

```

```

    }
    await mountApp("sparse-table");
    const full = bodyText();
    if (!/100 (коржей|копий|матриц)|сто (коржей|матриц)/.test(full))
        throw new Error("на странице билета 2 не объяснено");
    }
    if (!/не конкуренты|разные задачи/i.test(full)) {
        throw new Error("на странице билета 2 не сказано, что");
    }
    // аналогии: торт стоит РЯДОМ с вафельными трубочками
    for (const topic of ["sparse-table", "prefix-sums-2"])
        await mountApp(topic);
        const full = bodyText();
        if (!/Аналогия 1: Сетка из вафельных трубочек/.test(full))
            throw new Error(`на «${topic}» не объяснено, что`);
        if (!/Аналогия 2: Торт-пирамидка/.test(full))
            throw new Error(`на «${topic}» не объяснено, что`);
        if (!/крем обратно/.test(full))
            throw new Error(`на «${topic}» не объяснено, что`);
        if (!/трубочки можно не только докладывать, но и вынимать/.test(full))
            throw new Error(`на «${topic}» не объяснено, что`);
        if (!/время: построение  $O(N \cdot M \cdot \log N \cdot \log M)$ , запрос суммы  $O(1)$ /.test(full))
            throw new Error(`на «${topic}» у карточки торта`);
        if (!/память:  $N \times M \times \log N \times \log M$ /.test(full))
            throw new Error(`на «${topic}» у карточки торта`);
        if (!/время: построение  $O(N \cdot M)$ , запрос суммы  $O(1)$ /.test(full))
            throw new Error(`на «${topic}» у карточки трубочки`);
        if (!/память:  $N \times M$  — как оригинал, тонкая/.test(full))
            throw new Error(`на «${topic}» у карточки трубочки`);
    }
    return "билет 2:  $N \times M \times \log N \times \log M$  против  $N \times M$ ; билет 5: ";
});

```

/* — 10. Сквозной путь: свой код чужой размерности, Floyd 4x4

```

await test("К. свой код", "Флойд 4x4 из редактора вед");
await mountApp("floyd");
await click(findButton("Боковая панель Python"), "п");

```

```

const ta = textarea();
if (!ta) throw new Error("нет редактора");
typeInto(
  ta,
  [
    "a = [5, 1, 9, 3, 7, 2]",
    "n = len(a)",
    "LOG = 3",
    "i, j = 0, 0",
    "st = []",
    "for row in a:",
    "    st.append([row] + [None] * (LOG - 1))",
    "for j in range(1, LOG):",
    "    for i in range(n - (1 << j) + 1):",
    "        st[i][j] = min(st[i][j - 1], st[i + (1 << j) - 1])",
    "print(n, st[0][2], st[1][1])",
  ].join("\n")
);
const ready = await waitFor(() => bodyText().includes("Готово"));
if (!ready) throw new Error("своя таблица не построена");
if (!/6 1 1/.test(panelText())) throw new Error("отсутствует таблица");
await click(findButton("В конец трассы"), "в конец трассы");
const linked = await waitFor(() => /данные из компилятора/);
if (!linked) throw new Error("демонстрация не принята");
const defects = domDefects(document.querySelector("#defects"));
if (defects.length) throw new Error("артефакты в демонстрации");
return "свой массив из 6 чисел: st[0][2] = 1, st[1][1] = 1";
});

await test("К. свой код", "Косарайю: эталон сам считает");
await mountApp("scc-kosaraju");
await click(findButton("Боковая панель Python"), "панель Python");
if (!textarea()) throw new Error("нет редактора");
await click(findButton("Вставить эталонный код"), "вставить код");
const ready = await waitFor(() => bodyText().includes("Готово"));
if (!ready) throw new Error("эталон Косарайю не выполнен");
if (/Ошибка:/.test(panelText())) throw new Error("эталон не выполнен");
const out = panelText();

```

```

const unknown = liteLinks.filter((a) => !chapters.s
if (unknown.length) throw new Error(`в lite-версии
if (!/Блиц: вопросы по теме/.test(txt)) throw new E
if (!/Верно:/.test(txt)) throw new Error("в lite-ве
return `${txt.length} знаков, ссылок в lite ${liteL
});

await test("G. lite-режим", "HTML-книга: оглавление,
const { buildBookHtml } = await import("../src/u
const html = await buildBookHtml(chapters);
if (!html.includes('<nav id="toc"')) throw new Erro
const missing = chapters.filter((c) => !html.includ
if (missing.length) throw new Error(`в книге потеря
const tocLinks = (html.match(/href="#ch-/g) ?? []).
if (tocLinks < chapters.length) throw new Error(`сс
if (!/Словарик терминов пособия/.test(html)) throw
if (!/↑ к оглавлению/.test(html)) throw new Error("
if (html.length < 200_000) throw new Error(`книга п
return `${(html.length / 1024).toFixed(0)} КБ, глав
});

console.error = realConsoleError;
return results;
}

/** Сводка для оркестратора. */
export function summary() {
  const byGroup = new Map<string, { ok: number; fail: n
  for (const r of results) {
    const g = byGroup.get(r.group) ?? { ok: 0, fail: 0
    g[r.ok ? "ok" : "fail"]++;
    byGroup.set(r.group, g);
  }
  return { total: results.length, passed: results.filte
    .fromEntries(byGroup) };
}

/** Инициализация страницы из vizSync – нужна для прове

```



```

    'export { chapters, chapterTopics, SECTIONS, sectionOf, GLOSSARY } from "../src/data/glossary"
    'export { VIZ_REGISTRY } from "../src/components"
    'export { PAGE_SYNC } from "../src/data/vizSync"
    'export { quizzes } from "../src/data/quizzes";
    'export { MINI_DEMO_KINDS } from "../src/components"
  ].join("\n")
);
await esbuild.build({
  entryPoints: ["tmp/verify/links-entry.ts"],
  bundle: true,
  outfile: OUT,
  format: "esm",
  platform: "browser",
  target: "es2022",
  jsx: "automatic",
  logLevel: "error",
  define: {
    "process.env.NODE_ENV": '"development"',
    "import.meta.env.BASE_URL": '"/'',
    "import.meta.env.MODE": '"development"',
    "import.meta.env.DEV": "true",
    "import.meta.env.PROD": "false",
    "import.meta.env.SSR": "false",
  },
  loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },
});
const { chapters, chapterTopics, SECTIONS, sectionOf, GLOSSARY } = await import(`_${process.cwd()}/${OUT}?run=${Date.now()}`);

const chapterIds = new Set(chapters.map((c) => c.id));
const registryIds = Object.keys(VIZ_REGISTRY);

/** Демо без собственной страницы: открываются из подсказки */
const EXTRA_DEMOS = new Set(["stack-dfs", "queue-bfs", "graph-bfs", "graph-dfs"]);

/* 1. билеты 1-24 */
const covered = new Set();

```

```

const withoutLinks = [];
for (const chapter of chapters) {
  const html = chapter.content ?? "";
  let mentionsHere = 0;

  // 3. упоминания «билет N» — только внутри текста, не
  const textOnly = html
    .replace(/<pre[\s\S]*?</pre>/gi, " ")
    .replace(/<code[\s\S]*?</code>/gi, " ")
    .replace(/<[^>]*>/g, " ");
  for (const m of textOnly.matchAll(TICKET_RE)) {
    const from = Number(m[1]);
    const to = m[2] ? Number(m[2]) : from;
    mentionsHere += 1;
    for (let n = from; n <= to; n += 1) {
      ok(n >= 1 && n <= 24 && covered.has(n), `«${chapter
    }
  }
  ticketMentions += mentionsHere;

  // 4. явные переходы
  for (const m of html.matchAll(/data-goto="([^\"]+)"/g))
    explicitLinks += 1;
    ok(chapterIds.has(m[1]), `«${chapter.id}»: data-goto
  }
  for (const m of html.matchAll(/href="\?(?:lite=1&)?to
    explicitLinks += 1;
    ok(chapterIds.has(decodeURIComponent(m[1])), `«${ch
  }

  // 5. якоря
  for (const m of html.matchAll(/href="#([^\"]+)"/g)) {
    ok(html.includes(`id="${m[1]}"`), `«${chapter.id}»:
  }

  if (mentionsHere === 0 && !/data-goto=/.test(html)) w
}

```

```
-----
for (const id of withoutLinks) NOTES.push(`нет перекрёс
const noQuiz = chapters.filter((c) => !(quizzes[c.id] ?

console.log("----- ссылки и структура содержания —
console.log(` страниц: ${chapters.length} • разделов:
console.log(` упоминаний «билет N» в тексте: ${ticketM
console.log(` визуализаций в реестре: ${registryIds.le
  GLOSSARY.length}`);
console.log(` блиц есть у ${Object.keys(quizzes).length
if (NOTES.length) console.log(` заметки:\n    ${NOTES.
if (noQuiz.length) console.log(` без блица: ${noQuiz.j

if (FAILS.length) {
  console.log(`\nПРОВАЛЕНО ${FAILS.length}:`);
  FAILS.slice(0, 40).forEach((f) => console.log(` x ${f
  process.exit(1);
}
console.log("\nвсе ссылки целы: 0 битых переходов");
```

```
-----
ФАЙЛ 101/111: scripts/verify/check-reference-code.mjs
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 133 | РАЗМ
```

```
-----
/**
 * Контракт «эталонный код ↔ страница ↔ визуализация».
 *
 * npm run verify:code
 *
 * Каждый референсный Python-код из PAGE_SYNC выполняет
 * (pyodide из node_modules) под тем же трейсером, что
 * проверяется:
 *   1. код не падает и не обрывается по лимиту шагов;
 *   2. трасса не пустая (иначе панель показала бы пуст
 *   3. каждая переменная, обещанная странице, реально
 *      иначе визуализация не может за ним следовать;
 *   4. в снимках нет «мусорных» значений, которые урон
```

```

-----
py.setStderr({ batched: (m) => printed.push("[stderr] "

const only = process.argv.slice(2);
const ident = (t) => /^[A-Za-z_][A-Za-z0-9_]*$/.test(t)
/** «xk / x», «dist[v]», «(i-1)//2», «π» → набор python
const alts = (raw) => raw.split(/[\/[\],]|\\s+или\\s+/).m

const rows = [];
let problems = 0;
for (const [key, entry] of Object.entries(mod.sync)) {
  if (only.length && !only.some((o) => key.includes(o)))
    printed.length = 0;
  let result;
  try {
    result = JSON.parse(String(await py.runPythonAsync(
  } catch (error) {
    problems++;
    rows.push({ key, steps: 0, status: `СБОЙ ЗАПУСКА: $
    continue;
  }
  const seen = new Set();
  for (const step of result.steps) for (const name of 0)
  for (const name of Object.keys(result.finalValues ??

const issues = [];
if (result.error) issues.push(`ошибка Python: ${result
if (result.truncated) issues.push("трасса оборвана по
if (result.steps.length === 0) issues.push("нет ни од
const missing = (mod.declared[key] ?? [])
  .map(alts)
  .filter((a) => a.some(ident))
  .filter((a) => !a.some((x) => ident(x) && seen.has(
  .map((a) => a.join(" / "));
if (missing.length) issues.push(`код не создаёт обеща
/*
* Обещанная переменная не должна быть пустой на ВСЕХ
* объявил и не наполнил – визуализации нечего показы
* «зашитый» результат вместо работы алгоритма. Именн

```

```

===== "ок" ? "" : r.status}`});
}
console.log("=".repeat(96));
console.log(`всего: ${rows.length} | с проблемами: ${problems}`);
writeFileSync(`${OUT_DIR}/reference-code.json`, JSON.stringify(rows));
process.exit(problems > 0 ? 1 : 0);

```

ФАЙЛ 102/111: scripts/verify/dom-env.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 107 | РАЗМЕР: 1.5 КБ

```

/**
 * Браузерное окружение для проверок: jsdom + те API, которые есть в браузере
 * Создается ПОСЛЕ загрузки Pyodide (иначе pyodide решит, что это не браузер)
 */
import { JSDOM, VirtualConsole } from "jsdom";

const OBSERVER_STUB = class {
  observe() {}
  unobserve() {}
  disconnect() {}
  takeRecords() {
    return [];
  }
};

export function createDom({ url = "http://localhost/?to=reference-code" }) {
  const virtualConsole = new VirtualConsole();
  const jsdomErrors = [];
  virtualConsole.on("jsdomError", (error) => {
    jsdomErrors.push(error?.message || String(error));
    onError?.(error);
  });
  // console.error/warn приложения собираем отдельно – чтобы не мешало
  const consoleErrors = [];
  virtualConsole.on("error", (...args) => consoleErrors.push(...args));

```

```

    "navigator",
    "HTMLElement",
    "HTMLInputElement",
    "HTMLTextAreaElement",
    "HTMLButtonElement",
    "HTMLAnchorElement",
    "HTMLDivElement",
    "HTMLSelectElement",
    "Element",
    "Node",
    "Event",
    "CustomEvent",
    "MouseEvent",
    "KeyboardEvent",
    "InputEvent",
    "getComputedStyle",
    "requestAnimationFrame",
    "cancelAnimationFrame",
    "DOMParser",
    "SVGElement",
    "Text",
    "DocumentFragment",
    "NodeList",
    "HTMLCollection",
    "Range",
    "MutationObserver",
    "localStorage",
    "sessionStorage",
    "Blob",
    "File",
    "FileReader",
  ];
  for (const key of copy) {
    try {
      Object.defineProperty(globalThis, key, { value: w
    } catch {
      /* read-only в Node – не критично */
    }
  }

```

```

    logLevel: "warning",
  });
const { liveMatrix, pickLiveVar, clampIndex } = await import('live-matrix');

let passed = 0;
const failures = [];
function check(name, actual, expected) {
  const a = JSON.stringify(actual);
  const e = JSON.stringify(expected);
  if (a === e) passed++;
  else failures.push(`${name}\n          получено: ${a}\n          ожидалось: ${e}`);
}
const m = (v) => liveMatrix(v);
const shape = (v) => {
  const r = m(v);
  return r ? { rows: r.rows, cols: r.cols, label: r.size } : null;
};
const cells = (v) => {
  const r = m(v);
  if (!r) return null;
  return Array.from({ length: r.rows }, (_, i) => Array.from({ length: r.cols }, (_, j) => r.get(i, j)));
};

/* — liveMatrix: формы данных —————
check("квадратная 3×3 (список списков)", shape([[1, 2, 3], [4, 5, 6], [7, 8, 9]]), [[1, 2, 3], [4, 5, 6], [7, 8, 9]]);
check("не квадратная 2×4", shape([[1, 2, 3, 4], [5, 6, 7, 8]]), [[1, 2, 3, 4], [5, 6, 7, 8]]);
check("значения ячеек 2×2", cells([[0, 1], [2, 3]]), [[0, 1], [2, 3]]);
check("рванные строки: cols = максимум", shape([[1, 2, 3], [4, 5, 6], [7, 8, 9, 10]]), [[1, 2, 3], [4, 5, 6], [7, 8, 9, 10]]);
check("рванные строки: дырка = null", cells([[1, 2, 3], [4, 5, 6], [7, 8, 9, 10, null]]), [[1, 2, 3], [4, 5, 6], [7, 8, 9, 10, null]]);
check("at() вне диапазона → null", m([[1, 2], [3, 4]]).at(10, 10), null);
check("at() с отрицательным индексом → null", m([[1, 2], [3, 4]]).at(-10, -10), null);
check("список словарей с числовыми ключами", cells([{ 0: 1, 1: 2 }, { 0: 3, 1: 4 }]), [{ 0: 1, 1: 2 }, { 0: 3, 1: 4 }]);
check("словарь списков", cells({ 0: [1, 2], 1: [3, 4] }), { 0: [1, 2], 1: [3, 4] });
check("словарь словарей", cells({ 0: { 0: 7, 1: 8 }, 1: { 0: 9, 1: 10 } }), { 0: { 0: 7, 1: 8 }, 1: { 0: 9, 1: 10 } });
check('ключи-пары "(i, j)" после снапшота tuple', cells(tuple([0, 1], [2, 3])), tuple([0, 1], [2, 3]));
check('ключи-пары без скобок "0, 1"', cells({ "0, 0": 5, "0, 1": 6, "1, 0": 7, "1, 1": 8 }, { "0, 0": 5, "0, 1": 6, "1, 0": 7, "1, 1": 8 }));

/* — liveMatrix: что матрицей НЕ является —————

```

```
-----  
process.exit(failures.length ? 1 : 0);  
  
-----
```

ФАЙЛ 104/111: scripts/verify/probe-compiler.tsx

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 121 | РАЗМЕР: 1.2 КБ

```
-----  
/**  
 * Зонд состояния панели Python во времени: что происходит  
 * Запуск: node tmp/harness/run-probe.mjs  
 */  
import { act } from "react";  
import { createRoot } from "react-dom/client";  
import App from "../../src/App";  
  
const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));  
async function settle(ms = 40) {  
  await act(async () => {  
    await sleep(ms);  
  });  
}  
function panel(): HTMLElement | null {  
  return document.querySelector<HTMLElement>('aside[aria-label="Python"]');  
}  
function editor(): HTMLTextAreaElement | undefined {  
  return panel()?.querySelector<HTMLTextAreaElement>('textarea');  
}  
function footer(): string {  
  const divs = Array.from(panel()?.querySelectorAll("div"));  
  const f = divs.find((d) => d.className.includes("text"));  
  return (f?.textContent ?? "").replace(/\s+/g, " ");  
}  
function state(tag: string) {  
  const p = panel();  
  const ed = editor();  
  const txt = (p?.textContent ?? "").replace(/\s+/g, " ");  
  const step = txt.match(/war[^ ]*\s*\d+\s*\/\s*\d+/i);  
  return {  
    tag,  
    step,  
    txt,  
    ed,  
    p,  
  };  
}
```



```
await click(findButton(/Боковая панель Python/), "панель Python",
state("t=0"));
for (const t of [300, 1000, 3000, 8000]) {
  await settle(t - (t === 300 ? 0 : 0));
  state(`t+${t}`);
  if (/Трасса готова|Нет шагов|Пустой код|Ошибка/.test(state)) {
    return;
  }
}
```

```
process.stdout.write("\n=== 2. вводим код ТОЛЬКО с кодами\n");
const ed = editor();
if (!ed) {
  process.stdout.write(" редактора нет – жмём Esc и ждём\n");
  await act(async () => {
    document.dispatchEvent(new window.KeyboardEvent("keydown", {
      key: "Esc",
    }));
    await sleep(30);
  });
}
const ed2 = editor();
if (!ed2) throw new Error("редактор так и не появился\n");
typeInto(ed2, "# только комментарий\n# и ещё\n");
state("ввели");
for (const t of [200, 800, 2000, 6000]) {
  await settle(t);
  state(`t+${t}`);
}
```

```
process.stdout.write("\n=== 3. вводим рабочий код x=41\n");
const ed3 = editor();
if (!ed3) throw new Error("редактор пропал после комментария\n");
typeInto(ed3, "x = 41\nx += 1\nprint(x)\n");
state("ввели");
for (const t of [200, 800, 2000, 6000]) {
  await settle(t);
  state(`t+${t}`);
}
```

```
process.stdout.write("\n=== 4. очищаем редактор полностью\n");
const ed4 = editor();
```

Универсальное пособие • полный исходный код

помарочно по DOM-сценариям, `reference-code.json` — по

Что именно проверяется

`verify:unit` — `live-matrix.test.mjs`

Разбор данных, которые присылает компилятор, в матрицу 1000 размерности: список списков, рванные строки, словарь слов, `"(0, 3)"` (так снапшотятся tuple-ключи Python), `"inf"`, числа-строки, обрезка до 64x64. Отдельно — что матрицей, список и словарь по именам (`dist`, `prev`): иначе визуализация чужие данные за сетку. Плюс `pickLiveVar` (приоритет имени, индексы пользователя вне диапазона демо).

`verify:code` — `check-reference-code.mjs`

Каждый референсный код из `PAGE\_SYNC` выполняется настоящим `sys.settrace`-трейсером, что и панель компилятора. Протокол трасса не оборвана по лимиту шагов, шагов больше нуля (или пустой листинг), и **каждая переменная, обещанная строкой кодом** — иначе визуализация физически не может за ним следовать. Предупреждения интерпретатора, которые иначе пойдут в «Результат».

`verify:dom` — `browser-checks.tsx`

Настоящее приложение монтируется в jsdom (`App` целиком в первой главе, визуализация, панель Python), сценарии кликают кнопки в редакторе.

| Группа | Что делает |

| --- | --- |

| А. страницы (22 × 3) | каждая глава монтируется; бейдж автозапуска инициализации настоящим Python не роняет и не возвращает `[object Object]`; нет кнопок без имени |

| В. Флойд: размерности (10) | 5×5, 10×10 (больше демо — 100×100), 100×100 (обрезка), пустая матрица, чужое имя, список, 80×80 (обрезка), пустая матрица, чужое имя

| С. матричные страницы (6) | префиксные суммы 2D и разности

| D. графы путей (2) | Дейкстра/Форд-Беллман с вершинами, гашения гаснут и оживают обратно |

| Е. эталонный код (46) | прогон кода в CPython → финальный

* window.__algv0PythonRuntime и собирает реальные комп
*/

```
import * as esbuild from "esbuild";  
import { writeFileSync, mkdirSync } from "fs";  
import { loadPyodide } from "pyodide";  
import { createDom } from "./dom-env.mjs";
```

```
const OUT_DIR = "tmp/verify";  
const t0 = Date.now();  
const jsdomErrors = [];
```

```
log("гружаю CPython (pyodide из node_modules)...");  
const py = await loadPyodide({ indexURL: "node_modules/" });  
log(` Python готов за ${Date.now() - t0} мс`);
```

```
log("поднимаю jsdom...");  
const { window, document, consoleErrors } = createDom({  
window.__algv0PythonRuntime = py;
```

```
log("собираю приложение esbuild'ом..");  
mkdirSync(OUT_DIR, { recursive: true });  
await esbuild.build({  
  entryPoints: ["scripts/verify/browser-checks.tsx"],  
  bundle: true,  
  outfile: `${OUT_DIR}/bundle.mjs`,  
  format: "esm",  
  platform: "browser",  
  target: "es2022",  
  jsx: "automatic",  
  sourcemap: true,  
  // Vite-специфичные глобалы, которых нет вне dev-серв  
  define: {  
    "process.env.NODE_ENV": '"development"',  
    "import.meta.env.BASE_URL": '"/"',  
    "import.meta.env.MODE": '"development"',  
    "import.meta.env.DEV": "true",  
    "import.meta.env.PROD": "false",  
    "import.meta.env.SSR": "false",
```

Универсальное пособие • полный исходный код

```
-----  
process.exit(sum.failed > 0 || jsdomErrors.length > 0 ?
```

```
function log(message) {  
  process.stdout.write(`[verify:dom] ${message}\n`);  
}
```

```
-----  
ФАЙЛ 107/111: scripts/verify/run-probe.mjs
```

```
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 45 | РАЗМ
```

```
-----  
/**  
 * Зонд панели Python: печатает состояние редактора/тран  
 * Не тест, а диагностика — когда нужно понять, что пан  
 * ввода кода (автозапуск, переход в листинг, сообщения  
 *  
 *   npm run verify:probe  
 */
```

```
import * as esbuild from "esbuild";  
import { mkdirSync } from "fs";  
import { loadPyodide } from "pyodide";  
import { createDom } from "../dom-env.mjs";
```

```
const OUT_DIR = "tmp/verify";  
mkdirSync(OUT_DIR, { recursive: true });  
const log = (m) => process.stdout.write(`[verify:probe]
```

```
log("гружаю CPython...");  
const py = await loadPyodide({ indexURL: "node_modules/"  
log("поднимаю jsdom...");  
const { window, document } = createDom();  
window.__algv0PythonRuntime = py;
```

```
await esbuild.build({  
  entryPoints: ["scripts/verify/probe-compiler.tsx"],  
  bundle: true,  
  outfile: `${OUT_DIR}/probe-bundle.mjs`,
```

иначе деплой из новой рабочей ветки отклоняется

on:

push:

branches:

- main

Позволяет проверить Pages прямо из рабочей ветки

- "arena/**"

workflow_dispatch:

permissions:

contents: read

pages: write

id-token: write

concurrency:

group: pages

cancel-in-progress: true

jobs:

build:

name: Build static site

runs-on: ubuntu-latest

steps:

- name: Checkout

uses: actions/checkout@v6

- name: Setup Node.js

uses: actions/setup-node@v7

with:

node-version: 24

cache: npm

- name: Install dependencies

run: npm ci --no-audit --no-fund

- name: Build for the repository subpath

env:

Универсальное пособие • полный исходный код

ФАЙЛ 109/111: `.github/workflows/rebuild-pdf.yml`

КАТЕГОРИЯ: CI / автоматизация | СТР0К: 155 | РАЗМЕР: 6.1

name: Rebuild code PDF

Пайплайн пересборки PDF при каждом коммите:

#

исходный код → full_code_all_pages.txt (scrip

└─→ page-0001.png ... page-NNNN.png (

└─→ full_code_all_page

#

Готовые TXT и PDF коммитаются обратно в ветку (их отда

промежуточные PNG сохраняются как artifacts запуска.

on:

 push:

 branches:

 - "**"

 paths-ignore:

 # чтобы коммит самого workflow не запускал бескон

 - "public/export/**"

 - "**/*.md"

 workflow_dispatch:

 inputs:

 density:

 description: "DPI растеризации страниц (по умол

 required: false

 default: "150"

concurrency:

 group: rebuild-pdf-\${{ github.ref }}

 cancel-in-progress: true

permissions:

- name: Step 1 – код → txt
run: npm run export:txt
- name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${github.event.inputs.density}
run: npm run export:png
- name: Step 3 – png → pdf
run: npm run export:pdf
- name: Step 4 – собрать приложение и поднять preview
run: |
 # headless-браузер для снимков: chrome ставится автоматически
 # этот вызов – страховка, если кэш оказался пустым
 npx puppeteer browsers install chrome || true
 npm run build
 mkdir -p tmp
 nohup npx vite preview --host 127.0.0.1 --port 4173 &
 for i in \$(seq 1 40); do
 if curl -sf http://127.0.0.1:4173/ > /dev/null; then
 sleep 1
 fi
 done
- name: Step 5 – снимки визуализаций (headless-браузер)
run: npm run export:shots
continue-on-error: true
- name: Step 6 – PDF-конспект со снимками
run: |
 npm run export:guide
 # снимки публикуются вместе с приложением: HTML-список
 # в автономный файл прямо из /export/viz-shots/manifest.json
 mkdir -p public/export/viz-shots
 cp tmp/viz-shots/*.png tmp/viz-shots/manifest.json
- name: Type-check приложения
run: npx tsc --noEmit

```
-----
git add -- public/export/full_code_all_pages.
t/viz-shots
if git diff --cached --quiet; then
    echo "Экспорт не изменился — коммит не нужен"
    exit 0
fi
git commit -m "chore(export): пересборка full
git push origin "HEAD:${GITHUB_REF_NAME}"
-----
```

ФАЙЛ 110/111: .github/workflows/verify.yml

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 71 | РАЗМЕР: 2.3

name: Verify

Проверки приложения на каждом пуше и PR:

```
#
#   verify:unit   — разбор «живых» данных компилятора (l
#   verify:code  — все эталонные Python-коды страниц вы
#                   под трейсером панели и создают обеща
#   verify:dom    — приложение монтируется в jsdom: стра
#                   матрицы чужих размерностей, кнопки,
#
# Headless-браузер не нужен: jsdom + pyodide из node_modules
```

on:

push:

branches:

- "**"

paths-ignore:

- "public/export/**"

- "**/*.md"

pull_request:

workflow_dispatch:

concurrency:


```
run: npm run verify:dom
```

- name: Отчёт проверок
if: always()
uses: actions/upload-artifact@v4
with:
 name: verify-report
 path: tmp/verify/report.json
 if-no-files-found: ignore

РАЗДЕЛ: ПРОЧЕЕ

ФАЙЛ 111/111: .ai_skills/universal_guide_authoring/SKIL
КАТЕГОРИЯ: Прочее | СТРОК: 252 | РАЗМЕР: 13.3 KB

Universal Guide Authoring – шаблоны страниц пособия

Обязательный справочник для ****создания и правки контента** репозитории. На него ссылаются `AGENTS.md` и `GEMINI.md` собираем ****только**** из блоков ниже – не придумываем новый «голый» markdown.

Контент живёт в `src/data/\*\*/\*.ts` как массив объектов `src/data/content.ts` (при одинаковом `id` побеждает бо.

1. Объект главы

```
```ts
```

```
import { Chapter } from "../../types";
```

```
export const myChapters: Chapter[] = [
```

```
...определение, абзацы, стат-сетка, аналогии, спол
 </div>
 </div>
</section>
```
```

Цвет левой полосы и заголовка блока – семантика раздела

| Полоса / цвет | Смысл раздела |
|---|-----------------|
| --- --- | |
| `border-blue-500` + `text-blue-400` | определение, «о |
| `border-emerald-500` + `text-emerald-400` | алгоритм, |
| `border-indigo-500` + `text-indigo-400` | теория, док |
| `border-amber-500` + `text-amber-400` | ловушки, част |
| `border-rose-500` + `text-rose-400` | ограничения, «к |

3. Библиотека блоков (точные классы)

3.1 Определение / строгое правило – всегда по центру

```
```html
<div class="bg-slate-900 p-4 rounded-lg mb-6 text-cente
 <p class="text-lg text-emerald-300 italic mb-2">Строг
 <p class="text-xl font-mono text-white">if d[v] > d[u]
</div>
```
```

3.2 Обычный абзац

```
```html
<p class="text-slate-300 text-sm mb-4">
 Текст с акцентом и инлайн-кодом
 dist[v]
</p>
```
```

```

    </div>
</div>
```

```

Второй карточкой может быть «Аналогия 2», «Ограничения» «Когда не работает» — маккуп тот же (пунктирная rose-ра

### ### 3.5 Инфо-блок с таблицей

```

```html
<div class="bg-slate-800/70 p-5 rounded-lg border borde
  <p class="text-sm font-bold text-white mb-2">  Заголо
  <p class="text-slate-300 text-sm mb-3">Пояснение пере
  <table class="w-full text-xs text-left border-collaps
    <thead>
      <tr class="text-slate-500 border-b border-slate-70
        <th class="py-1.5 pr-2 font-bold">Колонка</th>
        <th class="py-1.5 pr-2 font-bold">Колонка</th>
      </tr>
    </thead>
    <tbody>
      <tr class="border-b border-slate-800 align-top">
        <td class="py-2 pr-3 font-bold">Значение</td>
        <td class="py-2 pr-3 font-mono">O(log n)</td>
      </tr>
    </tbody>
  </table>
</div>
```

```

### ### 3.6 Спойлер `<details>` — **\*\*обязателен\*\*** для кода, и

```

```html
<details class="bg-slate-900/40 rounded-lg border borde
  <summary class="p-4 font-bold text-slate-400 outline-l
    e-700/50">
    ▶ Код: Дейкстра на куче (Python)
  </summary>

```

по `id` главы после текста.

5. Контракт «демонстрация ↔ компилятор»

Каждая страница обязана иметь и визуализацию, и синхронизацию.

- `src/components/vizRegistry.tsx` – запись с ключом ****`id`**** (``title``, опциональный ``hint``, ``Component``).
- `src/data/vizSync.ts` – запись ``PAGE_SYNC[id]`` или ``PAGE_SYNC[title]`, где ``demoId`` – второй аргумент ``emitVizDemo(demoId, title)`` (вкладки одной демонстрации).
- Поля записи: ``vizTitle``, ``stepNote`` (что считается одной страницей), ``variables: [{ name, role, range }]``, ``code`` (референс на код).
- Переменные, которые читает визуализация, должны быть ``int``, ``list``, ``dict``, ``str``: экземпляры классов ``Int``, ``List``, ``Dict``, ``Str`` до графики не доезжают.
- Стартовый шаблон в редакторе компилятора – ``buildInit``, инициализация переменных до строки ``# ---`` – тело алгоритма.

Проверка перед коммитом: ``npm run dev`` → страница открылась, код, шаг демонстрации подсвечивает нужные переменные.

6. Экспорт в PDF (``npm run export:guide``)

Текст страниц парсится ``scripts/export/html-to-blocks.mjs``, результат сохраняется в ``public/export/guide_lite.pdf`` (текст + визуализации, без кода компилятора и без навигации). Чтобы экспорт был полным:

- заголовки – только ``h2/h3/h4``; списки – ``ul/ol/li``; теги кода – ``pre``; определения – обычный ``p`` (см. 3.1);
- ****смысл не должен держаться на эмодзи****: в PDF используются только ASCII-глифы – пиктограммы вырезаются, остаётся текст. «Операции на отрезках» доедет как «Операции на отрезках», а вот «Операции на отрезках / Операции на отрезках» потеряет смысл, пиктограммы не вырезаются.